Alex Goessmann

Artificial Intelligence in the Tensor Formalism

April 4, 2026

Springer Nature

Contents

Notation xi
1.1 Tensors xi
1.2 Tensor networks and contractions xii
1.3 Generic contractions xiii
1.4 Normalizations xiv
1.5 Function encoding and Computation-Activation Networks xiv

Part I Foundations of Artificial Intelligence

2 Propositional Logic 3
2.1 Propositonal Formulas 3
2.1.1 Knowledge bases 4
2.2 Entailment 4
2.2.1 Deciding Entailment by Contractions 5
2.2.2 Deciding Entailment by Partial Ordering 5
2.2.3 Redundancy of Entailed Formulas 6
2.2.4 Contraction Knowledge Base 6
2.3 Syntactical decompositions 7
2.3.1 Encoding of Booleans 8
2.3.2 Basis encodings 9
2.3.3 11
2.3.4 Circuit decomposition 11
2.3.5 Atomic Formulas 11
2.3.6 Syntactical Composition of Formulas 12
2.3.7 Minterm and maxterm decompositions 15
2.3.8 Outlook 17
2.4 Constraint Satisfaction Problems 17
2.4.1 Manipulation of constraint tensor networks 18
2.4.2 Examples 18
2.5 Inference algorithms 23
2.5.1 Monotonicity of Entailment 23
2.5.2 Syntactical inference algorithms 24
2.5.3 Constraint Propagation 24
2.5.4 Applications 27
2.5.5 Backtracking search 28
2.5.6 Discussion 28

3 Probability Theory . . . 29
3.1 Tensor Formalism in Probability Theory . . . 29
3.1.1 Joint Probability Distributions . . . 29
3.1.2 Base Measures . . . 30
3.1.3 Marginal Distribution . . . 31
3.1.4 Conditional Probabilities . . . 32
3.1.5 Bayes Theorem and the Chain Rule . . . 34
3.2 Independence as a Decomposition Mechanism . . . 34
3.3 Graphical Models . . . 38
3.3.1 Markov Networks . . . 38
3.3.2 Bayesian Networks . . . 40
3.3.3 Bayesian Networks as Markov Networks . . . 42
3.4 Factorization based on sufficient statistics . . . 44
3.5 Efficient contractions by message passing . . . 46
3.6 Queries . . . 48
3.6.1 Querying by Functions . . . 48
3.6.2 Mode Queries . . . 49
3.6.3 Energy Representations . . . 50
3.7 Sampling . . . 50
3.7.1 Forward Sampling . . . 51
3.7.2 Gibbs Sampling . . . 51
3.7.3 Simulated Annealing . . . 52
3.8 Maximum Likelihood Estimation . . . 53
3.8.1 Empirical Distributions . . . 53
3.8.2 Likelihood Loss . . . 55
3.8.3 Entropic Interpretation . . . 56

4 First-Order Logics . . . 59
4.1 Syntax and Semantics of first-order logic . . . 59
4.1.1 Syntax . . . 59
4.1.2 Tensor Representation of Semantics . . . 59
4.1.3 Worlds . . . 60
4.1.4 Two Tensor Structures . . . 60
4.2 Substitution Structure . . . 61
4.2.1 Grounding Tensors . . . 61
4.2.2 Basis encodings of terms . . . 61
4.2.3 Atomic sentences . . . 62
4.2.4 Formula Synthesis by Connectives . . . 62
4.2.5 Quantifiers . . . 63
4.2.6 Storage in Basis CP Decomposition . . . 65
4.2.7 Example: Relational Databases . . . 65
4.3 Knowledge bases . . . 66
4.3.1 Substitution . . . 66
4.4 Inference . . . 66
4.4.1 Unification . . . 67
4.4.2 Modus ponens . . . 67

5 Circuits and Automata . . . 69
5.1 Generalized basis encodings . . . 69
5.2 Circuit decompositions . . . 69
5.2.1 Function decomposition . . . 69
5.2.2 Function evaluation by message passing . . . 73

6 Causality . . . 77
6.1 Rung 1: Association . . . 77
6.2 Rung 2: Intervention . . . 77
6.2.1 Do calculus . . . 78
6.2.2 Intervention Queries . . . 78
6.2.3 Simplifications . . . 78
6.3 Rung 3: Counterfactuals . . . 80
6.3.1 Response Variables . . . 81
6.3.2 Counterfactual Twinned Network . . . 81

7 Quantum circuits . . . 83
7.1 Introduction . . . 83
7.1.1 Quantum and Tensor Parallelism . . . 83
7.2 Foundations . . . 83
7.2.1 Comparing tensor networks and quantum circuits . . . 84
7.2.2 Controlled Single Qubit Gates . . . 84
7.2.3 Measurements and Contractions . . . 85
7.3 Graph Controlled Circuits . . . 87
7.3.1 Definition . . . 87
7.3.2 Preparation of q-samples to Bayesian Networks . . . 87
7.3.3 Equivalence with Bayesian Networks . . . 90
7.4 Ancilla Augmentation . . . 91
7.4.1 Post-selection by Ancilla Variables . . . 91
7.4.2 Construction based on Uniform Distributions . . . 92
7.4.3 Best Acceptance Probability by ∞-Renyi Divergence . . . 94
7.4.4 Augmentation of Markov Networks . . . 95
7.5 Computation circuits . . . 98
7.5.1 Relation with basis encodings . . . 98
7.5.2 Polynomial sparsity . . . 99
7.5.3 Decomposition sparsity . . . 100
7.6 Function Encoding . . . 101
7.6.1 Basis encoding . . . 102
7.6.2 Sign encoding . . . 103
7.6.3 Conclusion . . . 104
7.7 Contractions based on Fourier Transforms . . . 105
7.7.1 Walsh-Hadamard transform . . . 105
7.7.2 Examples . . . 105
7.7.3 Generic Hidden Subgroup Problems . . . 107

Part II Computation-Activation Networks

8 Elementary Computation-Activation Networks . . . 111
8.1 Exponential Families . . . 111
8.1.1 Definition . . . 111
8.1.2 Tensor Network Representation . . . 114
8.1.3 Markov Networks as Exponential Families . . . 116
8.1.4 Representation of Generic Distributions . . . 118

9 Mean Polytopes and Maximum Entropy Distributions . . . 119
9.1 Polytopes of Mean Parameters . . . 119
9.1.1 Representation by Convex Hulls . . . 119
9.1.2 Representation as Intersecting Half-Spaces . . . 120
9.1.3 Characterization of the Interior . . . 121
9.1.4 Characterization of the Boundary by Faces . . . 122
9.1.5 Partition into Effective Interiors of Faces . . . 125

9.1.6 Centers . . . 125

10 Variational Inference . . . 127

11 Inductive Reasoning . . . 129

12 Probabilistic Guarantees . . . 131

13 Quantum Inference . . . 133
13.1 Introduction . . . 133
13.1.1 Circuit Preparation schemes . . . 133
13.1.2 Related Works . . . 134
13.2 Grover operator . . . 134
13.2.1 Definition . . . 134
13.2.2 Action on projected quantum states . . . 134
13.2.3 Circuit Realization of the reflection operators . . . 135
13.3 Preparation of q-samples to CompActNets . . . 137
13.3.1 Sampling from Computation-Activation Networks . . . 138
13.4 Quantum Rejection Sampling . . . 139
13.4.1 Acceptance Amplification . . . 140
13.4.2 Optimal rotation numbers . . . 140
13.4.3 Applications on CompActNets . . . 141
13.4.4 Examples . . . 141
13.5 Moment Estimation . . . 141
13.5.1 Moments as rotation angles . . . 142
13.5.2 Retrieving the moment from the phase . . . 143
13.5.3 Quantum Phase Estimation . . . 143
13.6 Moment Amplification . . . 143
13.6.1 Interpretation of Grover rotations . . . 143
13.6.2 Moment Matching . . . 145
13.7 Extension: Sampling from proposal distributions . . . 146

14 Probabilistic Relational Models . . . 147
14.1 Semantic Structure . . . 147
14.1.1 World Enumerating Variables . . . 147
14.1.2 Representation of Terms and Formulas . . . 148
14.1.3 Case of Propositional Logic . . . 148
14.1.4 One-hot Encoding of Worlds . . . 148
14.1.5 Probability Distributions . . . 149
14.2 Probabilistic Relational Models . . . 149
14.2.1 Hybrid First-Order Logic Networks . . . 149
14.2.2 Propositionalization . . . 150
14.2.3 Conditioning on an Importance Formula . . . 151
14.2.4 Decomposition of the Log Likelihood . . . 151
14.2.5 Reduction to Propositional Logic . . . 152
14.2.6 Sample Extraction from First-Order Logic Worlds . . . 155
14.3 Generation of First-Order Logic Worlds . . . 158
14.3.1 Samples by Single Objects . . . 158
14.3.2 Samples by Tuples of Objects . . . 160
14.4 Discussion . . . 161

Part III Contraction Calculus

Glossary . . . 165

Acronyms

TT	Tensor-Train format
CP	Canonical Polyadic format
CAN	Computation-Activation Network
HLN	Hybrid Logic Network, Computation-Activation Network in case of binary variables

Chapter 1
Notation

Abstract We introduce the notation used in this book.

In this section, we introduce the hypergraph-based tensor network formalism and define the most general tensor network architecture of CompActNets based on this formalism.

1.1 Tensors

Tensors are multiway arrays and a generalization of vectors and matrices to higher orders. We first provide a formal definition as real maps from index sets enumerating the coordinates of vectors, matrices, and higher-order tensors. To ease the notation, we abbreviate sets as $[d] = \{0, \ldots, d-1\}$, tuples of state indices by $x_{[d]} = (x_0, \ldots, x_{d-1})$ and tuples of variables by $X_{[d]} = (X_0, \ldots, X_{d-1})$.

Definition 1.1 (Tensor) For $k \in [d]$, let $m_k \in \mathbb{N}$ and let X_k be categorical variables taking values in $[m_k]$. A tensor $\tau\left[X_0, \ldots, X_{d-1}\right]$ of order d and leg dimensions $m_0, \ldots, m_{d-1}$ is defined through its coordinates

$$\tau\left[X_{[d]} = x_{[d]}\right] = \tau\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] \in \mathbb{R}$$

for index tuples

$$x_{[d]} = (x_0, \ldots, x_{d-1}) \in \bigtimes_{k \in [d]} [m_k] \,.$$

Tensors $\tau\left[X_{[d]}\right]$ are elements of the tensor space

$$\bigotimes_{k \in [d]} \mathbb{R}^{m_k} \,,$$

which is a linear space, enriched with the operations of coordinate-wise summation and scalar multiplication. We call a tensor $\tau\left[X_{[d]}\right]$ boolean, when all coordinates are in $\{0, 1\}$, and positive, when all coordinates are greater than 0.

We introduced tensors here in a non-canonical way based on categorical variables assigned to their axes. While this may look like syntactic sugar at this point, it allows us to define contractions without further specification of axes, based on comparisons of shared variables. We occasionally also allow for variables X taking values in infinite sets such as $\mathbb{R}$, in which case we denote the set of values to a variable by $\mathrm{val}\,(X)$.

Example 1.1 (Delta tensor) Given a tuple of variables $X_{[d]} = (X_0, \ldots, X_{d-1})$ with identical dimension m, where $d \geq 1$, the delta tensor is the element

$$\delta^{[d],m}\left[X_{[d]}\right] \in \bigotimes_{k \in [d]} \mathbb{R}^m$$

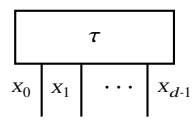
with coordinates

$$\delta^{[d],m}\left[X_{[d]} = x_{[d]}\right] = \begin{cases} 1 & \text{if} \quad x_0 = \ldots = x_{d-1} \\ 0 & \text{else} \end{cases} .$$

We depict this tensor by black dots, which sometimes appears as auxiliary elements in tensor network diagrams (see e.g. Figure 1.3). For $d = 1$, the delta tensor is the trivial vector, whose coordinates are constantly 1, which we denote by $\mathbb{I}[X]$.

1.2 Tensor networks and contractions

We use a standard visualization of tensors (dating back to [**?**]) by blocks with lines depicting the axes of the tensor. Additionally, we assign to each axis of the tensor the corresponding variable X_k:



We now associate categorical variables with nodes of a hypergraph and tensors with hyperedges, which are arbitrary subsets of nodes. Based on this association we continue with the definition of tensor networks.

Definition 1.2 (Tensor network) Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a hypergraph, let X_v for $v \in \mathcal{V}$ be categorical variables with dimensions $m_v \in \mathbb{N}$, and let

$$\tau^e\left[X_e\right] \in \bigotimes_{v \in e} \mathbb{R}^{m_v}$$

be tensors for $e \in \mathcal{E}$, where we denote by X_e the set of categorical variables X_v with $v \in e$. Then, we call the set

$$\tau^{\mathcal{G}}\left[X_{\mathcal{V}}\right] = \left\{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\right\}$$

the tensor network of the decorated hypergraph $\mathcal{G}$. The set of tensor networks on $\mathcal{G}$ such that all tensors have non-negative coordinates is denoted by $\mathcal{T}^{\mathcal{G}}$.

As examples we now present the CP and the TT formats in our hypergraph notation.

Example 1.2 (The CP *format)* The Candecomp-Parafac (CP) tensor format (see [**?**]) corresponds in our notation to a hypergraph (see Figure 1.1) defined by

- nodes $X_{[d]}$ and a single hidden variable I, decorated by dimensions $m_{[d]}$ and the CP-rank n, respectively
- edges $\left\{e_k = \{X_k, I\} \,:\, k \in [d]\right\}$ each decorated by a matrix $\tau^{e_k}\left[X_k, I\right] \in \mathbb{R}^{m_k \times n}$.

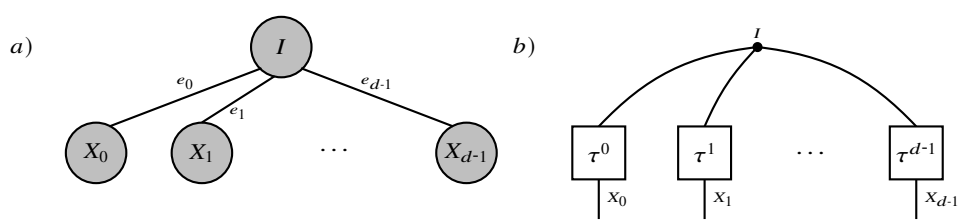


Fig. 1.1 Hypergraph to a CP format (see Example 1.2). a) Node-centric design. b) Corresponding tensor network on the edges of the hypergraph.

Example 1.3 (The TT *format)* The Tensor-Train (TT) format (see [**?**]) corresponds in our notation to a hypergraph (see Figure 1.2) defined by

- nodes $X_{[d]}$ and hidden variables $I_{[d-1]}$, each decorated by a dimension $m_{[d]}$ and $n_{[d-1]}$,
- edges

$$\left\{e_0 = \{X_0, I_0\}\right\} \cup \left\{e_k = \{I_{k-1}, X_k, I_k\} \,:\, k \in \{1, \ldots, d-2\}\right\} \cup \left\{e_{d-1} = \{I_{d-2}, X_{d-1}\}\right\}$$

  each decorated by a tensor of order 3 (respectively 2 for $k \in \{0, d-1\}$).

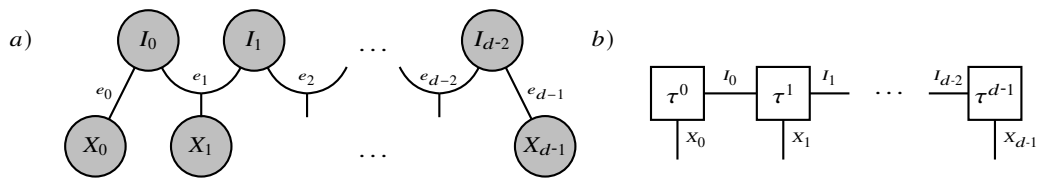


Fig. 1.2 Hypergraph to a TT format (see Example 1.3). a) Node-centric design. b) Corresponding tensor network on the edges of the hypergraph.

1.3 Generic contractions

Let us now exploit our graphical approach to tensor networks in the definition of contractions.

Definition 1.3 Let $\tau^{\mathcal{G}}$ be a tensor network on a decorated hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. For any subset $\mathcal{U} \subset \mathcal{V}$ we define the contraction of $\tau^{\mathcal{G}}$ with open variables $X_{\mathcal{U}}$ to be the tensor (for an example see Figure 1.3)

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}]} \in \bigotimes_{v \in \mathcal{U}} \mathbb{R}^{m_v}$$

with coordinates at indices $x_{\mathcal{U}} \in \bigtimes_{v \in \mathcal{U}} [m_v]$ by

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} = \sum_{x_{\mathcal{V} \setminus \mathcal{U}} \in \bigtimes_{v \in \mathcal{V} \setminus \mathcal{U}} [m_v]} \left(\prod_{e \in \mathcal{E}} \tau^e \left[X_e = x_e \right] \right).$$

When an open variable X does not appear in any tensor in a contraction, we define the contraction as a tensor product with the trivial tensor $\mathbb{I}[X]$ (see Example 1.1). To ease notation, we often omit the set notation by brackets $\{\cdot\}$.

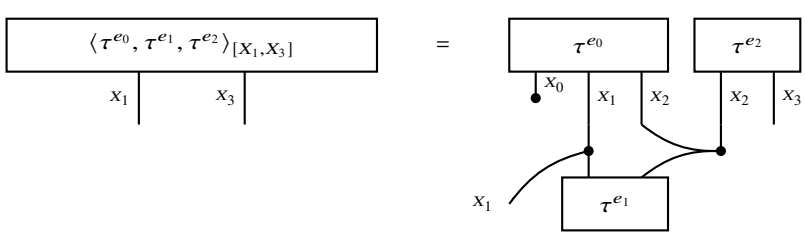


Fig. 1.3 Graphical depiction of a tensor network contraction with the open variables X_1, X_3. Open variables are depicted by those without a dot at the end of the line.

Example 1.4 (Tensor product) The simplest contraction is the tensor product, which maps a pair of two tensors with distinct variables onto a third tensor and has an interpretation by coordinate-wise products. Such a contraction corresponds with a tensor network of two tensors with disjoint variables. Let there be two tensors

$$\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]} \mathbb{R}^{m_k} \quad \text{and} \quad \tilde{\tau}\left[Y_{[p]}\right] \in \bigotimes_{\ell\in[p]} \mathbb{R}^{n_\ell}$$

with disjoint tuples of categorical variables assigned to their axes. Then their tensor product is the tensor

$$\left\langle \tau\left[X_{[d]}\right], \tilde{\tau}\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]},Y_{[p]}\right]} \in \left(\bigotimes_{k\in[d]} \mathbb{R}^{m_k}\right) \otimes \left(\bigotimes_{\ell\in[p]} \mathbb{R}^{n_\ell}\right)$$

with coordinates to tuples of $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ and $y_{[p]} \in \bigtimes_{\ell\in[p]}[n_\ell]$ as

$$\begin{aligned}&\left\langle \tau\left[X_{[d]}\right], \tilde{\tau}\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}=x_{[d]},Y_{[p]}=y_{[p]}\right]}\\ &\quad := \tau\left[X_{[d]} = x_{[d]}\right] \cdot \tilde{\tau}\left[Y_{[p]} = y_{[p]}\right] .\end{aligned}$$

1.4 Normalizations

Based on generic contractions, we now introduce the normalization of tensors, which introduces certain contraints on tensors to be depicted by directed hyperedges.

Definition 1.4 The normalization of a tensor $\tau\left[X_{\mathcal{V}}\right]$ on incoming nodes $\mathcal{V}^{\text{in}} \subset \mathcal{V}$ and outgoing nodes $\mathcal{V}^{\text{out}} \subset \mathcal{V}\backslash\mathcal{V}^{\text{in}}$ is the tensor $\langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}^{\text{out}}}|X_{\mathcal{V}^{\text{in}}}\right]}$ defined for $x_{\mathcal{V}^{\text{in}}}$ as

$$\langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}^{\text{out}}}|X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]} = \begin{cases} \frac{\langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{out}}},X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]}}{\langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]}} & \text{if} \quad \langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]} \neq 0 \\ \frac{1}{\prod_{v\in\mathcal{V}^{\text{out}}} m_v}\mathbb{I}\left[X_{\mathcal{V}^{\text{out}}}\right] & \text{else} \end{cases} .$$

We say that $\tau\left[X_{\mathcal{V}}\right]$ is normalized with incoming nodes $\mathcal{V}^{\text{in}} \subset \mathcal{V}$, if

$$\tau\left[X_{\mathcal{V}}\right] = \langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}\backslash\mathcal{V}^{\text{in}}}|X_{\mathcal{V}^{\text{in}}}\right]} .$$

In our graphical tensor notation, we depict normalized tensors by directed hyperedges (a), which are decorated by directed tensors (b), for example when $X_{\mathcal{V}^{\text{in}}} = (X_2, X_3)$ and $X_{\mathcal{V}\backslash\mathcal{V}^{\text{in}}} = (X_0, X_1)$:

1.5 Function encoding and Computation-Activation Networks

Towards presenting the function encoding schemes, we define one-hot encodings mapping the states of variables to basis tensors.

Definition 1.5 (One-hot encoding) To any variable X taking values in $[m]$, the one-hot encoding of any state $x \in [m]$ is the vector with coordinates

$$\epsilon_x\left[X = \tilde{x}\right] := \begin{cases} 1 & \text{if} \quad x = \tilde{x} \\ 0 & \text{else} . \end{cases}$$

To any tuple $X_{[d]}$ of variables taking values in $\bigtimes_{k\in[d]}[m_k]$, the one-hot encoding of a state tuple $x_{[d]}$ is the tensor product

$$\epsilon_{x_{[d]}}\left[X_{[d]}\right] := \bigotimes_{k\in[d]} \epsilon_{x_k}\left[X_k\right] .$$

We now use one-hot encodings to encode functions between state sets.

Definition 1.6 (Basis encoding of maps between state sets) Let there be two sets of variables $X_{[d]}$ and $Y_{[p]}$, and let there be a map

$$q : \bigtimes_{k\in[d]} [m_k] \to \bigtimes_{\ell\in[p]} [n_\ell]$$

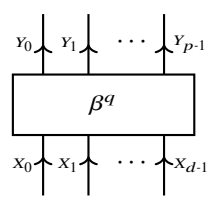
between their state sets. Then, the basis encoding of q is a tensor

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] \in \left(\bigotimes_{\ell\in[p]} \mathbb{R}^{n_\ell}\right) \otimes \left(\bigotimes_{k\in[d]} \mathbb{R}^{m_k}\right)$$

defined by

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] = \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \epsilon_{q\left(x_{[d]}\right)}\left[Y_{[p]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] .$$

Basis encodings are normalized tensors and are thus depicted as decorations of directed edges in hypergraphs:



We further generalize basis encodings to arbitrary functions between finite sets by the use of bijective image enumeration maps. Given an arbitrary set $\mathcal{U}$, we say a map

$$I : \bigtimes_{k\in[d]} [m_k] \to \mathcal{U}$$

is an enumeration map of $\mathcal{U}$ by d variables X_k, taking values in m_k. Given a function $q : \mathcal{U}^{\text{in}} \to \mathcal{U}^{\text{out}}$ between arbitrary sets and enumerating maps I_{in} and I_{out} for both sets, we define the basis encoding of q as

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] = \sum_{u\in\mathcal{U}^{\text{in}}} \epsilon_{I_{\text{out}}^{-1}(q(u))}\left[Y_{[p]}\right] \otimes \epsilon_{I_{\text{in}}^{-1}(u)}\left[X_{[d]}\right] ,$$

where $X_{[d]}, Y_{[p]}$ are variables taking values in $[\left|\mathcal{U}^{\text{in}}\right|]$ and $[\left|\mathcal{U}^{\text{out}}\right|]$. In Example 5.1 we present index enumeration maps for summations in m-adic integer representations. Based on these concepts, we define the most general tensor network architecture to be applied in the rest of this work.

Definition 1.7 (Computation-Activation Network (CompActNets)) Let there be a function $t : \bigtimes_{k\in[d]}[m_k] \to \mathbb{R}^p$ with basis encoding $\beta^t\left[Y_{[p]}, X_{[d]}\right]$, where $Y_{[p]}$ is a tuple of variables to an enumeration map of the image of t. Let there further be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with nodes $\mathcal{V}$ containing $[p]$. We define the by t computable and by $\mathcal{G}$ activated family of distributions by

$$\Lambda^{t,\mathcal{G}} = \left\{\left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \langle\xi\rangle_{\left[Y_{[p]}\right]}\right\rangle_{\left[X_{[d]}|\varnothing\right]} : \xi\left[Y_{\mathcal{V}}\right] \in \mathcal{T}^{\mathcal{G}}\right\} .$$

We refer to any member $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{t,\mathcal{G}}$ as a *Computation-Activation Network* (or shorter as a *CompActNet*). We call $\beta^t\left[Y_{[p]}, X_{[d]}\right]$ (and any decomposition of it) the *computation network* and $\xi\left[Y_{\mathcal{V}}\right]$ the *activation network*.

The elementary activated networks are representable by an elementary activation tensor with respect to the graph

$$\mathrm{EL} = \left(\mathcal{V}, \{\{v\} \,:\, v \in \mathcal{V}\}\right)$$

and we denote such networks by $\Lambda^{t,\mathrm{EL}}$. Any CompActNet is representable with respect to the maximal hypergraph

$$\mathrm{MAX} = \left(\mathcal{V}, \{\mathcal{V}\}\right).$$

We therefore have for any graph that $\Lambda^{t,\mathcal{G}} \subset \Lambda^{t,\mathrm{MAX}}$.

Part I
Foundations of Artificial Intelligence

Many concepts of Artificial Intelligence can be captured by the formalism of tensors, tensor networks and their contractions. We dedicate this part to anchoring the formalism in the probabilistic and logical foundations of artificial intelligence.

Chapter 2
Propositional Logic

Abstract We present the representation and the reasoning mechansims of propositional logics.

2.1 Propositonal Formulas

We choose a semantic-central definition and define propositional formulas as maps from states to The epistemological commitments of propositional logics are whether the state is True or False reflected by coordinates 1 or 0. Intuitively these maps mark, whether a specific world can be the state of a factored system.

2.1.0.1 Formulas as maps

We now choose a semantic centric approach to propositional logic, by defining formulas as boolean tensors. Then we investigate the corresponding syntax of formulas as specification of a tensor network decomposition of the basis encoding of formulas.

Logics is especially useful in interpreting boolean tensors representing Propositional Knowledge Bases, based on connections with abstract human thinking. To make this more precise, we associate each such tensor with a formula f being a composition of the atomic variables with logical connectives as we proof next.

Definition 2.1 A propositional formula $f\left[X_{[d]}\right]$ depending on d atoms X_k is a boolean-valued tensor

$$f\left[X_{[d]}\right] : \bigtimes_{k\in[d]} [2] \to \{0,1\} \subset \mathbb{R}\,.$$

We call a state $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ a model of a propositional formula f, if

$$f\left[X_{[d]} = x_{[d]}\right] = 1\,.$$

If there is a model to a propositional formula, we say the formula is satisfiable.

The propositional formulas coincide therefore with the boolean tensors (see Def. **??**).

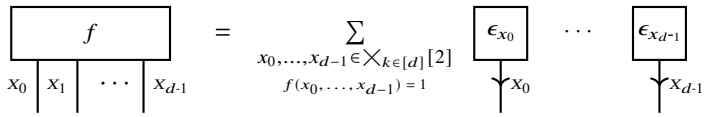
Since propositional formulas are binary valued tensors, the generic decomposition of Lem. **??** simplifies to

$$f\left[X_{[d]}\right] = \sum_{x_0,\ldots,x_{d-1}\in\bigtimes_{k\in[d]}[2]} f\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] \cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right] \tag{2.1}$$

$$= \sum_{x_0,\ldots,x_{d-1}\in\bigtimes_{k\in[d]}[2]\,:\,f[X_0=x_0,\ldots,X_{d-1}=x_{d-1}]=1} \epsilon_{x_0,\ldots,x_{d-1}}\left[X_{[d]}\right]\,. \tag{2.2}$$

Thus, any propositional formula is the sum over the one-hot encodings of its models. This is equal to the encoding of the set of models, which will be introduced in Chapter **??** (see Def. **??**).

We depict this decomposition in the diagrammatic notation by



We here chose a semantic approach to propositional logic in contrary to the standard syntactical approach. Instead of defining formulas by connectives acting on atomic formulas, we define them here as binary valued functions of the states of a factored system. They are interpreted by marking possible states as models, given the knowledge of f. The syntactical side will then be introduced later by studying decompositions of formulas.

2.1.1 Knowledge bases

Review the notation:

- $\mathcal{KB}$ as a set of formulas, i.e. a tensor network
- $\mathcal{KB}\left[X_{[d]}\right]$ as the contraction of the tensor network

Knowledge bases of propositional formulas are our first examples of tensor network decompositions.

Definition 2.2 A propositional knowledge base is a set $\mathcal{KB} = \{f_\ell\left[X_{e_\ell}\right] : \ell \in [p]\}$ of propositional formulas dependent on variables X_{e_ℓ}. The corresponding formula $\mathcal{KB}\left[X_\mathcal{V}\right]$ depending on $X_\mathcal{V} = \bigcup_{\ell\in[p]} X_{e_\ell}$ has coordinates to $x_\mathcal{V} \in \bigtimes_{v\in\mathcal{V}}[m_v]$

$$\mathcal{KB}\left[X_\mathcal{V} = x_\mathcal{V}\right] = \begin{cases} 1 & \text{if} \quad \forall \ell \in [p] : f_\ell\left[X_{e_\ell} = x_{e_\ell}\right] = 1 \\ 0 & \text{else}\,. \end{cases}$$

We will later investigate syntactical decompositions and denote

$$\mathcal{KB}\left[X_\mathcal{V}\right] = \bigvee_{\ell\in[p]} f_\ell\left[X_{e_\ell}\right]\,.$$

Theorem 2.1 (Tensor network representation of knowledge base) *Let there be a propositional knowledge base* $\mathcal{KB} = \{f_\ell : \ell \in [p]\}$. *Then the*

$$\mathcal{KB}\left[X_\mathcal{V}\right] = \left\langle f\left[X_{e_\ell}\right] : \ell \in [p]\right\rangle_{[X_\mathcal{V}]}\,.$$

Proof. We have for any $x_\mathcal{V}$ that

$$\mathcal{KB}\left[X_\mathcal{V} = x_\mathcal{V}\right] = \left\langle f\left[X_{e_\ell}\right] : \ell \in [p]\right\rangle_{[X_\mathcal{V}=x_\mathcal{V}]} = \prod_{\ell\in[p]} f\left[X_{e_\ell} = x_{e_\ell}\right]$$

where by x_{e_ℓ} we denote the reduction of the tuple $x_\mathcal{V}$ to the indices e_ℓ. It follows that $\mathcal{KB}\left[X_\mathcal{V} = x_\mathcal{V}\right] = 1$ if and only if $\forall \ell \in [p] : f_\ell\left[X_{e_\ell} = x_{e_\ell}\right] = 1$. □

2.2 Entailment

We here discuss entailment in propositinal logics based on contractions of formulas.

Entailment is the central consequence relation among logical formulas. Let us define this relation first based on the models of a knowledge base and a test formula.

Definition 2.3 (Entailment of propositional formulas) Given two propositional formulas $\mathcal{KB}$ and f we say that $\mathcal{KB}$ entails f, denoted by $\mathcal{KB} \models f$, if any model of $\mathcal{KB}$ is also a model of f, that is

$$\forall x_{[d]} \in \bigtimes_{k\in[d]} [2] : \left(\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1\right) \Rightarrow \left(f\left[X_{[d]} = x_{[d]}\right] = 1\right).$$

If $\mathcal{KB} \models \neg f$ holds, we say that $\mathcal{KB}$ contradicts f.

To use the tensor network formalism for the decision of entailment, we will in the following develop three equivalent criteria for entailment.

2.2.1 Deciding Entailment by Contractions

First of all, we can decide entailment based on vanishing contractions with the negated test formula.

Theorem 2.2 (Contraction Criterion of Entailment) *We have $\mathcal{KB} \models f$ if and only if*

$$\langle \mathcal{KB}, \neg f\rangle_{[\varnothing]} = 0\,.$$

Proof. "$\Leftarrow$": If for a $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have $\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1$ but not $\left(f\left[X_{[d]} = x_{[d]}\right] = 1\right)$, we would have $\left(\neg f\left[X_{[d]} = x_{[d]}\right] = 1\right)$ and

$$\langle \mathcal{KB}, \neg f\rangle_{[\varnothing]} = \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[2]} \mathcal{KB}\left[X_{[d]} = x_{[d]}\right] \cdot f\left[X_{[d]} = x_{[d]}\right] > 1\,.$$

Thus, whenever the contraction vanishes, we have

$$\forall x_{[d]} \in \bigtimes_{k\in[d]} [2] : \left(\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1\right) \Rightarrow \left(f\left[X_{[d]} = x_{[d]}\right] = 1\right).$$

"$\Rightarrow$": Conversely, if the contraction $\langle \mathcal{KB}, \neg f\rangle_{[\varnothing]}$ does not vanish, we would find $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ with $\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1$ and $\neg f\left[X_{[d]} = x_{[d]}\right] = 1$, therefore $f\left[X_{[d]} = x_{[d]}\right] = 0$. It follows that $\mathcal{KB} \models f$ does not hold. □

The contraction criterion can be extended to the decision of contradiction as well, since $\mathcal{KB} \models \neg f$ is equivalent to $\langle \mathcal{KB}, f\rangle_{[\varnothing]} = 0$. Therefore, entailment and contradiction can be decided simultaneously by a single contraction, as we state next.

Theorem 2.3 *Given propositional formulas $\mathcal{KB}$ and f we build*

$$\tau\left[Y_f\right] = \left\langle \mathcal{KB}\left[X_{[d]}\right], \beta^f\left[Y_f, X_{[d]}\right]\right\rangle_{[Y_f]}\,.$$

Then $\mathcal{KB} \models f$ is equivalent to $\tau\left[Y_f = 0\right] = 0$, and $\mathcal{KB} \models \neg f$ is equivalent to $\tau\left[Y_f = 1\right] = 0$.

Proof. This follows from Thm. 2.2 using that

$$\langle \mathcal{KB}, \neg f\rangle_{[\varnothing]} = \tau\left[Y_f = 0\right]$$

and

$$\langle \mathcal{KB}, f\rangle_{[\varnothing]} = \tau\left[Y_f = 1\right]\,.\square$$

2.2.2 Deciding Entailment by Partial Ordering

Logical entailment can be understood by subset relations of the models of the respective formulas. This perspective can be applied with subset encodings in Chapter **??**. The subset relation corresponds with partial ordering of its encoded

tensors, as will be shown in Thm. **??**. For two propositional formulas, we denote to this end $f \prec h$ (see Def. **??**), if and only if for all $x_{[d]} \in \bigtimes_{k\in[d]}[2]$

$$f\left[X_{[d]} = x_{[d]}\right] \leq h\left[x_{[d]}\right] .$$

Theorem 2.4 (Partial Ordering Criterion of Entailment) *We have $\mathcal{KB} \models f$ if and only if $\mathcal{KB}\left[X_{[d]}\right] \prec f\left[X_{[d]}\right]$.*

Proof. Since both $\mathcal{KB}$ and f are boolean tensors, we have for any $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ that

$$\mathcal{KB}\left[X_{[d]} = x_{[d]}\right], f\left[X_{[d]} = x_{[d]}\right] \in \{0,1\} .$$

Thus,

$$\forall x_{[d]} \in \bigtimes_{k\in[d]}[2] : \mathcal{KB}\left[X_{[d]} = x_{[d]}\right] \leq f\left[X_{[d]} = x_{[d]}\right]$$

is equivalent to

$$\forall x_{[d]} \in \bigtimes_{k\in[d]}[2] : \left(\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1\right) \Rightarrow \left(f\left[X_{[d]} = x_{[d]}\right] = 1\right) .$$

This states that $\mathcal{KB}\left[X_{[d]}\right] \prec f\left[X_{[d]}\right]$ is equivalent to $\mathcal{KB} \models f$. □

2.2.3 Redundancy of Entailed Formulas

Another interpretation of entailment is by redundancy of a formula in a Knowledge Base. This is especially interesting for the sparse representation of Knowledge Bases.

Theorem 2.5 (Redundancy Criterion of Entailment) *If and only if $\mathcal{KB} \models f$ we have*

$$\mathcal{KB}\left[X_{[d]}\right] = \langle \mathcal{KB}, f\rangle_{\left[X_{[d]}\right]} .$$

Proof. For any formula f we have

$$\mathbb{I}\left[X_{[d]}\right] = f\left[X_{[d]}\right] + \neg f\left[X_{[d]}\right]$$

and thus

$$\begin{aligned}\mathcal{KB}\left[X_{[d]}\right] &= \left\langle \mathcal{KB}\left[X_{[d]}\right], \mathbb{I}\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} \\ &= \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} + \left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} .\end{aligned}$$

Now, by Thm. 2.2 we have $\mathcal{KB} \models f$, if and only if $\left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = 0$, which is thus equal to

$$\mathcal{KB}\left[X_{[d]}\right] = \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} .\square$$

2.2.4 Contraction Knowledge Base

We exploit the contraction and redundancy criteria of entailment to sketch an implementation of a propositional Knowledge Base in Algorithm 1. Here the function ASK(f) returns, whether a formula f is entailed or contradicted by a Knowledge Base. If the formula is neither entailed or contradicted, we say it is contingent. If it is both, we have $\mathcal{KB}\left[X_{[d]}\right] = 0$ and thus an inconsistent Knowledge Base. Exploiting Thm. 2.3 we decide these situations based on a single contraction.

The function TELL(f) incorporates an additional formula f into a Knowledge Base $\mathcal{KB}$. Here we exploit Thm. 2.5 and do not add a formula, which is entailed in order to maintain a sparse representation. The function further refuses to add a formula, which would make the Knowledge Base inconsistent (returns Refused) and only changes the Knowledge Base in case of a contingent formula (returns Added).

		$\mathcal{KB} \models f$: False	$\mathcal{KB} \models f$: True
$\mathcal{KB} \models \neg f$	False	*"Contingent"*	*"Entailed"*
	True	*"Contracticted"*	*"Inconsistent"*

Fig. 2.1 Table of possible logical relations between a knowledge base $\mathcal{KB}$ and f, based on whether the knowledge base entails the formula ($\mathcal{KB} \models f$) and its negation ($\mathcal{KB} \models \neg f$).

Algorithm 1 Contraction Knowledge Base with operations ASK and TELL

ASK($\mathcal{KB}$, f)

Require: Knowledge base $\mathcal{KB}$, query formula f
Ensure: Decision which relation between $\mathcal{KB}$ and f holds (see Figure 2.1)

```
τ[Y_f] ← ⟨{h[X_[d]] : h ∈ KB}, β^f[Y_f, X_[d]]⟩_[Y_f]
if τ[Y_f = 0] = 0 and τ[Y_f = 1] = 0 then
    return "Inconsistent"
else if τ[Y_f = 0] = 0 then
    return "Entailed"
else if τ[Y_f = 1] = 0 then
    return "Contradicted"
else
    return "Contingent"
end if
```

TELL($\mathcal{KB}$, f)

Require: Knowledge base $\mathcal{KB}$, query formula f
Ensure: Decision whether a formula is added to the knowledge base (*"Added"*), or an exception in (*"Inconsistent"*, *"Redundant"* or *"Refused"*) is raised.

```
answer ← ASK(f)
if answer is "Inconsistent": then
    return "Inconsistent"
else if answer is "Entailed": then
    return "Redundant"
else if answer is "Contradicted": then
    return "Refused"
else if answer is "Contingent": then
    KB ← KB ∪ {f}
    return "Added"
end if
```

2.3 Syntactical decompositions

When syntax is available for formulas, we find tensor network decompositions. This is a first example of basis calculus.

2.3.1 Encoding of Booleans

Before investigating schemes for syntactical decompositions, we revisit the encoding of Booleans. Boolean variables are a basic data structure, valued in $\{\text{False}, \text{True}\}$.

2.3.1.1 Representation by Coordinates

To represent booleans by categorical variables X with two states we use the index interpretation function (see Def. **??** and for more details Def. **??** in Part **??**)

$$I \,:\, [2] \to \{\text{False}, \text{True}\}$$

defined as

$$I(1) = \text{True} \quad \text{and} \quad I(0) = \text{False}\,.$$

One motivation for this particular choice of the interpretation function I is the effective execution of the conjunction as we show in the next Lemma.

Lemma 2.1 *I is a homomorphism between the groups*

$$\big(\{0,1\}, \cdot\big) \quad \textit{and} \quad \big(\{\text{False}, \text{True}\}, \wedge\big)\,.$$

Proof. It suffices to notice, that for arbitrary $z_0, z_1 \in \{0,1\}$ we have

$$I(z_0 \cdot z_1) = I(z_0) \wedge I(z_1)\,.\square$$

Based on this homomorphism, contractions of boolean tensors, in which all variables are kept open, can be regarded as parallel calculations of the conjunction $\wedge$ encoded by I. This homomorphism is further applied in type conversion in dynamically-typed languages (e.g. in python [**?**]).

Operations like the negation fail to be linear and are only affine linear, since for $z \in \{\text{False}, \text{True}\}$ we have

$$I^{-1}(\neg z) = 1 - I^{-1}(z)\,. \tag{2.3}$$

Since any logical connective can be represented as a composition of conjunctions and negations, any logical connective corresponds with an affine linear function on the interpreted truth values. Direct applications of this insight to execute logical calculus will be discussed later in Sect. **??**. For our purposes here, we would like to execute logical connective based on single contractions and avoid summations over them. This is why we call the negation representation as in (2.3) the affine representation problem, which we aim to resolve in the following.

While in this work, we will always encode boolean states by I, other index interpretation functions could be chosen. For example, the interpretation

$$I_\vee \,:\, \{0,1\} \to \{\text{False}, \text{True}\}$$

defined as

$$I_\vee(0) = \text{True} \quad \text{and} \quad I_\vee(1) = \text{False}$$

results is a homomorphism between the groups

$$\big(\{0,1\}, \cdot\big) \quad \text{and} \quad \big(\{\text{False}, \text{True}\}, \vee\big)\,.$$

While placing the disjunction $\vee$ as the logical connective effectively executed by contractions, the negation will for arbitrary interpretations mapping onto $\{0,1\}$ remain the function

$$I_\vee^{-1}(\neg z) = 1 - I_\vee^{-1}(z)\,.$$

Thus, the problem of affine linear operations cannot be resolved by a clever choice of an interpretation function with image in $\{0, 1\}$.

2.3.1.2 Representation by Basis Vectors

While contractions can just perform conjunctions, we need a representation trick to extend the contraction expressivity to arbitrary connectives and resolve the affine representation problem. To this end we now compose I with the one-hot encoding ϵ and get an encoding

$$\epsilon \circ I^{-1} : \{\text{False}, \text{True}\} \to \{\epsilon_0 [X], \epsilon_1 [X]\},$$

where X is a categorical variable with $m = 2$. For any $z \in \{\text{False}, \text{True}\}$ we have

$$\epsilon \circ I^{-1}(z) = \begin{bmatrix} I^{-1}(\neg z) \\ I^{-1}(z) \end{bmatrix}.$$

Performing the negation now amounts to switching the coordinates of the encoded vector, which can be performed by contraction with a transposition matrix

$$\beta^{\neg} [Y_{\neg}, X] = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} [Y_{\neg}, X] ,$$

where in this notation we always understand the first variable X as the row index selector and the second variable $Y_{\neg}$ as the column index selector. We then have

$$\epsilon \circ I^{-1}(\neg z)[Y_{\neg}] = \left\langle \beta^{\neg} [Y_{\neg}, X], \epsilon \circ I^{-1}(z)[X] \right\rangle_{[Y_{\neg}]} .$$

We therefore arrived at our aim to resolve the affine representation problem and have found a procedure to represent logical negations by a contraction, which is a linear operation. Besides negations, we will show in this chapter, that arbitrary logical formulas can be represented by contractions.

2.3.1.3 Coordinate and Basis Calculus

Our findings on the encoding of booleans hint towards more general schemes to encode information into boolean tensors, which will be explored in more detail in Chapter **??** and Chapter **??**. When each coordinate in a boolean tensor represents one in $\{0, 1\}$ interpreted boolean we call the scheme coordinate calculus. In basis calculus on the other hand, booleans are represented by elements of $\{\epsilon_0 [X], \epsilon_1 [X]\}$. In that scheme, there are pairs of two coordinates (building slice vectors of the tensors), which are restricted to be different from each other. This amounts to posing a global directionality constraint on the boolean tensor, as will be shown in Thm. **??**.

2.3.2 Basis encodings

Add entailment by basis calculus diagram.

2.3.2.1 Definition

Definition 2.4

2.3.2.2 Formula representation by Computation-Activation Networks

We apply the coordinate and the basis encoding schemes to represent propositional formulas as tensors. In the coordinate encoding scheme, which we use by default, we understand [2] as subset of $\mathbb{R}$ and interpreting the formula directly as a tensor (as in Def. 2.1). To apply the basis encoding scheme, we understand [2] as the states of a system with factored representation by a single categorical variable. Although it comes with an additional boolean variable compared with coordinate encoding scheme, we in this work use the basis encoding scheme to find sparse tensor network decompositions based on syntactical formula decompositions. Furthermore, the basis encoding connects with the formalism of Computation-Activation Networks developed in Chapter **??**, as we show next.

Theorem 2.6 (Formula representation as Computation-Activation Network) *For each formula f we have*

$$f\left[X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_1\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]}$$

and

$$\neg f\left[X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_0\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]} ,$$

which are Computation-Activation Networks to the statistic $\{f\}$ and the hard activation tensor $\epsilon_1\left[Y_f\right]$, respectively $\epsilon_0\left[Y_f\right]$.

Proof. Given a factored system with d atoms $X_{[d]}$ and a propositional formula f, the basis encoding of f (see Def. 1.6 and Figure 2.2) is the tensor

$$\beta^f\left[Y_f, X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[2]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \epsilon_{f\left[X_{[d]}=x_{[d]}\right]}\left[Y_f\right] .$$

We reorder the summation into

$$\begin{aligned}\beta^f\left[Y_f, X_{[d]}\right] =& \epsilon_0\left[Y_f\right] \otimes \left(\sum_{x_{[d]}\,:\,f\left[x_{[d]}\right]=0} \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right) \\ &+ \epsilon_1\left[Y_f\right] \otimes \left(\sum_{x_{[d]}\,:\,f\left[x_{[d]}\right]=1} \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right)\end{aligned}$$

where the second term sums up the models of f and the first one the models of $\neg f$. Using linearity of contractions we get

$$\begin{aligned}\left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_1\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]} &= \left\langle \epsilon_0\left[Y_f\right], \epsilon_1\left[Y_f\right]\right\rangle_{[\varnothing]} \cdot \left(\sum_{x_{[d]}\,:\,f\left[x_{[d]}\right]=0} \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right) \\ &\quad + \left\langle \epsilon_1\left[Y_f\right], \epsilon_1\left[Y_f\right]\right\rangle_{[\varnothing]} \cdot \left(\sum_{x_{[d]}\,:\,f\left[x_{[d]}\right]=1} \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right) \\ = \sum_{x_{[d]}\,:\,f\left[x_{[d]}\right]=1} \epsilon_{x_{[d]}}\left[X_{[d]}\right] &= f\left[X_{[d]}\right] .\end{aligned}$$

Analogously we have

$$\left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_0\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]} = \sum_{x_{[d]}\,:\,f\left[x_{[d]}\right]=0} \epsilon_{x_{[d]}}\left[X_{[d]}\right] = \neg f\left[X_{[d]}\right] .\square$$

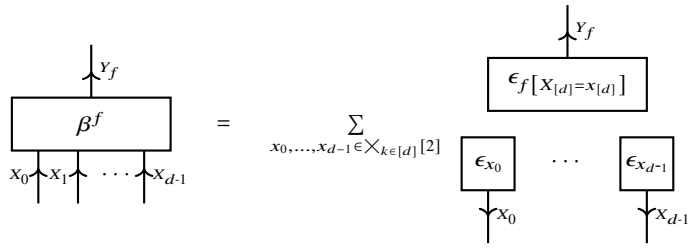


Fig. 2.2 Basis encoding of a propositional formula. The encoding is a sum of the one hot encodings of all states of the factored system in a tensor product with basis vectors, which encode whether the state is a model of the formula. The tensor is directed, since any contraction with an encoded state results in the basis vector evaluating the formula, which we called basis calculus.

Thm. 2.6 shows that besides the formula itself, we can represent the negation of the formula as a Computation-Activation Network with the same statistic $\{f\}$. In general, any boolean tensor which has a representation as a Computation-Activation Network needs to have a boolean activation tensor. In the case of the single-dimensional statistic $\{f\}$ we have three boolean activation vectors $\{\mathbb{I}\left[Y_f\right], \epsilon_1\left[Y_f\right], \epsilon_0\left[Y_f\right], 0\left[Y_f\right]\}$. While the Computation-Activation Networks with the activation cores $\epsilon_1\left[Y_f\right], \epsilon_0\left[Y_f\right]$ are by Thm. 2.6 the formula and its negation, the activation core $\mathbb{I}\left[Y_f\right]$ and $0\left[Y_f\right]$ represent the trivial $\mathbb{I}\left[X_{[d]}\right]$ and the vanishing tensor $0\left[X_{[d]}\right]$.

The contraction of the basis encoding with one-hot encodings to worlds $x_{[d]} \in \times_{k\in[d]}[2]$ is

$$\left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right\rangle_{[Y_f]} = \begin{cases} \epsilon_1\left[Y_f\right] & \text{if the world } x_{[d]} \text{ is a model of } f \\ \epsilon_0 Y_f & \text{else}\,. \end{cases}$$

We thus interpret the contractions as calculating in basis vectors, whether the world $x_{[d]}$ is a model of the formula f. This is the basis calculus scheme and will be investigated it in more detail in Chapter **??** (see Thm. **??**).

2.3.3

2.3.4 Circuit decomposition

Syntax provides a description of a formula as a circuit of connective formulas.

Basis encodings of propositional formulas are especially useful when representing function compositions by the representation of their components (see Thm. **??**). In propositional logics, the syntax of defining propositional formulas is oriented on compositions of formulas by connectives. In this section, we investigate the decomposition schemes of basis encodings into tensor networks of component encodings for binary tensors following propositional logic syntax.

2.3.5 Atomic Formulas

We call atomic formulas the most granular formulas, which are not split into compositions of other formulas. Our syntactic decomposition of propositional formulas will then investigate, how any propositional formula can be represented by these.

Definition 2.5 The tensors $f_k\left[X_{[d]}\right]$ defined for $x_{[d]} \in \times_{k\in[d]}[2]$ as

$$f_k\left[X_{[d]} = x_{[d]}\right] = x_k$$

are called atomic formulas.

Atomic formulas and their basis encodings have an especially compelling representation.

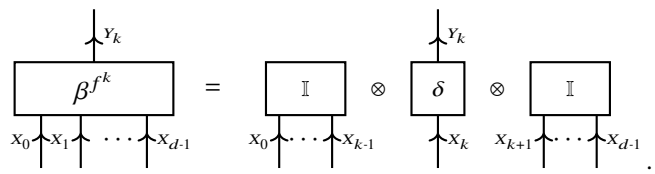
Lemma 2.2 *Any atomic formula* $f^k\left[X_{[d]}\right]$ *is represented as*

$$f^k\left[X_{[d]} = x_{[d]}\right] = \langle \epsilon_1\left[X_k\right]\rangle_{\left[X_{[d]}\right]} = \epsilon_1\left[X_k\right] \otimes \mathbb{I}\left[X_{[d]/\{k\}}\right] .$$

and has a basis encoding

$$\beta^{f^k}\left[Y_d, X_{[d]}\right] = \langle \delta\left[Y_k, X_k\right]\rangle_{\left[X_{[d]}\right]} = \delta\left[Y_k, X_k\right] \otimes \mathbb{I}\left[X_{[d]/\{k\}}\right] .$$

The decomposition is depicted in a network diagram as



Proof. We have by definition

$$\begin{aligned}\beta^{f^k}\left[Y_k, X_{[d]}\right] &= \sum_{x_0,\ldots,x_{d-1}\in\bigtimes_{k\in[d]}[2]} \epsilon_{x_0,\ldots,x_{d-1}}\left[X_{[d]}\right] \otimes \epsilon_{f_k[X_0=x_0,\ldots,X_{d-1}=x_{d-1}]}\left[Y_k\right] \\ &= \left(\epsilon_{0,0}\left[X_k, Y_k\right] + \epsilon_{1,1}\left[X_k, Y_k\right]\right) \otimes \mathbb{I}\left[X_l \,:\, l \neq k\right] \\ &= \langle \delta\left[X_k, Y_k\right]\rangle_{\left[X_{[d]}, Y_k\right]} .\square\end{aligned}$$

2.3.6 Syntactical Composition of Formulas

Propositional formulas are elements of tensor spaces with d axis. The number of coordinates thus grows exponentially with the number of atoms, which is

$$\dim\left(\bigotimes_{k\in[d]} \mathbb{R}^2\right) = 2^d .$$

When the number of atoms is large, the naive representation of formula tensors will be thus intractable. In contrast, typical logical formulas appearing in practical knowledge bases are sparse in the sense that they have short representations in a logical syntax. In logical syntax formulas are described by atomic formulas recursively combined via connectives, which are itself formulas of typically low order d. We now exploit such syntactical decompositions of formulas to derive tensor network decompositions.

Let there be formulas f and h depending on categorical variables $X_{[d]}$ and a binary connective

$$\circ \,:\, [2] \times [2] \to [2] .$$

Then we have (see Figure 2.3b)

$$\beta^{f\circ h}\left[X_{[d]}, X_{f\circ h}\right] = \left\langle \beta^{\circ}\left[Y_f, Y_h, X_{f\circ h}\right], \beta^{f}\left[X_{[d]}, Y_f\right], \beta^{h}\left[X_{[d]}, Y_h\right]\right\rangle_{\left[X_{[d]}, X_{f\circ h}\right]} .$$

For any unary connective $\circ : [2] \to [2]$ we analogously have (see Figure 2.3a)

$$\beta^{\circ f}\left[Y_{\circ f}, X_{[d]}\right] = \left\langle \beta^{\circ}\left[Y_{\circ f}, Y_f\right], \beta^{f}\left[Y_f, X_{[d]}\right]\right\rangle_{\left[Y_{\circ f}, X_{[d]}\right]} .$$

Lemma 2.3 *Let there be a* r*-ary connective*

$$\circ \,:\, \bigtimes_{l\in[r]} [2] \to [2]$$

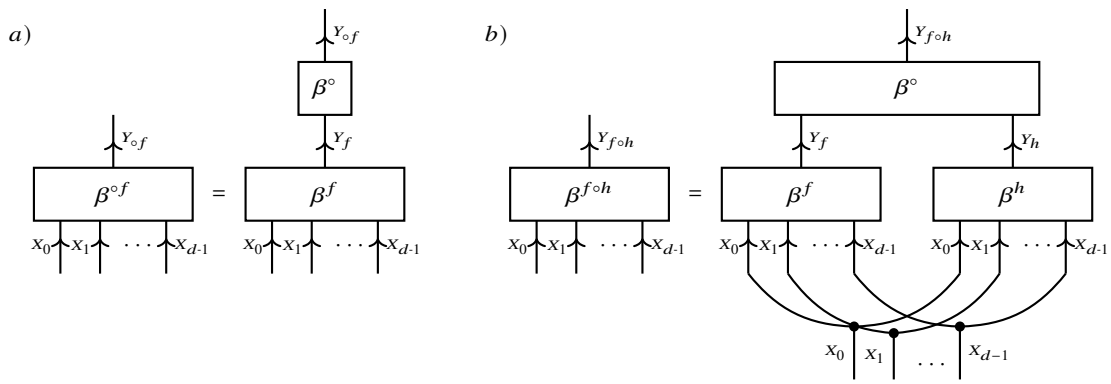


Fig. 2.3 Representation of syntactical compositions of formulas by basis encodings. a) Unary connective $\circ$ acting on a formula f. b) Binary connective $\circ$ composing formulas f, h. In more generality, the basis encoding of arbitrary syntactical compositions is a contraction of the basis encoding of the components.

and for each $l \in [r]$ a formula $f_l\left[X_{[d]}\right]$. For the composition formula $f\left[X_{[d]}\right]$ defined by

$$f\left[X_{[d]} = x_{[d]}\right] = \circ\left(f_0\left[X_{[d]} = x_{[d]}\right], \ldots, f_{r-1}\left[X_{[d]} = x_{[d]}\right]\right)$$

we have

$$\beta^{f\left[X_{[d]}\right]}\left[Y_f, X_{[d]}\right] = \left\langle \{\beta^{\circ}\left[Y_{f\left[X_{[d]}\right]}, Y_f, Y_{[r]}\right]\} \cup \{\beta^{f_l}\left[Y_l, X_{[d]}\right] \,:\, l \in [r]\}\right\rangle_{\left[Y_{f\left[X_{[d]}\right]}, X_{[d]}\right]} .$$

Proof. Here we used the main property of basis encodings, that basis encodings of composition are equal to contracted basis encodings of the components. We will show this property in more generality in Thm. **??**. □

These properties motivate a hypergraph-based definition of syntactical compositions of a propositional formula.

Definition 2.6 A syntactical circuit is a directed acyclic hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ (see Def. **??**) such that

- each hyperedge $e = (e^{\text{in}}, e^{\text{out}})$ has exactly one outgoing node, i.e. $\left|e^{\text{out}}\right| = 1$
- each node $v \in \mathcal{V}$ carries a boolean variable Y_v and appears at most once as the outgoing node of a hyperedge
- each hyperedge $(e^{\text{in}}, \{v\})$ with $e^{\text{in}} \neq \emptyset$ is decorated by a logical connective

$$\circ_v\left[Y_{e^{\text{in}}}\right] : \bigtimes_{v \in e^{\text{in}}} [2] \to [2]$$

- the node not appearing as an outgoing node are labeled by $[d]$

We say that the syntactical circuit is single-rooted, if exactly one node $\tilde{v}$ does not appear as an incoming node of a hyperedge. In this case this unique node is called the root node. We assign atomic formulas to the nodes $[d]$ and recursively assign to each further node v a node formula

$$f_v\left[X_{[d]} = x_{[d]}\right] = \circ_v\left[\left[f_{\bar{v}}\left[X_{[d]} = x_{[d]}\right] \,:\, \bar{v} \in e^{\text{in}}\right]\right] \quad \forall x_{[d]} \in \bigtimes_{k \in [d]} [m_k] ,$$

where e^{in} are the incoming nodes in the unique hyperedge with outgoing nodes $\{v\}$. We call the formula $f\left[X_{[d]}\right] := f_{\tilde{v}}\left[X_{[d]}\right]$ to the root note $\tilde{v}$ the syntactical composition of $\mathcal{G}$ and $\mathcal{G}$ is a syntactical decomposition of f.

We are now prepared to show the main result of this chapter, the representation of formulas as contracted basis encoding of their decomposition.

Theorem 2.7 *For any syntactical circuit $\mathcal{G}$ with composition f we have*

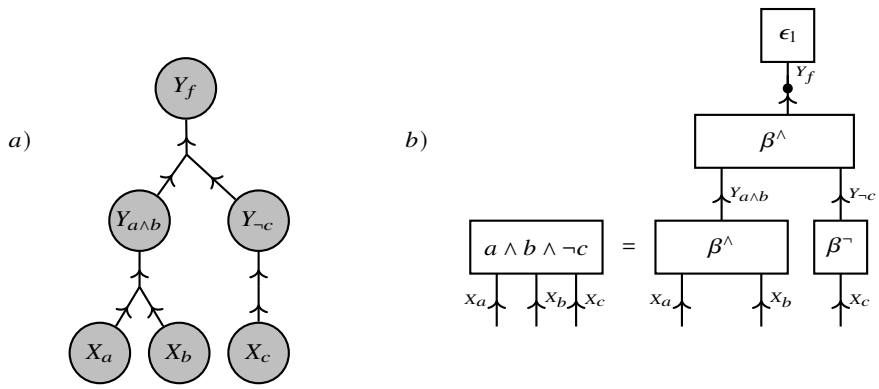


Fig. 2.4 Decomposition of the formula tensor to $f = a \wedge b \wedge \neg c$ into unary (matrix) and binary (third order tensor) cores. a) Syntactical decomposition of f as a hypergraph $\mathcal{G}$ according to Def. 2.6. b) Tensor network of basis encodings of the connectives decorating the hyperedges, which represents by Thm. 2.7 the formula f.

$$f\left[X_{[d]}\right] = \Big\langle \left\{\beta^{\circ_v}\left[Y_v, Y_{e^{\text{in}}}\right] \,:\, (e^{\text{in}}, \{v\}) \in \mathcal{E}\right\} \cup \\ \{\delta\left[Y_k, X_k\right] \,:\, k \in [d]\} \cup \{\epsilon_1\left[Y_{\bar{v}}\right]\}\Big\rangle_{\left[X_{[d]}\right]} .$$

Proof. We show inductively over the nodes, that

$$\beta^{f_v}\left[Y_v, X_{[d]}\right] = \Big\langle \left\{\beta^{\circ_{\bar{v}}}\left[Y_{\bar{v}}, Y_{e^{\text{in}}}\right] \,:\, \bar{v} \prec v,\ (e^{\text{in}}, \{\bar{v}\}) \in \mathcal{E}\right\} \cup \{\beta^{\circ_v}\left[Y_v, Y_{e^{\text{in}}}\right]\} \cup \tag{2.4}$$

$$\{\delta\left[Y_k, X_k\right] \,:\, k \in [d]\}\Big\rangle_{\left[X_{[d]}\right]} . \tag{2.5}$$

By $\prec$ we denote the partial ordering of nodes by directed paths in $\mathcal{G}$, i.e. $\bar{v} \prec v$ holds if there is a directed path from $\bar{v}$ to v in the hypergraph $\mathcal{G}$. We start the induction at the nodes $[d]$ not appearing as outgoing nodes, where (2.4) holds by Lem. 2.2.

If for a node $v \in \mathcal{V}$ (2.4) holds at all $\bar{v} \in e^{\text{in}}$, we apply Lem. 2.3 to the connective $\circ_v$ and the formulas $f_{\bar{v}}\left[X_{[d]}\right]$ of the parents $\bar{v} \in e^{\text{in}}$ and get

$$\begin{aligned}\beta^{f_v}\left[Y_v, X_{[d]}\right] &= \Big\langle \{\beta^{\circ_v}\left[Y_{f_v}, Y_f, Y_{[r]}\right]\} \cup \{\beta^{f_l}\left[Y_l, X_{[d]}\right] \,:\, l \in [r]\}\Big\rangle_{\left[Y_f\left[X_{[d]}\right], X_{[d]}\right]} \\ &= \Big\langle \left\{\beta^{\circ_{\bar{v}}}\left[Y_{\bar{v}}, Y_{e^{\text{in}}}\right] \,:\, \bar{v} \prec v,\ (e^{\text{in}}, \{\bar{v}\}) \in \mathcal{E}\right\} \cup \{\beta^{\circ_v}\left[Y_v, Y_{e^{\text{in}}}\right]\} \cup \\ &\qquad \{\delta\left[Y_k, X_k\right] \,:\, k \in [d]\}\Big\rangle_{\left[X_{[d]}\right]} .\end{aligned}$$

Thus (2.4) also holds for v. Since the hypergraph is acyclic the induction reached all nodes and (2.4) holds in particular for the root node $\tilde{v}$. The claim then follows with Thm. 2.6 applied on $f = f_{\tilde{v}}$. □

An example of a syntactical decomposition is provided in Figure 2.4a, and the corresponding tensor network representation in Figure 2.4b. Examples of unary ($d = 1$) and binary ($d = 2$) connectives are further provided in Example 2.1.

Example 2.1 We use the following connectives:

- negation $\neg : [2] \to [2]$ by the vector

$$\neg[Y_f] = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- conjunctions $\wedge : [2] \times [2] \to [2]$

$$\wedge[Y_f, Y_h] = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

- disjunctions $\vee : [2] \times [2] \to [2]$

$$\vee[Y_f, Y_h] = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

- exact disjunction $\oplus : [2] \times [2] \to [2]$

$$\oplus[Y_f, Y_h] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- implications $\Rightarrow: [2] \times [2] \to [2]$

$$\Rightarrow [Y_f, Y_h] = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

- biimplication $\Leftrightarrow: [2] \times [2] \to [2]$

$$\Leftrightarrow [Y_f, Y_h] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Remark 2.1 (d-ary connectives such as $\wedge$ and $\vee$) Since the decomposition of basis encoding can be applied to generic function compositions (see Thm. **??**), we can also allow for d-ary connectives

$$\circ : \bigtimes_{k \in [d]} [2] \to [2] \,.$$

The connectives $\wedge$ and $\vee$ satisfy associativity and have thus straightforward generalizations to the d-ary case. This is because associativity can be exploited to represent the basis encoding by any tree-structured composition of binary $\wedge$ and $\vee$ connectives.

2.3.7 Minterm and maxterm decompositions

We have seen how syntactical decompositions of formulas into connectives acting can be exploited to find effective representations by tensor networks. We now provide a generic scheme to find syntactical decompositions to arbitrary propositional formulas.

Definition 2.7 (Terms and Clauses) Given a subset $A \subset [d]$ and $y_A \in \bigtimes_{k \in A} [2]$, the corresponding term is the formula

$$Z_A^{\wedge}\left[X_{[d]}\right] = \left(\bigwedge_{k \in A \,:\, y_k = 0} \neg f^k \left[X_k\right] \right) \wedge \left(\bigwedge_{k \in A \,:\, y_k = 1} f^k \left[X_k\right] \right)$$

and the corresponding clause is the formula

$$Z_A^{\vee}\left[X_{[d]}\right] = \left(\bigvee_{k \in A \,:\, y_k = 0} f^k \left[X_k\right] \right) \vee \left(\bigvee_{k \in A \,:\, y_k = 1} \neg f^k \left[X_k\right] \right)$$

$$Z_{\mathcal{V}^0}^{\vee}\left[\mathcal{V}^1\right] X_{[d]} = \left(\bigvee_{k \in \mathcal{V}^0} f^k \right) \vee \left(\bigvee_{k \in \mathcal{V}^1} \neg f^k \right) ,$$

We call the term a minterm and the clause a maxterm, if $A = [d]$.

Use the (A, y_A) parametrization instead of the nodes 0 1 parametrization in the following.
Terms and Clauses have for any index tuple $x_{[d]}$ a polynomial representation by

$$Z^{\wedge}_{\mathcal{V}^0}\mathcal{V}^1[X_{[d]} = x_{[d]}] = \left(\prod_{k\in\mathcal{V}^0}(1-x_k)\right)\left(\prod_{k\in\mathcal{V}^1}x_k\right)$$

and

$$Z^{\vee}_{\mathcal{V}^0}\mathcal{V}^1[X_{[d]} = x_{[d]}] = 1-\left(\prod_{k\in\mathcal{V}^0}(1-x_k)\right)\left(\prod_{k\in\mathcal{V}^1}x_k\right).$$

Lemma 2.4 *Terms are contractions of one-hot encodings, that is for any disjoint subsets $\mathcal{V}^0, \mathcal{V}^1 \subset [d]$ we have*

$$Z^{\wedge}_{\mathcal{V}^0}\mathcal{V}^1[X_{[d]}] = \left\langle \epsilon_{\{x_k=0:k\in\mathcal{V}^0\}\cup\{x_k=1:k\in\mathcal{V}^1\}}\right\rangle_{[X_{[d]}]}.$$

Clauses are subtractions of one-hot encodings from the trivial tensor, that is for any disjoint subsets $\mathcal{V}^0, \mathcal{V}^1 \subset [d]$ we have

$$Z^{\vee}_{\mathcal{V}^0}\mathcal{V}^1[X_{[d]}] = \mathbb{I}\left[X_{[d]}\right] - \left\langle \epsilon_{\{x_k=0:k\in\mathcal{V}^0\}\cup\{x_k=1:k\in\mathcal{V}^1\}}\right\rangle_{[X_{[d]}]}.$$

The reference of the formulas in the case $\mathcal{V}^0\dot\cup\mathcal{V}^1 = [d]$ as minterms and maxterms is due to the fact, that minterms are formulas with unique models and maxterms are formulas with a unique world not satisfying the formula. We use this insight and enumerate maxterms and minterms by the index $x \in \bigtimes_{k\in[d]}[2]$ of the unique world where the minterm is satisfied, respectively the maxterm is not satisfied. For any $\mathcal{V}^0\dot\cup\mathcal{V}^1 = [d]$ we take the index tuple $x_0, \ldots, x_{d-1}$ where $x_k = 0$ if $k \in \mathcal{V}^0$ and $x_k = 1$ if $k \in \mathcal{V}^1$ and define

$$Z^{\vee}_{x_0,\ldots,x_{d-1}} = Z^{\vee}_{\mathcal{V}^0}\mathcal{V}^1 \quad \text{and} \quad Z^{\wedge}_{x_0,\ldots,x_{d-1}} = Z^{\wedge}_{\mathcal{V}^0}\mathcal{V}^1.$$

Corollary 2.1 *Minterms are basis elements of the tensor space, that is for any $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have*

$$Z^{\wedge}_{x_{[d]}} = \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

Maxterms are subtraction of basis elements from the trivial tensor, that is for any $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have

$$Z^{\vee}_{x_{[d]}} = \mathbb{I}\left[X_{[d]}\right] - \epsilon_{x_{[d]}}\left[X_{[d]}\right].$$

Proof. Follows from Lem. 2.4, since when $\mathcal{V}^0 \cup \mathcal{V}^1 = [d]$ the contraction of the one-hot encodings coincides with the one-hot encoding of a fully specified state. □

Based on this insight, we can decompose any propositional formula into a conjunction of maxterms or a disjunction of minterms as we show next.

Theorem 2.8 *For any boolean tensor $\tau\left[X_{[d]}\right] \in \bigotimes_{k\in[d]}\mathbb{R}^2$ we have*

$$\tau\left[X_{[d]}\right] = \left(\bigvee_{x_{[d]}:\,\tau\left[X_{[d]}=x_{[d]}\right]=1} Z^{\wedge}_{\{k\,:\,x_k=0\}}\{k\,:\,x_k=1\}\right)[X_{[d]}]$$

and

$$\tau\left[X_{[d]}\right] = \left(\bigwedge_{x_{[d]}:\,\tau\left[X_{[d]}=x_{[d]}\right]=0} Z^{\vee}_{\{k\,:\,x_k=0\}}\{k\,:\,x_k=1\}\right)[X_{[d]}].$$

Proof. To show the representation by minterms we use the decomposition

$$\tau\left[X_{[d]}\right] = \sum_{x_{[d]}:\,\tau\left[X_{[d]}=x_{[d]}\right]=1}\epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

and notice that each term in the disjunction modifies the formula by adding respective world $x_{[d]}$ to the models of the formula. To show the representation by maxterms we use the decomposition

$$\tau\left[X_{[d]}\right] = \mathbb{I}\left[X_{[d]}\right] \quad - \sum_{x_{[d]}\,:\,\tau\left[X_{[d]}=x_{[d]}\right]=0} \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

and notice that each term in the conjunction modifies the formula by removing the respective world $x_{[d]}$ from the models of the formula. Thus, both decompositions are propositional formulas with the same set of models as the formula τ and are thus identical to τ. □

The decompositions found in Thm. 2.8 are also called canonical normal forms to propositional formulas $\tau\left[X_{[d]}\right]$.

Remark 2.2 (Efficient Representation in Propositional Syntax) The decomposition in Thm. 2.8 is a basis CP decomposition of the binary tensor and will further be investigated in Chapter **??**. The formulas constructed in the proof of Thm. 2.8 are however just one possibility to represent a formula tensor in propositional syntax. Typically there are much sparser representations for many formula tensors, in the sense that less connectives and atomic symbols are required. Having such a sparser syntactical description of a propositional formula can be exploited to find a shorter conjunctive normal form of the formula and construct a sparse polynomial based on similar ideas as in Thm. 2.8. We will provide such constructions in Chapter **??**, where we show that dropping the demand of directionality and investigating binary CP Decompositions will improve the sparsity of the polynomial formula representation.

2.3.8 Outlook

While we in this chapter investigated representation schemes for single propositional formulas, we will further study the representation of knowledge bases consisting in multiple formulas in Sect. **??**. Further, we will build hybrid models bridging the concepts of probability distributions and propositional logics in Sect. **??**. Propositional formulas will therein serve as features and base measures for exponential families.

2.4 Constraint Satisfaction Problems

Let us now explore a more general class of logical inference problems and discuss probabilistic entailment within that class. We then provide further examples based on categorical constraints. Following Chapter 5 in [**?**], we now define Constraint Satisfaction Problems.

Definition 2.8 Let there be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and $\tau^{\mathcal{G}}$ be a tensor network of boolean constraint tensors $\tau^e\left[X_e\right]$ to each $e \in \mathcal{E}$, that is

$$\tau^{\mathcal{G}} = \left\{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\right\}.$$

We then call $\tau^{\mathcal{G}}$ a *constraint network* and the tensor

$$\tau^{\mathcal{G}}\left[X_{\mathcal{V}}\right] = \left\langle \tau^{\mathcal{G}} \right\rangle_{\left[X_{\mathcal{V}}\right]}$$

its contraction. The *constraint satisfaction problem (CSP)* to the constraint network $\tau^{\mathcal{G}}$ is the decision whether there is a state $x_{\mathcal{V}}$ such that

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{\left[X_{\mathcal{V}}=x_{\mathcal{V}}\right]} = 1\,.$$

We say the CSP is satisfiable, when there is such a state, and unsatisfiable if not.

Example 2.2 Knowledge base satisfaction as a constraint satisfaction problem Given a knowledge base $\mathcal{KB} = \{f_\ell \,:\, \ell \in [p]\}$ deciding whether the knowledge base is satisfiable is a constraint satisfaction problem. Here the by the formula f_ℓ affected variables X_{e_ℓ} build the hyperedges, which are decorated by the constraint tensors coinciding with the formulas $f\left[X_{e_\ell}\right]$.

2.4.1 Manipulation of constraint tensor networks

Decide whether tensor networks should be denoted hyperedge-based and not graph based.

Theorem 2.9 (Invariance under adding subcontractions, Rule (i)**)** *Let there be a constraint network* $\tau^{\mathcal{E}}$ *on the hypergraph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ *and choose an arbitrary subset* $\tilde{\mathcal{E}} \subset \mathcal{E}$ *and* $\mathcal{U} \subset \mathcal{V}$*. We further choose a reduced subset* $\mathcal{E}^{\text{reduced}} \subset \mathcal{E}$ *with* $\mathcal{E} \backslash \tilde{\mathcal{E}} \subset \mathcal{E}^{\text{reduced}}$ *and for any* $e \in \mathcal{E} \backslash \mathcal{E}^{\text{reduced}}$ *we have* $e \subset \mathcal{U}$*. Then we have*

$$\left\langle \tau^{\mathcal{E}} \right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{\mathcal{E}^{\text{reduced}}} \cup \{\mathbb{I}_{\neq 0} \circ \left\langle \tau^{\tilde{\mathcal{E}}} \right\rangle_{[\mathcal{U}]}\} \right\rangle_{[X_{\mathcal{V}}]} .$$

Theorem 2.10 (Invariance under splitting disentangled edges, Rule (ii)**)** *Let there be a constraint network* $\tau^{\mathcal{E}}$ *and let for some* $e \in \mathcal{E}$ *with* $e = e_0 \dot{\cup} e_1$ *exist tensors* $\tau^{e_0}\left[X_{e_0}\right]$ *and* $\tau^{e_1}\left[X_{e_1}\right]$ *obeying*

$$\tau^{e}\left[X_e\right] = \tau^{e_0}\left[X_{e_0}\right] \otimes \tau^{e_1}\left[X_{e_1}\right] .$$

Then

$$\left\langle \tau^{\mathcal{E}} \right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{(\mathcal{E} \cup \{e_0, e_1\}) \backslash \{e\}} \right\rangle_{[X_{\mathcal{V}}]} .$$

Reformulate inference algorithms based on the following operations: Leaving the contraction invariant:

i Adding the support of a sub-contraction of a graph $\tilde{\mathcal{G}} \subset \mathcal{G}$ and nodes $\mathcal{U} \subset \mathcal{V}$, and drop some $e \in \tilde{\mathcal{G}}$ with $e \subset \mathcal{U}$.

ii Partition an edge e into two edges e_0, e_1 decorated by $\tau^{e_0}\left[X_{e_0}\right], \tau^{e_1}\left[X_{e_1}\right]$, if

$$\tau^{e}\left[X_e\right] = \tau^{e_0}\left[X_{e_0}\right] \otimes \tau^{e_1}\left[X_{e_1}\right]$$

Monotonic decrease of solutions:

- Adding constraint tensors. Thus, the network is satisfiable if an increased one is satisfiable. If a decreased one is unsatisfiable, then also the network is unsatisfiable.

  Elementary networks are satisfiable when not containing the zero tensor.
  Need partial ordering for the monotonicity?

2.4.2 Examples

We now investigate examples of constraint networks and their manipulations.

2.4.2.1 Clauses

An interesting class of constraint networks are those where each constraint is a clause. Many logical inference algorithms are understood as manipulations of the constraint networks [**?**]. Before presenting the basic syntactical inference rules, let us derive contraction properties of clauses.

Lemma 2.5 *For any clause* $Z^{\vee}_{(A,y_A)}\left[X_A\right]$ *and any* $k \in A$ *and* $\tilde{y}_k \in [2]$ *we have*

$$\left\langle Z^{\vee}_{(A,y_A)}\left[X_A\right], \epsilon_{y_k}\left[X_k\right] \right\rangle_{[X_A]} = \begin{cases} Z^{\vee}_{(A,y_A)} X_{A\backslash\{k\}} \otimes \epsilon_{y_k}\left[X_k\right] & \text{if} \quad y_k = \tilde{y}_k \\ \mathbb{I}\left[X_{A\backslash\{k\}}\right] \otimes \epsilon_{y_k}\left[X_k\right] & \text{if} \quad y_k \neq \tilde{y}_k \end{cases} .$$

Example 2.3 (Modus ponens) When there is an edge

$$e = \mathcal{V}^{\text{in}} \cup \mathcal{V}^{\text{out}}$$

with

$$\tau^e\left[X_e\right] = \left(\bigwedge_{v\in\mathcal{V}^{\text{in}}} X_v\right) \Rightarrow \left(\bigwedge_{v\in\mathcal{V}^{\text{out}}} X_v\right)$$

and for each $v \in \mathcal{V}^{\text{in}}$ an edge $\{v\}$ with $\tau^{\{v\}}\left[X_v\right] = \epsilon_1\left[X_v\right]$, then:

- Choose $\tilde{\mathcal{G}} = \left(e, \{e\} \cup \bigcup_{v\in\mathcal{V}^{\text{in}}}\{v\}\right), \mathcal{U} = e$
- Apply *i* rule to replace $\tilde{\mathcal{G}}$ with $\tau^e\left[X_e\right] = \bigtimes_{v\in e}\epsilon_1\left[X_v\right]$
- Apply *ii* rule to split into edges $\{v\}$ for $v \in e$ with $\epsilon_1\left[X_v\right]$

Notice, that any basis vector is also a clause. We further derive a contraction property of arbitrary clauses with negated literals.

Lemma 2.6 *Let there be two clauses* $Z^{\vee}_{(A^0,y^0_{A^0})}\left[X_{A^0}\right]$, $Z^{\vee}_{(A^1,y^1_{A^1})}\left[X_{A^1}\right]$ *and let us assume there is* $k \in (A^0 \cap A^1)$ *with*

$$y^0_k \neq y^1_k\,.$$

Then

$$\begin{aligned}
&\mathbb{I}_{\neq 0} \circ \left\langle Z^{\vee}_{(A^0,y^0_{A^0})}\left[X_{A^0}\right], Z^{\vee}_{(A^1,y^1_{A^1})}\left[X_{A^1}\right]\right\rangle_{\left[X_{(A^0\cup A^1)\setminus\{k\}}\right]} \\
&= \begin{cases} \mathbb{I}\left[X_{(A^0\cup A^1)\setminus\{k\}}\right] & \textit{if} \quad \exists\tilde{k} \in (A^0\cup A^1)\setminus\{k\} : y^0_{\tilde{k}} \neq y^1_{\tilde{k}} \\ Z^{\vee}_{(A^0\cup A^1)\setminus\{k\}, y_{(A^0\cup A^1)\setminus\{k\}}}\left[X_{(A^0\cup A^1)\setminus\{k\}}\right] & \textit{else}\,. \end{cases}
\end{aligned}$$

Proof. Since contraction is the sum of the contractions with $\epsilon_1\left[X_k\right]$ and $\epsilon_1\left[X_k\right]$, shrinking the right and the left clause. Can argue with the unique assignments to the left and right shrinked variables such that vanishing coordinate is possible. Together, if they agree, they build the unique assignment where the resolvent is vanishing. □

Example 2.4 (Resolution) When there are two clauses with a complementary literal can apply *i* with the complementary literal variable left out of the open variables and get the resolvent clause. Notice that the old clauses cannot always be dropped, since they are no subsets of $\mathcal{U}$. This is different to the modus ponens inference rule.

2.4.2.2 Categorical Constraints

These are the exactly one constraints, used in Sudoku.

Definition 2.9 (Categorical Constraint and Atomization Variables) Given a list $X_0, \ldots, X_{m-1}$ of boolean variables and a categorical variable X with dimension m a categorical constraint is a tensor $\beta^e\left[X_{[m]}, X\right]$ with coordinates

$$\beta^e\left[X_{[m]} = x_{[m]}, X = x\right] = \begin{cases} 1 & \text{if} \quad x_{[d]} = \epsilon_x \\ 0 & \text{else}\,. \end{cases} \quad \left(\text{i.e. } \forall k \in [m] : (x = k) \Leftrightarrow (x_k = 1)\right)$$

We then call the variables $X_0, \ldots, X_{m-1}$ the atomization variables to the categorical variable X.

We notice that the categorical constraint tensor is the basis encoding of the one-hot map on $[m]$. The restriction to the image coordinate k of the one-hot map is the function

$$e^k\left(\tilde{k}\right) = \begin{cases} 1 & \text{if} \quad \tilde{k} = k \\ 0 & \text{else}\,. \end{cases}$$

The basis encoding of the one-hot map is decomposed as

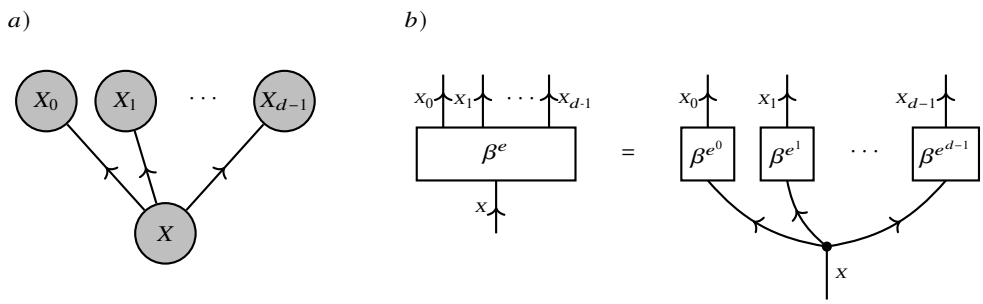


Fig. 2.5 Representation of a categorical constraint in a CP Format tensor network. a) Representation of the dependency of the graphical model. b) Tensor Representation with further network decomposition.

$$\beta^{e^k}\left[X_k, X\right] = \epsilon_1\left[X_k\right] \otimes \epsilon_k\left[X\right] + \epsilon_0\left[X_k\right] \otimes \left(\mathbb{I}\left[X\right] - \epsilon_k\left[X\right]\right). \tag{2.6}$$

We thus have (see Figure 2.5)

$$\beta^e\left[X_{[m]}, X\right] = \left\langle \left\{ \beta^{e^k}\left[X_k, X\right] \,:\, x \in [m] \right\} \right\rangle_{[X, X_0, \ldots, X_{m-1}]} .$$

Add Sudoku from the NeSy paper, call the boolean variables atomization variables.

Example 2.5 (Sudoku representation) We index the rows and the columns by tuples $(r0, r1)$ and $(co, c1)$, where $r0, r1, c0, c1 \in [n]$. The first index indicates the block and the second counts the row or column inside that block. For each $r0, r1, c0, c1 \in [n]$ and $i \in [n^2]$ we then define an atomic variable $X_{r0,r1,c0,c1,i} \in \{0, 1\}$ indicating whether in the row $(r0, r1)$ and column $(co, c1)$ the number i is written. The Sudoku rules then amount to the formula

$$\begin{aligned}\mathcal{KB}^n := &\left(\bigwedge_{r0,r1,c0,c1\in[n]} \left(\bigoplus_{i\in[n^2]}^{(1)} X_{r0,r1,c0,c1,i}\right)\right) \wedge \left(\bigwedge_{r0,r1\in[n],i\in[n^2]} \left(\bigoplus_{c0,c1\in[n]}^{(1)} X_{r0,r1,c0,c1,i}\right)\right) \wedge \\ &\left(\bigwedge_{c0,c1\in[n],i\in[n^2]} \left(\bigoplus_{c0,c1\in[n]}^{(1)} X_{r0,r1,c0,c1,i}\right)\right) \wedge \left(\bigwedge_{r0,c0\in[n],i\in[n^2]} \left(\bigoplus_{r1,c1\in[n]}^{(1)} X_{r0,r1,c0,c1,i}\right)\right),\end{aligned}$$

where $\bigoplus^{(1)}$ is the n^2-ary exclusive or connective (that is 1 if and only if exactly one of the arguments is 1). The four outer brackets in $\mathcal{KB}$ mark the constraints that at each position exactly one number is assigned, further that in each row each number is assigned once, and similar for the columns and the squares of the board. When solving a specific Sudoku instance, one typically knows from an initial board assignment E^{start} a collection of atomic variables, which hold, and needs to find further atomic variables, which are entailed. This means, we need to decide for each $(r_0, r_1, c_0, c_1, i) \notin E^{\text{start}}$ whether the Sudoku rules and the initial board imply that the atomic variable $X_{r0,r1,c0,c1,i}$ (i.e. assignment to the board) is true

$$\mathcal{KB}^n \wedge \left(\bigwedge_{(r_0,r_1,c_0,c_1,i)\in E^{\text{start}}} X_{r0,r1,c0,c1,i}\right) \models X_{r0,r1,c0,c1,i}$$

or false

$$\mathcal{KB} \wedge \left(\bigwedge_{(r_0,r_1,c_0,c_1,i)\in E^{\text{start}}} X_{r0,r1,c0,c1,i}\right) \models \neg X_{r0,r1,c0,c1,i} .$$

If and only if the Sudoku has a unique solution given the initial board assignment E^{start}, exactly one of these entailment statements holds for each $(r_0, r_1, c_0, c_1, i) \notin E^{\text{start}}$. Deciding which is equivalent to solving the Sudoku.

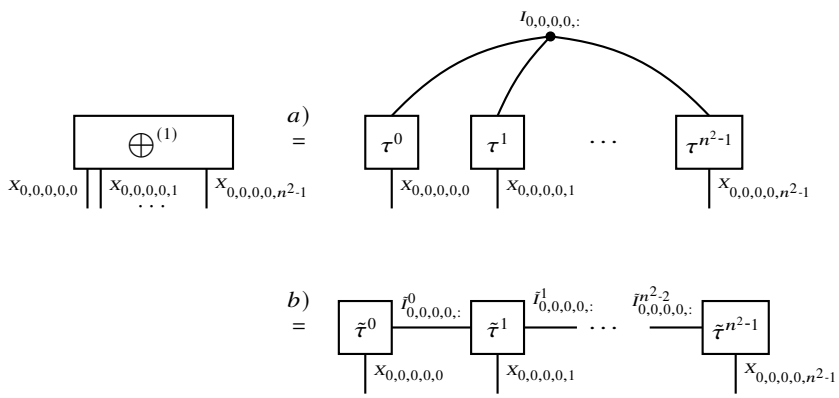


Fig. 2.6 Decomposition of the position constraint $\bigoplus^{(1)}$ at position $(r0, r1, c0, c1) = (0, 0, 0, 0)$ into a) a CP decomposition with hidden variable $I_{0,0,0,0,:}$ and b) a TT decomposition with $d-1$ hidden variables $I^k_{0,0,0,0,:}$, $k \in [d-1]$.

We now exploit Thm. **??** to find efficient tensor network representation of the Sudoku knowledge base from Example **??**. We directly get, that the knowledge base $\mathcal{KB}^n$ of Sudoku rules is a tensor network of the $4 \cdot n^4$ constraint formulas using the n^2-ary connective $\bigoplus^{(1)}$, and the evidence E^{start} can be encoded by vectors $\epsilon_1\left[X_{(r_0,r_1,c_0,c_1,i)}\right]$. To get a representation by matrices instead of tensors of order n^2, we introduce a hidden variable I taking values in $[n^2]$ for each of the constraints. With the usage of matrices

$$\tau^k\left[X_k, I\right] = \epsilon_0\left[X_k\right] \otimes \mathbb{I}\left[I\right] + \left(\epsilon_1\left[X_k\right] - \epsilon_0\left[X_k\right]\right) \otimes \epsilon_k\left[I\right]$$

we have the decomposition

$$\bigoplus^{(1)}[X_{[n^2]}] = \left\langle \{\tau^k\left[X_k, I\right] \,:\, k \in [n^2]\}\right\rangle_{\left[X_{[n^2]}\right]},$$

which is a CP decomposition (see Example 1.2) depicted in Figure 2.6 a).

Alternatively there is a TT decomposition (see Example 1.3) of the constraint $\bigoplus^{(1)}$, which we depict in Figure 2.6 b). We introduce for $k \in [d-1]$ hidden variables I^k of dimension 2, which are interpreted as the indicator, whether one of the variables $X_{[k]}$ is true. Following this interpretation we introduce TT cores

$$\begin{aligned}\tilde{\tau}^0\left[X_0, \tilde{I}^0\right] &= \epsilon_1\left[X_0\right] \otimes \epsilon_1\left[\tilde{I}^0\right] + \epsilon_0\left[X_0\right] \otimes \epsilon_0\left[\tilde{I}^0\right]\\ \tilde{\tau}^{d-1}\left[\tilde{I}^{d-2}, X_{d-1}\right] &= \epsilon_0\left[\tilde{I}^{d-2}\right] \otimes \epsilon_1\left[X_{d-1}\right] + \epsilon_1\left[\tilde{I}^{d-2}\right] \otimes \epsilon_0\left[X_{d-1}\right]\end{aligned}$$

and for $k \in \{1, \ldots, d-2\}$

$$\tilde{\tau}^k\left[\tilde{I}^{k-1}, X_k, \tilde{I}^k\right] = \delta\left[\tilde{I}^{k-1}, \tilde{I}^k\right] \otimes \epsilon_0\left[X_k\right] + \epsilon_0\left[\tilde{I}^{k-1}\right] \otimes \epsilon_1\left[X_k\right] \otimes \epsilon_1\left[\tilde{I}^{k-1}\right].$$

We notice, that the TT decomposition of the constraint $\bigoplus^{(1)}$ introduces $d-1$ many hidden variables of dimension 2, whereas the CP decomposition introduces a single hidden variable of dimension d. However, in the following we will further apply the CP decomposition.

Given evidence E^{start} we denote the Sudoku knowledge base $\mathcal{KB}^{n,E^{\text{start}}}$. We model the Sudoku knowledge base $\mathcal{KB}^{n,E^{\text{start}}}$ as a tensor network on a hypergraph $\mathcal{G}^{\text{Sudoku},n}$ consistent in

- $n^6 + 4 \cdot n^4$ nodes by n^6 categorical variables $X_{(r0,r1,c0,c1,i)}$ and by $4 \cdot n^4$ decomposition variables to the constraints
- $5 \cdot n^6$ edges

$$\begin{aligned}\mathcal{E} = \bigcup_{r0,r1,c0,c1\in[n]} \big\{&\{X_{(r0,r1,c0,c1,i)}\}, \{X_{(r0,r1,c0,c1,i)}, I_{r0,r1,c0,c1,:}\}, \{X_{(r0,r1,c0,c1,i)}, I_{r0,r1,:,:,:,i}\},\\ &\{X_{(r0,r1,c0,c1,i)}, I_{:,:,c0,c1,i}\}, \{X_{(r0,r1,c0,c1,i)}, I_{r0,:,c0,:,i}\}\big\}\end{aligned}$$

We denote the decomposition variables to the position, row, column and square constraints by $I_{r0,r1,c0,c1,:}$, $I_{r0,r1,:,:,i}$, $I_{:,:,c0,c1,i}$ and $I_{r0,:,c0,:,i}$.

Each edge containing a decomposition variable is decorated by a matrix $\tau^k\,[X, I]$ corresponding to a core in the CP decomposition of a constraint. Here k is determined by the tuple $(r0, r1, c0, c1, i)$ and the type of the constraint (for example, for the variable $X_{(0,1,1,2,1)}$ and the row constraint $I_{(0,1,:,:,1)}$ we have $k = 1 \cdot n + 2$. We further assign to each edge containing a single variable $\{X_{(r0,r1,c0,c1,i)}\}$ either the vector $\epsilon_1\left[X_{(r0,r1,c0,c1,i)}\right]$ if $(r0, r1, c0, c1, i) \in E^{\text{start}}$ or the trivial vector $\mathbb{I}\left[X_{(r0,r1,c0,c1,i)}\right]$.

For a more concrete example, let $n = 2$ and

$$\begin{aligned} E^{\text{start}} = \{&(0,0,0,0,0), (0,0,1,0,2), (0,0,1,1,1), (0,1,0,1,1),\\ &(1,0,1,0,3), (1,1,0,0,3), (1,1,0,1,2)\}\,. \end{aligned}$$

We visualize this evidence by writing $i + 1$ in a grid cell $(r0, r1, c0, c1)$ to indicate that $(r0, r1, c0, c1, i) \in E^{\text{start}}$:

1		3	2
	2		
		4	
4	3		

Example 2.6 (Queens assignment game) For arbitrary $n_0, n_1 \in \mathbb{N}$ we have a chess board of $n_0 \times n_1$ positions with n_0 columns and n_1 rows. We want to assign r queens onto positions of the chess board, such that each column and each row contains at most one queen.

We assign to each position (i, j) of the chess board a boolean variable $X_{i,j} \in \{0, 1\}$, which indicates whether there is a queen at position (i, j). The constraints that each row i and each column j contains at most one queen can then be expressed by the constraints $\tau^{\text{row},i}\left[X_{i\times[n_1]}\right]$ defined by coordinates

$$\tau^{\text{row},i}\left[X_{i\times[n_1]} = x_{i\times[n_1]}\right] = \begin{cases} 1 & \text{if} \quad \sum_{j\in[n_1]} x_{i,j} \le 1 \\ 0 & \text{else} \end{cases}$$

$$\tau^{\text{col},j}\left[X_{[n_0]\times j} = x_{[n_0]\times j}\right] = \begin{cases} 1 & \text{if} \quad \sum_{i\in[n_0]} x_{i,j} \le 1 \\ 0 & \text{else}\,. \end{cases}$$

We introduce decomposition variables $I_{\text{row},i}$ and $I_{\text{col},i}$ of dimension $n_{\text{row},i} = n_1 + 1$ and $n_{\text{col},i} = n_0 + 1$. The first n_1 (respectively n_0) states shall be interpreted by a queen in the row (repectively column) at the corresponding position. The last state is interpreted by a missing queen in that row (respectively column). This interpretation is enforced by the matrices

$$\tau^{\text{row},i,j}\left[X_{i,j}, I_{\text{row},i}\right] = \epsilon_1\left[X_{i,j}\right] \otimes \epsilon_j\left[I_{\text{row},i}\right] + \epsilon_0\left[X_{i,j}\right] \otimes \left(\mathbb{I}\left[I_{\text{row},i}\right] - \epsilon_j\left[I_{\text{row},i}\right]\right)$$

and we have the property

$$\tau^{\text{row},i}\left[X_{i\times[n_1]}\right] = \left\langle \{\tau^{\text{row},i,j}\left[X_{i,j}, I_{\text{row},i}\right] \,:\, j \in [n_1]\}\right\rangle_{\left[X_{i\times[n_1]}\right]}\,.$$

The tensor of possible assignments to the queens game

$$\left\langle \{\tau^{\text{row},i,j}\left[X_{i,j}, I_{\text{row},i}\right] \,:\, i \in [n_0],\ j \in [n_1]\} \cup \{\tau^{\text{col},i,j}\left[X_{i,j}, I_{\text{col},i}\right] \,:\, i \in [n_0],\ j \in [n_1]\}\right\rangle_{\left[X_{[n_0]\times[n_1]}\right]}\,.$$

The number of possible assignments of queens to a $n_0 \times n_1$ board is the contraction without open variables.

2.5 Inference algorithms

Contractions of sub-networks are the local inference steps. We review algorithms like AC-3 in that perspective and give with knowledge propagation a generic inference scheme.

When having a Constraint Satisfaction Problem on a large number of variables, which are densely connected by constraint tensors, direct exploitation of the global entailment criterion in Thm. 2.2 will be infeasible. An alternative to deciding entailment by global operations is the use of local operations. Here we interpret a part of the network (for example a single core) as an own knowledge base (with atomic formulas being the roots of the directed subgraph, that is potentially differing with the atoms in the global perspective) and perform entailment with respect to that.

2.5.1 Monotonicity of Entailment

Remove markov networks for monotonocity statements. Understand as the proof of the rule i.

Vanishing local contractions provide sufficient but not necessary criterion to decide entailment, as we show in the next theorem.

Theorem 2.11 (Monotonicity of Entailment) *For any Markov Network $\mathbb{P}^{\mathcal{G}}$ on the decorated hypergraph $\mathcal{G}$ and any subgraph $\tilde{\mathcal{G}}$, we have for any formula that $\mathbb{P}^{\mathcal{G}} \models f$ if $\mathbb{P}^{\tilde{\mathcal{G}}} \models f$.*

To prove the theorem, we first establish the following lemma that states if a contraction of non-negative tensors vanishes, the vanishing of a contraction over a subset of these tensors is a sufficient criterion.

Lemma 2.7 *For any non-negative tensor network $\tau^{\mathcal{G}}$ on $\mathcal{G}$ and $\tilde{\mathcal{E}} \subset \mathcal{E}$ we have the following. For $\tilde{\mathcal{G}} = (\mathcal{U}, \tilde{\mathcal{E}})$ with $\mathcal{U} = \cup_{e \in \tilde{\mathcal{E}}} e$ and the tensor network $\tau^{\tilde{\mathcal{G}}}$ with tensors coinciding on $\tilde{\mathcal{E}}$ with those in $\tau^{\mathcal{G}}$ we have*

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[\varnothing]} = 0$$

if $\left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[\varnothing]} = 0$.

Proof. Since the tensor network $\tau^{\tilde{\mathcal{G}}}$ is non-negative, we have whenever $\left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[\varnothing]} = 0$ that

$$\left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[X_{\mathcal{U}}]} = 0\,[X_{\mathcal{U}}]\ .$$

It follows with the commutation of contractions (see Thm. **??** in Chapter **??**), that

$$\begin{aligned}
\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{V}}]} &= \left\langle \{\tau^e \ : \ e \in \mathcal{E}/\tilde{\mathcal{E}}\} \cup \{\left\langle \tau^{\tilde{\mathcal{G}}} \right\rangle_{[X_{\mathcal{U}}]}\} \right\rangle_{[X_{\mathcal{V}}]} \\
&= \left\langle \{\tau^e \ : \ e \in \mathcal{E}/\tilde{\mathcal{E}}\} \cup \{0\,[X_{\mathcal{U}}]\} \right\rangle_{[X_{\mathcal{V}}]} \\
&= 0
\end{aligned}$$

Thus, also the contraction of $\tau^{\mathcal{G}}$ vanishes in this case. □

Proof of Thm. 2.11. We use Lem. 2.7 on the subset $\tau^{\tilde{\mathcal{G}}}$ of the cores $\tau^{\mathcal{G}}$ to the Markov Network $\mathbb{P}^{\mathcal{G}}$, which itself defines the Markov Network $\mathbb{P}^{\tilde{\mathcal{G}}}$. Whenever $\mathbb{P}^{\tilde{\mathcal{G}}} \models f$ for a formula f, then we have by Thm. 2.2

$$\left\langle \tau^{\tilde{\mathcal{G}}} \cup \{\neg f\} \right\rangle_{[\varnothing]} = 0\,.$$

It follows with Lem. 2.7 that also

$$\left\langle \tau^{\mathcal{G}} \cup \{\neg f\} \right\rangle_{[\varnothing]} = 0\,.$$

and therefore $\mathbb{P}^{\mathcal{G}} \models f$. □

Remark 2.3 To make use of Thm. 2.11 we can exploit any entailment criterion. However, there is no general statement about entailment possible, when the local entailment does not hold. Thm. 2.11 therefore just provides a sufficient but not necessary criterion of entailment with respect to $\mathbb{P}^{\mathcal{G}}$.

2.5.2 Syntactical inference algorithms

Clauses are tensors, which either shrink or trivialize when contracted with basis vectors.

2.5.2.1 Forward Chaining in Horn Logics

When all clauses definite and of length at most 2, then both the $1_{\mathcal{V}}$ and $0_{\mathcal{V}}$ satisfy the constraints. Forward chaining as copying length 1 clauses to other clauses and contract them. When terminated all clauses are of lenght at most 2, and definite if they initially where definite (then any clause of length 1 is a fact).

2.5.2.2 Resolution

Theorem 2.12 (Grand resolution theorem) *When a constraint tensor network of clauses is unsatisfiable, then the empty clause is in the closure under resolution.*

Proof. We give a standard proof [**?**] with the perspective of contractions. Having a clause tensor network we add basis vectors to get a elementary tensor network, which is satisfiable when we started with the closure under resolution. □

2.5.2.3 Generic inference rules

More generally, whenever

$$\bigwedge_{f \in \mathcal{F}} f \models h$$

then the support of the sub contraction of $\mathcal{F}$ leaving the variables of h open produces a formula entailing h.

2.5.3 Constraint Propagation

Understand constraint propagation as an iterative application of the (i) manipulation rule where messages are computed and contracted into memory cores. (Each step applies the rule twice: New message core computed and fused into old message core.) Call κ memory core, which is updated by a message.

To store preliminary conclusions, we define auxiliary message cores storing constraints on variables $e \subset \mathcal{V}$. They are understood as logical formulas to the atomization variables $(X_e = x_e)$ of the respective formulas

$$\kappa^e\left[X_e\right] = \bigvee_{x_e \,:\, \kappa^e[X_e = x_e]} \bigwedge_{v \in e} \left(X_e = x_e\right) \ .$$

Definition 2.10 Let $\tau^{\mathcal{G}}$ be a constraint satisfaction problem. We say that a message core $\kappa^e\left[X_e\right]$ is sound for $\tau^{\mathcal{G}}$, if

$$\mathbb{I}_{\neq 0} \circ \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_e]} \prec \kappa^e\left[X_e\right]$$

and complete for $\tau^{\mathcal{G}}$ if in addition

$$\mathbb{I}_{\neq 0} \circ \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_e]} = \kappa^e \left[X_e \right] \ .$$

We now provide a solution algorithm for constraint satisfaction problems by propagating local contractions. The dynamic programming paradigm is implemented by the storage of partial entailment results in message cores. We then iterate over local entailment checks, where we recursively add further entailment checks to be redone due to additional knowledge. This local entailment scheme is called Knowledge Propagation and described in a generic way in Algorithm 2.

We understand the support of local contractions as messages, which are tensors $\kappa^e \left[X_e \right]$ associated with message edges e. These message cores are propagated through the network, and summarize neighboring constraints.

Adjust constraint propagation to match tnreason message passing schemes: Queue containing $\tilde{\mathcal{G}}$ and open nodes $\mathcal{U}$.

Algorithm 2 Constraint Propagation

Require: Constraint tensor network $\tau^{\mathcal{G}}$ on $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, message edges $\mathcal{E}^k$ and a set $\mathcal{U}$ of subsets of $\mathcal{E}$ for local propagation
Ensure: Message cores $\kappa^e \left[X_e \right]$ for $e \in \mathcal{E}^k$ with $\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_e]} \prec \kappa^e \left[X_e \right]$

Initialize for all $e \in \mathcal{E}^k$:

$$\kappa^e \left[X_e \right] = \mathbb{I} \left[X_e \right]$$

Initialize a queue

$$Q = \left\{ (\mathcal{E}^{inference}, e_m) \ : \ \mathcal{E}^{inference} \in \mathcal{U}, \ e_m \in \mathcal{E}^k, \ e_m \cap \Big(\bigcup_{e \in \mathcal{E}^{inference}} e \Big) \neq \emptyset \right\}$$

while Q is not empty **do**
  Choose a set of edges from the queue

$$\mathcal{E}^{\text{inference}}, e_m \leftarrow Q.\text{pop}()$$

  Contract

$$\tau \left[X_{e_m} \right] = \mathbb{I}_{\neq 0} \circ \left\langle \{ \tau^{\tilde{e}} \left[X_{\tilde{e}} \right] \ : \ \tilde{e} \in \mathcal{E}^{\text{inference}} \} \cup \{ \kappa^e \left[X_e \right] \ : \ e \in \mathcal{E}^k, \ e \cap \bigcup_{\tilde{e} \in \tilde{\mathcal{E}}} \tilde{e} \neq \emptyset \} \right\rangle_{[X_{e_m}]}$$

  if $\tau \left[X_e \right] \neq \kappa^e \left[X_e \right]$ **then**

$$\kappa^e \left[X_e \right] \leftarrow \tau \left[X_e \right]$$

    for all $\tilde{\mathcal{E}} \in \mathcal{U}$ with $e \cap \bigcup_{\tilde{e} \in \tilde{\mathcal{E}}} \tilde{e} \neq \emptyset$ **do**

$$Q.\text{push}(\tilde{\mathcal{E}})$$

    end for
  end if
end while
return $\{ \kappa^e \left[X_e \right] \ : \ e \in \mathcal{E}^k \}$

Each chosen subset $\tilde{\mathcal{E}} \in \mathcal{U}$ is understood as a local knowledge base, which is then applied for local entailment. The message cores are understood as messages, which propagate information from different regions of a tensor network (see Chapter **??**).

There are different ways of implementing Algorithm 2, by choosing the set $\mathcal{U}$ of constraint sets $\tilde{\mathcal{E}}$ and domain $\mathcal{E}^k$.

Example 2.7 (AC-3 as a Constraint propagation scheme) The AC-3 algorithm (see [**?**]) is a specific instance, where message cores are assigned to single variables and propagation is performed on single constraint cores. We assume that each edge has cardinality 2, that is $\mathcal{G}$ is a graph. Here the inference clusters are

$$\mathcal{U} = \left\{ \{e\} \ : \ e \in \mathcal{E} \right\} .$$

The message edges are

$$\mathcal{E}^k = \left\{\{v\} \,:\, v \in \mathcal{V}\right\}.$$

At each local inference step to the edge $e = \{v_0, v_1\}$ we compute

$$\left\langle \kappa^{\{v_0\}}\left[X_{v_0}\right], \tau^e\left[X_{v_0,v_1}\right], \kappa^{\{v_0\}}\left[X_{v_1}\right]\right\rangle_{[X_{v_0}]} .$$

Theorem 2.13 *At any state of the constraint propagation Algorithm 2, we have that each message core $\kappa^{\tilde{e}}$ is sound for $\tau^{\mathcal{G}}$. After each update in Algorithm 2, $\kappa^{\tilde{e}}$ is further monotonically decreasing with respect to the partial ordering.*

Proof. This follows from soundness of the i manipulation rule. We show the first claim by induction over the update steps in Algorithm 2. At the start, where $\kappa^e\left[X_e\right] = \mathbb{I}\left[X_e\right]$, we trivially have

$$\left\langle \tau^{\mathcal{G}} \cup \{\kappa^e\left[X_e\right] \,:\, e \in \mathcal{E}^k\}\right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{\mathcal{G}} \cup \{\mathbb{I}\left[X_e\right] \,:\, e \in \mathcal{E}^k\}\right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{\mathcal{G}}\right\rangle_{[X_{\mathcal{V}}]} .$$

Let us now assume, that for a state of cores $\{\kappa^e \,:\, e \in \mathcal{E}^k\}$ the first claim holds and let $\tilde{\mathcal{E}} \subset \mathcal{E}$ be chosen for the update of $\kappa^{\tilde{e}}$. By the invariance under adding the support of subcontractions, which we will proof in more detail as Thm. **??** in Chapter **??**, we have for the update

$$\tilde{\kappa}^{\tilde{e}}\left[X_{\tilde{e}}\right] = \mathbb{I}_{\neq 0} \circ \left\langle \{\tau^e \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\kappa^e \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{\tilde{e} \in \tilde{\mathcal{E}}} \tilde{e} \neq \emptyset\}\right\rangle_{[X_{\tilde{e}}]}$$

that

$$\left\langle \tau^{\mathcal{G}} \cup \{\kappa^e\left[X_e\right] \,:\, e \in \mathcal{E}^k\}\right\rangle_{[X_{\mathcal{V}}]} = \left\langle \tau^{\mathcal{G}} \cup \{\kappa^e\left[X_e\right] \,:\, e \in \mathcal{E}^k\} \cup \{\tilde{\kappa}^{\tilde{e}}[X_{\tilde{e}}]\}\right\rangle_{[X_{\mathcal{V}}]} .$$

Thus, the first claim holds also after the update of the core to $\tilde{e}$.

We further have with the monotonicity of boolean contraction (see Thm. **??**) that for any update of $\kappa^{\tilde{e}}$ by $\tilde{\kappa}^{\tilde{e}}$

$$\tilde{\kappa}^{\tilde{e}}\left[X_{\tilde{e}}\right] = \mathbb{I}_{\neq 0} \circ \left\langle \{\tau^e \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\kappa^e \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{e \in \tilde{\mathcal{E}}} e \neq \emptyset\}\right\rangle_{[X_{\tilde{e}}]} \prec \kappa^{\tilde{e}}\left[X_{\tilde{e}}\right] .$$

Thus, each message core is monotonously decreasing at each update, with respect to the partial tensor ordering.

From the first claim we further have for any $\tilde{e} \in \tilde{\mathcal{E}}$

$$\left\langle \{\kappa^{\tilde{e}}\left[X_{\tilde{e}}\right]\} \cup \tau^{\mathcal{G}} \cup \left\{\kappa^e \,:\, e \in \mathcal{E}^k / \{\tilde{e}\}\right\}\right\rangle_{[X_{\tilde{e}}]} = \left\langle \tau^{\mathcal{G}}\right\rangle_{[X_{\tilde{e}}]}$$

And thus in combination with the monotonicity of boolean contraction (see Thm. **??**) that

$$\mathbb{I}_{\neq 0}\left(\left\langle \tau^{\mathcal{G}}\right\rangle_{[X_{\tilde{e}}]}\right) \prec \kappa^{\tilde{e}}\left[X_{\tilde{e}}\right] . \square$$

Let us now show that constraint propagation always terminates. We can further characterize the message cores at termination.

Definition 2.11 We say that a set of message cores $\{\kappa^e \,:\, e \in \mathcal{E}^k\}$ is consistent with a set $\{\tau^e \,:\, e \in \tilde{\mathcal{E}}\}$, if for any $e \in \mathcal{E}^k$

$$\kappa^e\left[X_e\right] = \mathbb{I}_{\neq 0} \circ \left\langle \{\tau^e \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\kappa^e \,:\, e \in \mathcal{E}^k\}\right\rangle_{[X_e]} .$$

This property is similar to the completeness of a message core, when interpreting the other message cores and the constraints $\{\tau^e \,:\, e \in \tilde{\mathcal{E}}\}$ as posing a Constraint Satisfaction Problem.

Theorem 2.14 *constraint propagation Algorithm 2 always terminates. At termination we further for each $\tilde{\mathcal{E}} \in \mathcal{U}$ and $e \in \mathcal{E}^k$ with $e \cap \bigcup_{e \in \tilde{\mathcal{E}}} e \neq \emptyset$, that the message cores $\{\kappa^e \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{e \in \tilde{\mathcal{E}}} e \neq \emptyset\}$ are consistent with $\{\tau^e \,:\, e \in \tilde{\mathcal{E}}\}$.*

Proof. For each message core, there are finitely many boolean tensor precessing it with respect to the partial order. Therefore, since they are monotonously decreasing, each message core can only be varied finitely many times during

the algorithm. In total the algorithm can run only finitely many times in the second for loop, where new sets of edges are pushed into the queue. Therefore the while loop will always terminate.

When after a single pass through the while loop with chosen $\tilde{\mathcal{E}} \in \mathcal{U}$, the set $\tilde{\mathcal{E}}$ is not pushed back into Q, we have for any $e \in \mathcal{E}^k$ with $e \cap \bigcup_{e \in \tilde{\mathcal{E}}} e \neq \emptyset$ that

$$\kappa^e \left[X_e\right] = \mathbb{I}_{\neq 0} \circ \left\langle \{\tau^e \,:\, e \in \tilde{\mathcal{E}}\} \cup \{\kappa^e \,:\, e \in \mathcal{E}^k,\, e \cap \bigcup_{e \in \tilde{\mathcal{E}}} e \neq \emptyset\} \right\rangle_{[X_e]} .$$

Whenever the contraction on the right hand side changes during the algorithm, the set $\tilde{\mathcal{E}}$ is pushed into Q. At termination of the algorithm, Q is empty, and the claimed consistency therefore has to hold. □

We can exploit the constraint propagation Algorithm 2 for the solution of Constraint Satisfaction Problems, by taking $\tau^{\mathcal{G}}$ as the tensor network of constraint tensors. Whenever a message core vanishes, we can conclude that the Constraint Satisfaction Problems is not satisfiable, as we show next.

Corollary 2.2 *Let us for a Constraint Satisfaction Problem encoded by $\tau^{\mathcal{G}}$ run constraint propagation Algorithm 2. Whenever for any $\tilde{e} \in \mathcal{E}$ we have $\kappa^{\tilde{e}} \left[X_{\tilde{e}}\right] = 0 \left[X_{\tilde{e}}\right]$, then the Constraint Satisfaction Problem is not satisfiable.*

Proof. Whenever $\kappa^{\tilde{e}} \left[X_{\tilde{e}}\right] = 0 \left[X_{\tilde{e}}\right]$, then we have by Thm. 2.13

$$\mathbb{I}_{\neq 0} \left(\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\tilde{e}}]} \right) \prec 0 \left[X_{\tilde{e}}\right]$$

and therefore

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[\emptyset]} = 0 .\square$$

2.5.4 Applications

Let us exemplify the usage of constraint propagation on Constraint Satisfaction Problems posed by entailment queries on Markov Networks.

Corollary 2.3 *Let Algorithm 2 be run on the cores $\tau^{\mathcal{G}} \cup \{\beta^f\}$ with an arbitrary design of $\tilde{\mathcal{E}}$. Whenever for a formula $f \left[X_{\mathcal{U}}\right]$ and a κ^e we have*

$$\left\langle \kappa^e, \beta^f \right\rangle_{\left[Y_f = 0\right]} = 0$$

then the Markov Network $\tau^{\mathcal{G}}$ probabilistically entails f. If on the contrary

$$\left\langle \kappa^e, \beta^f \right\rangle_{\left[Y_f = 1\right]} = 0$$

then the Markov Network $\tau^{\mathcal{G}}$ probabilistically entails $\neg f$, that is probabilistically contradicts f.

Proof. This follows from Thm. 2.13 ensuring the soundness of constraint propagation and the sufficiency of local entailment. □

Example 2.8 (Batch decision of entailment) Let $\mathcal{F}$ be a set of formulas and $\mathbb{P}^{\mathcal{G}} \left[X_{[d]}\right]$ a Markov Network, for which it shall be decided, which formulas in $\mathcal{F}$ are entailed, contradicted or contingent. We can in addition to the cores of the Markov Network create the cores $\{\beta^f \left[Y_f, X_{[d]}\right] \,:\, f \in \mathcal{F}\}$ and prepare the message cores

$$\kappa^{\{f\}} \left[Y_f\right] .$$

To decide entailment batchwise, constraint propagation Algorithm 2 can be run. Whenever during the algorithm we have that for a f, then Cor. 2.3 implies that if

$$\kappa^{\{f\}}\left[Y_f\right] = \begin{cases} \epsilon_1\left[Y_f\right] & \text{then, the formula is entailed by } \mathbb{P}^{\mathcal{G}}\,. \\ \epsilon_0\left[Y_f\right] & \text{then, the formula is contradicted by } \mathbb{P}^{\mathcal{G}}\,. \\ \mathbb{I}\left[Y_f\right] & \text{then no conclusion can be drawn.} \end{cases}$$

Note, that $\kappa^{\{f\}}\left[Y_f\right] = 0\left[Y_f\right]$ can not happen, since this would mean that $\mathbb{I}_{\neq 0}\left(\mathbb{P}^{\mathcal{G}}\right)$ is inconsistent. Thus, at any stage of Algorithm 2, one of the three holds.

2.5.5 Backtracking search

Understand as adding basis vectors which are locally possible (i.e. can satisfy affected constraints).

When the constraint propagation Algorithm 2 converges in a given implementation and no message core vanishes, we can however not conclude that the Constraint Satisfaction Problem is not satisfiable. However, for any index tuple $x_{\mathcal{V}}$ to be a solution of the CSP to $\tau^{\mathcal{G}}$, we have the necessary condition

$$\forall \tilde{e} \in \tilde{\mathcal{E}} : \kappa^{e}\left[X_{\tilde{e}} = x_{\mathcal{V}}|_{\tilde{\mathcal{E}}}\right] = 1\,,$$

where by $x_{\mathcal{V}}|_{\tilde{\mathcal{E}}}$ we denote the restriction of the index tuple $x_{\mathcal{V}}$ to the variables included in $\tilde{\mathcal{E}}$. One can use this insight as a starting point for backtracking search, where the assignments to variables $X_{\mathcal{U}}$ are iteratively guessed, based on the restriction that each constraint is locally satisfialbe, i.e. .

$$\forall \tilde{e} \in \tilde{\mathcal{E}} : \left\langle \kappa^{e}\left[X_{\tilde{e}\cap\mathcal{U}} = x_{\mathcal{V}}|_{\tilde{e}\cap\mathcal{U}}, X_{\tilde{e}/\mathcal{U}}\right]\right\rangle_{[\varnothing]} \neq 0\,.$$

One can understand the guess of an assignment x_v to a variable X_v, as it is done during backtracking search, as an inclusion of a constraint

$$\kappa^{\{v\}}\left[X_v\right] = \epsilon_{x_v}\left[X_v\right]\ .$$

Therefore, constraint propagation Algorithm 2 can be integrated with backtracking search, with iterations between propagations of knowledge and guessing of additional variables.

2.5.6 Discussion

In our perspective, the number of possible sub-networks to be contracted is the bottleneck, instead of the demand to perform such contractions. The choice and the scheduling of these sub-contractions is most important for the design of efficient inference schemes.

Chapter 3
Probability Theory

Abstract In this chapter, we will establish relations between the formalism of tensor networks and basic concepts of probability theory. We will first introduce joint distributions by tensors and connect their marginalizations and conditionings to basic tensor operations. Sparse representations of these tensors by tensor networks will be motivated based on independence properties. These mechanisms lead towards graphical models, for which we provide tensor network representations.

Topics:

- Chapter I.2 except for exponential families (later in book II.0)
- Sufficient statistics from sketches

3.1 Tensor Formalism in Probability Theory

Let us relate the main terminology of probability theory with the tensor network formalism. As such, any joint distribution of finitely valued variables is a tensor. Further, the operations of marginalization and conditioning of distributions are equal to contractions and normalizations of tensors.

3.1.1 Joint Probability Distributions

Let there be uncertainties of the assignments x_k to the categorical variables X_k of a system. In this scenario we understand each X_k as random variables, which together have a joint distribution defined by the uncertainties of the state assignments. The uncertainty of a state $x_{[d]}$ is then quantified by probabilities

$$\mathbb{P}\left[X_{[d]} = x_{[d]}\right] \in [0, 1] \,.$$

Joint probability distributions are therefore naturally represented by tensors, whose coordinates store the probabilities to each state. These tensors have additional properties as we state in the next definition.

Definition 3.1 (Joint Probability Distribution) Let there be for each $k \in [d]$ a categorical variable X_k taking values in $[m_k]$. A joint probability distribution of these categorical variables is a function

$$\mathbb{P}\left[X_{[d]}\right] \,:\, \bigtimes_{k\in[d]} [m_k] \to \mathbb{R}$$

which is non-negative, that is for any $x_{[d]} \in \bigtimes_{k\in[d]} [m_k]$

$$\mathbb{P}\left[X_{[d]} = x_{[d]}\right] \geq 0 \,,$$

and which is normalized, that is

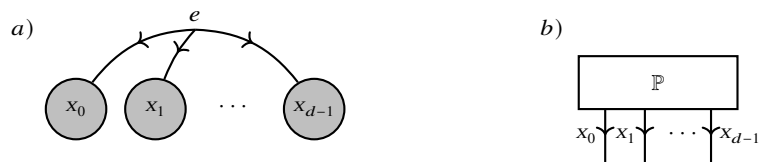


Fig. 3.1 Probability distributions of variables $X_0, \ldots, X_{d-1}$, sketched a) by a directed edge e with all variables outgoing, which is decorated b) by a directed tensor $\mathbb{P}\left[X_{[d]}\right]$.

$$\left\langle \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 1 \,.$$

In a slight abuse of notation, we directly applied the coordinate encoding scheme (see Def. **??** and for more details Chapter **??**) to represent a probability distribution as a tensor. Along this scheme, any probability distribution is a weighted sum of one-hot encodings (see Lem. **??**) by

$$\mathbb{P}\left[X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right] \,,$$

where we understand $\mathbb{P}\left[X_{[d]} = x_{[d]}\right]$ as the probability of the categorical variables to take the state $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$. The normalization condition $1 = \left\langle \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$ has a more convenient equivalence by the coordinate sum

$$1 = \left\langle \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \,,$$

and thus ensures that all probabilities sum to 1, which is necessary for the probabilistic interpretation. While the assumptions of non-negative coordinates in Def. 3.1 reflects the first probability axiom of Kolmogorov, the assumption of normalization, that is a contraction 1, implements the second axiom (see for example [**?**]). Since probability distributions contract to 1, they are directed (see Def. **??**) with all distributed variables outgoing and no incoming variables (see Figure 3.1).

3.1.2 Base Measures

From a measure theoretic perspective, probabilities are measurable functions called probability densities, which integrals are 1 (see for example [**?**]). In our case of finite dimensional state spaces of factored systems, we implicitly used the trivial tensor $\mathbb{I}\left[X_{[d]}\right]$ as a base measure, which measures subsets of states by their cardinality and is therefore referred to as state counting base measure. The distributions $\mathbb{P}\left[X_{[d]}\right]$ can then be understood as probability densities with respect to this state counting base measure. We in this work will also consider more general base measures $\nu\left[X_{[d]}\right]$, which we restrict to be boolean, that is $\nu\left[X_{[d]} = x_{[d]}\right] \in \{0, 1\}$ for all states $x_{[d]}$. When understanding $\mathbb{P}\left[X_{[d]}\right]$ as a probability density with respect to $\nu\left[X_{[d]}\right]$, any probabilistic interpretation will be through the contraction $\left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]}$ and the normalization condition reads as

$$\left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 1 \,.$$

Since we restrict to boolean base measures, the contraction effectively manipulates the tensor $\mathbb{P}$ by setting the coordinates $\mathbb{P}\left[X_{[d]} = x_{[d]}\right]$ to zero, when $\nu\left[X_{[d]} = x_{[d]}\right] = 0$. Therefore, multiple tensors $\mathbb{P}$ will have the same probabilistic interpretation, when $\nu\left[X_{[d]}\right] \neq \mathbb{I}\left[X_{[d]}\right]$. To avoid this ambiguity, we introduce the notation of representability with respect to a base measure ν, by demanding that such coordinates are zero.

Definition 3.2 We say that a probability distribution $\mathbb{P}$ is representable with respect to a boolean base measure ν, if for all $x_{[d]}$ with $\nu\left[X_{[d]} = x_{[d]}\right] = 0$ we have $\mathbb{P}\left[X_{[d]} = x_{[d]}\right] = 0$. We denote the set of by ν representable distributions by $\Lambda^{\delta,\mathrm{MAX},\nu}$.

When a probability distribution $\mathbb{P}$ is representable with respect to a boolean base measure ν, we have the invariance

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]}$$

and can therefore safely ignore the base measures. This enables the characterization of by ν representable distributions by

$$\Lambda^{\delta,\mathrm{MAX},\nu} = \Big\{\mathbb{P}\left[X_{[d]}\right] : \Big(\forall x_{[d]} \in \bigtimes_{k\in[d]} [m_k] : \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \geq 0\Big), \\ \left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = \mathbb{P}\left[X_{[d]}\right]\Big\}.$$

Starting with Chapter 2 we will further investigate boolean tensors and relate them with propositional formulas. In Chapter **??** we will connect the representation and positivity with respect to boolean base measures with the formalism of entailment. The notation $\Lambda^{\delta,\mathrm{MAX},\nu}$ of by ν representable distributions will later in this chapter relate with distributions computable given a statistic.

We now investigate, which base measures ν can be chosen for a probability distribution $\mathbb{P}$, such that $\mathbb{P}$ is representable by ν. Here we want to find a ν, which is in a sense to be defined minimal amount the base measures, such that $\mathbb{P}$ is representable with respect to them. For this minimality criterion we will develop in Chapter **??** orders based on entailment and show the minimality in Thm. **??**. Here, we just introduce the minimality criterion as positivity of a distribution with respect to a base measure.

Definition 3.3 We say that a probability distribution $\mathbb{P}\left[X_{[d]}\right]$ is positive with respect to a boolean base measure $\nu\left[X_{[d]}\right]$, if the distribution is representable by ν (i.e. $\langle \mathbb{P}, \nu\rangle_{[\varnothing]} = 1$) and for all $x_{[d]}$ with $\nu\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] = 1$ we have $\mathbb{P}\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] > 0$. We further say that a distribution is positive, if it is positive with respect to the trivial base measure $\mathbb{I}\left[X_{[d]}\right]$.

3.1.3 Marginal Distribution

Contractions of probability distributions are related to marginalizations as we introduce next.

Definition 3.4 (Marginal Probability) Given a distribution $\mathbb{P}\left[X_0, X_1\right]$ of the categorical variables X_0 and X_1 the marginal distribution of the categorical variable X_0 is the tensor

$$\mathbb{P}\left[X_0\right] \in \mathbb{R}^{m_0}$$

defined by the contraction

$$\mathbb{P}\left[X_0\right] = \langle \mathbb{P}\left[X_0, X_1\right]\rangle_{[X_0]} \ .$$

To connect with a more standard defining equation of marginal distributions, let us notice that for any $x_0 \in [m_0]$

$$\mathbb{P}\left[X_0 = x_0\right] = \langle \mathbb{P}\left[X_0, X_1\right]\rangle_{[X_0 = x_0]} = \sum_{x_1 \in [m_1]} \mathbb{P}\left[X_0 = x_0, X_1 = x_1\right] \ .$$

Thus, each coordinate of the marginal distribution is the sum of the joint probability of compatible states. We say that the variable X_1 is marginalized out, when building the marginal distribution $\mathbb{P}\left[X_0\right]$ of X_0. Let us now justify this terminology and show, that any marginal distribution is a probability distribution as introduced in Def. 3.1.

Theorem 3.1 *Any marginal distribution is a probability distribution.*

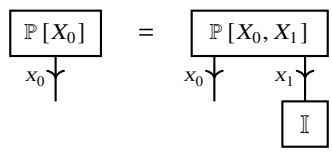
Proof. We further have that any marginal distribution is normalized, since by the commutativity of contractions (see for more details Thm. **??** in Part **??**)

$$\langle \mathbb{P}\left[X_0\right]\rangle_{[\varnothing]} = \left\langle \langle \mathbb{P}\left[X_0, X_1\right]\rangle_{[X_0]}\right\rangle_{[\varnothing]} = \langle \mathbb{P}\left[X_0, X_1\right]\rangle_{[\varnothing]} = 1 \ .$$

Fig. 3.2 Depiction of conditional probability distributions a) by an edge with the incoming variable X_1 and the outgoing variable X_1, which is decorated by b) the directed tensor $\mathbb{P}[X_0|X_1]$.

Further any coordinate is non-negative, since it is a sum of non-negative coordinates. It follows from Def. 3.1, that any marginal distribution is a probability distribution. □

In a tensor network diagram we often represent variables X_1 not appearing as open variables of a contraction as contracted with the trivial tensor $\mathbb{I}[X_1]$. Following this notation, we depict the marginal distribution in Def. 3.4 by



Since we have shown, that marginal distributions are themself probability distributions, they inherit the outgoing directionality in tensor network diagrams.

We notice that Def. 3.4 generalizes to marginalizations of arbitrary sets of variables, when having a distribution $\mathbb{P}\left[X_{[d]}\right]$ of an arbitrary number of categorical variables. It suffices for this to interpret X_0 and X_1 as collections of variables, which indices take the states of the respective factored systems.

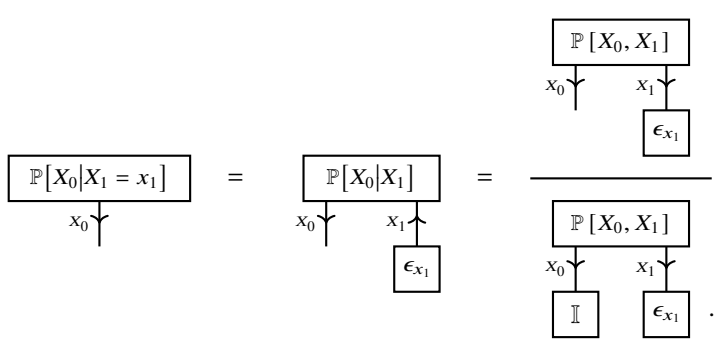
3.1.4 Conditional Probabilities

Normalizations of probability distributions result in conditional distributions as we define next.

Definition 3.5 (Conditional Probability) Let $\mathbb{P}[X_0, X_1]$ be a distribution of the categorical variables X_0 and X_1, such that $\mathbb{P}[X_0, X_1]$ is normalizable on $\{X_1\}$. Then the distribution of X_0 conditioned on X_1 is defined by

$$\mathbb{P}\left[X_0 \middle| X_1\right] = \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0|X_1]} \ .$$

Since conditional probabilities are normalizations of probability tensors, they are directed and therefore depicted by directed hyperedges (see Figure 3.2). For any $x_1 \in [m_1]$ we depict the slice $\mathbb{P}\left[X_0 \middle| X_1 = x_1\right]$ defined by a normalization operation as



As we have done before for marginal distribution, we relate Def. 3.5 with a more convenient coordinatewise definition of conditional probabilities. For any indices $x_0 \in [m_0]$ and $x_1 \in [m_1]$ we have

$$\mathbb{P}\left[X_0 = x_0 \middle| X_1 = x_1\right] = \frac{\mathbb{P}[X_0 = x_0, X_1 = x_1]}{\langle \mathbb{P}[X_0, X_1] \rangle_{[X_1 = x_1]}} = \frac{\mathbb{P}[X_0 = x_0, X_1 = x_1]}{\sum_{x_0 \in [m_0]} \mathbb{P}[X_0 = x_0, X_1 = x_1]} \ .$$

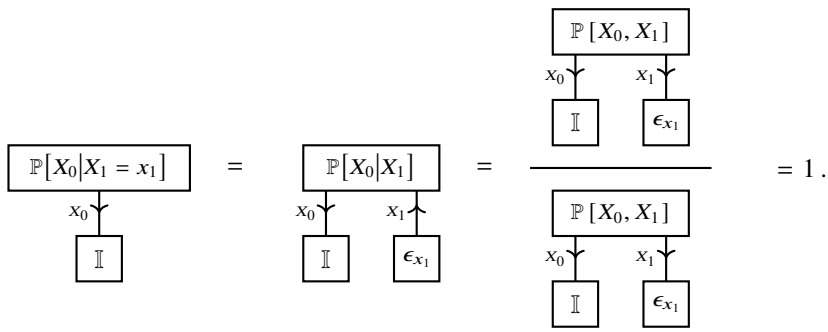
The distribution of X_0 conditioned on X_1 is the normalized collection of slice of the probability distribution $\mathbb{P}[X_0, X_1]$. Each slice of the conditioned distribution with respect to incoming variables is a probability distribution itself, as we show next.

Theorem 3.2 *For any $x_1 \in [m_1]$ the tensor $\mathbb{P}\big[X_0 \big| X_1 = x_1\big]$ is a probability tensor.*

Proof. As a normalization of a non-negative tensor, the conditional probability $\mathbb{P}\big[X_0 \big| X_1 = x_1\big]$ and any of its slices is also a non-negative tensor. Further, we have for any $x_1 \in [m_1]$

$$\begin{aligned}\big\langle \mathbb{P}\big[X_0 \big| X_1 = x_1\big]\big\rangle_{[\varnothing]} &= \sum_{x_0 \in [m_0]} \mathbb{P}\big[X_0 = x_0 \big| X_1 = x_1\big] \\ &= \frac{\sum_{x_0 \in [m_0]} \mathbb{P}\left[X_0 = x_0, X_1 = x_1\right]}{\sum_{x_0 \in [m_0]} \mathbb{P}\left[X_0 = x_0, X_1 = x_1\right]} \\ &= 1\,,\end{aligned}$$

and therefore each slice is normalized. We can visualize this calculation exploiting our diagramatic notation as



Since for any $x_1 \in [m_1]$ the slice $\mathbb{P}\big[X_0 \big| X_1 = x_1\big]$ is non-negative and contracts to 1, we conclude that it is a probability distribution. □

We further show, that exactly the directed tensors with non-negative coordinates are conditional probability tensors.

Theorem 3.3 *Any tensor with non-negative coordinates is a conditional distribution, if and only if it is directed with the conditioned variables incoming and the other outgoing.*

Proof. "⇒": By Thm. 3.2 a conditional probability tensor $\mathbb{P}\big[X_0 \big| X_1\big]$ is the normalization of a tensor and by Thm. **??** a directed tensor. Since probability tensors have only non-negative coordinates, their contractions with one-hot encodings also have only non-negative coordinates and also their normalizations.

"⇐": Conversely, let $\tau\left[X_{\mathcal{V}}\right]$ be a directed tensor with $\mathcal{V}^{\text{in}}$ incoming and $\mathcal{V}^{\text{out}}$ outgoing and non-negative coordinates. Then

$$\mathbb{P}\left[X_{\mathcal{V}}\right] = \frac{1}{\prod_{v \in \mathcal{V}^{\text{in}}} m_v} \cdot \tau\left[X_{\mathcal{V}}\right] \tag{3.1}$$

is a probability tensor, since

$$\sum_{x_{\mathcal{V}^{\text{in}}}} \sum_{x_{\mathcal{V}^{\text{out}}}} \mathbb{P}\left[X_{\mathcal{V}} = x_{\mathcal{V}}\right] = \sum_{x_{\mathcal{V}^{\text{in}}}} \sum_{x_{\mathcal{V}^{\text{out}}}} \frac{1}{\prod_{v \in \mathcal{V}^{\text{in}}} m_v} \cdot \tau\left[X_{\mathcal{V}} = x_{\mathcal{V}}\right] = \sum_{x_{\mathcal{V}^{\text{in}}}} \frac{1}{\prod_{v \in \mathcal{V}^{\text{in}}} m_v} = 1\,.$$

The conditional probability $\mathbb{P}\big[X_{\mathcal{V}^{\text{out}}} \big| X_{\mathcal{V}^{\text{in}}}\big]$ coincides with τ, since

$$\begin{aligned}\mathbb{P}\big[X_{\mathcal{V}^{\text{out}}} \big| X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\big] &= \frac{\mathbb{P}\left[X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]}{\sum_{x_{\mathcal{V}^{\text{out}}}} \mathbb{P}\left[X_{\mathcal{V}^{\text{out}}} = x_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]} \\ &= \frac{\tau\left[X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]}{\sum_{x_{\mathcal{V}^{\text{out}}}} \tau\left[X_{\mathcal{V}^{\text{out}}} = x_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]} = \tau\left[X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}} = x_{\mathcal{V}^{\text{in}}}\right]\,,\end{aligned}$$

where in the last equation we used that the denominator is by definition trivial since τ is normalized. □

Thm. 3.3 specifies a broad class of tensors to represent conditional probabilities. In combination with Thm. **??**, which states that basis encodings are directed, we get that any basis encoding of a function is a conditional probability tensor.

3.1.5 Bayes Theorem and the Chain Rule

So far, we have connected concepts of probability theory such as marginal and conditional probabilities with contractions and normalizations of tensors. We will now proceed to show that basic theorems of probability theory translate into more general contraction equations.

Theorem 3.4 (Bayes Theorem) *For any probability distribution* $\mathbb{P}[X_0, X_1]$ *with positive* $\mathbb{P}[X_1]$ *we have*

$$\mathbb{P}[X_0, X_1] = \left\langle \mathbb{P}\left[X_0 \middle| X_1\right], \mathbb{P}[X_1] \right\rangle_{[X_0, X_1]} .$$

Proof. This theorem follows from the more generic contraction equation Thm. **??** to be shown in Chapter **??**. We note that by positivity of $\mathbb{P}[X_1]$, the tensor network $\mathbb{P}$ is normalizable with respect to X_1. Thm. **??** therefore implies choosing $\mathcal{V} = \{0, 1\}$, $\mathcal{V}^{\text{in}} = \{1\}$ and $\mathcal{V}^{\text{out}} = \{0\}$, that For our tensor

$$\begin{aligned}\mathbb{P}[X_0, X_1] &= \left\langle \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0|X_1]}, \langle \mathbb{P}[X_0, X_1] \rangle_{[X_1]} \right\rangle_{[X_0, X_1]} \\ &= \left\langle \mathbb{P}\left[X_0 \middle| X_1\right], \mathbb{P}[X_1] \right\rangle_{[X_0, X_1]} .\square\end{aligned}$$

Following the insight of the Bayes Thm. 3.4, probability distributions of arbitrary numbers of variables can be decomposed as a contraction of conditional probabilities, as we show in the next theorem.

Theorem 3.5 (Chain Rule) *For any probability distribution* $\mathbb{P}\left[X_{[d]}\right]$ *we have*

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}[X_0]\} \cup \left\{\mathbb{P}\left[X_k \middle| X_0, \ldots, X_{k-1}\right] \,:\, k \in [d],\, k \geq 1\right\} \right\rangle_{\left[X_{[d]}\right]} ,$$

provided that all conditional probability distributions exist.

Proof. The claim can be derived by an iterative application of the Bayes Thm. 3.4 theorem. We will proof this statement in more generality in Chapter **??** as Thm. **??**, deducing it from the generalization of the Bayes Thm. 3.4 by Thm. **??**. The claim here then follows from Thm. **??** using $\mathcal{V} = [d]$ and $\tau[X_{\mathcal{V}}] = \mathbb{P}\left[X_{[d]}\right]$, since

$$\begin{aligned}\mathbb{P}\left[X_{[d]}\right] &= \left\langle \{\mathbb{P}[X_0]\} \cup \{\left\langle \mathbb{P}\left[X_{[d]}\right] \right\rangle_{[X_k|X_0,\ldots,X_{k-1}]} \,:\, k \in [d],\, k \geq 1\} \right\rangle_{\left[X_{[d]}\right]} \\ &= \left\langle \{\mathbb{P}[X_0]\} \cup \left\{\mathbb{P}\left[X_k \middle| X_0, \ldots, X_{k-1}\right] \,:\, k \in [d],\, k \geq 1\right\} \right\rangle_{\left[X_{[d]}\right]} .\square\end{aligned}$$

We observe, that the chain rule provides a generic decomposition scheme of probability distributions into conditional distributions. The conditional distribution to $k = d - 1$, which appears in the chain decomposition, is in the same tensor space as the decomposed distribution $\mathbb{P}\left[X_{[d]}\right]$. To achieve our main goal of tensor network decompositions, which is an efficient storage format of the decomposed tensor, we need to further sparsify the appearing conditional probabilities (to be more precise, we aim at basis+ CP decompositions, to be introduced in Chapter **??**). These simplification require additional assumptions on independencies in the distribution, which we will introduce in the next section.

3.2 Independence as a Decomposition Mechanism

The number of coordinates in a tensor representation of probability distributions is the product

$$\prod_{k \in [d]} m_{X_k} \ ,$$

and therefore scales exponentially in the number of coordinates. To find efficient representation schemes of probability distributions by tensor networks, we need to exploit additional properties of the distribution. In this section, we introduce the independence and the computation mechanisms towards tensor network decompositions of probability distributions.

Independence leads to severe sparsifications of conditional probabilities and is therefore the key assumption to gain sparse decompositions of probability distributions. Before showing such decomposition schemes, we first provide a coordinatewise definition of independent variables.

Definition 3.6 (Independence) We say that X_0 is independent of X_1 with respect to a distribution $\mathbb{P}[X_0, X_1]$, if for any values $x_0 \in [m_0]$ and $x_1 \in [m_1]$ the distribution satisfies

$$\mathbb{P}[X_0 = x_0, X_1 = x_1] = \mathbb{P}[X_0 = x_0] \cdot \mathbb{P}[X_1 = x_1] \ .$$

In this case we denote $(X_0 \perp X_1)$.

We state next an equivalent independence criterion based on a contraction equation of probability distributions.

Theorem 3.6 (Independence Criterion as a Contraction Equation) *The variable X_0 is independent from X_1 with respect to a probability distribution $\mathbb{P}[X_0, X_1]$, if and only if*

$$\mathbb{P}[X_0, X_1] = \left\langle \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0]}, \langle \mathbb{P}[X_0, X_1] \rangle_{[X_1]} \right\rangle_{[X_0, X_1]} \ .$$

Proof. By Thm. 3.1 we know that marginal probabilities are equivalent to contracted probability distributions, i.e. $\mathbb{P}[X_0] = \langle \{\mathbb{P}\} \rangle_{[X_0]}$. By orthogonality of one-hot encodings we have that

$$\forall x_0 \in [m_0], x_1 \in [m_1] : \mathbb{P}[X_0 = x_0, X_1 = x_1] = \mathbb{P}[X_0 = x_0] \cdot \mathbb{P}[X_1 = x_1]$$

is equivalent to

$$\begin{aligned} &\sum_{x_0 \in [m_0]} \sum_{x_1 \in [m_1]} \mathbb{P}[X_0 = x_0, X_1 = x_1] \cdot \epsilon_{x_0}[X_0]\, \epsilon_{x_1}[X_1] \\ &= \sum_{x_0 \in [m_0]} \sum_{x_1 \in [m_1]} \mathbb{P}[X_0 = x_0] \cdot \mathbb{P}[X_1 = x_1] \cdot \epsilon_{x_0}[X_0]\, \epsilon_{x_1}[X_1] \ . \end{aligned}$$

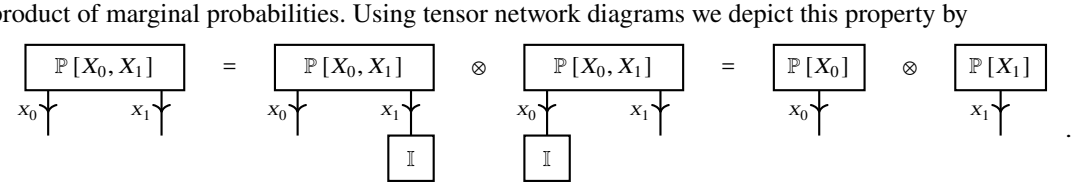
We reorder the summations and arrive at

$$\begin{aligned} &\sum_{x_0 \in [m_0]} \sum_{x_1 \in [m_1]} \mathbb{P}[X_0 = x_0, X_1 = x_1] \cdot \epsilon_{x_0, x_1}[X_0, X_1] \\ &= \left(\sum_{x_0 \in [m_0]} \mathbb{P}[X_0 = x_0]\, \epsilon_{x_0}[X_0] \right) \cdot \left(\sum_{x_1 \in [m_1]} \mathbb{P}[X_1 = x_1] \cdot \epsilon_{x_1}[X_1] \right) \end{aligned}$$

which is by Lem. **??** equal to the claim

$$\mathbb{P}[X_0, X_1] = \left\langle \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0]}, \langle \mathbb{P}[X_0, X_1] \rangle_{[X_1]} \right\rangle_{[X_0, X_1]} \ . \square$$

Two jointly distributed variables are by Thm. 3.6 independent, if and only if their joint distribution $\mathbb{P}[X_0, X_1]$ is the tensor product of marginal probabilities. Using tensor network diagrams we depict this property by



Let us notice, that the assumption of independence reduces the degrees of freedom from $m_0 \cdot m_1 - 1$ to $(m_0 - 1) + (m_1 - 1)$. The decomposition into marginal distributions furthermore exploits this reduced freedom and provides an efficient

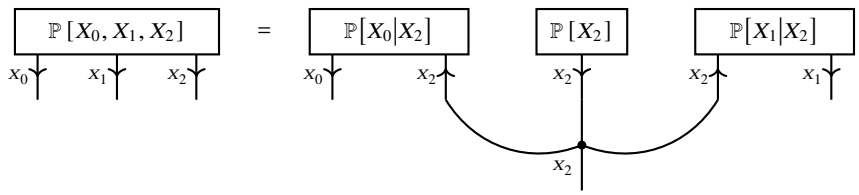


Fig. 3.3 Diagrammatic visualization of the contraction equation in Cor. 3.1. Conditional independence of X_0 and X_1 given X_2 holds if the contraction on the right side is equal to the probability tensor on the left side.

storage. Having a joint distribution of multiple variables, which disjoint subsets are independent, we can iteratively apply the decomposition scheme. As a result, the degrees of freedom scaling exponential in the number of distributed variables would be reduced to a linear scaling, by the assumption of independence.

Independence is, as we observed, a strong assumption, which is often too restrictive. It is furthermore an undesired property, when in a supervised learning scenario a target variable has to be predicted based on known feature variables. Conditional independence instead is a less demanding assumption, which still implies efficient tensor network decompositions schemes. We introduce conditional independence as independence of variables with respect to conditional distributions.

Definition 3.7 (Conditional Independence) Given a joint distribution of variables X_0, X_1 and X_2, such that $\mathbb{P}[X_2]$ is positive. We say that X_0 is independent of X_1 conditioned on X_2 if for any states $x_0 \in [m_0], x_1 \in [m_1]$ and $x_2 \in [m_2]$

$$\mathbb{P}\big[X_0 = x_0, X_1 = x_1 \big| X_2 = x_2\big] = \mathbb{P}\big[X_0 = x_0 \big| X_2 = x_2\big] \cdot \mathbb{P}\big[X_1 = x_1 \big| X_2 = x_2\big] .$$

In this case we denote $(X_0 \perp X_1) \mid X_2$.

Conditional independence stated in Def. 3.7 has a close connection with independence stated in Def. 3.6. To be more precise, X_0 is independent of X_1 conditioned on X_2, if and only if X_0 is independent of X_1 with respect to any slice $\mathbb{P}\big[X_0, X_1 \big| X_2 = x_2\big]$ of the conditional distribution $\mathbb{P}\big[X_0, X_1 \big| X_2\big]$. Analogously to Thm. 3.6 for independence, we further find a decomposition criterion for conditional independence. Since conditional independence can be regarded as a property of conditional probabilities, this decomposition criterion also involves conditional probabilities.

Theorem 3.7 (Conditional Independence as a Contraction Equation) *Given a distribution $\mathbb{P}$ of variables X_0, X_1 and X_2, the variable X_0 is independent of X_1 conditioned on X_2, if and only if the equation*

$$\mathbb{P}\big[X_0, X_1 \big| X_2\big] = \big\langle \mathbb{P}\big[X_0 \big| X_2\big], \mathbb{P}\big[X_1 \big| X_2\big] \big\rangle_{[X_0, X_1, X_2]}$$

holds.

Proof. With the same argumentation as in the proof of Thm. 3.6, we notice that the contraction equation holds, if and only if for any $x_0 \in [m_0]$, $x_1 \in [m_1]$ and $x_2 \in [m_2]$

$$\mathbb{P}\big[X_0 = x_0, X_1 = x_1 \big| X_2 = x_2\big] = \mathbb{P}\big[X_0 = x_0 \big| X_2 = x_2\big] \cdot \mathbb{P}\big[X_1 = x_1 \big| X_2 = x_2\big] .$$

This is equivalent to conditional independence by Def. 3.7. □

We can further exploit conditional independence to find tensor network decompositions of probabilities, as we show as the next corollary.

Corollary 3.1 *If and only if X_0 is independent of X_1 conditioned on X_2 the probability distribution $\mathbb{P}$ satisfies (see Figure 3.3)*

$$\mathbb{P}[X_0, X_1, X_2] = \big\langle \mathbb{P}\big[X_0 \big| X_2\big], \mathbb{P}\big[X_1 \big| X_2\big], \mathbb{P}[X_2] \big\rangle_{[X_0, X_1, X_2]} .$$

Proof. With the Bayes Thm. 3.4 it holds that

$$\mathbb{P}[X_0, X_1, X_2] = \big\langle \mathbb{P}\big[X_0, X_1 \big| X_2\big], \mathbb{P}[X_2] \big\rangle_{[X_0, X_1, X_2]} .$$

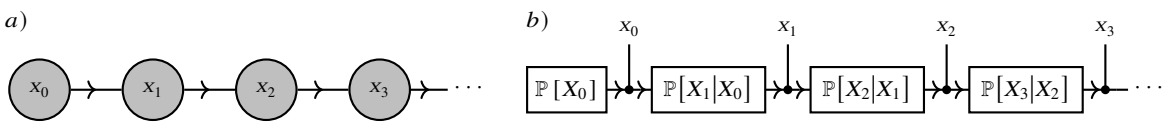


Fig. 3.4 Depiction of a Markov Chain Decomposition by a a) hypergraph with the nodes $\mathcal{V} = [d]$ and edges $\mathcal{E} = \big\{\{0\} \cup \{\{k, k+1\} : k \in [d],\ k > 1\}\big\}$ and b) a decorating Tensor Network representing the sparsified conditional probabilities.

Decomposing the first tensor in the contraction, Thm. 3.7 implies, that X_0 is independent of X_1 conditioned on X_2, if and only if

$$\mathbb{P}[X_0, X_1, X_2] = \left\langle \mathbb{P}\big[X_0\big|X_2\big], \mathbb{P}\big[X_1\big|X_2\big], \mathbb{P}[X_2] \right\rangle_{[X_0, X_1, X_2]} . \square$$

Let us now recall our motivation of the study of conditional independence, namely to find sparsifications of conditional probabilities as those appearing in chain decompositions Thm. 3.5. As we state as the next theorem, such sparsifications follow from conditional independence.

Theorem 3.8 *Whenever X_0 is independent of X_1 given X_2, we have for any $x_1 \in [m_1]$*

$$\mathbb{P}\big[X_0\big|X_1 = x_1, X_2\big] = \mathbb{P}\big[X_0\big|X_2\big] .$$

Proof. By the Bayes Thm. 3.4 we have for any indices to the variables

$$\mathbb{P}\big[X_0 = x_0\big|X_1 = x_1, X_2 = x_2\big] = \frac{\mathbb{P}\big[X_0 = x_0, X_1 = x_1\big|X_2 = x_2\big]}{\left\langle \mathbb{P}\big[X_0, X_1 = x_1\big|X_2 = x_2\big] \right\rangle_{[\varnothing]}}$$

If X_0 is independent of X_1 given X_2 it follows that

$$\begin{aligned}\mathbb{P}\big[X_0 = x_0\big|X_1 = x_1, X_2 = x_2\big] &= \frac{\mathbb{P}\big[X_0 = x_0\big|X_2 = x_2\big] \cdot \mathbb{P}\big[X_1 = x_1\big|X_2 = x_2\big]}{\left\langle \mathbb{P}\big[X_0, X_1 = x_1\big|X_2 = x_2\big] \right\rangle_{[\varnothing]}} \\ &= \mathbb{P}\big[X_0 = x_0\big|X_2 = x_2\big] . \square\end{aligned}$$

Following our motivation of sparse decompositions, we now combine this result with the generic chain rule, to show Markov Chain decompositions.

Theorem 3.9 (Markov Chain) *Let there be a set of variables X_k where $k \in [d]$, and let us denote for $k \in [d]$ by $X_{[k]}$ the collection of variables $X_0, \ldots, X_{k-1}$. Let us assume, that for any $k \in [d]$ with $k \geq 2$ the variable X_k is independent of $X_{[k-1]}$ conditioned on X_{k-1}, then*

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}[X_0]\} \cup \{\mathbb{P}\big[X_k\big|X_{k-1}\big] \ : \ k \in [d],\ k \geq 1\} \right\rangle_{[X_{[d]}]}$$

We depict this decomposition in Figure 3.4.

Proof. By the chain rule shown in Thm. 3.5 we have

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}\big[X_k\big|X_{[k]}\big] \ : \ k \in [d]\} \right\rangle_{[X_{[d]}]}$$

Using that X_k is conditionally independent of $X_{[k-1]}$ conditioned on X_{k-1} we further have by Thm. 3.8

$$\mathbb{P}\big[X_k\big|X_{[k]}\big] = \mathbb{P}\big[X_k\big|X_{k-1}\big] \otimes \mathbb{I}\left[X_{[k-1]}\right] .$$

Composing both equalities and omitting the trivial tensors shows the claim. $\square$

The assumption of X_k being independent of $X_{[k-1]}$ conditioned on X_{k-1} is called the Markov property and the corresponding collection of random variables is called a Markov Chain. Thm. 3.9 states an efficient decomposition of

the probability distribution into a concatenated product of matrices representing conditional probability distributions. Marginal distributions of Markov Chains can therefore consecutively be computed by matrix-vector products, that is for $k \in [d]$ with $k \geq 1$

$$\mathbb{P}\left[X_k\right] = \left\langle \mathbb{P}\left[X_k \middle| X_{k-1}\right], \mathbb{P}\left[X_{k-1}\right] \right\rangle_{[X_k]} .$$

The conditional probability matrices are therefore called stochastic transition matrices.

We notice that the decomposition scheme of Thm. 3.9 hints at an efficient representation of $\mathbb{P}\left[X_{[d]}\right]$ based on transition matrices. While $\mathbb{P}\left[X_{[d]}\right]$ is a tensor in a space of dimension

$$\prod_{k \in [d]} m_k ,$$

the sum of the dimension of the transition matrices is

$$m_0 + \sum_{k \in [d] : k \geq 1} m_k \cdot m_{k-1} .$$

We therefore observe a linear increase of the storage demand of the transition matrices in the order d, whereas a naive storage of $\mathbb{P}\left[X_{[d]}\right]$ by its coordinates would have an exponentially demand.

The Markov Chain serves as a toy example drawing on a restrictive chain arrangement of conditional independencies. In the following section, we will investigate decomposition schemes, which relax this assumption and draw on more general collections of conditional independencies. The computation of marginal distribution by consecutive transition matrix multiplications will then be replaced by more general tensor network contractions.

3.3 Graphical Models

Graphical models are specific instances of exponential families [**?**, **?**]. They combine both the independence approach and the computation approach to tensor network representations of probability distributions. Graphical models provide a generic framework to relate conditional dependency assumptions on a distribution with tensor network decompositions. Following the tensor network formalism we in this section introduce graphical models based on hypergraphs. First, we study Markov Networks in most generality and then connect with conditional probabilities in the discussion of Bayesian Networks.

3.3.1 Markov Networks

We now define Markov Networks based on hypergraphs, to establish a direct connection with tensor network decorating the hypergraph. In a more canonical way, Markov Networks are instead defined by graphs, where instead of the edges the cliques are decorated by factor tensors (see for example [**?**]).

Definition 3.8 (Markov Network) Let $\tau^{\mathcal{G}}$ be a tensor network of non-negative tensors decorating a hypergraph $\mathcal{G}$. Then the Markov Network $\mathbb{P}^{\mathcal{G}}$ to $\tau^{\mathcal{G}}$ is the probability distribution of X_v defined by the tensor

$$\mathbb{P}^{\mathcal{G}}\left[X_{\mathcal{V}}\right] = \frac{\langle \{\tau^e \,:\, e \in \mathcal{E}\} \rangle_{[X_{\mathcal{V}}]}}{\langle \{\tau^e \,:\, e \in \mathcal{E}\} \rangle_{[\varnothing]}} = \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{V}} | \varnothing]} .$$

We call the denominator

$$\mathcal{Z}\left(\tau^{\mathcal{G}}\right) = \langle \{\tau^e \,:\, e \in \mathcal{E}\} \rangle_{[\varnothing]}$$

the partition function of the tensor network $\tau^{\mathcal{G}}$.

The marginalization of a Markov Network to $\tau^{\mathcal{G}}$ on subsets of variables $X_{\mathcal{U}}$ is

$$\mathbb{P}^{\mathcal{G}}\left[X_{\mathcal{U}}\right] = \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}|\varnothing]} .$$

This can be derived from Thm. **??**, which established an equivalence of contractions with sequences of consecutive contractions.

Further, the distribution of $X_{\mathcal{U}}$ conditioned on $X_{\mathcal{W}}$, where $\mathcal{U}, \mathcal{W}$ are disjoint subsets of $\mathcal{V}$, is

$$\mathbb{P}^{\mathcal{G}}\left[X_{\mathcal{U}}|X_{\mathcal{W}}\right] = \left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}|X_{\mathcal{W}}]} .$$

While we have directly defined Markov Networks as decomposed probability distributions, we now want to derive assumptions on a distribution assuring that such decompositions exist. As we will see, the sets of conditional independencies encoded by a hypergraph are captured by its seperation properties, as we define next.

Definition 3.9 (Separation of Hypergraph) A path in a hypergraph is a sequence of nodes v_k for $k \in [d]$, such that for any $k \in [d-1]$ we find a hyperedge $e \in \mathcal{E}$ such that $(v_k, v_{k+1}) \subset e$. Given disjoint subsets A, B, C of nodes in a hypergraph $\mathcal{G}$ we say that C separates A and B with respect to $\mathcal{G}$, when any path starting at a node in A and ending in a node in B contains a node in C.

To characterize Markov Networks in terms of conditional independencies we need to further define the property of clique-capturing. This property of clique-capturing established a correspondence of hyperedges with maximal cliques in the more canonical graph-based definition of Markov Networks [**?**].

Definition 3.10 (Clique-Capturing Hypergraph) We call a hypergraph $\mathcal{G}$ clique-capturing, when each subset $\mathcal{U} \subset \mathcal{V}$ is contained in a hyperedge, if for any $a, b \in \mathcal{U}$ there is a hyperedge $e \in \mathcal{E}$ with $a, b \in e$.

Let us now show a characterization of Markov Networks in terms of conditional independencies, which is analogous to Thm. 3.11.

Theorem 3.10 (Hammersley-Clifford) *Given a clique-capturing hypergraph $\mathcal{G}$, the set of positive Markov Networks on the hypergraph coincides with the set of positive probability distributions, such that each for each disjoint subsets of variables A, B, C we have X_A is independent of X_B conditioned on X_C, when C separates A and B in the hypergraph.*

Proof. "$\Rightarrow$": Let there be a hypergraph $\mathcal{G}$, a Markov Network $\tau^{\mathcal{G}}$ on $\mathcal{G}$ and nodes $A, B, C \subset \mathcal{V}$, such that C separates A from B. Let us denote by $\mathcal{V}_0$ the nodes with paths to A, which do not contain a node in C, and by $\mathcal{V}_1$ the nodes with paths to B, which do not contain a node in C. Further, we denote by $\mathcal{E}_0$ the hyperedges which contain a node in $\mathcal{V}_0$ and by $\mathcal{E}_1$ the hyperedges which contain a node in $\mathcal{V}_1$. By assumption of separability, both sets $\mathcal{E}_0$ and $\mathcal{E}_1$ are disjoint and no node in A is in a hyperedge in $\mathcal{E}_1$, respectively no node in B is in a hyperedge in $\mathcal{E}_0$, . We then have

$$\begin{aligned}
\langle\{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\}\rangle_{[X_A, X_B|X_C = x_C]} &= \left\langle \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\} \cup \{\epsilon_{x_C}\} \right\rangle_{[X_A, X_B|\varnothing]} \\
&= \left\langle \{\tau^e \,:\, e \in \mathcal{E}_0\} \cup \{\epsilon_{x_C}\} \right\rangle_{[X_A|\varnothing]} \\
&\quad \otimes \left\langle \{\tau^e \,:\, e \in \mathcal{E}_1\} \cup \{\epsilon_{x_C}\} \right\rangle_{[X_B|\varnothing]} .
\end{aligned}$$

By Thm. 3.7, it now follows that X_A is independent of X_B conditioned on X_C.

"$\Leftarrow$": The converse direction, i.e. that positive distributions respecting the conditional independence assumptions are representable as Markov Networks, is known as the Hammersley-Clifford Theorem (see [**?**]), which we will proof later in Sect. **??** of Chapter **??**. □

From the proof of Thm. 3.10 Markov Networks with zero coordinates still satisfy the conditional independence assumption. However, the reverse is not true, that is there are distributions with vanishing coordinates, which satisfy the conditional independence assumptions, but cannot be represented as a Markov Network (see Example 4.4 in [**?**]).

Example 3.1 (Independent boolean variables) Let there be d boolean variables $X_{[d]}$, which are i.i.d. drawn from a positive distribution $\mathbb{P}\left[X\right]$. From the pairwise independencies of X_k it follows with the Hammersley-Clifford Factorization Thm. **??** that the distribution is representable by an elementary tensor network, that is

$$\mathbb{P}\left[X_{[d]}\right] = \bigotimes_{k\in[d]} \mathbb{P}^k\left[X_k\right] .$$

The corresponding hypergraph is the elementary graph, with respect to which any two disjoint subsets of nodes are separated (see Figure 3.5).

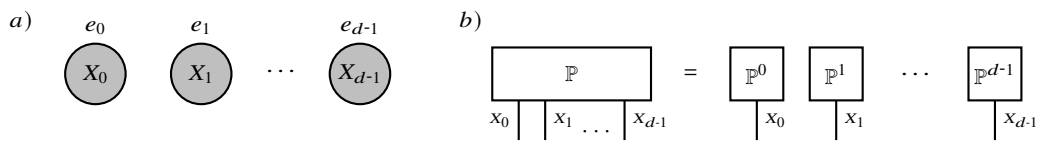


Fig. 3.5 Decomposition of a probability distribution with independent variables (see Example 3.1). The independencies are captured by the elementary hypergraph a), whose edges contain single nodes. The corresponding tensor $\mathbb{P}\left[X_{[d]}\right]$ is then represented by a Markov network on the elementary hypergraph, where each factor is the marginal distribution of the corresponding variable as visualized in b).

Example 3.2 We consider a classical example of a graphical model (see [**?**, Example 4.3]): A student of intelligence (X_I) and SAT score (X_S) is assigned a test of difficulty (X_D), for which he gets a grade (X_G) depending on which he gets a recommendation letter (X_L) by its teacher. We make the following modelling assumptions:

- "The SAT score depends only on the students intelligence": $\left(X_S \perp X_{\{D,G,L\}}\right) \mid X_I$.
- "The recommendation letter depends only on the grade": $\left(X_L \perp X_{\{D,I,S\}}\right) \mid X_G$.

The associated hypergraph capturing these conditional independencies is drawn in Figure 3.6 a).

3.3.2 Bayesian Networks

Compared to Markov Networks, Bayesian Networks impose further conditions on tensor networks representing a distribution. They assume a directed hypergraph and each tensor decorating the edges to be normalized according to the direction. We will observe, that if the hypergraph is in addition acyclic, then each tensor core coincides with the conditional distribution of the underlying Markov Network. To introduce Bayesian Networks, we extend Def. **??** by introducing the property of acyclicity for hypergraphs.

Definition 3.11 A directed path is a sequence $v_0, \dots v_r$ such that for any $l \in [r]$ there is an hyperedge $e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E}$ such that $v_l \in e^{\text{in}}$ and $v_{l+1} \in e^{\text{out}}$. We call the hypergraph $\mathcal{G}$ acyclic, if there is no path with $r > 0$ such that $v_0 = v_r$. Given a directed hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ we define for any node $v \in \mathcal{V}$ its parents by

$$\text{Pa}(v) = \left\{\tilde{v} \,:\, \left(\exists e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} : \tilde{v} \in e^{\text{in}},\, v \in e^{\text{out}}\right)\right\}$$

and its non-descendants $\text{NonDes}(v)$ as the set of nodes $\tilde{v}$, such that there is no directed path from v to $\tilde{v}$.

Based on these additional graphical properties, we now define Bayesian Networks.

Definition 3.12 (Bayesian Network) Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed acyclic hypergraph with edges of the form

$$\mathcal{E} = \{(\text{Pa}(v), \{v\}) \,:\, v \in \mathcal{V}\} .$$

A *Bayesian Network* is a decoration of each edge $(\text{Pa}(v), \{v\})$ by a conditional probability distribution

$$\mathbb{P}\left[X_v \middle| X_{\text{Pa}(v)}\right]$$

which represents the probability distribution

$$\mathbb{P}\left[X_{\mathcal{V}}\right] = \left\langle \{\mathbb{P}\left[X_v \middle| X_{\text{Pa}(v)}\right] \,:\, v \in \mathcal{V}\}\right\rangle_{[X_{\mathcal{V}}]} .$$

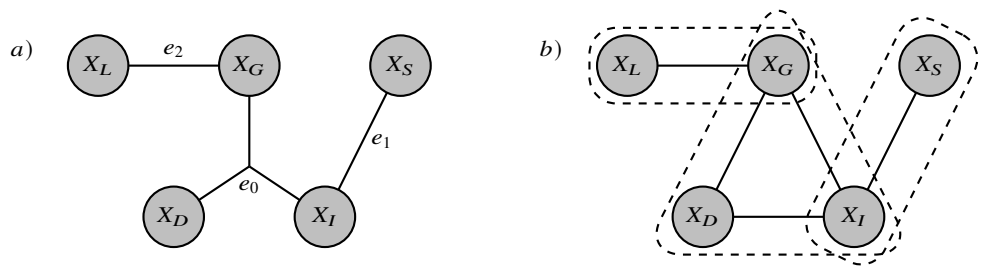


Fig. 3.6 Hypergraph a) capturing the conditional independencies of the student example. The cliques of the node adjacency graph are highlighted in b) and coincide with hyperedges of the hypergraph. The hypergraph is therefore clique-capturing (see Def. 3.10).

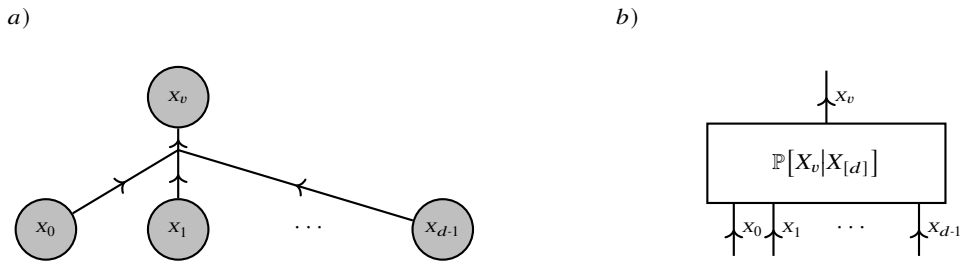


Fig. 3.7 Example of a Factor of a Bayesian Network to the node X_v with parents $X_0, \ldots, X_{d-1}$, as an directed edge a) which is decorated by a directed tensor b).

By definition each tensor decorating a hyperedge is directed with $X_{\mathrm{Pa}(v)}$ incoming and X_v outgoing. Thus, the directionality of the hypergraph is reflected in each tensor decorating a directed hyperedge. This allows us to verify with Thm. **??** that their contraction defines a probability distribution.

Marginalization of a Bayesian Network are still Bayesian Networks on a graph where the edges directing to variables, which are not marginalized over, are replaced by directed edges to the children. Conditioned Bayesian Network do not have a simple Bayesian Network representation, which is why we will treat them as Markov Networks to be introduced next.

Theorem 3.11 (Independence Characterization of Bayesian Networks) *A probability distribution $\mathbb{P}[X_{\mathcal{V}}]$ has a representation by a Bayesian Network on a directed acyclic graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, if and only if for any $v \in \mathcal{V}$ the variables X_v are independent on* $\mathrm{NonDes}(v)$ *conditioned on* $\mathrm{Pa}(v)$.

Proof. We choose a topological order $\prec$ on the nodes of $\mathcal{G}$, which exists since $\mathcal{G}$ is acyclic.

"$\Rightarrow$": Let us assume, that the conditional independencies are satisfied and apply the chain rule with respect to that ordering to get

$$\mathbb{P}[X_{\mathcal{V}}] = \left\langle \mathbb{P}\left[X_v \middle| X_{\tilde{v}} : \tilde{v} \prec v\right] \right\rangle_{[X_{\mathcal{V}}]} .$$

Since $\prec$ is a topological ordering we have

$$\mathrm{Pa}(v) \subset \{\tilde{v} : \tilde{v} \prec v\}$$

We apply the assumed conditional independence with Thm. 3.8 and get

$$\mathbb{P}[X_{\mathcal{V}}] = \left\langle \mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right] \right\rangle_{[X_{\mathcal{V}}]} .$$

"$\Leftarrow$": To show the converse direction, let there be a Bayesian Network $\mathbb{P}[X_{\mathcal{V}}]$ on $\mathcal{G}$. To show for any node v, that X_v is independent of $\mathrm{NonDes}(v)$ conditioned on $\mathrm{Pa}(v)$, we reorder the tensors in the contraction

$$
\begin{aligned}
&\mathbb{P}\left[X_v, X_{\mathrm{NonDes}(v)} \middle| X_{\mathrm{Pa}(v)} = x_{\mathrm{Pa}(v)}\right] \\
&\quad = \left\langle \{\mathbb{P}\left[X_{\tilde{v}} \middle| X_{\mathrm{Pa}(\tilde{v})}\right] \,:\, \tilde{v} \in \mathcal{V}\} \right\rangle_{\left[X_v, X_{\mathrm{NonDes}(v)} | X_{\mathrm{Pa}(v)} = x_{\mathrm{Pa}(v)}\right]} \\
&\quad = \left\langle \{\mathbb{P}\left[X_{\tilde{v}} \middle| X_{\mathrm{Pa}(\tilde{v})}\right] \,:\, \tilde{v} \in \mathcal{V}\} \cup \{\epsilon_{x_{\mathrm{Pa}(v)}}\} \right\rangle_{\left[X_v, X_{\mathrm{NonDes}(v)} | \varnothing\right]} \\
&\quad = \left\langle \{\mathbb{P}\left[X_{\tilde{v}} \middle| X_{\mathrm{Pa}(\tilde{v})}\right] \,:\, \tilde{v} \in \mathrm{NonDes}(v)\} \cup \{\epsilon_{x_{\mathrm{Pa}(v)}}, \mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right]\} \right\rangle_{\left[X_v, X_{\mathrm{NonDes}(v)} | \varnothing\right]} \\
&\quad = \left\langle \{\mathbb{P}\left[X_{\tilde{v}} \middle| X_{\mathrm{Pa}(\tilde{v})}\right] \,:\, \tilde{v} \in \mathrm{NonDes}(v)\} \cup \{\epsilon_{x_{\mathrm{Pa}(v)}}\} \right\rangle_{\left[X_{\mathrm{NonDes}(v)} | \varnothing\right]} \\
&\qquad \cdot \left\langle \{\mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right], \epsilon_{x_{\mathrm{Pa}(v)}}\} \right\rangle_{\left[X_v | \varnothing\right]} \\
&\quad = \left\langle \{\mathbb{P}\left[X_{\mathrm{NonDes}(v)} \middle| X_{\mathrm{Pa}(v)} = x_{\mathrm{Pa}(v)}\right], \mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)} = x_{\mathrm{Pa}(v)}\right]\} \right\rangle_{\left[X_v, X_{\mathrm{NonDes}(v)}\right]}
\end{aligned}
$$

Here we have dropped in the third equation all tensors to the descendants, since their marginalization is trivial (which can be shown by a leaf-stripping argument). In the fourth equation we made use of the fact, that any directed path between the non-descendants and the node is through the parents of the node. By Thm. 3.7, it now follows that X_v is independent of $\mathrm{NonDes}(v)$ conditioned on $\mathrm{Pa}(v)$. □

3.3.3 Bayesian Networks as Markov Networks

Markov Networks are more flexible compared with Bayesian Networks, since any Bayesian Network is a Markov Network by ignoring the directionality of the hypergraph and understanding the conditional distributions as generic tensor cores. In the next theorem we provide the conditions for the interpretation of a Markov Network as a Bayesian Network.

Theorem 3.12 *Let $\tau^{\mathcal{G}}$ be a tensor network on a directed acyclic hypergraph, such that the edges are of the structure*

$$\mathcal{E} = \{(\mathrm{Pa}(v), \{v\}) \,:\, v \in \mathcal{V}\}$$

and each tensor τ^e respects the directionality of the graph, that is each $\tau^{(\mathrm{Pa}(v),\{v\})}$ is directed with the variables to $\mathrm{Pa}(v)$ incoming and v outgoing. Then $\mathcal{Z}\left(\tau^{\mathcal{G}}\right) = 1$ and for each $v \in \mathcal{V}$ we have

$$\tau^{(\mathrm{Pa}(v),\{v\})} = \left\langle \tau^{\mathcal{G}} \right\rangle_{\left[X_v | X_{\mathrm{Pa}(v)}\right]} .$$

In particular, $\tau^{\mathcal{G}}$ is a Bayesian Network.

Proof. We show the claim by induction over the cardinality of $\mathcal{V}$.

$|\mathcal{V}| = 1$: In this case we find a unique node $v \in \mathcal{V}$ and have $\mathcal{E} = \{(\varnothing, \{v\})\}$. The tensor $\tau^{(\varnothing,\{v\})}$ is then normalized with no incoming variables and we thus have

$$\mathcal{Z}\left(\tau^{\mathcal{G}}\right) = \left\langle \tau^{\mathcal{G}} \right\rangle_{[\varnothing]} = \left\langle \tau^{(\varnothing,\{v\})} \right\rangle_{[\varnothing]} = 1$$

and

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_v | \varnothing]} = \tau^{(\varnothing,\{v\})} .$$

$|\mathcal{V}| - 1 \longrightarrow |\mathcal{V}|$: Let there now be a directed hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and let us now assume, that the theorem holds for any tensor networks with node cardinality $|\mathcal{V}| - 1$. Since the hypergraph is acyclic, we find a root $v \in \mathcal{V}$ such that $v \notin \mathrm{Pa}(\tilde{v})$ for $\tilde{v} \in \mathcal{V}$. We denote $\tau^{\tilde{\mathcal{G}}}$ the tensor network on the hypergraph $\tilde{\mathcal{G}} = \{\mathcal{V}/\{v\}, \mathcal{E}/\{(\mathrm{Pa}(v), \{v\})\}\}$ with decorations inherited from $\tau^{\mathcal{G}}$. With Thm. **??**, the directionality of $\tau^{(\mathrm{Pa}(v),\{v\})}$ and the induction assumption on $\tau^{\tilde{\mathcal{G}}}$ we have

$$\begin{aligned}\left\langle \tau^{\tilde{\mathcal{G}}} \cup \left\{\tau^{(\mathrm{Pa}(v),\{v\})}\right\}\right\rangle_{[\varnothing]} &= \left\langle \tau^{\tilde{\mathcal{G}}} \cup \left\{\left\langle \tau^{(\mathrm{Pa}(v),\{v\})}\right\rangle_{\left[X_{\mathrm{Pa}(v)}\right]}\right\}\right\rangle_{[\varnothing]} \\ &= \left\langle \tau^{\tilde{\mathcal{G}}} \cup \left\{\mathbb{I}\left[X_{\mathrm{Pa}(v)}\right]\right\}\right\rangle_{[\varnothing]} \\ &= 1\end{aligned}$$

and thus a trivial partition function. Since v does not appear in $\tilde{\mathcal{G}}$, we have for any index $x_{\mathrm{Pa}(\mathcal{V})}$

$$\left\langle \tau^{\mathcal{G}}\right\rangle_{\left[X_v, X_{\mathrm{Pa}(v)}=x_{\mathrm{Pa}(v)}\right]} = \left\langle \tau^{(\mathrm{Pa}(v),\{v\})}\right\rangle_{\left[X_v, X_{\mathrm{Pa}(v)}=x_{\mathrm{Pa}(v)}\right]} \cdot \left\langle \tau^{\tilde{\mathcal{G}}}\right\rangle_{\left[X_{\mathrm{Pa}(v)}=x_{\mathrm{Pa}(v)}\right]}$$

and thus, since $\tau^{(\mathrm{Pa}(v),\{v\})}$ is directed, that

$$\left\langle \tau^{\mathcal{G}}\right\rangle_{\left[X_v | X_{\mathrm{Pa}(v)}\right]} = \tau^{(\mathrm{Pa}(v),\{v\})} . \square$$

Thm. 3.12 states that Bayesian Networks are a subset of Markov Networks. While Markov Network allow generic tensor cores, Bayesian Networks impose a local directionality condition on each tensor core by demanding it to be a conditional probability tensor. In our diagrammatic notation, the local normalization of Bayesian Networks is highlighted by the directionality of the hypergraph. Generic Markov Networks are on undirected hypergraphs, where in general no local directionality condition is assumed. As a consequence, tasks such as the determination of the partition functions or calculation of conditional distributions involve global contractions.

Example 3.3 (Hidden Markov Models) Hidden Markov Models are examples of Bayesian Networks, constructed as follows. Let us recall Markov Chains as investigated in Thm. 3.9 and extend them by observation variables E_k for $k \in [d]$, representing limited observations of the state variables X_k. To be more precise, we assume the following conditional independencies:

- As for Markov Chains, we assume that for $k \in [d]$ with $k \geq 1$ the variable X_k is independent of $X_{[k-1}$ and $E_{[k-1]}$ conditioned on X_{k-1}
- In addition, for we assume that for $k \in [d]$ the observation variable E_k is independent of $X_{[k}$ and $E_{[k]}$ conditioned on X_k

From this conditional independence assumption, we apply the Chain Rule Thm. 3.5 given the order of variables

$$X_0, E_0, X_1, E_1, \ldots, X_{d-1}, E_{d-1}$$

and get

$$\begin{aligned}\mathbb{P}\left[X_{[d]}, E_{[d]}\right] = \Big\langle &\{\mathbb{P}\left[X_0\right], \mathbb{P}\left[E_0 \middle| X_0\right]\} \\ &\cup \{\mathbb{P}\left[X_k \middle| X_{[k]}, E_{[d]}\right] \,:\, k \in [d]\} \\ &\cup \{\mathbb{P}\left[E_k \middle| X_{[k+1]}, E_{[d]}\right] \,:\, k \in [d]\}\Big\rangle_{\left[X_{[d]}, E_{[d]}\right]} .\end{aligned}$$

We now apply the conditional independence assumptions to sparsify the appearing conditional distributions by application of Thm. 3.8. This results in the decomposition (see Figure 3.8b)

$$\begin{aligned}\mathbb{P}\left[X_{[d]}, E_{[d]}\right] = \Big\langle &\{\mathbb{P}\left[X_0\right], \mathbb{P}\left[E_0 \middle| X_0\right]\} \\ &\cup \{\mathbb{P}\left[X_k \middle| X_{k-1}\right] \,:\, k \in [d]\} \\ &\cup \{\mathbb{P}\left[E_k \middle| X_k\right] \,:\, k \in [d]\}\Big\rangle_{\left[X_{[d]}, E_{[d]}\right]} .\end{aligned}$$

In addition to the stochastic transition matrices $\mathbb{P}\left[X_k \middle| X_{k-1}\right]$ appearing in Markov Chains, we further have stochastic observation matrices $\mathbb{P}\left[E_k \middle| X_k\right]$ for $k \in [d]$. Their contraction with marginal distribution of the respective state varibles delivers the marginal distribution of the observation matrix by

$$\mathbb{P}\left[E_k\right] = \left\langle \mathbb{P}\left[E_k \middle| X_k\right], \mathbb{P}\left[X_k\right]\right\rangle_{\left[E_k\right]}$$

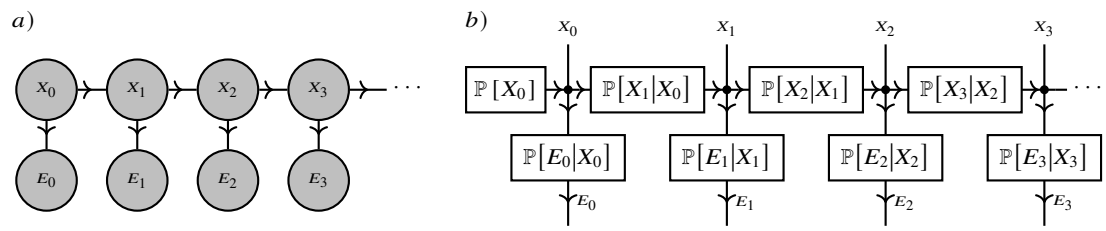


Fig. 3.8 Decomposition of a probability distribution in the Hidden Markov Model, consistent of state variables X_k and observation variables E_k. Given the models conditional independence assumptions, the distribution is a Bayesian Network on the directed hypergraph a). The hypergraph is decorated by the network of conditional probability tensors b), which are interpreted as stochastic transition matrices $\mathbb{P}\left[X_k \middle| X_{k-1}\right]$ and stochastic observation matrices $\mathbb{P}\left[E_k \middle| X_k\right]$.

We notice that this is a Bayesian Network on a directed acyclic hypergraph $\mathcal{G}$ (see Figure 3.8a) consistent in nodes $\{X_{[d]}\} \cup \{E_{[d]}\}$ to each state and observation variables, and the directed hyperedges by

- $(\emptyset, \{X_0\})$, decorated by the intial marginal distribution of X_0
- $(\{X_{k-1}\}, \{X_k\})$ for $k \in [d]$ with $k \geq 1$, decorated by stochastic transition matrices
- $(\{X_k\}, \{E_k\})$ for $k \in [d]$, decorated stochastic observation matrices

While we have derived this directed graph structure directly based on the chain rule decomposition with sparsified conditional distributions, it also follows from the more generic hypergraph characterization of Bayesian Networks through separability by Thm. 3.11.

3.4 Factorization based on sufficient statistics

Missing: Definition of family of distributions

Let us now introduce sufficient statistics towards studying further tensor network decompositions of probability distributions.

Definition 3.13 Let $\mathbb{P}[X, Z]$ be a joint distribution of the m-dimensional variable X and the n-dimensional variable Z and let

$$t : [m] \to [n]$$

be a statistic. We are interested in the distribution $\mathbb{P}[X, Z, Y_t] = \langle \mathbb{P}[X, Z], \beta^t [Y_t, X] \rangle_{[X,Z,Y_t]}$. We say that t is a sufficient statistic for Z if X is independent of Z conditioned on Y_t.

Example 3.4 (Sufficient statistics for the probability) Let Z be the value $\mathbb{P}\left[X_{[d]} = x_{[d]}\right]$ when drawing $X_{[d]}$ from $\mathbb{P}\left[X_{[d]}\right]$. Then t is a sufficient statistic for $Z = \mathbb{P}\left[X_{[d]}\right]$ if for all y in the image of t we have

$$\mathbb{P}\left[X_{[d]} = x_{[d]} \middle| Y_t = y\right] = \begin{cases} \frac{1}{\left|\{x_{[d]} : t\left(x_{[d]}\right) = y\}\right|} & \text{if} \quad t(x_{[d]}) = y \\ 0 & \text{else} \end{cases} .$$

When knowing the value $t\left(x_{[d]}\right)$ of the sufficient statistic at a given index $x_{[d]}$, we then also know the probability $\mathbb{P}\left[X_{[d]} = x_{[d]}\right]$. The function t is thus a sufficient statistic for $Z = \mathbb{P}\left[X_{[d]}\right]$ if and only if there is a tensor $\xi\left[Y_{[p]}\right]$ with

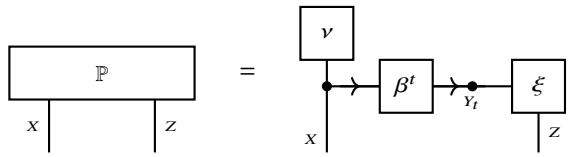
$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \beta^t \left[Y_{[p]}, X_{[d]}\right], \xi\left[Y_{[p]}\right] \right\rangle_{\left[X_{[d]}\right]} .$$

Example 3.4 hints at a connection between sufficient statistics and decompositions into CompActNets. More generally, such decompositions are provided by the Fisher-Neyman Factorization Theorem.

Theorem 3.13 (Fisher-Neyman Factorization Theorem) *Let $\mathbb{P}$ be a joint distribution of variables X, Z with values* $\mathrm{val}(X)$, $\mathrm{val}(Z)$. *Let there further be a finite set* $\mathrm{val}(Y_t)$*, which is enumerated with a variable Y_t. Then $t : \mathrm{val}(X) \to \mathrm{val}(Y_t)$ is a sufficient statistic for Z if and only if there are tensors $\nu[X]$ and $\xi[Y_t, Z]$ such that*

$$\mathbb{P}[X, Z] = \left\langle \xi[Y_t, Z], \beta^t[Y_t, X], \nu[X] \right\rangle_{[X,Z]} .$$

We depict this equation diagrammatically by



Proof. Shown in the appendix, see Thm. **??**. □

Note that the definition of sufficient statistic does not make use of the marginal distribution $\mathbb{P}[Z]$. We therefore can define sufficient statistics also for families of distributions $\mathbb{P}\left[X \middle| Z\right]$ with respect to arbitrary non-degenerate marginal distributions $\mathbb{P}[Z]$. We then use Thm. 3.13 to embed such families in CompActNets.

Corollary 3.2 *Let $\mathbb{P}\left[X_{[d]} \middle| Z\right]$ be an arbitrary family of distributions of $X_{[d]}$ and t a sufficient statistic for Z. Then, there is a tensor $\nu\left[X_{[d]}\right]$ and an activation tensor $\xi\left[Y_{[p]}, Z\right]$ such that for any $z \in \mathrm{val}(Z)$ we have*

$$\mathbb{P}\left[X_{[d]} \middle| Z = z\right] \in \Lambda^{t,\mathrm{MAX},\xi} .$$

The Factorization Theorem of Fisher-Neyman provides the fundamental motivation for the CompActNets architecture. Any decomposition of $\beta^t\left[Y_{[p]}, X_{[d]}\right]$ is called a *computation network* and common to all members of a family with sufficient statistic t. The activation tensor $\xi\left[Y_{[p]}, Z = z\right]$, whose decomposition is called the *activation network*, is specific to each member of the family. We now show in two examples how families of distributions can be represented in CompActNets by sufficient statistics.

Example 3.5 (Order statistic for boolean variables) Let there be d boolean variables $X_{[d]}$ and a family $\mathbb{P}\left[X_{[d]} \middle| Z\right]$ of distributions. The order statistic assigns the ordered tuple to each tuple $x_{[d]}$. The ordered tuple effectively counts the number of 1 coordinates in the tuple $x_{[d]}$, that is the statistic

$$t^+ : \bigtimes_{k \in [d]} [m_k] \to [p] , \quad t^+(x_{[d]}) = |\{k : x_k = 1\}| .$$

When the order statistic is sufficient for Z, the detailed order of the outcomes is uninformative about the member z from which the random variables have been drawn. By the Fisher-Neyman Factorization Thm. 3.13, t^+ is a sufficient statistic if and only if there are tensors $\nu\left[X_{[d]}\right]$ and $\xi[Y_+, \Theta]$ such that for each $\theta \in \Theta$

$$\mathbb{P}\left[X_{[d]} \middle| Z = z\right] = \left\langle \xi[Y_+, Z = z], \beta^{t^+}\left[Y_+, X_{[d]}\right], \nu\left[X_{[d]}\right] \right\rangle_{\left[X_{[d]}\right]} .$$

For each member z of the family, the probability of each sequence $x_{[d]}$ is thus the product of a base measure factor $\nu\left[X_{[d]} = x_{[d]}\right]$ and a factor $\xi\left[Y_+ = +(x_{[d]}), Z = z\right]$ depending only on the count $+(x_{[d]})$ of 1 coordinates in $x_{[d]}$. We later continue this example in Example 8.1, where further interpretations to the case of i.i.d. variables are provided.

Example 3.6 (Graphical models as a special case of CompActNets) For graphical models we take the *identity statistic*

$$\delta\left(x_{[d]}\right) = x_{[d]}$$

so that the image coordinates coincide with the variables and there are no non-trivial computation cores. The associated basis encoding is just the identity tensor

$$\beta^\delta\left[Y_{[d]}, X_{[d]}\right] = \delta\left[X_{[d]}, Y_{[d]}\right] .$$

Therefore, for any activation tensor $\xi\left[Y_{[p]}\right]$ we obtain

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \xi\left[Y_{[p]}\right], \beta^{\delta}\left[Y_{[d]}, X_{[d]}\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]} = \left\langle \xi\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]} .$$

Put differently, in the graphical–model case the activation tensor coincides with the joint distribution tensor. In this setting, structural properties of the distribution such as (conditional) independences can be read off as algebraic factorization patterns of the activation (and hence joint) tensor.

3.5 Efficient contractions by message passing

Contractions of tensor networks are generally hard to solve. Here, we investigate message passing algorithms, which decompose global contractions into a sequence of local contractions, whose results are passed as messages through the tensor network. The resulting algorithm is called the Tree Belief Propagation (see Algorithm 3). While various scheduling strategies for the message passing exist, we focus on the case of tree hypergraphs, for which exactness and efficiency can be shown. We denote $\mathcal{E}^{\rightarrow}$ to be all tuples (e_0, e_1) of hyperedges $e_0, e_1 \in \mathcal{E}$ such that $e_0 \neq e_1$ and $e_0 \cap e_1 \neq \varnothing$. For our purposes we call a hypergraph $\mathcal{G}$ a tree when the graph with nodes by the hyperedges $\mathcal{E}$ and edges by $\mathcal{E}^{\rightarrow}$ is a tree (for an example see Figure 3.9).

Algorithm 3 Tree Belief Propagation

Require: Tensor network $\tau^{\mathcal{G}}$ on a hypergraph $\mathcal{G}$
Ensure: Messages $\{\chi_{e_0\rightarrow e_1}\left[X_{e_0\cap e_1}\right] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$

Initialize a message scheduler $S = \{(e_0, e_1) \in \mathcal{E}^{\rightarrow} : e_0 \text{ a leaf in the tree } (\mathcal{E}, \mathcal{E}^{\rightarrow})\}]$
while S not empty **do**
  Pop a (e_0, e_1) pair from S
  Compute the message

$$\chi_{e_0\rightarrow e_1}\left[X_{e_0\cap e_1}\right] = \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2\rightarrow e_0}\left[X_{e_2\cap e_0}\right] : (e_2, e_0) \in \mathcal{E}^{\rightarrow},\ e_2 \neq e_1\}\right\rangle_{\left[X_{e_0\cap e_1}\right]}$$

  Update S by all messages (e_1, e_3) which have not yet been sent, if all messages (e_2, e_1) with $e_2 \neq e_3$ have been sent.
end while
return Messages $\{\chi_{e_0\rightarrow e_1}\left[X_{e_0\cap e_1}\right] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$

For an implementation of Algorithm 3 in the python package tnreason, see Sect. **??**.

The following theorem states that the contraction of a whole tensor network can be replaced by local contractions with messages. Since contracting the whole network can be infeasible, this shows that calculating the messages with the Algorithm 3 can be advantageous.

Theorem 3.14 *Let $\tau^{\mathcal{G}}$ be a tensor network on a tree hypergraph $\mathcal{G}$ (i.e. the graph $(\mathcal{E}, \mathcal{E}^{\rightarrow})$ is a tree). The messages in the tree belief propagation Algorithm 3 are contracted to local marginals, meaning that for each $e_0 \in \mathcal{E}$ we have*

$$\left\langle \tau^{\mathcal{G}}\right\rangle_{\left[X_{e_0}\right]} = \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2\rightarrow e_0}\left[X_{e_2\cap e_0}\right] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}\}\right\rangle_{\left[X_{e_0}\right]} .$$

We show Thm. 3.14 based on the following lemma. We denote for each pair (e_0, e_1) the subset $\mathcal{E}^{\rightarrow(e_0,e_1)} \subset \mathcal{E}$ as the subset of edges $e \in \mathcal{E}$, for which each path in $(\mathcal{E}, \mathcal{E}^{\rightarrow})$ to e_1 passes through e_0. The tree hypergraph property makes this definition equivalent to an existing path through e_0, which is used in the proof of the following lemma. Note that by construction $e_0 \in \mathcal{E}^{\rightarrow(e_0,e_1)}$.

Lemma 3.1 *For any tensor network on a tree hypergraph, Algorithm 3 terminates in the tree-based implementation and returns final messages*

$$\chi_{e_0\rightarrow e_1}\left[X_{e_0\cap e_1}\right] = \left\langle \{\tau^{e}\left[X_{e}\right] : e \in \mathcal{E}^{\rightarrow(e_0,e_1)}\}\right\rangle_{\left[X_{e_0\cap e_1}\right]} .$$

Proof. We show this property by induction over the size of the edge sets $\mathcal{E}^{\to(e_0,e_1)}$ for pairs $(e_0, e_1) \in \mathcal{E}^{\to}$, such that $\left|\mathcal{E}^{\to(e_0,e_1)}\right| \leq n$. Note that since always $e_0 \in \mathcal{E}^{\to(e_0,e_1)}$ we have that $n \geq 1$.

$n = 1$: In this case we have $\mathcal{E}^{\to(e_0,e_1)} = \{e_0\}$ and e_0 is a leaf of the tree-hypergraph $\mathcal{G}$. The claimed message property holds thus by definition.

$n \to n + 1$: Assume that the message obeys the claimed property for edge sets with cardinality up to n. If there is no edge set with cardinality $n + 1$, the property holds also for those with cardinality up to $n + 1$. If there is an edge set $\mathcal{E}^{\to(e_0,e_1)}$ with size $n + 1$, we have

$$\mathcal{E}^{\to(e_0,e_1)} = \{e_0\} \cup \left(\bigcup_{e_2 \in \mathcal{E}^{\to}} \mathcal{E}^{\to(e_2,e_0)} \right) .$$

The message $\chi_{e_0 \to e_1}$ is sent once all messages $\chi_{e_2 \to e_0}$ to $(e_2, e_0) \in \mathcal{E}^{\to(e_0,e_1)}$ arrived. By definition we have that

$$\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] = \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2 \to e_0}\left[X_{e_2 \cap e_0}\right] \,:\, (e_2, e_0) \in \mathcal{E}^{\to},\, e_2 \neq e_1\} \right\rangle_{\left[X_{e_0 \cap e_1}\right]} .$$

Now we use the induction assumption on $\mathcal{E}^{\to(e_2,e_0)}$ (since its cardinality is at most n) and get

$$\begin{aligned}
\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] &= \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \left(\bigcup_{(e_2,e_0) \in \mathcal{E}^{\to},\, e_2 \neq e_1} \left\langle \{\tau^{e_3}\left[X_{e_3}\right] \,:\, e_3 \in \mathcal{E}^{\to(e_2,e_0)}\} \right\rangle_{\left[X_{e_2 \cap e_0}\right]} \right) \right\rangle_{\left[X_{e_0 \cap e_1}\right]} \\
&= \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \left(\bigcup_{(e_2,e_0) \in \mathcal{E}^{\to},\, e_2 \neq e_1} \{\tau^{e_3}\left[X_{e_3}\right] \,:\, e_3 \in \mathcal{E}^{\to(e_2,e_0)}\} \right) \right\rangle_{\left[X_{e_0 \cap e_1}\right]} \\
&= \left\langle \{\tau^{e}\left[X_{e}\right] \,:\, e \in \mathcal{E}^{\to(e_0,e_1)}\} \right\rangle_{\left[X_{e_0 \cap e_1}\right]} .
\end{aligned}$$

Here, we used the commutation of contraction property in the second equation, which is justified by the assumed tree property of the hypergraph. Thus, the message property holds also for any edge sets of size $n + 1$.

By induction, the claimed message property therefore holds for all final messages. □

Proof of Thm. 3.14. Since the hypergraph is by assumption a tree, we can partition $\mathcal{E}$ into disjoint subsets $\{e_0\}$ and $\mathcal{E}^{\to(e_2,e_0)}$ for $(e_2, e_0) \in \mathcal{E}^{\to}$. We then have

$$\begin{aligned}
\left\langle \tau^{\mathcal{G}} \right\rangle_{\left[X_{e_0}\right]} &= \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \left\{ \left\langle \tau^{e}\left[X_{e}\right] \,:\, e \in \mathcal{E}^{\to(e_2,e_0)} \right\rangle_{\left[X_{e \cap e_2}\right]} \,:\, (e_2, e_0) \in \mathcal{E}^{\to} \right\} \right\rangle_{\left[X_{e_0}\right]} \\
&= \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2 \to e_0}\left[X_{e_2 \cap e_0}\right] \,:\, (e_2, e_0) \in \mathcal{E}^{\to}\} \right\rangle_{\left[X_{e_0}\right]} ,
\end{aligned}$$

where we used Lem. 3.1 in the second equation. □

We illustrate the usage of Algorithm 3 on the Markov network of Example 3.2.

Example 3.7 (Continuation of Example 3.2) We exemplify the Belief Propagation Algorithm 3 on the Markov network in the student example (see Example 3.2). The directions of the messages result from the hyperedge overlaps (see Figure 3.9 a) and the resulting directions $\mathcal{E}^{\to}$ are sketched in Figure 3.9 b). The messages to $\{(e_2, e_0), (e_0, e_2)\}$ are vectors of X_G and the messages $\{(e_0, e_1), (e_1, e_0)\}$ are vectors of X_I.

Since the hyperedges are minimally connected, we can implement Algorithm 3 by a tree scheduler S:

- The scheduler is initialized with messages from leafs, in our example $\{(e_2, e_0), (e_1, e_0)\}$.
- Each message is placed exactly once on S, when at a hyperedge all but the reverse message have been received. In our example, after execution of (e_2, e_0) the message (e_0, e_1) is placed on S and after execution of (e_1, e_0) the message (e_0, e_2).

In this implementation, Algorithm 3 terminates after $|\mathcal{E}^{\to}| = 4$ iterations of the while loop. The exact marginals of the edge variables are then

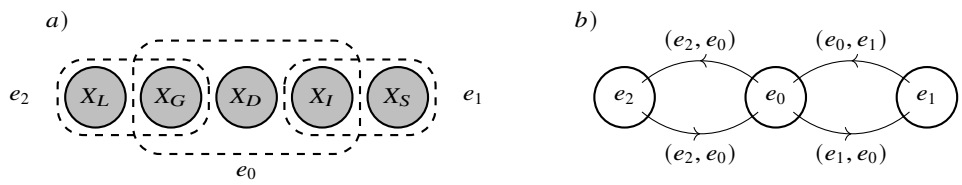


Fig. 3.9 a) Sketch of the overlap of the edges, resulting in the message directions b) $\mathcal{E}^{\rightarrow} = \{(e_2, e_0), (e_0, e_2), (e_0, e_1), (e_1, e_0)\}$.

$$\begin{aligned}
\mathbb{P}\left[X_L, X_G\right] &= \left\langle \tau^{e_2}\left[X_L, X_G\right], \chi_{e_0 \to e_2}\left[X_G\right]\right\rangle_{[X_L, X_G|\varnothing]}, \\
\mathbb{P}\left[X_G, X_D, X_I\right] &= \left\langle \tau^{e_0}\left[X_G, X_D, X_I\right], \chi_{e_2 \to e_0}\left[X_G\right], \chi_{e_1 \to e_0}\left[X_I\right]\right\rangle_{[X_G, X_D, X_I|\varnothing]}, \\
\mathbb{P}\left[X_I, X_S\right] &= \left\langle \tau^{e_1}\left[X_I, X_S\right], \chi_{e_0 \to e_1}\left[X_I\right]\right\rangle_{[X_I, X_S|\varnothing]} .
\end{aligned}$$

3.6 Queries

The efficient retrieval of information stored in probability distributions has to exploit the available decomposition schemes. To avoid the instantiation of a distribution based on its decomposition, we directly define deductive reasoning schemes by contractions, which can be executed using the available decomposition.

3.6.1 Querying by Functions

We now formalize queries by retrieving expectations of functions given a distribution specified by probability tensors. We exploit basis calculus in defining categorical variables Y_q to tensors q, which are enumerating the set im (q). More details on this scheme are provided in Chapter **??**, see Def. **??** therein.

Definition 3.14 The marginal query of a probability distribution $\mathbb{P}\left[X_{[d]}\right]$ by a query statistic

$$q : \bigtimes_{k \in [d]} [m_k] \to \mathcal{U}^q$$

is the vector $\mathbb{P}\left[Y_q\right]$, where Y_q is an image enumerating variable (see Chapter **??** for more details), defined as the contraction

$$\mathbb{P}\left[Y_q\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \beta^q\left[Y_q, X_{[d]}\right]\right\rangle_{[Y_q]} .$$

If $\mathcal{U}^q \subset \mathbb{R}$, and the statistic q is therefore a tensor, we further define the expectation query of $\mathbb{P}\left[X_{[d]}\right]$ by q as

$$\mathbb{E}\left[q\right] = \left\langle q(X_{[d]}), \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .$$

Given another query statistic $g : \bigtimes_{k \in [d]} [m_k] \to \mathcal{U}^g$, which image is enumerated by a variable Y_g, the conditional query of the probability distribution $\mathbb{P}\left[X_{[d]}\right]$ by the statistic q conditioned on g is the matrix $\mathbb{P}\left[Y_q \middle| Y_g\right] \in \mathbb{R}^{|\mathrm{im}(q)|} \otimes \mathbb{R}^{|\mathrm{im}(g)|}$ defined as the normalization

$$\mathbb{P}\left[Y_q \middle| Y_g\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \beta^q\left[Y_q, X_{[d]}\right], \beta^g\left[Y_g, X_{[d]}\right]\right\rangle_{[Y_q|Y_g]} .$$

We notice that marginal and conditional queries generalize the schemes of marginalization and conditioning on the distributed variables. As such, marginal distributions are marginal queries with respect to a identity query statistic acting

on the respective variables. Conditional distributions can be similarly retrieved by two identity statistics, representing the incoming and outgoing variables.

Expectation queries return the expectation of a real-valued feature $q : \bigtimes_{k\in[d]}[m_k] \to \mathbb{R}$. When understanding q as a random variable given the probability measure $\mathbb{P}\left[X_{[d]}\right]$, the expectation query returns the expectation of that random variable. Expectation queries are further contractions of marginal queries with the identity function restricted on the image of q, since

$$\mathbb{E}\left[q\right] = \left\langle \mathbb{P}\left[Y_q\right], \mathrm{Id}|_{\mathrm{im}(q)} Y_q \right\rangle_{[\varnothing]} .$$

This contraction equation follows from the more general Cor. **??**, which will be shown in Chapter **??**.

Expectation queries compute mean parameters to a distribution, see Def. 9.1. Given a statistic t and a distribution $\mathbb{P}\left[X_{[d]}\right]$, each feature t_ℓ for $\ell \in [p]$ corresponds with a mean parameter

$$\mathbb{E}\left[q\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], t_\ell \right\rangle_{[\varnothing]} = \mu\left[L = \ell\right] .$$

3.6.2 Mode Queries

A different kind of queries are mode queries, which we formalize by the searches of the indices to maximal coordinates in a tensor.

Definition 3.15 Given a tensor $\tau\left[X_{[d]}\right]$ the mode query is the problem

$$\mathrm{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \tau\left[X_{[d]} = x_{[d]}\right] . \tag{3.2}$$

We can pose mode queries as convex optimization problems. To this end we recall, that the set of all probability distributions is a convex hull of the one-hot encoded states, that is

$$\Lambda^{\delta,\mathrm{MAX},\mathbb{I}} = \mathrm{conv}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] : x_{[d]} \in \bigtimes_{k\in[d]}[m_k]\right) .$$

With this we have

$$\begin{aligned}\max_{x_{[d]}} \tau\left[X_{[d]} = x_{[d]}\right] &= \max_{x_{[d]}} \left\langle \tau\left[X_{[d]}\right], \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} \\ &= \max_{\mu\left[X_{[d]}\right]\in\Lambda^{\delta,\mathrm{MAX},\mathbb{I}}} \left\langle \tau\left[X_{[d]}\right], \mu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .\end{aligned}$$

We note that the maximization over $\Lambda^{\delta,\mathrm{MAX},\mathbb{I}}$ is a convex optimization problem and the maxima are taken at

$$\mathrm{argmax}_{\mu\left[X_{[d]}\right]\in\Lambda^{\delta,\mathrm{MAX},\mathbb{I}}} \left\langle \tau\left[X_{[d]}\right], \mu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \mathrm{conv}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] : x_{[d]} \in \mathrm{argmax}_{x_{[d]}} \tau\left[X_{[d]} = x_{[d]}\right]\right) .$$

We will further apply this generic trick to approach mode queries when studying inference problems for exponential families in the following sections.

Let us note, that we can also approach modified mode queries, where we restrict to $\mathcal{U} \subset \bigtimes_{k\in[d]}[m_k]$, namely

$$\mathrm{argmax}_{x_{[d]}\in\mathcal{U}} \tau\left[X_{[d]} = x_{[d]}\right] .$$

We can choose the base measure by the subset encoding (see Chapter **??**) of $\mathcal{U}$, namely

$$\nu\left[X_{[d]}\right] = \sum_{x_{[d]}\in\mathcal{U}} \epsilon_{x_{[d]}}\left[X_{[d]}\right] ,$$

and conclude that

$$\operatorname{argmax}_{\mu\left[X_{[d]}\right]\in\Lambda^{\delta,\mathrm{MAX},\nu}}\left\langle\tau\left[X_{[d]}\right],\mu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}=\operatorname{conv}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right]\,:\,x_{[d]}\in\operatorname{argmax}_{x_{[d]}\in\mathcal{U}}\tau\left[X_{[d]}=x_{[d]}\right]\right)\,.$$

3.6.3 Energy Representations

For exponential families (see Sect. 8.1) we have observed, that often energy tensors have feasible tensor network representations, whereas the corresponding probability distributions can get infeasible. We therefore investigate here schemes to answer queries based on the energy tensor instead of the distribution.

Theorem 3.15 *Let $\phi\left[X_{[d]}\right]$ be an energy tensor and $\mathbb{P}\left[X_{[d]}\right]=\left\langle\exp\left[\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$ the corresponding distribution. For disjoint subsets $A,B\subset[d]$ with $A\cup B=[d]$ and any x_B we have*

$$\mathbb{P}\left[X_A\middle|X_B=x_B\right]=\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\rangle_{[X_A|\varnothing]}\,.$$

Proof. To show the theorem, we use a generic simplification property of coordinatewise transforms, which we will show as Lem. **??** in Chapter **??** and get

$$\langle\exp\left[\phi\left[X_A,X_B\right]\right]\rangle_{[X_A,X_B=x_B]}=\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\rangle_{[X_A]}$$

Based on this we get

$$\begin{aligned}\mathbb{P}\left[X_A\middle|X_B=x_B\right]&=\langle\exp\left[\phi\left[X_A,X_B\right]\right]\rangle_{[X_A|X_B=x_B]}\\&=\frac{\langle\exp\left[\phi\left[X_A,X_B\right]\right]\rangle_{[X_A,X_B=x_B]}}{\langle\exp\left[\phi\left[X_A,X_B\right]\right]\rangle_{[X_B=x_B]}}\\&=\frac{\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\rangle_{[X_A]}}{\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\rangle_{[\varnothing]}}\\&=\langle\exp\left[\phi\left[X_A,X_B=x_B\right]\right]\rangle_{[X_A|\varnothing]}\,.\square\end{aligned}$$

Importantly, Thm. 3.15 does not generalize to situations, where $A\cup B\neq[d]$, since summation over the indices of the variables $[d]/A\cup B$ and contraction do not commute. In this more generic situation, we would need to sum over exponentiated coordinates, that is

$$\mathbb{P}\left[X_A\middle|X_B=x_B\right]=\left\langle\sum_{x_{[d]/A\cup B}\in[m_{[d]/A\cup B}]}\exp\left[\phi\left[X_A,X_B=x_B,X_{[d]/A\cup B}=x_{[d]/A\cup B}\right]\right]\right\rangle_{[X_A|\varnothing]}\,.$$

Mode queries on probability distributions in an energy representation can always be reduces to mode queries on the energy tensor. This is due to the monotonicity of the exponential function, which implies

$$\begin{aligned}\operatorname{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\mathbb{P}\left[X_{[d]}=x_{[d]}\right]&=\operatorname{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\left\langle\exp\left[\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}=x_{[d]}|\varnothing\right]}\\&=\operatorname{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\exp\left[\phi\left[X_{[d]}=x_{[d]}\right]\right]\\&=\operatorname{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\phi\left[X_{[d]}=x_{[d]}\right]\,.\end{aligned}$$

Since we are only interested in identifying the index of the maximum coordinate, and not its value, we have further dropped the normalization term by partition functions. When one instead need to get the value of the maximal, the partition function cannot be ignored.

3.7 Sampling

Let us here investigate how to draw samples from a probability distribution, based on queries on it. Naive methods, such as drawing a random number in $[0,1]$, adding iteratively the coordinates and stopping when the sum exceeds the random

variables, are infeasible when having large tensor orders causing exponential increases of the coordinate number. We recall, that the number of coordinates of $\mathbb{P}\left[X_{[d]}\right]$ is $\prod_{k\in[d]} m_k$, which increases exponentially in the number d of the variables. Efficient methods instead have to exploit tensor network decompositions of the decompositions.

3.7.1 Forward Sampling

The first insight to derive efficient sampling algorithms is to sample a single variable in each step. Forward sampling (see Algorithm 4) exploits to this end the generic chain decomposition (see Thm. 3.5) of a probability distribution, namely

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}\left[X_0\right]\} \cup \left\{\mathbb{P}\left[X_k \middle| X_0, \ldots, X_{k-1}\right] \,:\, k \in [d],\, k \geq 1\right\}\right\rangle_{\left[X_{[d]}\right]} ,$$

It then samples iteratively a state x_k for the variable X_k conditioned on the previously sampled states, that is from the conditonal distribution

$$\mathbb{P}\left[X_k \middle| X_{[k]} = x_{[k]}\right] .$$

The generic chain decomposition thereby ensures that probability of getting a state $x_{[d]}$ by this procedure coincides with $\mathbb{P}\left[X_{[d]} = x_{[d]}\right]$.

Algorithm 4 Forward Sampling

Require: Probability distribution $\mathbb{P}$
Ensure: Exact sample $x_0, \ldots, x_{d-1}$ of $\mathbb{P}$

for all $k \in [d]$ **do**
Draw $x_k \in [m_k]$ from the conditional distribution

$$\mathbb{P}\left[X_k \middle| X_{[k]} = x_{[k]}\right]$$

end for
return $x_0, \ldots, x_{d-1}$

Forward Sampling is especially efficient, when sampling from a Bayesian Network respecting the topological order of its nodes. More technically, when the parents $\mathrm{Pa}(k)$ of a node k are contained in the preceding variables $[k]$, we apply the conditional independence assumption (more precisely Thm. 3.8 in combination with Thm. 3.11) to get

$$\mathbb{P}\left[X_k \middle| X_{[k]} = x_{[k]}\right] = \mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)} = x_{\mathrm{Pa}(k)}\right] .$$

Since this conditional probability coincides with a local tensor in the Bayesian Network, we can avoid to contract the network for preparing the conditional distribution. Different to more general Markov Networks, forward sampling from Bayesian Network can therefore be done efficiently by reduction to conditional queries answerable using local tensors. We note that it is important to sample in the topological order induced by the underlying directed hypergraph, since the computation of generic conditional distributions is also for Bayesian Networks NP-hard (see Chapter 13 in [**?**]). Sampling along the topological variable order requires only tractable to answer conditional queries on the Bayesian Network.

3.7.2 Gibbs Sampling

While we have seen that forward sampling can be performed efficiently on Bayesian Networks, Gibbs sampling can be also performed efficiently for Markov Networks. Gibbs sampling Algorithm 5 overcomes the intractability problems of sampling steps during forward sampling at the expense of repetitions of the sampling step. When performing finite repetitions, Gibbs sampling in general samples from an approximate distribution to the one desired. It can be shown,

that these approximate distribution tend to one desired in the asymptotic limit of infinite repetitions of the sampling step (see Chapter 12 in [**?**]).

Algorithm 5 Gibbs Sampling

Require: Probability distribution $\mathbb{P}$
Ensure: Approximative sample $x_0, \ldots, x_{d-1}$ of $\mathbb{P}$

```
for all k ∈ [d] do
    Draw x_k from an initialization distribution.
end for
while Stopping criterion is not met do
    for all k ∈ [d] do
        Draw x_k from the conditional distribution
```

$$\mathbb{P}\left[X_k \middle| X_{[d]/\{k\}} = x_{[d]/\{k\}}\right]$$

```
    end for
end while
return x_0, ..., x_{d-1}
```

The central problem of forward sampling on Markov Networks has been the need for global contractions to answer the required conditional queries, which originates from large numbers of variables to be marginalized out. When avoiding the marginalization of variables, and conditioning on them instead, global contractions can be avoided. To be more precise, for any tensor network $\tau^{\mathcal{G}}$ on $\mathcal{G} = ([d], \mathcal{E})$ and any $k \in [d]$ we have

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{\left[X_k, X_{[d]/\{k\}}\right]} = \left\langle \left\{ \tau^e \left[X_k, X_{e/\{k\}} = x_{e/\{k\}}\right] \,:\, e \in \mathcal{E},\, k \in e \right\} \right\rangle_{[X_k]} \cdot \prod_{e \in \mathcal{E},\, k \notin e} \tau^e \left[X_e = x_e\right] .$$

As a consequence, we get for the Markov Network $\mathbb{P} = \mathbb{P}^{\mathcal{G}}$ to $\tau^{\mathcal{G}}$, that

$$\begin{aligned} \mathbb{P}\left[X_k \middle| X_{[d]/\{k\}} = x_{[d]/\{k\}}\right] &= \left\langle \tau^{\mathcal{G}} \right\rangle_{\left[X_k, X_{[d]/\{k\}} | \varnothing\right]} \\ &= \left\langle \left\{ \tau^e \left[X_k, X_{e/\{k\}} = x_{e/\{k\}}\right] \,:\, e \in \mathcal{E},\, k \in e \right\} \right\rangle_{[X_k | \varnothing]} . \end{aligned}$$

The conditional queries on a Markov Network asked in Gibbs Sampling can therefore be answered by contractions only of those tensors containing the variable X_k. To find further locally answerable conditional queries, we need to condition only on the neighbored variables, referred to as Markov blanket, such that the other variables are conditionally independent. This follows from the characterization of the conditional independences in Markov Networks, which has been shown in Thm. 3.10, and can be used to design further tractable sampling schemes for Markov Networks.

We can further answer these conditional queries efficiently, when we perform Gibbs sampling on a probability distribution in an energy representation, that is $\mathbb{P}\left[X_{[d]}\right] = \left\langle \exp\left[\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]} | \varnothing\right]}$. Using Thm. 3.15, we have

$$\mathbb{P}\left[X_k \middle| X_{[d]/\{k\}} = x_{[d]/\{k\}}\right] = \left\langle \exp\left[\phi\left[X_k, X_{[d]/\{k\}} = x_{[d]/\{k\}}\right]\right]\right\rangle_{[X_A | \varnothing]}$$

We note, that the main property of the conditional query exploited here, is that all variables but the one sampled appear as a condition and none is marginalized out. In the scheme of forward sampling, where most of the variables are marginalized out in many queries, we cannot apply this trick and would have to perform sums over exponentiated coordinates to the variables marginalized out.

3.7.3 Simulated Annealing

Simulated annealing is an adapted sampling scheme targeting mode queries rather than generating representative samples from a distribution. It employs an annealing process that gradually transforms the probability distribution by increasingly favoring high-likelihood configurations, thereby improving the chances of sampling a solution to a mode query. To

be more precise, let there be a distribution in energy representation, that is $\mathbb{P}\left[X_{[d]}\right] = \left\langle \exp\left[\phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$. We introduce a parameter $\beta \in \mathbb{R}$, which we understand as the inverse temperature, and anneal the distribution through scaling its energy by this parameter. In the limit of $\beta \to \infty$, for each state $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ the annealed distribution behaves as

$$\left\langle \exp\left[\beta \cdot \phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}=x_{[d]}|\varnothing\right]} \to \left\langle \mathbb{I}_{\mathrm{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\phi\left[X_{[d]}=x_{[d]}\right]}\right\rangle_{\left[X_{[d]}=x_{[d]}|\varnothing\right]} .$$

In this limit, the annealed distribution tends to the uniform distribution of the maximal coordinates, that is the uniform distribution of the set

$$\mathrm{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\phi\left[X_{[d]} = x_{[d]}\right] = \mathrm{argmax}_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\mathbb{P}\left[X_{[d]} = x_{[d]}\right] .$$

i To integrate annealing into Gibbs sampling, one chooses a parameter β for each repetition of a sampling step and sample from the conditioned annealed distribution $\left\langle \exp\left[\beta \cdot \phi\left[X_{[d]}\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$. Through increasing β during the algorithm, the samples are drawn towards states with larger coordinates in $\mathbb{P}\left[X_{[d]}\right]$. However, when β is large, the sampling procedure can get stuck in local maxima, whereas small β are in favor of overcoming such. The inverse temperature is thus understood as a tradeoff parameter between the exploration of new regions of the state space and increasing the coordinate of the sample by local coordinate optimization. It is therefore typically chosen low in the beginning of the sampling algorithm and then sequentially increased to find maximal coordinates. Due to this typical increasement of the inverse temperature strategy, the algorithm is referred to as simulated annealing.

3.8 Maximum Likelihood Estimation

So far we have been concerned with deductive reasoning task, that is retrieve information from a given distribution or drawing a sample. We now turn to inductive reasoning tasks, where a probability distribution is estimated given data. To present the generic framework of maximum likelihood estimation in the tensor network contraction formalism, we introduce the likelihood loss exploiting the structure of empirical distribution, and then provide interpretations in terms of entropies.

3.8.1 Empirical Distributions

To prepare for reasoning on data, we now derive tensor network representation for empirical distributions, which are defined based on observed states $\left((x_0^j, \ldots, x_{d-1}^j) \,:\, j \in [m]\right)$ of a factored system.

Definition 3.16 Given a dataset $\left((x_0^j, \ldots, x_{d-1}^j) \,:\, j \in [m]\right)$ of samples of the factored system we define the sample selector map

$$D \,:\, [m] \to \bigtimes_{k\in[d]} [m_k]$$

elementwise by

$$D(j) = \left(x_0^j, \ldots, x_{d-1}^j\right) .$$

The empirical distribution to the sample selector map D is the probability distribution

$$\mathbb{P}^D\left[X_{[d]}\right] := \left\langle \beta^D\left[X_{[d]}, J\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]} ,$$

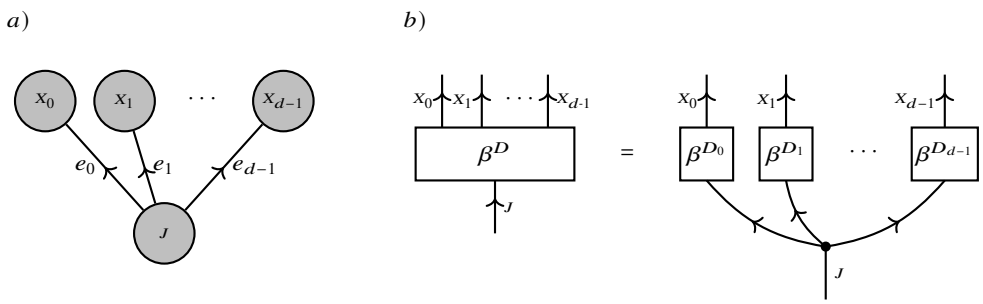


Fig. 3.10 Decomposition of the basis encoding of a sample selector map to a dataset $\left((x_0^j, \ldots, x_{d-1}^j) \,:\, j \in [m]\right)$. a) Interpretation as a sample selection variable J selecting states for the variables $X_{[d]}$ according to the enumerated dataset. b) Corresponding decomposition of the basis encoding β^D into a tensor network in the basis CP Format (see Sect. **??**), where $\tau^{e_k} = \beta^{D_k}$.

where we introduced as single incoming for the basis encoding of the sample selector map the sample selecting variable J taking values in $[m]$.

The basis encoding of the sample selector map has a decomposition by

$$\beta^D \left[X_{[d]}, J\right] = \sum_{j \in [m]} \epsilon_{x_0^j, \ldots, x_{d-1}^j} \left[X_{[d]}\right] \otimes \epsilon_j \left[J\right] .$$

Each coordinate $x_{[d]}$ of the empirical distribution can thus be calculated by

$$\begin{aligned} \mathbb{P}^D \left[X_{[d]} = x_{[d]}\right] &= \frac{1}{\left\langle \beta^D \right\rangle_{[\varnothing]}} \left(\sum_{j \in [m]} \epsilon_{x_0^j, \ldots, x_{d-1}^j} \left[X_{[d]} = x_{[d]}\right] \right) \\ &= \frac{\left| \left\{ j \in [m] \,:\, (x_0^j, \ldots, x_{d-1}^j) = (x_0, \ldots, x_{d-1}) \right\} \right|}{m} . \end{aligned}$$

We can therefore interpret each coordinate of the empirical distribution as the relative frequency of the corresponding state in the observed data.

The basis encoding of the sample selector map is a sum of one-hot encodings of the data indices and the corresponding sample states. Such sums of basis tensors will be further investigated in Sect. **??** as basis CP decompositions. We now exploit this structure to find efficient tensor network decompositions (see Figure 3.10) based on matrices encoding its variables.

Theorem 3.16 *Given a data map $D : [m] \to \bigtimes_{k \in [d]} [m_k]$ we define for $k \in [d]$ its coordinate maps*

$$D_k \,:\, [m] \to [m_k]$$

by

$$D_k(j) = x_k^j .$$

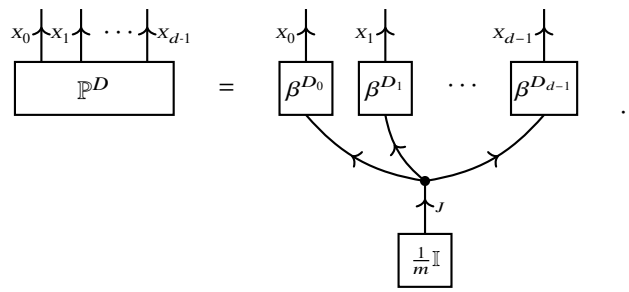
We then have

$$\beta^D \left[X_{[d]}, J\right] = \left\langle \{\beta^{D^k} \left[X_k, J\right] \,:\, k \in [d]\} \right\rangle_{\left[X_{[d]}, J\right]}$$

and

$$\mathbb{P}^D \left[X_{[d]}\right] = \left\langle \beta^D \left[X_{[d]}, J\right], \frac{1}{m} \mathbb{I}\left[J\right] \right\rangle_{\left[X_{[d]}\right]} = \left\langle \beta^{D_0}\left[X_0, J\right], \ldots, \beta^{D_{d-1}}\left[X_{d-1}, J\right], \frac{1}{m} \mathbb{I}\left[J\right] \right\rangle_{\left[X_{[d]}\right]} .$$

In a contraction diagram this decomposition is represented as



Proof. The first claim is a special case of Thm. **??**, to be shown in Chapter **??**. To show the second claim we notice

$$\left\langle \beta^D \right\rangle_{[\varnothing]} = \sum_{j \in [m]} \left\langle \beta^D \left[X_{[d]}, J = j \right] \right\rangle_{[\varnothing]} = m \,.$$

With the first claim it now follows that

$$\begin{aligned} \mathbb{P}^D \left[X_{[d]} \right] = \left\langle \beta^D \right\rangle_{[X_{[d]} | \varnothing]} &= \frac{\left\langle \beta^D \right\rangle_{[X_{[d]}]}}{\left\langle \beta^D \right\rangle_{[\varnothing]}} \\ &= \left\langle \left\{ \beta^{D^k} \left[X_k, J \right] \,:\, k \in [d] \right\} \cup \left\{ \frac{1}{m} \mathbb{I} \left[J \right] \right\} \right\rangle_{[X_{[d]}]} \,. \square \end{aligned}$$

The cores β^{D_k} are matrices storing the value of the categorical variable X_k in the sample world indexed by j.

From the proof of Thm. 3.16 we notice that the scalar $\frac{1}{m}$ could be assigned with any core in a representation of $\mathbb{P}^D$, and the core $\mathbb{I}\left[J\right]$ is thus redundant in the contraction representation. However, creating the core $\frac{1}{m}\mathbb{I}\left[J\right]$ provides us with a simple interpretation of the empirical distribution. We can understand $\frac{1}{m}\mathbb{I}\left[J\right]$ as the uniform probability distribution over the samples, which is by the map D forwarded to a distribution over $\bigtimes_{k \in [d]} [m_k]$. The one-hot encoding of each sample is itself a probability distribution, which is understood as conditioned on the respective state of the sample selection variable J. The conditional distribution β^D therefore forwards the uniform distribution of the samples to a distribution of the variables $X_{[d]}$. In the perspective of a Bayesian Network (see Figure 3.10), the variable J serves as single parent for each categorical variable X_k.

3.8.2 Likelihood Loss

The likelihood of a probability distribution $\mathbb{P}\left[X_{[d]}\right]$ to produce an observed sample is

$$\mathbb{P}\left[X_{[d]} = D(j)\right] \,.$$

We further introduce the likelihood of $\mathbb{P}\left[X_{[d]}\right]$ with respect to a dataset as

$$\mathbb{P}\left[\left((x_0^j, \ldots, x_{d-1}^j) \,:\, j \in [m]\right)\right] := \prod_{j \in [m]} \mathbb{P}\left[X_{[d]} = D(j)\right] \,.$$

The likelihood draws on the assumption, that each datapoint in the dataset has been drawn independently from the same distribution. When this generating distribution coincides with $\mathbb{P}\left[X_{[d]}\right]$, then the probability of generating a dataset by this scheme is the likelihood. In inductive reasoning, the true distribution $\mathbb{P}\left[X_{[d]}\right]$ is unknown and needs to be approximatively estimated based on data and a learning hypothesis. We will therefore compute and compare the likelihood of distributions, which in general do not coincide with the distribution generating the data. It is thus important to not understand the likelihood as a probability, which is only true for the generating distribution, as pointed out in Chapter 2 in [**?**].

Let us now transform the likelihood to find an representation involving the empirical distribution (see Def. 3.16), for which efficient tensor network decompositions have been derived in Thm. 3.16. Applying a scaled logarithm we get

$$\begin{aligned}\frac{1}{m}\cdot\ln\left[\mathbb{P}\left[\left((x_0^j,\ldots,x_{d-1}^j)\ :\ j\in[m]\right)\right]\right] &= \frac{1}{m}\cdot\ln\left[\prod_{j\in[m]}\mathbb{P}\left[X_{[d]}=D(j)\right]\right]\\ &= \frac{1}{m}\sum_{j\in[m]}\ln\left[\mathbb{P}\left[X_{[d]}=D(j)\right]\right]\\ &= \left\langle\ln\left[\mathbb{P}\left[X_{[d]}\right]\right],\mathbb{P}^D\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\ .\end{aligned}$$

Let us notice, that this transform of the likelihood is monotoneous and therefore does not influence the position of the maximum, when optimizing the likelihood. Motivated by this property, we use the transformed form to define the loss-likelihood loss and maximum likelihood estimation.

Definition 3.17 The log-likelihood loss of a distribution $\mathbb{P}$ given a dataset

$$\left((x_0^j,\ldots,x_{d-1}^j)\ :\ j\in[m]\right)$$

is the functional

$$\mathcal{L}_D\left(\mathbb{P}\right)=-\left\langle\ln\left[\mathbb{P}\left[X_{[d]}\right]\right],\mathbb{P}^D\left[X_{[d]}\right]\right\rangle_{[\,]}\ .$$

Having a hypothesis $\Gamma\subset\Lambda^{\delta,\mathrm{MAX},\mathbb{I}}$, that is a set of probability distributions, the maximum likelihood estimation is the problem

$$\mathrm{argmin}_{\mathbb{P}\in\Gamma}\mathcal{L}_D\left(\mathbb{P}\right)\ . \qquad (\mathrm{P}^{\mathrm{M}}_{\Gamma,\mathbb{P}^D})$$

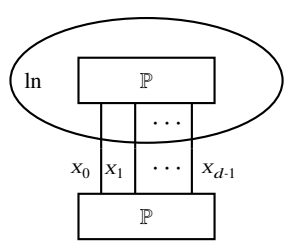
3.8.3 Entropic Interpretation

The Shannon entropy, which has been introduced in the seminal paper [**?**], is a foundational concept in various research fields beyond statistical learning, such as information theory or statistical physics. While a detailed discussion is out of the scope of this work, we here only provide computation schemes of the entropy based on contractions of distributions.

Definition 3.18 (Shannon entropy) The Shannon entropy of a distribution $\mathbb{P}\left[X_{[d]}\right]$ is the quantity

$$\mathbb{H}\left[\mathbb{P}\right]=\sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}\mathbb{P}\left[X_{[d]}=x_{[d]}\right]\cdot\left(-\ln\left[\mathbb{P}\left[X_{[d]}=x_{[d]}\right]\right]\right)=\left\langle\mathbb{P},-\ln\left[\mathbb{P}\right]\right\rangle_{[\varnothing]}\ .$$

We represent the Shannon entropy by the tensor network diagram

$\mathbb{H}\left[\mathbb{P}\right]\ =$ 

where we denote a coordinatewise transform by the logarithm as an ellipsis (see Sect. **??**).

We here make the convention $\ln[0]=-\infty$ and $0\cdot\ln[0]=0$, to have the Shannon entropy well-defined for distributions with non-trivial support.

Among the distributions in the same tensor space, the uniform distribution maximizes the Shannon entropy

$$\mathbb{H}\left[\langle\mathbb{I}\rangle_{[X_{[d]}|\varnothing]}\right]=\sum_{k\in[d]}\ln\left[m_k\right]$$

and the one-hot encodings to states $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ minimize the Shannon entropy

$$\mathbb{H}\left[\epsilon_{x_{[d]}}\left[X_{[d]}\right]\right] = 0\,.$$

The Shannon entropy measures the information content of a distribution and is therefore a central tool for regularization in statistical learning (see for an introduction Chapter 2 in [**?**]). We therefore exploit this information content as a regularizer to identify a distribution among those coinciding in the answer to a collection of expectation queries. To be more precise, let there be for $\ell \in [p]$ query tensors q^ℓ (see Def. 3.14) and $\mathbb{P}^D$ an empirical distribution. The problem of maximal entropy with respect to coinciding expectation queries with $\mathbb{P}^D$ is then posed as

$$\text{argmax}_{\mathbb{P}\in\Lambda^{\delta,\text{MAX},\mathbb{I}}}\mathbb{H}\left[\mathbb{P}\right] \quad \text{subject to} \quad \forall \ell \in [p] : \left\langle \mathbb{P}, q^\ell \right\rangle_{[\varnothing]} = \left\langle \mathbb{P}^D, q^\ell \right\rangle_{[\varnothing]}$$

where $\Lambda^{\delta,\text{MAX},\mathbb{I}}$ is the set of probability distributions given a factored system. We study instances of this maximal entropy problem later in Sect. **??**, where we show that its solution is a member of the exponential family, which statistic is build by the query tensors. We will further provide connections between the problems of maximal entropy and maximum likelihood estimation.

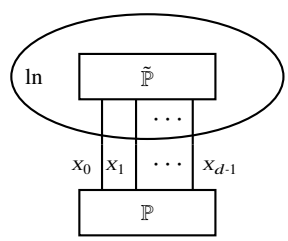
While the Shannon entropy is a property of a single distribution, the cross-entropy is a straight forward generalization towards pairs of distributions. We first introduce this quantity and then interpret the log-likelihood loss based on the cross-entropy.

Definition 3.19 (Cross entropy and Kullback Leibler divergence) The cross-entropy between two distributions $\mathbb{P}\left[X_{[d]}\right]$ and $\tilde{\mathbb{P}}[X_{[d]}]$ defined with respect to the same factored system is the quantity

$$\mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] = \sum_{x_0,\ldots,x_{d-1}} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \cdot \left(-\ln\left[\tilde{\mathbb{P}}[X_{[d]} = x_{[d]}]\right]\right) = \left\langle \mathbb{P}, -\ln\left[\tilde{\mathbb{P}}\right]\right\rangle_{[\varnothing]}\,.$$

The cross-entropy is captured by the tensor network diagram

$\mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] =$



The Kullback-Leiber divergence or relative entropy between $\mathbb{P}\left[X_{[d]}\right]$ and $\tilde{\mathbb{P}}[X_{[d]}]$ is the quantity

$$\mathrm{D}_{\mathrm{KL}}\left[\mathbb{P}||\tilde{\mathbb{P}}\right] = \mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] - \mathbb{H}\left[\mathbb{P}\right]\,.$$

Let us notice, that we have $\mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] = \infty$ if and only if there is a state $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ such that $\mathbb{P}\left[X_{[d]} = x_{[d]}\right] > 0$ and $\tilde{\mathbb{P}}[X_{[d]} = x_{[d]}] = 0$.

The Gibbs inequality (for a proof see for example Chapter 2 in [**?**]) states that for any distributions $\mathbb{P}\left[X_{[d]}\right]$ and $\tilde{\mathbb{P}}[X_{[d]}]$ we have

$$\mathbb{H}\left[\mathbb{P}, \tilde{\mathbb{P}}\right] \geq \mathbb{H}\left[\mathbb{P}\right]\,,$$

where equality holds if any only if $\mathbb{P} = \tilde{\mathbb{P}}$ This ensures, that the Kullback-Leiber Divergence between any distributions is positive and vanishes if and only if both distributions coincide.

We in the next lemma provide an entropic interpretation of maximum likehlihood estimation as defined in Def. 3.17.

Lemma 3.2 *The maximum likelihood estimation* $\mathrm{P}^{\mathrm{M}}_{\Gamma,\mathbb{P}^D}$ *is equivalent to the minimization of cross-entropy and Kullback-Leibler divergence, that is*

$$\text{argmin}_{\mathbb{P}\in\Gamma}\mathcal{L}_D\left(\mathbb{P}\right) = \text{argmin}_{\mathbb{P}\in\Gamma}\mathbb{H}\left[\mathbb{P}^D, \mathbb{P}\right] = \text{argmin}_{\mathbb{P}\in\Gamma}\mathrm{D}_{\mathrm{KL}}\left[\mathbb{P}^D||\mathbb{P}\right]\,.$$

Proof. Comparing the log-likelihood loss in Def. 3.17 with the cross-entropy in Def. 3.19, we get

$$\mathcal{L}_D\left(\mathbb{P}\right) = \mathbb{H}\left[\mathbb{P}^D, \mathbb{P}\right]$$

which established the equivalence of maximum likelihood estimation and cross-entropy minimization. Further, since

$$\mathrm{D}_{\mathrm{KL}}\left[\mathbb{P}^D || \mathbb{P}\right] = \mathbb{H}\left[\mathbb{P}^D, \mathbb{P}\right] - \mathbb{H}\left[\mathbb{P}^D\right]$$

and $\mathbb{H}\left[\mathbb{P}^D\right]$ is a constant offset in the objective, maximum likelihood estimation is equivalent to the minimization of the Kullback-Leibler divergence. □

More general than Maximum Likelihood Estimation, we define the moment projection of an arbitrary distribution $\mathbb{P}^*$ onto a set Γ of probability distributions as the problem

$$\mathrm{argmin}_{\mathbb{P}\in\Gamma}\mathbb{H}\left[\mathbb{P}^*, \mathbb{P}\right] \ . \tag{$\mathrm{P}^{\mathrm{M}}_{\Gamma,\mathbb{P}^*}$}$$

With Lem. 3.2 we have established, that maximum likelihood estimation is the moment projection of the empirical distribution onto a set Γ.

We further define the information projection an arbitrary distribution $\mathbb{P}^*$ onto a set Γ of probability distributions as the problem

$$\mathrm{argmin}_{\mathbb{P}\in\Gamma}\mathrm{D}_{\mathrm{KL}}\left[\mathbb{P} || \mathbb{P}^*\right] \ . \tag{$\mathrm{P}^{\mathrm{I}}_{\Gamma,\mathbb{P}^*}$}$$

The relative entropy is not symmetric, thus the information projection ($\mathrm{P}^{\mathrm{I}}_{\Gamma,\mathbb{P}^*}$) and the moment projections do not coincide in general. The differences of both are discussed in Chapter 8 in [**?**].

Chapter 4
First-Order Logics

Abstract To introduce the syntax and semantics of first-order logics, we in this chapter introduce the substitution tensor structure. This allows to capture structured representations of systems, beyond the factored representations introduced in the first two chapters.

4.1 Syntax and Semantics of first-order logic

The framework of first-order logic generalizes propositional logic analogously to the generalization of factored system representations towards structured system representations [**?**]. This more expressive framework is designed to reason about relations and functions between objects. To capture this framework by tensors we here first define the syntax and then investigate tensor representations of the semantics.

4.1.1 Syntax

A first-order logic theory consists in a finite set of constant symbols $\mathcal{U}$, a set of function symbols $\{v_l : l \in [r]\}$, a set of predicate symbols $\{g_k : k \in [d]\}$ and an arity map assigning the arity n_l to the function v_l and the arity n_k to the predicate g_k.

4.1.2 Tensor Representation of Semantics

We here follow the model-theoretic semantics of first-order logic, where sets of possible worlds are considered. We only treat in this work database semantics, where we assume that the domain $\mathcal{U}$ of each world has a one-to-one mapping to the constants of the theory and is therefore constant among the worlds. Database semantics is central to combine the semantic structure with the substitution structure. Under this assumption, the dimension of the substitution structure does not depend on the specific world, and we can combine both structures in a single tensor space to be defined in the next sections.

To each world there is a world domain $\mathcal{U}$ of objects, which we assume to be finite. We exploit the set-encoding formalism discussed in more detail in Chapter **??** and use bijective index interpretation maps

$$I : [r] \to \mathcal{U} .$$

A so-called term variable O takes states $o \in [r]$, which represent objects

$$I(o) \in \mathcal{U} .$$

The relations between objects are described by n-ary predicates g. Given a specific world x_W the truth of relations is represented by boolean tensors

$$g|_{x_W} \; : \; \bigtimes_{l \in [n]} [r] \to \{0, 1\} \, .$$

Given a tuple $o_0, \ldots, o_{n-1} \in \bigtimes_{l \in [n]} [r]$ the boolean

$$g|_{x_W} \, [O_0 = o_0, \ldots, O_{n-1} = o_{n-1}] \in \{0, 1\}$$

is called a grounding and encodes, whether the relation g is satisfied in the world x_W for the objects $I(o_0), \ldots, I(o_{n-1})$.

Functions in first-order logic are object-valued maps

$$v|_{x_W} \; : \; \mathcal{U}^r \to \mathcal{U} \, .$$

While relations are represented by their coordinate encodings, functions are represented by directed basis encodings

$$\beta^{v|_{x_W}} \left[O_v, O_{[n]} \right]$$

where to each object tuple $I(o_0), \ldots, I(o_{n-1})$ the vector $\beta^{v|_{x_W}} \left[O_v, O_{[n]} = o_{[n]} \right]$ is the one-hot encoding of the object, that is

$$\beta^{v|_{x_W}} \left[O_v, O_{[n]} = o_{[n]} \right] = \epsilon_{v|_{x_W}[I(o_0), \ldots, I(o_{n-1})]} \left[O_v \right] \, .$$

4.1.3 Worlds

Worlds in First-Order Logics contain varying numbers of objects. We use domain variables O to enumerate the sets of objects.

Worlds are represented by three sets of tensors:

- One-hot encodings of constants
- Coordinate encodings for each predicate
- Basis encoding of each function

4.1.4 Two Tensor Structures

In comparison with propositional logic, first-order logic bears two natural tensor structures.

- **Semantic structure:** As in propositional logic, we enumerate possible worlds by a collection of variables X. This is a representation of the model-theoretic semantics, where subsets of worlds are represented by boolean tensors.
- **Substitution structure:** Different to propositional logic, a formula can have variables and the evaluation of a formula on a world is represented by its grounding tensor. Given a world which contains objects in the domain $\mathcal{U}$, we can substitute variables by objects in the domain. In this way, each predicate with r variables is represented in that world as a boolean tensor of order r.

While the semantic structure appears already in factored representations of systems, the substitution structure arises in more generality when treating structured representations of systems [**?**].

4.2 Substitution Structure

We here investigate the structure of worlds in first-order logics and how they can be represented by tensors. As we already argued, groundings of sentences can be aligned in boolean tensors, to be called grounding tensors. We here investigate sparse representation schemes for this substitution structure. In particular we apply basis calculus to find sparse decompositions of the grounding tensors

The semantic structure, an attempt to represent in tensors multiple worlds at once, is to be investigated in Chapter 14.

4.2.1 Grounding Tensors

Given a first-order logic world x_W, arbitrary formulas are interpreted in terms of the satisfactions of their groundings. We define their semantic first, and then relate their syntactical decomposition to tensor networks, similar to our approach to propositional logic in Chapter 2.

Definition 4.1 (Grounding of a first-order formula given a world) Given a specific world x_W, with a domain $\mathcal{U}$ enumerated by $[r]$, the grounding of a formula q with variables O_q is the tensor

$$q|_{x_W}\left[O_q\right] : \bigtimes_{l\in[O_q]} [r] \to \{0,1\}\,.$$

Each coordinate represents thereby the boolean, whether the substitution of the variables in the formula is satisfied given a world x_W, that is

$$q|_{x_W}\left[O_q = o_q\right] = 1$$

if and only if the substitution of q with the variables O_q replaced by the objects $I(o_l)$ is satisfied on the world x_W.

We in the following investigate how these tensors are represented by tensor networks involving the world defining tensors.

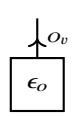
4.2.2 Basis encodings of terms

Terms are object valued functions, which r_v different arguments are also object valued and called variables. We represent terms by the corresponding basis encodings

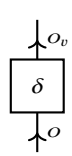
$$\beta^v\left[O_v, O_{[r_v]}\right]$$

Basic blocks:

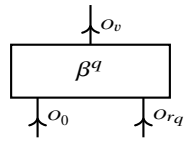
- Constant



- Variable

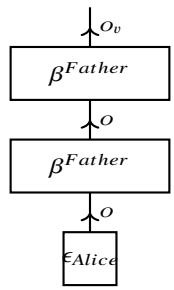


- Function of terms



Contractions of those terms along directed acyclic graphs with single root nodes represent more complex terms. As such, basis encodings of functions can be contracted with constants, variables or other functions to build up basis encodings of more complex terms.

Example: Father of the father of alice is represented by the contraction

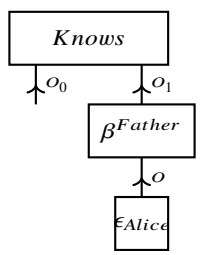


We notice, that the basis encoding of each term is decomposed into a network of world defining tensors, namely constant and function defining tensors.

4.2.3 Atomic sentences

Atomic sentences are contractions of coordinate encodings of predicates with basis encodings of terms.

Example: Knowing the father of alice



Complex sentences are built by

- Quantifier free sentences are contractions of basis encodings to atomic sentences with basis encodings of the connectives.
- Quantifiers act as non-linear transforms of tensors.

4.2.4 Formula Synthesis by Connectives

In order to have a sound semantic, the grounding of first-order logic formulas is determined by the syntax of the formula, i.e. a decomposition of the formula into connectives and quantifiers acting on atomic formulas.

Quantifier-free formulas are connectives acting on atomic formulas. We can describe them as in the case of propositional logic in the β-formalism. While the atomic formulas where delta tensors copying states, they are more involved here.

Theorem 4.1 *For any connective $\circ$ and formulas q_1 and q_2 we have*

$$(q_1 \circ q_2)|_{x_W} \left[O_{q_1} \cup O_{q_2}\right] \tag{4.1}$$
$$= \left\langle \beta^{q_1|_{x_W}} \left[Y_{q_1}, O_{q_1}\right], \beta^{q_2|_{x_W}} \left[Y_{q_2}, O_{q_2}\right], \beta^{\circ} \left[Y_{q_1 \circ q_2}, Y_{q_1}, Y_{q_2}\right], \epsilon_1 \left[Y_{q_1 \circ q_2}\right] \right\rangle_{\left[O_{[n]}\right]} . \tag{4.2}$$

Proof. This directly follows from Thm. **??**. □

Here, variables can be shared by the connected formulas, therefore the variables in the combined formula are unions of the possible not disjoint variables of the connected formulas.

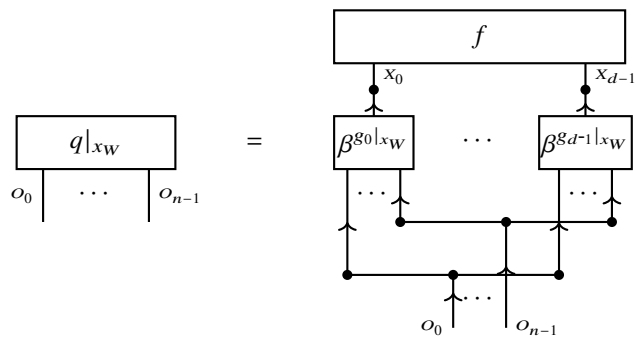
When interpreting the head variables of relational encoded atomic formulas as the atoms of a propositional theory, we find a propositional formula f associated with any decomposable first-order logic formula.

Definition 4.2 Given a formula q in first-order logic, we say that a propositional formula $f\left[X_{[d]}\right]$ is the propositional equivalent to q given atomic formulas g_k in first-order logic, when for any world x_W we have

$$q|_{x_W}\left[O_q\right] = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{g_k}\right] : k \in [d]\} \cup \{f\left[X_{[d]}\right]\}\right\rangle_{\left[O_q\right]} .$$

We here denote the head variables of the basis encoding to $\beta^{g_k|_{x_W}}$ by X_k to highlight their interpretation as propositional atoms.

We depict the relation of a grounding tensor to a propositional formula as:



4.2.5 Quantifiers

Existential and universal quantifiers appear in generic first-order logic and are besides substitutions further means to reduce the number of variables in a formula.

The semantics of existential quantification consists in a formula being true, if at least one state of the quantified variable is true, as we define next.

Definition 4.3 Given a grounding tensor

$$q|_{x_W}\left[O_0, \ldots, O_{n-1}\right]$$

the existential and universal quantification with respect to the first variable are the tensors

$$\left(\exists_{o_0} q\right)|_{x_W}\left[O_1, \ldots, O_{n-1}\right] \quad \text{and} \quad \left(\forall_{o_0} q\right)|_{x_W}\left[O_1, \ldots, O_{n-1}\right]$$

with coordinates as follows. For an assignment $o_1, \ldots, o$ to the non-quantified variables we have

$$\left(\exists_{o_0} q\right)|_{x_W}\left[O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right] = 1$$

if and only if there is an assignment $o_0 \in [r_0]$ such that

$$q|_{x_W}\left[O_0 = o_0, O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right] = 1 .$$

Conversely, we have for the universal quantification that

$$\left(\forall_{o_0} q\right)|_{x_W} [O_1 = o_1, \ldots, O_{n-1} = o_{n-1}] = 1$$

if and only if for any assignment $o_0 \in [r_0]$ we have

$$q|_{x_W} [O_0 = o_0, O_1 = o_1, \ldots, O_{n-1} = o_{n-1}] = 1 \,.$$

Let us now show, that existential and universal quantification are coordinatewise transforms (see Def. **??**) of contracted grounding tensors. To this end, let us introduce the greater-z indicator $\mathbb{I}_{>z}$, where $z \in \mathbb{R}$, as the function

$$\mathbb{I}_{>z} : \mathbb{R} \to \{0, 1\} \quad , \quad \mathbb{I}_{>z}\,(x) = \begin{cases} 1 & \text{if} \quad x > z \\ 0 & \text{else} \end{cases} \,.$$

Theorem 4.2 *For any formula q with variables $O_{[n]}$ we have*

$$(\exists o_0 q)|_{x_W} [O_1, \ldots, O_{n-1}] = \mathbb{I}_{>0} \left(\left\langle q|_{x_W} \right\rangle_{[O_1, \ldots, O_{n-1}]} \right) [O_1, \ldots, O_{n-1}]$$

and

$$(\forall o_0 q)|_{x_W} [O_1, \ldots, O_{n-1}] = \mathbb{I}_{>r-1} \left(\left\langle q|_{x_W} \right\rangle_{[O_1, \ldots, O_{n-1}]} \right) [O_1, \ldots, O_{n-1}]$$

Proof. We proof the claimed equalities to arbitrary slices of the remaining variables, which amount to arbitrary substitutions of the formulas. For any indices $o_1 \in [r_1], \ldots, o_{n-1} \in [r_{n-1}]$ We notice that

$$\begin{aligned} \left\langle q|_{x_W} \right\rangle_{[O_1 = o_1, \ldots, O_{n-1} = o_{n-1}]} &= \sum_{o_0 \in [r_0]} q|_{x_W} [O_0 = o_0, \ldots, O_{n-1} = o_{n-1}] \\ &= \left| o_0 \in [r_0] \;:\; q|_{x_W} [O_0 = o_0, \ldots, O_{n-1} = o_{n-1}] = 1 \right| \,. \end{aligned}$$

We can thus understand the contracted grounding tensor as storing in its coordinates the count of the coordinate extensions to the zeroth variable, such that the grounding tensor is satisfied. This is analogous to our interpretation of contracted propositional formulas as world counts. From this it is obvious, that the existential quantification is satisfied, if the count is different from zero, which is captured by the coordinatewise transform with $\mathbb{I}_{>0}$. We therefore arrive at

$$\left(\exists_{o_0} q\right)|_{x_W} [O_1 = o_1, \ldots, O_{n-1} = o_{n-1}] = \mathbb{I}_{>0} \left(\left\langle q|_{x_W} \right\rangle_{[O_1, \ldots, O_{n-1}]} \right) [O_1 = o_1, \ldots, O_{n-1} = o_{n-1}] \,.$$

The first claim follows, since the assignment to the non-quantified variables was arbitrary. The universal quantification is satisfied, when all extensions are satisfied, and the count is r. Since r is the maximal count, this is captured by the coordinatewise transform with $\mathbb{I}_{>r-1}$ and we get

$$(\forall o_0 q)|_{x_W} [O_1 = o_1, \ldots, O_{n-1} = o_{n-1}] = \mathbb{I}_{>r-1} \left(\left\langle q|_{x_W} \right\rangle_{[O_1, \ldots, O_{n-1}]} \right) [O_1 = o_1, \ldots, O_{n-1} = o_{n-1}] \,.$$

With the same argument, the second claim is established. □

We can extend this discussion towards more generic counting quantifiers, of which the existential and the universal quantifier are extreme cases. One can define quantifiers by demanding that at least $z \in \mathbb{N}$ compatible groundings are satisfied, and show that they amount to coordinatewise transforms with $\mathbb{I}_{>z}$. What is more, quantifiers demanding that at most $z \in \mathbb{N}$ are satisfied would be representable by transforms with an analogously defined function $\mathbb{I}_{\leq z}$. Such customized quantifiers appear for example in the OWL 2 standard of description logics (see [**?**] and Sect. **??**).

As will be discussed in Chapter **??**, any coordinatewise transform can be performed by a contraction of a basis encoding of the tensor with a head vector prepared by the transform function (see Thm. **??**). In the case here, a direct implementation would require a dimension of these head variables by r, which can be infeasible when having large object sets.

To summarize, let us assume a formula is in its prenex normal form, that is a collection of quantifiers are acting on a quantifier free part. We can represent its grounding tensor in three steps:

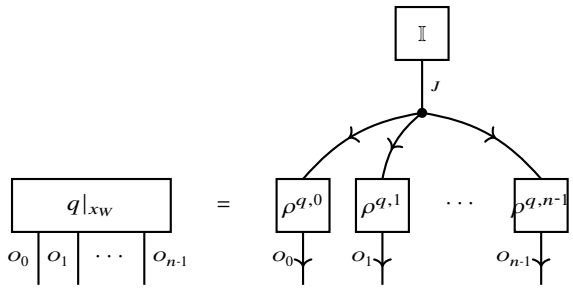


Fig. 4.1 Basis CP Decomposition of the grounding of q, following the scheme of Thm. **??**. Instead of direct storage of the grounding tensor $q|_{x_W}$, the non-zero coordinates are enumerated by a variable J and the corresponding coordinates stored in leg-matrices $\rho^{q,l}$.

- Instantiations of the atom groundings with the assigned variables, as contractions of the basis encoding of the world tensor with atom selecting tensors.
- Propositional formula acting on the head variables of the predicate instantiations, representing the connectives combining the formula.
- Quantifiers as a composition of contractions closing the quantified variable and coordinatewise transforms with the respective greater-than indicators.

4.2.6 Storage in Basis CP Decomposition

In many situations, grounding cores are sparse and representations as single tensor cores comes with a drastic overhead. We often encounter sparse grounding tensors, where the number of non-zero coordinates (to be investigated by basis CP ranks in Chapter **??**) satisfies

$$\ell_0\left(q|_{x_W}\right) << r^{|O_q|}\,.$$

In this case, since most coordinates vanish, the basis CP decomposition (see Sect. **??**) enables a representation of the grounding with significantly lower storage demand, see Thm. **??**. This is particularly useful for representing large relational databases, where each object has only a few relations with others, while the majority of possible relations remains unsatisfied. We depict such CP decomposition of a formula grounding in Thm. 4.1.

Most logical syntaxes exploit ℓ_0-sparsity, explicitly storing only known assertions. The interpretation of unspecified assertions depends on the underlying assumptions. Under the Closed World Assumption, for example, all unspecified assertions are assumed to be false.

4.2.7 Example: Relational Databases

A database is understood as a specific first-order logic world, and are operations on such a single world. Queries are described by a formula p, which are asked against a specific world x_W to retrieve the grounding $p|_{x_W}$. The variables of such formulas are called projection variables. The answer $p|_{x_W}$ of a query is most conveniently represented as a list of solution mappings from the projection variables to objects in the world, such that the query formula is satisfied. Answering a query by solution mappings corresponds with finding the basis CP Decomposition (see Sect. **??**) of $p|_{x_W}$. We can understand these solution mappings as stored in the leg-matrices $\rho^{q,l}$ (see Figure 4.1).

Let us give with the outer join an example of a popular operation to define queries, which efficient execution and storage can be improved based on considerations in the tensor network formalism.

Definition 4.4 (Outer join) Let there be a world x_W and formulas g_ℓ depending on variables $O_{\mathcal{V}^\ell}$, which have grounding tensors by

$$g_\ell|_{x_W}\left[O_v\right] \,:\, \bigtimes_{v\in\mathcal{V}^\ell}\left[r_v\right] \to \{0,1\}\,.$$

Then their (outer) JOIN is defined as the grounding of their conjunctions, as

$$\mathrm{JOIN}\left(g_0,\ldots,g_{p-1}\right)|_{x_W}\left[\bigcup_{\ell\in[p]} O_{\mathcal{V}^\ell}\right] = \left\langle g_\ell|_{x_W}\left[O_{\mathcal{V}^\ell}\right] \,:\, \ell\in[p]\right\rangle_{\left[\bigcup_{l\in[p]} O_{\mathcal{V}^\ell}\right]}\,.$$

We can understand the JOIN of groundings by a factor graph, where each grounding tensor decorates the hyperedge to the node set $\mathcal{V}^\ell$. The projection variable assignment to each formula combined in a JOIN operation provide a basic tensor network format to store the output of the operation. There are thus situations, in which the solution map storage corresponding with a CP Decomposition comes with unnecessary overheads compared with other formats.

We can also understand the JOIN operation as a coordinatewise transform (see Def. **??**) with the product as transform function. To make this connection solid, one would need to extend each joined formula trivially to the variables appearing in other formulas.

The efficiency of evaluating the contraction to a JOIN operation might be improved by understanding it as an Constraint Satisfaction Problem (see Chapter **??**). When applying efficient Message Passing algorithms such as Knowledge Propagation (see Algorithm 2), the groundings can be sparsified by local constraint propagation operations before turning to more global and more demanding contraction operations. Here the groundings $g_\ell|_{x_W}$ would be used to initialize Knowledge Cores κ^e and sequentially sparsified during the algorithm.

4.3 Knowledge bases

FOL knowledge bases without existential quantifiers are understood as contraction equations. That is, exactly those worlds are models, for which

$$\left\langle\left\{q_\ell\left[O_{[r_{q_\ell}]}\right] \,:\, \ell\in[p]\right\}\right\rangle_{[O_{\mathcal{V}}]} = \mathbb{I}\left[O_{\mathcal{V}}\right]\,.$$

4.3.1 Substitution

Understood as slice restriction of the equation: If the whole tensor is trivial, each slice is trivial. Formal as a constraint network manipulation: Adding a sub-contraction (but with basis vector in addition!).

4.4 Inference

Substitutions are used to lift inference to propositional logics:

- Represent the formulas to be infered with the same computation network of atomic formulas.
- Represent the formulas with an entangled activation network (whereas before they were disentangled, since refering to different atomic sentences).

We understand inference as manipulations in the activation network.

- **Unification**: Substitution of variables to represent multiple sentences by same activation network
- **Propositional inference**: Manipulations in activation network, as for constraint networks

4.4.1 Unification

Unification of atomic sentences: Find a substitution to the variables, such that both sentences are represented by the same computation network.

Compressing of complex sentences:

- Find a unification of the atomic sentences in both complex sentences
- Represent the substituted complex sentences by a constraction of the activation networks

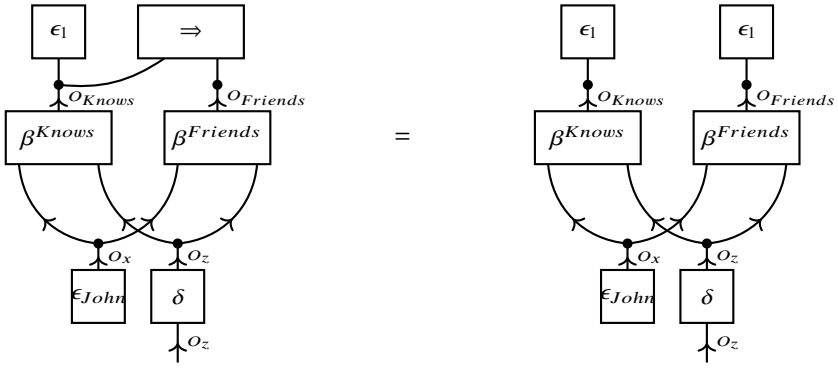
4.4.2 Modus ponens

Example 4.1 For example we have

- John knows everyone: Knows(John,z)
- People who know each other are friends: Knows(x,y) ⇒ Friends(x,y)

The unification of Knows(John,z) and Knows(x,y) is given by the substitution $x \mapsto$ John and $y \mapsto z$. We apply the unification on the second rule and get Knows(John,y) ⇒ Friends(John,y).

Now we do modus ponens on the activation network and get Friends(John,z).



Chapter 5
Circuits and Automata

Abstract We here present tensor network representation of the circuit model of computation.

5.1 Generalized basis encodings

Here: Introduce arbitrary set enumeration variables and basis encodings of functions between sets.

5.2 Circuit decompositions

The neural paradigm of artificial intelligence exploits the decomposition of functions into neurons, which are aligned in a directed acyclic graph. We show in this section how functions decomposable into neurons can be represented by tensor networks. To this end, we formalize discrete neural models by decomposition graphs and formally prove the corresponding decomposition of their basis encodings.

5.2.1 Function decomposition

As a main principle of tensor decompositions, we now show that basis encodings of composition functions are contractions of the basis encodings of their components.

Lemma 5.1 *Let $f\left[X_{[d]}\right]$ be a composition of a p-ary connective function h and functions $f_\ell\left[X_{[d]}\right]$, where $\ell \in [p]$, that is for $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have*

$$f(x_{[d]}) = h\left(f_0\left[x_{[d]}\right], \ldots, f_{p-1}\left[x_{[d]}\right]\right) .$$

Then, we have (see Figure 5.1)

$$\beta^f\left[Y_f, X_{[d]}\right] = \left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell}\left[Y_\ell, X_{[d]}\right] : \ell \in [p]\}\right\rangle_{\left[Y_f, X_{[d]}\right]} .$$

Proof. For any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ we have

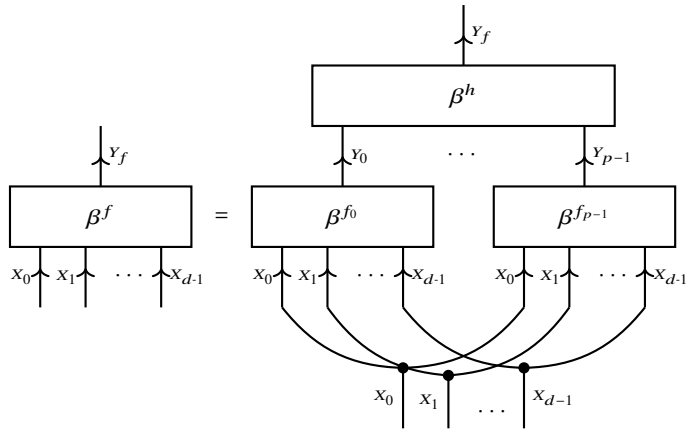


Fig. 5.1 Tensor network decomposition of the basis encoding of a function f, which is the composition of the functions $f_0, \ldots, f_{p-1}$ with a function h.

$$\begin{aligned}
&\left\langle \{\beta^h \left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell} \left[Y_\ell, X_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f, X_{[d]}=x_{[d]}]} \\
&\quad = \left\langle \{\beta^h \left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell} \left[Y_\ell, X_{[d]} = x_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f]} \\
&\quad = \left\langle \{\beta^h \left[Y_f, Y_{[p]}\right]\} \cup \{\epsilon_{f_\ell(x_{[d]})} \left[Y_\ell\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f]} \\
&\quad = \epsilon_{f(x_{[d]})} \left[Y_f\right] \\
&\quad = \beta^f \left[Y_f, X_{[d]} = x_{[d]}\right] .
\end{aligned}$$

Thus, the tensors on both sides of the equation coincide in all slices to $X_{[d]}$ and are thus equal. □

We now define a more generic decomposition of discrete functions.

Definition 5.1 A *circuit* is a directed acyclic hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ such that the following holds.

- Each node $v \in \mathcal{V}$ is decorated by a set $\mathcal{U}^v$ of finite cardinality m_v, a variable X_v, and an index interpretation function

$$I_v \,:\, [m_v] \to \mathcal{U}^v .$$

- Each directed hyperedge $(e^{\text{in}}, e^{\text{out}})$ has at least one outgoing node, that is $e^{\text{out}} \neq \emptyset$, and is decorated by an activation function

$$g^e \,:\, \bigtimes_{v \in e^{\text{in}}} \mathcal{U}^v \to \bigtimes_{v \in e^{\text{out}}} \mathcal{U}^v .$$

- Each node $v \in \mathcal{V}$ appears at most once as an outgoing node.
- The nodes not appearing as an outgoing node are enumerated by $v^{\text{in}}_{[d]}$. We abbreviate the corresponding variables by $X_{v^{\text{in}}_{[d]}} = X_{[d]}$.
- The nodes not appearing as an incoming node are enumerated by $v^{\text{out}}_{[p]}$. We abbreviate the variables by $X_{v^{\text{out}}_{[\ell]}} = Y_{[p]}$.

We assign for each $k \in [d]$ restriction functions

$$q_{v^{\text{in}}_k} \,:\, \bigtimes_{\tilde{k} \in [d]} \mathcal{U}^{v^{\text{in}}_{\tilde{k}}} \to \mathcal{U}^{v^{\text{in}}_k} \quad , \quad x_{[d]}|_k = x_k$$

to the nodes $v^{\text{in}}_{[d]}$ and recursively assign to each further node v a node function

$$q_v \,:\, \bigtimes_{k\in[d]} \mathcal{U}^{v_k^{\text{in}}} \to \mathcal{U}^v \quad , \quad q_v(x_{[d]}) = g^{e_v}\left(\bigtimes_{\tilde{v}\in e^{\text{in}}} q_{\tilde{v}}(x_{[d]})\right)\Big|_v ,$$

where e_v is to each $v \in e^{\text{in}}$ the unique hyperedge with outgoing nodes $\{v\}$. We then call the function

$$q_{\mathcal{G}} \,:\, \bigtimes_{k\in[d]} \mathcal{U}^{v_k^{\text{in}}} \to \bigtimes_{\ell\in[p]} \mathcal{U}^{v_\ell^{\text{out}}} \quad , \quad q_{\mathcal{G}} = \bigtimes_{\ell\in[p]} q_{v_\ell^{\text{out}}}$$

the composition function to the circuit $\mathcal{G}$.

The neural paradigm in AI can be modeled by the existence of circuits for functions on large sets. We now show how circuits enable the sparse representation of composition functions by tensor networks.

Theorem 5.1 *For any circuit $\mathcal{G}$ with composition formula $q_{\mathcal{G}}$, we have*

$$\beta^{q_{\mathcal{G}}}\left[Y_{[p]}, X_{[d]}\right] = \left\langle\left\{\beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \,:\, e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E}\right\}\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} .$$

Proof. We show by induction over the number of edges in $\mathcal{G}$ that, for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ and $v \in \mathcal{V}$, we have

$$\left\langle\left\{\beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \,:\, e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E}\right\}\right\rangle_{\left[X_{\mathcal{V}\setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}\right]} = \bigotimes_{v\in\mathcal{V}\setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] . \tag{5.1}$$

$|\mathcal{E}| = 1$: If there is a single edge $e = (e^{\text{in}}, e^{\text{out}})$ in $\mathcal{E}$, we have $X_{[d]} = X_{e^{\text{in}}}$ and $\mathcal{V}\setminus v_{[d]}^{\text{in}} = e^{\text{out}}$. In this case (5.1) holds since

$$\beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}} = x_{e^{\text{in}}}\right] = \bigotimes_{v\in\mathcal{V}\setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .$$

$(|\mathcal{E}| = n) \Rightarrow (|\mathcal{E}| = n+1)$: Let us now assume that (5.1) holds for all graphs with $|\mathcal{E}| \leq n$ and let $\mathcal{G}$ be a graph with $|\mathcal{E}| = n+1$. We find an edge $\tilde{e} = (e^{\text{in}}, e^{\text{out}})$ such that all nodes in e^{in} are only appearing as outgoing nodes in other edges.

$$\begin{aligned}
&\left\langle\left\{\beta^{g^e}\left[X_e\right] \,:\, e \in \mathcal{E}\right\}\right\rangle_{\left[X_{\mathcal{V}\setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}\right]} \\
&= \left\langle \beta^{\tilde{e}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right], \left\langle\left\{\beta^{g^e}\left[X_e\right] \,:\, e \in \mathcal{E}\setminus\{\tilde{e}\}\right\}\right\rangle_{\left[X_{\mathcal{V}\setminus\{v_{[d]}^{\text{in}}\cup e^{\text{out}}\}}, X_{[d]} = x_{[d]}\right]} \right\rangle_{\left[X_{\mathcal{V}\setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}\right]} \\
&= \left\langle \beta^{\tilde{e}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right], \bigotimes_{v\in\mathcal{V}\setminus\{v_{[d]}^{\text{in}}\cup e^{\text{out}}\}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \right\rangle_{\left[X_{\mathcal{V}\setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}\right]} \\
&= \bigotimes_{v\in\mathcal{V}\setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .
\end{aligned}$$

Here, in the second equation we used the induction hypothesis on the subgraph $(\mathcal{V}, \mathcal{E}\setminus\{\tilde{e}\})$ with $|\mathcal{E}\setminus\{\tilde{e}\}| = n$ and a generic contraction property of basis encodings in the third equation.

Thus, (5.1) holds always and we have for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ that

$$\begin{aligned}\left\langle\left\{\beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \,:\, e=(e^{\text{in}}, e^{\text{out}}) \in \mathcal{E}\right\}\right\rangle_{\left[Y_{[p]}, X_{[d]}=x_{[d]}\right]} &= \left\langle \bigotimes_{v \in \mathcal{V} \setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right]\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} \\ &= \bigotimes_{v \in v^{\text{out}}_{[p]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \\ &= \epsilon_{q_{\mathcal{G}}}\left[Y_{[p]}\right] \\ &= \beta^{q_{\mathcal{G}}}\left[Y_{[p]}, X_{[d]}=x_{[d]}\right] .\end{aligned}$$

Keeping $X_{[d]}$ open, the claim is established. □

When neurons have tunable parameters, we can discretize those by sets $\mathcal{U}^k$ and understand them as additional input variables.

Example 5.1 (Sum of integers in m-adic representation) We develop a tensor network representation of integer summations on the set $[m^d] = \{0, \ldots, m^d - 1\}$, where $m, d \in \mathbb{N}$,

$$+ \,:\, [m^d] \times [m^d] \to [m^{d+1}] \quad , \quad +(i, \tilde{i}) = i + \tilde{i} ,$$

which have a m-adic representation of length d. We define an index interpretation map

$$I \,:\, \bigtimes_{k \in [d]} [m] \to [m^d] \quad , \quad I(x_{[d]}) = \sum_{k \in [d]} x_k \cdot m^k ,$$

which enables the parameterization of $[m^d]$ as the states of d categorical variables $X_{[d]}$ of dimension m. We analogously represent a second set $[m^d]$ by variables $\tilde{X}_{[d]}$ and the set $[m^{d+1}]$ of possible sums by $Y_{[d+1]}$. The basis encoding of the sum is then

$$\beta^+\left[Y_{[d+1]}, X_{[d]}, Y_{[d]}\right] = \sum_{x_{[d]}, \tilde{x}_{[d]}} \epsilon_{I^{-1}(I(x_{[d]})+I(\tilde{x}_{[d]}))}\left[Y_{[d+1]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \epsilon_{\tilde{x}_{[d]}}\left[\tilde{X}_{[d]}\right] .$$

Note that the tensor space of β^+ is of dimension $m^{3 \cdot d+1}$ increasing exponentially in d. Feasible representation of this tensor for large d therefore requires tensor network decompositions, which we now provide based on a decomposition hypergraph. The targeted function to be decomposed is the representation of the integer sum by

$$+^m \,:\, \left(\bigtimes_{k \in [d]} [m]\right) \times \left(\bigtimes_{k \in [d]} [m]\right) \to \bigtimes_{k \in [d+1]} [m] \quad , \quad +^m(x_{[d]}, \tilde{x}_{[d]}) = I^{-1}(I(x_{[d]}) + I(\tilde{x}_{[d]})) .$$

We build a decomposition hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ (see Def. **??**) consisting of $4 \cdot d$ nodes (see Figure 5.2a) . The nodes carry the $(3 \cdot d + 1)$ variables $X_{[d]}, Y_{[d]}, Y_{[d+1]}$ of dimension m constructed above and $d - 1$ auxiliary variables $Z_{[d-1]}$ of dimension 2 representing carry bits. The directed hyperedges of $\mathcal{G}$ are

$$\begin{aligned}\mathcal{E} = &\left\{(\{X_0, \tilde{X}_0\}, \{Y_0, Z_0\})\right\} \cup \left\{(\{Z_{k-1}, X_k, \tilde{X}_k\}, \{Y_k, Z_k\}) \,:\, k \in \{1, \ldots, d-2\}\right\} \\ &\cup \left\{(\{Z_{d-2}, X_{d-1}, \tilde{X}_{d-1}\}, \{Y_{d-1}, Y_d\})\right\}\end{aligned}$$

and are decorated by local summation functions

$$\tilde{+} \,:\, [2] \times [m] \times [m] \to [m] \times [2] \quad , \quad \tilde{+}(z, x, \tilde{x}) = \left((z + x + \tilde{x}) \mod m, \left\lfloor \frac{z + x + \tilde{x}}{m} \right\rfloor\right) .$$

Since to the first hyperedge we do not have a carry bit, the decorating function is the restriction of the first argument to 0.

It is known that the composition of the local summations $\tilde{+}$ is the global summation $+^m$ of integers in m-adic representation. Thus, the composition function $q_{\mathcal{G}}$ is $+^m$. By Thm. 5.1 we have a decomposition of the basis encoding to $q_{\mathcal{G}}$ (see Figure 5.2b) as

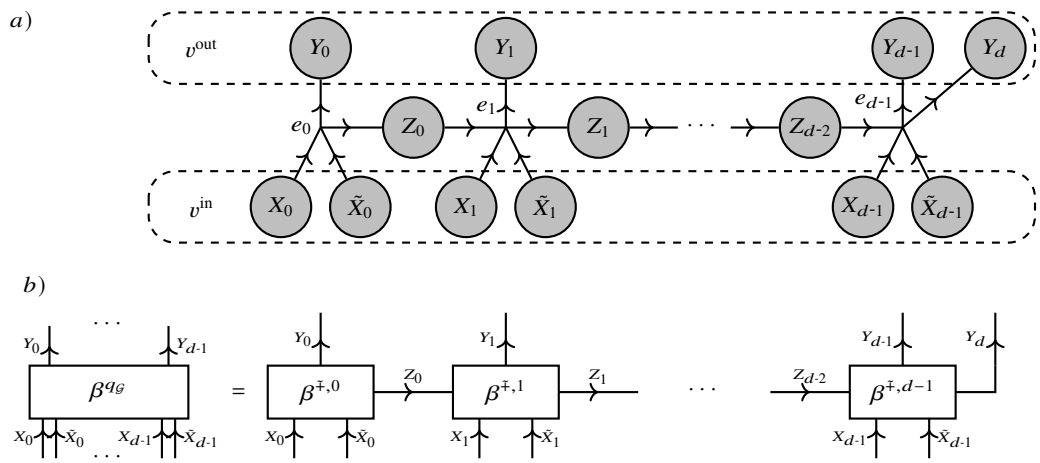


Fig. 5.2 Example of a decomposition hypergraph to the sum of integers (see Example 5.1). a) Hypergraph of directed edges e_k for $k \in [d]$, each decorated by an integer summation + preparing an index Y_k of the resulting sum. b) Corresponding tensor network decomposition of the basis encoded composition function, which is the sum of integers in m-adic representation.

$$\begin{aligned}\beta^{+^m}\left[Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}\right] = \Big\langle &\{\beta^{\mp,0}\left[Y_0, Z_0, X_0, \tilde{X}_0\right]\} \cup \\ &\{\beta^{\mp,k}\left[Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}\right] \,:\, k \in \{1, \dots, d-2\}\} \cup \\ &\{\beta^{\mp,d-2}\left[Y_{d-1}, Y_d, X_{d-1}, \tilde{X}_{d-1}, Z_{d-2}\right]\}\Big\rangle_{\left[Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}\right]}.\end{aligned}$$

5.2.2 Function evaluation by message passing

We are now concerned with an efficient inference algorithm based on tensor network contractions. To evaluate a function given as a tensor network decomposition of its basis encoding, the whole network has to be contracted. As this can be infeasible for large networks, a message passing algorithm based on local contractions can be applied, compare Algorithm 3 for a message passing algorithm for tensor networks on a on tree hypergraph.

Algorithm 6 Directed Belief Propagation

Require: Tensor network $\tau^{\mathcal{G}}$ on a directed hypergraph $\mathcal{G}$
Ensure: Messages $\{\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] \,:\, (e_0, e_1) \in \mathcal{E}^{\to}\}$

Prepare directed message directions

$$\mathcal{E}^{\to} = \left\{\left((e_0^{\text{in}}, e_0^{\text{out}}), (e_1^{\text{in}}, e_1^{\text{out}})\right) \,:\, e_0^{\text{in}} \cap (e_1^{\text{in}}, e_1^{\text{out}}) = \varnothing,\ e_1^{\text{out}} \cap (e_0^{\text{in}}, e_0^{\text{out}}) = \varnothing,\ e_0^{\text{out}} \cap e_1^{\text{in}} \neq \varnothing\right\}$$

Initialize a message queue $S = \{(e_2, e_0) \,:\, e_2 \text{ has empty incoming nodes}\}$
while S not empty **do**
  Pop a (e_0, e_1) pair from S
  Update the message

$$\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] = \left\langle\{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2 \to e_0}\left[X_{e_2 \cap e_0}\right] \,:\, (e_2, e_0) \in \mathcal{E}^{\to},\ e_2 \neq e_1\}\right\rangle_{\left[X_{e_0 \cap e_1}\right]}$$

  Update S by all messages (e_1, e_3) which have not yet been sent, if all messages (e_2, e_1) have been sent.
end while
return Messages $\{\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] \,:\, (e_2, e_0) \in \mathcal{E}^{\to}\}$

We apply the Directed Belief Propagation, Algorithm Algorithm 6, on a circuit, where we add hyperedges to each leaf node and assign one-hot encodings of input states. We then show that the messages are the one-hot encodings to the evaluations of the node functions.

Theorem 5.2 *Let $\mathcal{G}$ be a decomposition graph and let us add hyperedges containing single input nodes, which are decorated by one-hot encodings. Then the messages computed in Algorithm 6 are characterized by*

$$\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] = \bigotimes_{v \in e_0 \cap e_1} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .$$

Proof. We show the theorem inductively over the messages computed in Algorithm 6. The first message is sent from an input edge $\{[k]\}$ to an edge e of the decomposition graph and is by assumption the one-hot encoding of an input state $\epsilon_{x_k}\left[X_k\right]$.

We now assume that all previous messages satisfy the claimed equation at an arbitrary stage of the algorithm. The message computed in the while loop is then a contraction of one-hot encodings with basis encodings and

$$\begin{aligned}
\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] &= \left\langle \{\beta^{g^{e_0}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right]\} \cup \{\chi_{e_2 \to e_0}\left[X_{e_2 \cap e_0}\right] \,:\, (e_2, e_0) \in \mathcal{E}^{\to}\}\right\rangle_{\left[X_{e_0 \cap e_1}\right]} \\
&= \left\langle \{\beta^{g^{e_0}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right]\} \cup \{\epsilon_{q_v(x_{[d]})}\left[X_v\right] \,:\, v \in e^{\text{in}}\}\right\rangle_{\left[X_{e_0 \cap e_1}\right]} \\
&= \bigotimes_{v \in e_0 \cap e_1} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .
\end{aligned}$$

Thus, the new message is also the tensor product of the one-hot encodings of the evaluated node functions. By induction, the property is therefore true for all messages. □

We notice that we can interpret any directed acyclic hypergraph for which each node appears exactly once as an outgoing node and which is decorated by boolean and directed tensors $\tau^{\mathcal{G}}$. Edges with empty incoming sets are carrying one-hot encodings of input states and all further edges carry functions.

Example 5.2 (Continuation of Example 5.1) We now show how Algorithm 6 can be exploited to compute an efficient message passing algorithm for the digits of the m-adic sum. Given two numbers in m-adic representation by the tuples $x_{[d]}$ and $\tilde{x}_{[d]}$, we add the hyperedges with empty incoming nodes and single outgoing node

$$\bigcup_{k \in [d]} \left\{(\emptyset, \{X_k\}), (\emptyset, \{\tilde{X}_k\})\right\}$$

to the hypergraph of Example 5.1 and decorate them by the digit one-hot encodings $\epsilon_{x_k}\left[X_k\right]$ and $\epsilon_{\tilde{x}_k}\left[\tilde{X}_k\right]$ (see Figure 5.3). We then apply the Directed Belief Propagation Algorithm 6. The initial messages queue then consists of the messages from the digit encoding. As sketched in Figure 5.3, to each digit there are three messages (with the exception of the first being two), which can be scheduled in consecutive epochs $\chi^{(k,[3])}$. We then have by Thm. 5.2 for $k \in [d-1]$ that

$$\left\langle \beta^{\tilde{+},k}\left[Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}\right], \chi^{(k-1,2)}[Z_{k-1}], \chi^{(k,0)}[X_k], \chi^{(k,1)}[\tilde{X}_k]\right\rangle_{[Z_k]} = \epsilon_{z_k}\left[Z_k\right] ,$$

where z_k is the value of the k-th carry bit. The k-th digit of the sum y_k can further be obtained by the contraction

$$\left\langle \beta^{\tilde{+},k}\left[Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}\right], \chi^{(k-1,2)}[Z_{k-1}], \chi^{(k,0)}[X_k], \chi^{(k,1)}[\tilde{X}_k]\right\rangle_{[Y_k]} = \epsilon_{y_k}\left[Y_k\right] .$$

Note that the hypergraph representing this instance is a tree and by Thm. 3.14 also the message passing scheme of Algorithm 3 is guaranteed to produce the exact contractions. We can exploit this fact for example in the efficient computation of averages of the summation digits, when we have an elementary distribution of input digits. We emphasize that the directed belief propagation Algorithm 3 is exact even if the hypergraph fails to be a tree, provided that we have directed and boolean tensors..

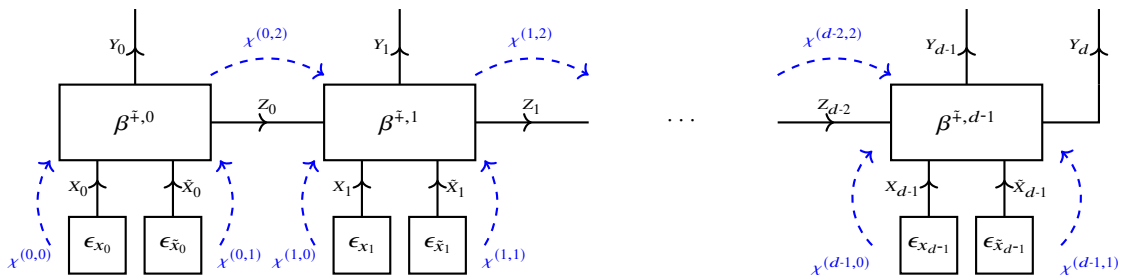


Fig. 5.3 Computation of the integer sum in m-adic representation by the Directed Belief Propagation Algorithm 6 (see Example 5.2). The summands are represented by one-hot encodings of the digits $x_{[d]}$ and $\tilde{x}_{[d]}$, from which the messages start. The k-th digit (for $k \in \{0, \ldots, d-1\}$) of the sum is computed based on the first messages of the epoch labeled by $\chi^{(k,[2])}$, The third message $\chi^{(k,2)}$ in each epoch communicates the carry bit to the next digit summation core. In the last message epoch the digit $d-1$ and d are computed based.

Chapter 6
Causality

Abstract We here describe how to represent Causal AI models in the tnreason formalism.

We find tensor network models for the different rungs of the *ladder of causation* [**?**]. The notes are organized to follow the three rungs of the ladder:

- **Rung 1: Association** – captured by Bayesian Networks
- **Rung 2: Intervention** – captured by do-calculus, for which we introduce intervention variables
- **Rung 3: Counterfactuals** – captured by functional causal models (see Chapter 21 in [**?**]) involving response variables

Following the formalism of Chapter 21 in [**?**]:

- Mutilation of the Bayesian network is captured by contractions with one-hot encodings to do-variables D_k.
- Intervention queries are formalized by contractions with one-hot encodings to all do-variables
- Response variables are formalized by selection variables
- Counterfactual queries are modeled as intervention queries on counterfactual twin networks (see [**?**, Chapter 21])

6.1 Rung 1: Association

See main report: Bayesian Networks are tensor networks and probabilistic queries can be answered by their contractions.

6.2 Rung 2: Intervention

So far, there are several hypergraphs to model a probability distributions by a Bayesian Network. We now want to understand the directed hyperedges as causal relationships between the variables, beyond encoding of conditional independencies (see Example 6.1). To this end, we model the effects of interventions, which will distinguish between multiple Bayesian Networks modeling the same joint distribution.

Example 6.1 (Patient treatment outcome) We model a relationship about the treatment of a sick patient and the outcome of the treatment. Whether a treatment is conducted is modelled by the binary variable X_T and whether the patient improves after the treatment by the variable X_0.

There are in general two possibilities of representing the joint distribution of X_T and X_O by a Bayesian Network:

- Treatment directs to outcome:

$$\mathcal{G} = (\{X_T, X_O\}, \{(\emptyset, X_T), (\{X_T\}, \{X_O\})\})$$

- Outcome directs to treatment:

$$\mathcal{G} = (\{X_T, X_O\}, \{(\emptyset, X_T), (\{X_O\}, \{X_T\})\})$$

In the typical intuition, we expect the treatment to have a causal effect on the outcome and not the other way round. From a Bayesian Network perspective, both models are however equivalent and represent the same joint distribution.

Interventions on Bayesian Networks are modeled by the do-calculus [**?**], which we capture here based on do-variables.

6.2.1 Do calculus

We introduce the do-variables D_k taking values in $[m_k + 1]$ for each variable X_k of dimension m_k, which model the effects of interventions. The states are interpreted as

- $d_k \in [m_k]$: An intervention setting X_k to the value d_k
- $d_k = m_k$: No intervention on X_k

Definition 6.1 The causally augmented conditional probability core is defined as

$$\begin{aligned}\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] =& \mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right] \otimes \epsilon_{m_k}\left[D_k\right] \\ &+ \sum_{d_k \in [m_k]} \epsilon_{d_k}\left[X_k\right] \otimes \epsilon_{d_k}\left[D_k\right] \otimes \mathbb{I}\left[X_{\mathrm{Pa}(k)}\right] .\end{aligned}$$

The causally augmented Bayesian Network is the tensor network

$$\left\langle \{\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] \,:\, k \in [d]\} \right\rangle_{\left[X_{[d]}, D_{[d]}\right]} .$$

The Bayesian Networks investigated so far are retrieved by contractions with $\epsilon_{m_k}\left[D_k\right]$. This corresponds with the situation, where no interventions are performed on any variable.

Lemma 6.1 *For any Bayesian Network $\mathbb{P}\left[X_{[d]}\right]$ on a directed acyclic hypergraph $\mathcal{G} = ([d], \mathcal{E})$ we have*

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] \,:\, k \in [d]\} \cup \{\epsilon_{m_k}\left[D_k\right] \,:\, k \in [d]\} \right\rangle_{\left[X_{[d]}\right]} .$$

Proof. By construction we have

$$\mathbb{P}^c\left[X_k \middle| D_k = m_k, X_{\mathrm{Pa}(k)}\right] = \mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right] .$$

Performing this on each causally augmented conditional distribution retrieves thus the original Bayesian Network. □

6.2.2 Intervention Queries

In a Bayesian Network, when intervening on a variable X_v, the corresponding conditional probability gets trivialized. The probability tensor is then captured by

$$\mathbb{P}\left[X_{\mathcal{U}} \middle| \mathrm{do}\left(X_{\mathcal{W}} = x_{\mathcal{W}}\right)\right] = \left\langle \{\mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right] \,:\, v \notin \mathcal{W}\} \cup \{\frac{1}{m_v}\mathbb{I}\left[X_v, X_{\mathrm{Pa}(v)}\right] \,:\, v \in \mathcal{W}\} \cup \{\epsilon_{x_v}\left[X_v\right] \,:\, v \in \mathcal{W}\} \right\rangle_{\left[X_{\mathcal{U}}\right]} .$$

Since $\frac{1}{m_v}\mathbb{I}\left[X_v, X_{\mathrm{Pa}(v)}\right]$ is directed with $X_{\mathrm{Pa}(v)}$ incoming and X_v outgoing, the partition function of the network stays 1 and the normalization is captured by the contraction.

6.2.3 Simplifications

In general, when all variables are observable, intervention queries can be expressed by a collection of conditional queries (since any cpd of a Bayesian network is a conditional query, and we contract all except those with interventions on the

head variables). In realistic scenarios, however, not all variables are observable (for example, the smoking gene has been only assumed, but no observations have been made). We are interested in situations, where intervention queries can be answered by conditional queries of observable variables. We can proof them in the tensor network formalism based on network separations, which contribution to contractions are scalar multiplications, which therefore are dropped in normalizations.

An example for a simplification is given by the randomized controlled trial.

Example 6.2 (Randomized controlled trial) The randomized controlled trial is a joint distribution of variables

- Confounding variable X_C summarizing other factors affecting the outcome.
- Treatment assignment X_T with values in $\{0, 1\}$. Here, 1 indicates assignment to the treatment group and 0 to the control group. The experiment is conducted such that the treatment variable is independent of the confounding variable X_C.
- Outcome X_O .

We want to estimate the causal effect of the treatment on the outcome, namely calculate the intervention query

$$\mathbb{P}\left[X_O \middle| D_T = 1\right] .$$

By design of the randomized control, the confounding variable has no causal effect on X_T. Thus, we have

$$\mathbb{P}\left[X_O \middle| D_T = 1\right] = \mathbb{P}\left[X_O \middle| X_T = 1\right]$$

that is, the intervention query is equal to the conditional probability.

6.2.3.1 Backdoor Criterion

More general than in the randomized controlled trial we have to deal with confounding variables. The Backdoor Criterion and the Frontdoor Criterion exploit mediator variables X_M, which are also observable and block certain causal paths to unobserved confounding variables X_U.

Lemma 6.2 (Backdoor Criterion) *When X_M blocks any backdoor path from X_0 to X_1 and no node in X_M is a descendant of X_0, then*

$$\mathbb{P}^c\left[X_1 \middle| D_0 = x_0\right] = \left\langle \mathbb{P}\left[X_1 \middle| X_0 = x_0, X_M\right], \mathbb{P}\left[X_M\right] \right\rangle_{[X_1]} .$$

Proof. By assumption, we can contract all variables beside $X_{[2]}, X_M$ into cpds, and have a simplified causal model (see Figure 6.1a). We have

$$\mathbb{P}^c\left[X_0 \middle| D_0 = x_0, X_M\right] = \epsilon_{x_0}\left[X_0\right] \otimes \mathbb{I}\left[X_M\right]$$

and thus

$$\mathbb{P}^c\left[X_1 \middle| D_0 = x_0\right] = \left\langle \mathbb{P}\left[X_1 \middle| X_0 = x_0, X_M\right], \mathbb{P}\left[X_M\right] \right\rangle_{[X_1]} .$$

□

Lemma 6.3 (Backdoor Criterion) *When X_M blocks any front door path from X_0 to X_1 and there is no confounder between X_0 and X_M, as well as between X_M and X_1, then*

$$\mathbb{P}^c\left[X_1 \middle| D_0 = x_0\right] = \left\langle \mathbb{P}\left[X_M \middle| X_0 = x_0\right], \left\langle \mathbb{P}\left[X_1 \middle| X_M, X_0\right], \mathbb{P}\left[X_0\right] \right\rangle_{[X_1, X_M]} \right\rangle_{[X_1]} .$$

Note that the inner contraction has X_0 as a closed variable.

Proof. We show the claim for the situation in Figure 6.2

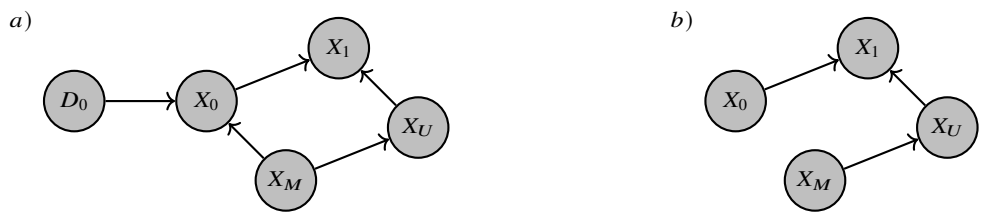


Fig. 6.1 Causal graph for the Backdoor Criterion. a) The mediator variable X_M blocks any backdoor path from X_0 to X_1, potentially through a confounding variable X_U. b) The effect of the intervention on X_0 reduces the Bayesian network effectively to the shown subgraph.

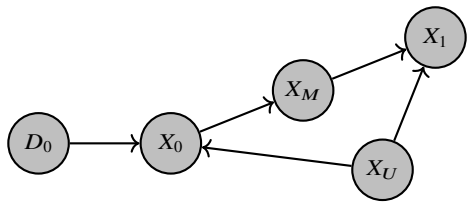


Fig. 6.2 Causal graph for the Frontdoor Criterion.

$$\begin{aligned}\mathbb{P}^c\left[X_1 \middle| D_0 = x_0\right] &= \left\langle \mathbb{P}\left[X_M \middle| X_0 = x_0\right], \mathbb{P}\left[X_1 \middle| X_M, X_U\right], \mathbb{P}\left[X_U\right]\right\rangle_{[X_1]} \\ &= \left\langle \mathbb{P}\left[X_M \middle| X_0 = x_0\right], \left\langle \mathbb{P}\left[X_1 \middle| X_M, X_U\right], \mathbb{P}\left[X_U\right]\right\rangle_{[X_1, X_M]}\right\rangle_{[X_1]} .\end{aligned}$$

We note that

$$\begin{aligned}\left\langle \mathbb{P}\left[X_1 \middle| X_M, X_U\right], \mathbb{P}\left[X_U\right]\right\rangle_{[X_1, X_M = x_M]} &= \mathbb{P}^c\left[X_1 \middle| D_M = x_M\right] \\ &= \left\langle \mathbb{P}\left[X_1 \middle| X_M, X_0\right], \mathbb{P}\left[X_0\right]\right\rangle_{[X_1, X_M = x_M]}\end{aligned}$$

Here we used in the second equation that the variable X_0 blocks all backdoor paths from X_M to X_1 (i.e. the backdoor criterion). Combining both equations proves the claim. □

6.3 Rung 3: Counterfactuals

Following the formalism of Chapter 21 in [**?**], counterfactual queries are modeled as intervention queries on counterfactual twin networks. Counterfactual twin networks are two copies of the original causal model, one representing the factual world and one representing the counterfactual world. They further share response variables, which capture the functional relationships between variables in the causal model. These response variables are modeled as selection variables in the tnreason formalism.

Example 6.3 (Patient counterfactual query) Let us assume, that we know whether a patient did not get assigned to the treatment group, and we know whether he did improve. We want to ask, whether he would have improved, if he had been assigned to the treatment group. This is not an intervention query, but a counterfactual query, since we reason about a different outcome, although we already know that the patient has not been treated.

The causally augmented Bayesian Network is not rich enough to answer such queries. Let us consider a situation where X_T is chosen uniformly from $[2]$. There might be situations, where the probabilistic (and intervention) queries are the same, but the counterfactual queries differ:

(i) The outcome is independent of the treatment.
(ii) With probability 0.5, the treatment always leads to improvement and missing the treatment not. And also with probability 0.5, the treatment never leads to improvement, but missing the treatment does.

Fig. 6.3 Introduction of response variables: a) as a Bayesian network, b) decoration by the basis encoding of the corresponding function.

Intuitively, in the situation (i), the counterfactual query should return probability $1/2$ for improvement under treatment. In the situation (ii), the counterfactual query returns the same value with probability 1 as observed in the real world. Both cases are however indistinguishable by causal models itself, since in both cases the causally augmented conditional probabilities are

$$\mathbb{P}^c\left[X_T \middle| D_T\right] = \frac{1}{2}\mathbb{I}\left[X_T\right] \otimes \epsilon_2\left[D_T\right] + \left(\sum_{d_T \in [2]} \epsilon_{d_T}\left[D_T\right] \otimes \epsilon_{d_T}\left[X_T\right]\right)$$

and

$$\mathbb{P}^c\left[X_O \middle| D_O, X_T\right] = \frac{1}{2}\mathbb{I}\left[X_O, X_T\right] \otimes \epsilon_2\left[D_O\right] + \left(\sum_{d_O \in [2]} \epsilon_{d_O}\left[D_O\right] \otimes \epsilon_{d_O}\left[X_O\right] \otimes \mathbb{I}\left[X_T\right]\right).$$

To answer counterfactual queries, we therefore have to also reason about the mechanisms behind the causal relationships, which we do by introducing response variables in the following.

6.3.1 Response Variables

We introduce response variables L_k, such that X_k has a deterministic dependence on L_k and the parents of a variable to the variable itself. The dependence is captured by a function

$$q_k \,:\, [p_k] \times \left(\bigtimes_{\tilde{k} \in \mathrm{Pa}(k)} [m_{\tilde{k}}]\right) \to [m_k]\,.$$

The response augmented conditional probability core is defined as (see Figure 6.3):

$$\mathbb{P}^f\left[X_k \middle| L_k, X_{\mathrm{Pa}(k)}\right] = \beta^{q_k}\left[X_k, L_k, X_{\mathrm{Pa}(k)}\right]$$

6.3.2 Counterfactual Twinned Network

To answer counterfactual queries, we now construct the counterfactual twinned network (see Figure 6.4).

Definition 6.2 Let there be a response augmented Bayesian Network on a directed acyclic hypergraph $\mathcal{G} = ([d], \mathcal{E})$ with response variables $L_{[d]}$. Further another hypergraph $\tilde{\mathcal{G}}$ and a Bayesian Network on $\tilde{\mathcal{G}}$ modelling the joint distribution of the response variables $L_{[d]}$. Be build counterfactual copies $\tilde{X}_k$ of each variable X_k, which are controlled by the same response variables L_k as the factual variables X_k. The counterfactual twinned network is then the tensor network consists of the causal and response augmented conditional probability cores to $X_{[d]}$, the conditional probability cores of the response variables $L_{[d]}$ and the response augmented conditional probability cores to the counterfactual variables $\tilde{X}_{[d]}$.

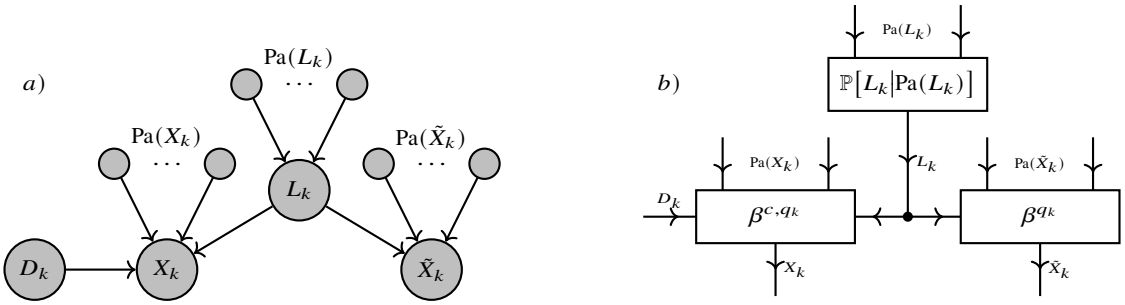


Fig. 6.4 Counterfactual twinned network: a) as a Bayesian network. b) Decorations by causal augmented basis encodings by a do-variable for X_k, a conditional probability distribution for L_k and a basis encoding for the counterfactual variable $\tilde{X}_k$.

Chapter 7
Quantum circuits

Abstract Quantum mechanics is formulated in tensor spaces and quantum circuits are naturally tensor networks. Quantum Circuits are naturally tensor networks. This motivates the investigation, which contractions can be efficiently performed on a Quantum Computer.

7.1 Introduction

Quantum circuits are contractions of gate tensors and therefore tensor networks, and measurement probabilities are given by contractions.

7.1.1 Quantum and Tensor Parallelism

By its central axioms, quantum mechanics of multiple qubits is formulated by tensors capturing states and discrete time evolutions. Motivated by the structural similarity, we investigate how quantum circuits can be utilized for the tensor-network based approach towards efficient and explainable AI in the tnreason framework.

Potential Advantage: *Quantum Parallelism* (see [**?**, Section 3.2.5]).

- Evaluation of multiple function values by single circuit evaluation, we will relate it with the contraction of β^f here.
- Contraction perspective: Loop-tolerant efficient contractions.
- However: Need a generic encoding scheme to exploit this advantage, which is not known yet.
- We here only provide a scheme based on post-selection, which provides a quantum advantage only through amplitude amplification. Without amplitude amplification and post-selection of samples, the encoded distribution is always uniform.
- Further challenge: Application in NISQ devises [**?**], instead of fault-tolerant quantum computers.

Comparison with classical side, which we can call *Tensor Network Parallelism*: Message-passing schemes for efficient contractions, but exact in limited cases.

7.2 Foundations

We present the basic notation of quantum circuits using the tensor network notation used in the tnreason formalism.

7.2.1 Comparing tensor networks and quantum circuits

First of all, we need to extend to complex tensors, which are maps

$$\tau : \bigtimes_{k\in[d]} [2] \to \mathbb{C}$$

with image in $\mathbb{C}$ instead of $\mathbb{R}$ as in the report.

A coarse comparison of the nomenclature used for quantum circuits and tensor networks:

Quantum Circuit	**Tensor Network**
Qubit	Boolean Variable
Quantum Gate	Unitary Tensor
Quantum Circuit	Tensor Network of unitaries on a directed graph

Some constraints appear for a tensor network to be a quantum circuit

- **Incoming-Outgoing structure:** Variable appear at most once as incoming and at most once as outgoing variables. Those not appearing as outgoing (respectively as incoming) are the input and the output variables of the whole circuit.
- **Unitarity of each gate:** That is the variables of each tensor are bipartite into sets A^{in} and A^{out} of same cardinality and the basis encoding with respect to this bipartition, that is

$$T_{\text{in}\to\text{out}}[X^{\text{in}}, X^{\text{out}}] : \bigotimes_{k\in A^{\text{in}}} \mathbb{C}^2 \to \bigotimes_{k\in A^{\text{out}}} \mathbb{C}^2 ,$$

  is a unitary map, that is

$$(T_{\text{in}\to\text{out}})^H \circ (T_{\text{in}\to\text{out}}) = \left\langle T_{\text{in}\to\text{out}}[X^{\text{in}}, Y], \overline{T}_{\text{in}\to\text{out}}[Y, X^{\text{out}}]\right\rangle_{[X^{\text{out}}, X^{\text{in}}]} = \delta\left[X^{\text{out}}, X^{\text{in}}\right] .$$

- **Acyclicity:** Incoming and outgoing variables of each tensor core provide a direction of each edge tensor. With respect to this directionality the graph underlying the tensor network has to be acyclic.

The unitary tensors can be aligned layerwise, if and only if the last two assumption hold, i.e. the directed graph is acyclic and each variable appears at most once as an incoming and at most once as an outgoing variable.

7.2.2 Controlled Single Qubit Gates

We define the rotation gate around the Y-axis by an angle α as

$$R_Y(\alpha)\left[A^{\text{in}}, A^{\text{out}}\right] := \begin{pmatrix} \cos\left(\frac{\alpha}{2}\right) & -\sin\left(\frac{\alpha}{2}\right) \\ \sin\left(\frac{\alpha}{2}\right) & \cos\left(\frac{\alpha}{2}\right) \end{pmatrix} \left[A^{\text{in}}, A^{\text{out}}\right] .$$

Further we define the Pauli-X:

$$\sigma_1\left[A^{\text{in}}, A^{\text{out}}\right] := \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \left[A^{\text{in}}, A^{\text{out}}\right] .$$

Controlled single qubit gates are defined using control qubits, where the gate is applied to the target qubit if the control qubits are in a specific state and the identity is applied otherwise. In the tensor network diagrams, we do not distinguish between incoming and outgoing control qubit variables, since the control acts as a Dirac tensor. Thus, controlled unitary with target qubit X_t and control qubits X_c are represented by tensors

$$\mathcal{U}\left[X_{t,\text{in}}, X_{t,\text{out}}, X_c\right]$$

where for each state x_c to the control variables we have that

$$\mathcal{U}\left[X_{t,\text{in}}, X_{t,\text{out}}, X_c = x_c\right]$$

is a unitary matrix acting on the leg space of the target variable.

7.2.3 Measurements and Contractions

The main difficulty of using quantum circuits as contraction providers is that we can only extract information through measurements. Therefore measurement is the only way to execute contractions of the circuit, which come with restrictions when interested in contraction with open variables.

7.2.3.1 Computational basis measurement

The computational basis measurement of the qubits X_A of a Quantum State $\psi\left[X_{[d]}\right]$ is equal to drawing samples from a distribution

$$\mathbb{P}\left[X_A\right] = \left\langle \psi\left[X_{[d]}\right], \psi^*\left[X_{[d]}\right]\right\rangle_{[X_A]} .$$

Here $\psi^*\left[X_{[d]}\right]$ is the complex conjugate of $\psi\left[X_{[d]}\right]$. When ψ is prepared by a quantum circuit acting on a initial state, the complex conjugate is the hermitean conjugate of the circuit acting on the complex conjgate of the initial state.

We abbreviate these contractions by extending the contraction diagrams with measurement symbols (see Figure 7.1).

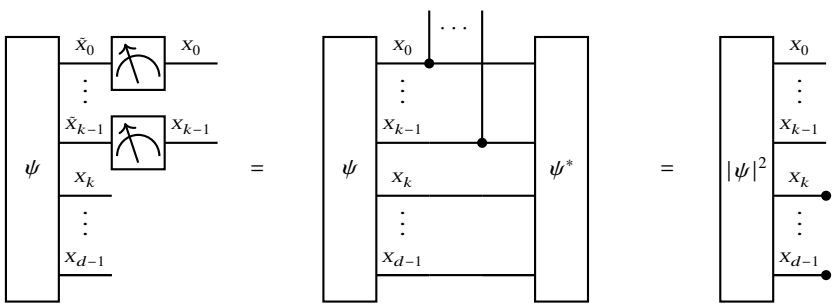


Fig. 7.1 Computational Basis Measurement of a quantum state ψ. The measurement symbols on the left side indicate the measured qubits and the first equation is understood as a definition. In the second equation we sketch, that the measurement distribution is equal to the contraction of the square absolute transform of ψ to the measured variables.

Each complex-valued tensor $\psi\left[X_{[d]}\right]$ has a decomposition into a phase tensor $\phi\left[X_{[d]}\right]$ and an absolute tensor $|\psi|\left[X_{[d]}\right]$ defined by

$$\psi\left[X_{[d]}\right] = \left\langle \exp\left[i \cdot \phi\left[X_{[d]}\right]\right], \left|\psi\left[X_{[d]}\right]\right|\right\rangle_{[X_{[d]}]} .$$

The measurement distribution depends only on $|\phi|$, that is

$$\mathbb{P}\left[X_{[d]}\right] = \left|\psi\left[X_{[d]}\right]\right|^2 .$$

Note, that this measurement distribution is not sensitive to the phases of the amplitudes.

When only a subset of variables is measured, the distribution is the contraction of the absolute square transform (these operations do not commute)

$$\mathbb{P}\left[X_{\mathcal{U}}\right] = \left\langle \left|\psi\left[X_{[d]}\right]\right|^2\right\rangle_{[X_{\mathcal{U}}]} .$$

When we are interested in the preparation of quantum states with a specific computational basis measurement distribution, we can restrict to states with vanishing phase cores, that is

$$\psi\left[X_{[d]}\right] = \left\langle \exp\left[i \cdot 0\left[X_{[d]}\right]\right], |\psi|\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = |\psi|\left[X_{[d]}\right].$$

7.2.3.2 Phase sensitive measurement

When applying a Walsh-Hadamard transform on the qubits $\mathcal{V}\backslash\mathcal{U}$ to be closed (see Figure 7.2), the probability of measuring the closed qubits in the ground state is

$$\mathbb{P}\left[X_{\mathcal{V}\backslash\mathcal{U}} = 0_{\mathcal{V}\backslash\mathcal{U}}, X_{\mathcal{U}}\right] = \frac{1}{2^{|\mathcal{U}|}} \cdot \left|\left\langle \psi\left[X_{[d]}\right]\right\rangle_{[X_{\mathcal{U}}]}\right|^2.$$

We thus sample from the contracted quantum state. Note that in this way, the relative phases of the amplitudes are contracted before the absolutes are taken. In this way, the measurement distribution is sensitive to the relative phases. This property is exploited in the Deutsch-Jozsa algorithm, which utilizes signs of the amplitudes.

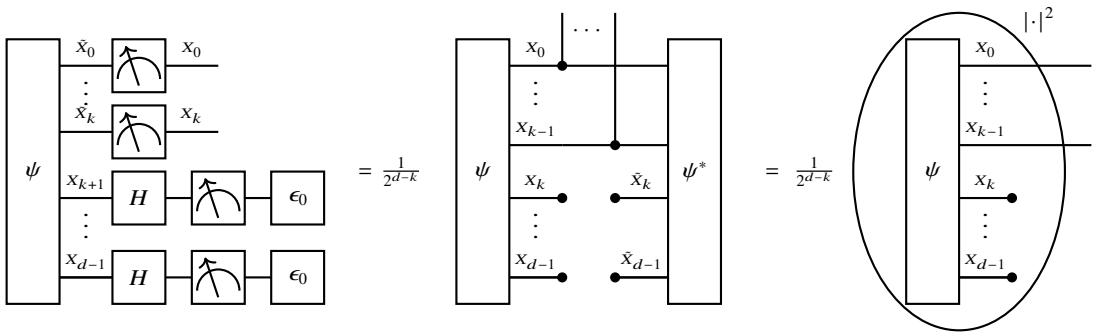


Fig. 7.2 Computational Basis Measurement of a quantum state ψ, after the Walsh-Hadamard Transform on the closed qubits. By the ellipse we indicate a coordinatewise transform to the square of the absolute, after the contraction.

7.2.3.3 POVM measurements

The most general measurement formalism is through a POVM, a set

$$\{E_y[X^{\text{in}}, X^{\text{out}}] \,:\, y \in [r]\}$$

of positive operators with

$$\sum_{y\in[r]} E_y[X^{\text{in}}, X^{\text{out}}] = \delta\left[X^{\text{in}}, X^{\text{out}}\right]$$

Measuring a pure state $|\psi\rangle$ we get the outcome y with probability

$$\left\langle \psi\left[X^{\text{out}}\right], E_y[X^{\text{in}}, X^{\text{out}}], \psi\left[X^{\text{in}}\right]\right\rangle_{[\varnothing]}$$

Introducing Y as an enumeration we define the tensor

$$E[Y, X^{\text{in}}, X^{\text{out}}] = \sum_{y\in[r]} \epsilon_y\left[Y\right] \otimes E_y[X^{\text{in}}, X^{\text{out}}].$$

The condition on E is then

$$\left\langle E[Y, X^{\text{in}}, X^{\text{out}}]\right\rangle_{\left[X^{\text{in}}, X^{\text{out}}\right]} = \delta\left[X^{\text{in}}, X^{\text{out}}\right] .$$

The measurement distribution of a pure state is then

$$\left\langle \psi\left[X^{\text{out}}\right], E[Y, X^{\text{in}}, X^{\text{out}}], \psi\left[X^{\text{in}}\right]\right\rangle_{[Y]} .$$

When having a mixed state defined by a density matrix $\rho[X^{\text{in}}, X^{\text{out}}]$, the distribution is

$$\left\langle E[Y, X^{\text{in}}, X^{\text{out}}], \rho[X^{\text{in}}, X^{\text{out}}]\right\rangle_{[Y]} .$$

7.3 Graph Controlled Circuits

All circuits presented in this work are graph controlled circuits, which are a directed class of quantum circuits. We here show, that their measurement distributions are equivalent with Bayesian networks.

7.3.1 Definition

We build on the definition of uniformly controlled unitaries (see [**?**]), and study their alignment along directed acyclic hypergraphs. We define graph-controlled circuits using hypergraphs, which nodes are associated with the qubits of the circuit. The hyperedges are tuples of two subsets of the qubits, namely the incoming qubits representing control and the outgoing qubit representing the target of a unitary. To each hyperedge we then associate a uniformly controlled unitary.

Definition 7.1 (Graph-Controlled Circuit) Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed acyclic hypergraph, where each hyperedge has exactly one outgoing node and all nodes appear exactly once as outgoing nodes of an hyperedge. Then a by $\mathcal{G}$ controlled circuit is a decoration of the edges $e = (e^{\text{in}}, \{v\}) \in \mathcal{E}$ by uniformly controlled unitaries

$$\mathcal{U}^e\left[X_{v,\text{in}}, X_{v,\text{out}}, X_{e^{\text{in}},\text{out}}\right] .$$

Example 7.1 Each activation circuit is a graph-controlled circuit, where the corresponding hypergraph consists in $\mathcal{V} = X_{[d]} \cup \{A\}$ and a single edge $\mathcal{E} = (X_{[d]}, \{A\})$.

Since the hypergraph $\mathcal{G}$ to a graph-controlled circuit is acyclic, we find an order of the qubits, such that to each qubit appearing in the outgoing qubits of a hyperedge is ordered after the incoming qubits. In such way, we have a unique definition of the graph-controlled circuit as a concatenation of the controlled unitaries respecting a directionality order. When there are multiple orders respecting the directionality, then the concatenation respecting the orders all lead to an equivalent circuit.

Example 7.2 (Generic State Representation) In [**?**] a preparation scheme for arbitrary states with real and positive amplitudes by graph-controlled circuits is presented. Therein, the hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is constructed as follows:

- Nodes $\mathcal{V} = [d]$
- Edges $\mathcal{E} = \{([k], \{k\}) \,:\, k \in [d]\}$

It is possible to realize each controlled unitary by a sequence of single qubit rotations and C-NOTs (see [**?**]), where the angles are computed using gray codes.

7.3.2 Preparation of q-samples to Bayesian Networks

Here: Bayesian networks with boolean variables

7.3.2.1 Activation circuits

We now study circuits preparing q-samples of Bayesian Networks.

Activation circuits are uniformly controlled unitaries (see [**?**]), where the rotation axis is chosen as the y-axes of the Bloch sphere, and the angles are computed by the function $h(\cdot)$ acting on a function value to be encoded.

We define the angle preparing function on $p \in [0,1]$ by

$$h(p) = 2 \cdot \cos^{-1}\left(\sqrt{1-p}\right) .$$

For any $p \in [0,1]$ we then have

$$\left\langle \epsilon_0 \left[A^{\text{in}}\right], R_Y\left(h(p)\right)\left[A^{\text{in}}, A^{\text{out}}\right]\right\rangle_{[A^{\text{out}}]} = \begin{pmatrix} \sqrt{1-p} \\ \sqrt{p} \end{pmatrix} \left[A^{\text{out}}\right] .$$

Definition 7.2 (Activation circuit) Given a function

$$f : \bigtimes_{k \in [d]} [m_k] \to [0,1]$$

its activation circuit is the uniformly controlled unitary $\mathcal{V}^f\left[A^{\text{in}}, A^{\text{out}}, X_{[d]}\right]$ defined as

$$\mathcal{V}^\tau\left[A^{\text{in}}, A^{\text{out}}, X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k \in [d]} [m_k]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes R_Y\left(h(f\left[x_{[d]}\right])\right)\left[A^{\text{in}}, A^{\text{out}}\right] .$$

When we have a probability tensor, its activation circuit be prepared, since all values are in $[0,1]$. Note that for rejection sampling, only the quotients of the values are important, we can therefore scale the value by a scalar such that the mode is 1.

Tensors $\tau\left[X_{[d]}\right]$ with non-negative coordinates can be encoded after dividing them by their maximum, that is the activation circuit of the function

$$f\left[x_{[d]}\right] = \frac{\tau\left[X_{[d]} = x_{[d]}\right]}{\max_{\tilde{y}_{[p]}} \tau\left[X_{[d]} = \tilde{y}_{[p]}\right]}$$

When the maximum of the tensor is not known, it can be replaced by an upper bound (reducing the acceptance rate of the rejection sampling).

7.3.2.2 Q-samples of Bayesian Networks

Q-samples of Bayesian Networks can be prepared by quantum circuits, where each conditional probability distribution is prepared by an activation circuit.

Theorem 7.1 ([?]) *Let there be a Bayesian Network $\mathbb{P}\left[X_{[d]}\right]$ on a directed acyclic hypergraph $\mathcal{G} = ([d], \mathcal{E})$ with dimensions $m_k = 2$ and decorations of the edges $\mathcal{E} = \{(\text{Pa}(k), \{k\}) : k \in [d]\}$ by conditional probabilities*

$$\mathbb{P}\left[X_k \middle| X_{\text{Pa}(k)}\right] .$$

To each $k \in [d]$ we construct the function

$$q_k : \bigtimes_{\tilde{k} \in \text{Pa}(k)} [2] \to [2] \quad , \quad q_k(x_{\text{Pa}(k)}) = \mathbb{P}\left[X_k = 1 \middle| X_{\text{Pa}(k)} = x_{\text{Pa}(k)}\right] .$$

Then the concatenation of the activation circuits (see Figure 7.3)

$$\mathcal{V}^{q_k}\left[X_k^{\text{in}}, X_k, X_{\text{Pa}(k)}\right]$$

respecting the order $[d]$ *prepares the q-sample to the Bayesian Network* $\mathbb{P}\left[X_{[d]}\right]$ *when acting on the initial state* $\bigotimes_{k\in[d]} \epsilon_0 \left[X_k^{\text{in}}\right]$.

Proof. Let $\psi\left[X_{[d]}\right]$ be the state prepared by the activation circuits to the conditional probability distributions. We need to show that the measurement distribution of ψ is the Bayesian Network. For any $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have

$$\begin{aligned}\left|\psi\left[X_{[d]} = x_{[d]}\right]\right|^2 &= \prod_{k\in[d]} \left(\mathcal{V}^{q_k}\left[X_k^{\text{in}} = 0, X_k = x_k, X_{\text{Pa}(k)} = x_{\text{Pa}(k)}\right]\right)^2 \\ &= \prod_{k\in[d]} \mathbb{P}\left[X_k = x_k \middle| X_{\text{Pa}(k)} = x_{\text{Pa}(k)}\right] \\ &= \mathbb{P}\left[X_{[d]} = x_{[d]}\right] .\end{aligned}$$

□

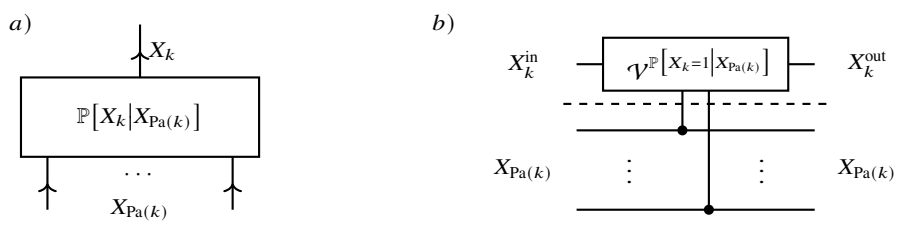


Fig. 7.3 Representation of directed and positive tensor by a controlled rotation. a) Conditional probability tensor $\mathbb{P}\left[X_k \middle| X_{\text{Pa}(k)}\right]$ being a tensor in a Bayesian Network. b) Circuit Encoding as a controlled rotation, which is the Activation circuit of the tensor $\mathbb{P}\left[X_k = 1 \middle| X_{\text{Pa}(k)}\right]$.

Example 7.3 (Encoding of discretized distributions) We here review the preparation of a discretized distribution from [**?**] as an application of activation circuits. Let $\mathbb{Q}[X]$ be a continuous probability distribution of a variable X taking values in $[0, 1]$. For any $d \in \mathbb{N}$ we define its discretization of order d as a discrete probability distribution $\mathbb{P}\left[X_{[d]}\right]$ with coordinates

$$\mathbb{P}^d\left[X_{[d]} = x_{[d]}\right] = \mathbb{Q}\left[X \in [\sum_{k\in[d]} 2^{-(k+1)} x_k, 2^{-d} + \sum_{k\in[d]} 2^{-(k+1)} x_k]\right] .$$

Then we have for any $p \in \mathbb{N}$ with $p \leq d$

$$\mathbb{P}^p\left[X_{[p]}\right] = \left\langle \mathbb{P}\left[X_{[d]}\right]\right\rangle_{\left[X_{[p]}\right]} .$$

The discretization order is thus increased by the contraction

$$\mathbb{P}^{d+1}\left[X_{[d+1]}\right] = \left\langle \mathbb{P}^d\left[X_{[d]}\right], \mathbb{P}^{d+1}\left[X_d | X_{[d]}\right]\right\rangle_{\left[X_{[d+1]}\right]} .$$

Having an activation circuit preparing $\mathbb{P}^d$ we can therefore prepare $\mathbb{P}^{d+1}$ by adding the activation circuit for the function

$$x_{[d]} \to \mathbb{P}^{d+1}\left[X_d = 1 | X_{[d]} = x_{[d]}\right] .$$

Note that the function evaluation amounts to the integration of $\mathbb{Q}$ as

$$\mathbb{P}^{d+1}\left[X_d = 1 | X_{[d]} = x_{[d]}\right] = \frac{\mathbb{Q}\left[X \in [2^{-(d+1)} + \sum_{k\in[d]} 2^{-(k+1)} x_k, 2^{-d} + \sum_{k\in[d]} 2^{-(k+1)} x_k]\right]}{\mathbb{Q}\left[X \in [\sum_{k\in[d]} 2^{-(k+1)} x_k, 2^{-d} + \sum_{k\in[d]} 2^{-(k+1)} x_k]\right]} .$$

7.3.3 Equivalence with Bayesian Networks

Above we have shown, how Bayesian Networks can be prepared by activation circuits, which are a class of graph-controlled circuits. We now show, that the measurement distribution of any graph-controlled circuit is a Bayesian Network. It then follows that the set of distributions prepared by graph-controlled circuits is exactly the set of Bayesian Networks.

This is closely connected to the chain decomposition of probability distributions, and their construction. In [**?**] Bayesian Networks are prepared by graph-controlled circuits. When in the graph $\mathcal{G}$ each node appears at most once as outgoing node, it is also the hypergraph to a family of Bayesian Networks. The measurement distributions of a state prepared by a $\mathcal{G}$-controlled Circuit acting on disentangled initial states are exactly the Bayesian Networks with respect to $\mathcal{G}$.

Theorem 7.2 *Let $\mathcal{G}$ be a directed acyclic hypergraph, such that node appears at most once as an outgoing node. The measurement distributions of the by $\mathcal{G}$ controlled circuits acting on disentangled initial states are equal to the Bayesian Networks on $\mathcal{G}$.*

Lemma 7.1 *Let $\mathcal{G}$ be a directed acyclic hypergraph, such that node appears at most once as an outgoing node. Then any Bayesian network on $\mathcal{G}$ can be prepared by a $\mathcal{G}$-controlled circuit with activation circuits of the conditional probability tensors.*

Proof. Let $\mathbb{P}\left[X_{[d]}\right]$ be a Bayesian network on the graph $\mathcal{G}$. Enumerate the nodes $\mathcal{V}$ of the $\mathcal{G}$ by $[d]$, such that for each $k \in [d]$ we have $\mathrm{Pa}(k) \subset [k]$. Then define a $\mathcal{G}$-controlled circuit, by choosing for each $k \in [d]$ controlled unitaries which satisfy

$$\mathcal{U}^k\left[X_k^{\mathrm{in}} = 0, X_{v,\mathrm{out}}, X_{\mathrm{Pa}(k),\mathrm{out}}\right] = \sqrt{\mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right]}\,.$$

Here we specified only the action of the controlled unitary on the basis vector $\epsilon_0\left[X_k\right]$, the action on $\epsilon_1\left[X_k\right]$ can be chosen by an arbitrary orthogonal unit vector. For more explicit construction, see the activation circuits. Any such defined $\mathcal{G}$-controlled circuit acting on the initial state $\bigotimes_{k\in[d]}\epsilon_0\left[X_k\right]$ prepares a quantum state $\psi\left[X_{[d]}\right]$ with measurement distribution

$$|\psi|^2\left[X_{[d]}\right]\,.$$

Given arbitrary $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ we have

$$|\psi|^2\left[X_{[d]} = x_{[d]}\right] = \prod_{k\in[d]}\mathbb{P}\left[X_k = x_k \middle| X_{\mathrm{Pa}(k)} = x_{\mathrm{Pa}(k)}\right] = \mathbb{P}\left[X_{[d]} = x_{[d]}\right]\,.$$

Here we used in the last equation, that $\mathbb{P}\left[X_{[d]}\right]$ is a Bayesian network. Since the equivalence holds for any coordinate, this establishes the equivalence of the measurement distribution of $\psi\left[X_{[d]}\right]$ and $\mathbb{P}\left[X_{[d]}\right]$. □

While we have already shown by Lem. 7.1 that arbitrary Bayesian Networks can be prepared by graph-controlled circuits, we now show the converse, namely that any distribution prepared by graph-controlled circuits is a Bayesian Network. To this end, we use the characterization of Bayesian Networks by the conditional independencies they encode (see [**?**]). To be more precise, we need to show that any node variable is conditionally independent of its non-descendants given its parents.

Lemma 7.2 *Let $(\mathcal{G},\mathcal{U})$ be a $\mathcal{G}$-controlled circuit acting on a disentangled initial state and $\mathbb{P}\left[X_{\mathcal{V}}\right]$ the corresponding measurement distribution. Then we have for each $v \in \mathcal{V}$ the conditional independence*

$$\left(X_v \perp X_{\mathrm{NonDes}(v)}\right) \,\middle|\, X_{\mathrm{Pa}(v)}\,.$$

Proof. We choose to a given $v \in \mathcal{V}$ an enumeration $[d]$ of the nodes, such that for each $k \in [d]$ we have $\mathrm{Pa}(k) \subset [k]$ and for the enumerator $\tilde{k}$ of v we further have $\mathrm{NonDes}(\tilde{k}) \subset [\tilde{k}]$. Let $\mathbb{P}\left[X_{[d]}\right]$ be the measurement distribution of the $\mathcal{G}$-controlled circuit acting on a disentangled initial state $\bigotimes_{k\in[d]}\psi^k\left[X_k\right]$ and choose arbitrary $x_{[k]}$. We then have

$$\begin{aligned}
\mathbb{P}\left[X_{\tilde{k}}, X_{[k]} = x_{[k]}\right] &= \left\langle \left(\bigcup_{k\in[d]} \{\mathcal{U}^k, \mathcal{U}^{k,\dagger}, \psi^k, \psi^{k,*}\right) \} \cup \left(\bigcup_{k\in[\tilde{k}]} \epsilon_{x_k}\left[X_k^{\text{out}}\right]\right)\right\rangle_{[X_{\tilde{k}}]} \\
&= \left\langle \bigcup_{k\in[\tilde{k}]} \{\mathcal{U}^k, \mathcal{U}^{k,\dagger}, \psi^k, \psi^{k,*}, \epsilon_{x_k}\left[X_k^{\text{out}}\right]\}\right\rangle_{[X_{\tilde{k}}]} \\
&= \left|\left\langle \mathcal{U}^{\tilde{k}}\left[X_{\tilde{k},\text{in}}, X_{\tilde{k},\text{out}}\right], X_{\text{Pa}(\tilde{k}),\text{out}} = x_{\text{Pa}(\tilde{k}),\text{out}}\right\rangle_{[X_{\tilde{k},\text{out}}]}\right|^2 \\
&\quad \cdot \prod_{k\in[\tilde{k}]} \left(\left\langle \psi^k\left[X_k^{\text{in}}\right], \mathcal{U}^k\left[X_k^{\text{in}}, X_k^{\text{out}}, X_{\text{Pa}(k),\text{out}} = x_{\text{Pa}(k),\text{out}}\right]\right\rangle_{[\varnothing]}\right)^2
\end{aligned}$$

Here we used in the second equation the unitarity of the controlled unitaries to $k \notin [\tilde{k}]$. Since the indices $x_{[\tilde{k}]/\text{Pa}(\tilde{k})} = x_{\text{NonDes}(\tilde{k})}$ appear only in the constant term, we conclude

$$\mathbb{P}\left[X_{\tilde{k}} \middle| X_{[k]}\right] = \mathbb{P}\left[X_{\tilde{k}} \middle| X_{\text{Pa}(\tilde{k})}\right] \otimes \mathbb{I}\left[X_{\text{NonDes}(\tilde{k})}\right] ,$$

which establishes the conditional independence $\left(X_v \perp X_{\text{NonDes}(v)}\right) \mid X_{\text{Pa}(v)}$. □

Proof of Thm. 7.2. The theorem follows directly from Lem. 7.1 and Lem. 7.2, using that Bayesian Networks are characterized by the conditional independence of each variable to its non-descendants given its parents. □

Example 7.4 (Chain decomposition) The hypergraph in Example 7.2 corresponds with a family of Bayesian Networks subsuming any probability distribution. This can be verified based on the chain decomposition of a generic distribution.

Another question is, whether each quantum state, which measurement distribution is a Bayesian Network can be prepared by a $\mathcal{G}$-controlled circuit. This is not always the case, since the phase tensor does not influence the measurement distribution. Any phase tensor, of a by $\mathcal{G}$-controlled circuit prepared state has however a decomposition

$$\phi\left[X_{[d]}\right] = \sum_{k\in[d]} \phi^k\left[X_k, X_{\text{Pa}(k)}\right] \otimes \mathbb{I}\left[X_{[d]/\{\{k\}\cup\text{Pa}(k)\}}\right] ,$$

where the phase cores ϕ^k can be read of the controlled unitaries. When there are phase cores which do not have such a decomposition, the corresponding states are not representable. [**?**] shows, that when the graph is chosen as in Example 7.2, then also the phases can be represented.

7.4 Ancilla Augmentation

We provide here some further theory underlying the ancilla augmentation of Computation-Activation Networks.

Graph controlled circuits prepare Bayesian Networks, and do not apply to undirected graphical models such as Markov Networks and Computation-Activation Networks. We therefore introduce in the following ancilla augmentation, which is a method to represent Markov Networks as conditioned Bayesian Network. This enables usage of the previously used preparation schemes for Bayesian Networks.

7.4.1 Post-selection by Ancilla Variables

First of all we define ancilla augmentation and then show how they can be prepared as measurement distributions using activation circuits.

Definition 7.3 (Ancilla Augmentation of a Distribution) Let $\mathbb{P}\left[X_{[d]}\right]$ be a probability distribution of variables $X_{[d]}$. Another distribution $\tilde{\mathbb{P}}[X_{[d]}, A_{[p]}]$ of variables $X_{[d]}$ and ancilla variables $A_{[p]}$ is called an ancilla augmentation of $\mathbb{P}\left[X_{[d]}\right]$, if there is $\lambda > 0$ with

$$\lambda \cdot \mathbb{P}\left[X_{[d]}\right] = \tilde{\mathbb{P}}[X_{[d]}, A_{[p]} = 1_{[p]}] \,.$$

We refer to λ as the corresponding acceptance probability.

Note that $\lambda \leq 1$ since

$$\lambda = \left\langle \lambda \cdot \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \left\langle \tilde{\mathbb{P}}[X_{[d]}, A_{[p]} = 1_{[p]}]\right\rangle_{[\varnothing]} \leq \left\langle \tilde{\mathbb{P}}[X_{[d]}, A_{[p]}]\right\rangle_{[\varnothing]} = 1 \,.$$

Sampling from the distribution can be done by rejection sampling, also called post selection, on the ancilla augmented distribution in the following way. When drawing a sample $(x_{[d]}, a)$ from $\tilde{\mathbb{P}}[X_{[d]}, A]$ and rejecting it whenever $a = 0$, the accepted samples are distributed as $\mathbb{P}\left[X_{[d]}\right]$. We then have λ as the probability of accepting a sampling in this rejection sampling method. The acceptance probability is critical in determining the efficiency of this approach, since we need to draw $\sim \frac{1}{\lambda}$ samples from $\tilde{\mathbb{P}}[X_{[d]}, A]$ to get an accepted sample.

Since

$$\tilde{\mathbb{P}}[X_{[d]}, A] = \left\langle \tilde{\mathbb{P}}[X_{[d]}], \tilde{\mathbb{P}}[A|X_{[d]}]\right\rangle_{\left[X_{[d]}, A\right]} \tag{7.1}$$

the described sampling procedure is equal to drawing $x_{[d]}$ from the marginal

$$\tilde{\mathbb{P}}[X_{[d]}]$$

and accepting the sample with probability

$$\tilde{\mathbb{P}}[A = 1|X_{[d]}] \,.$$

7.4.2 Construction based on Uniform Distributions

We construct ancilla augmentations based on the decomposition (7.1) where the marginal of $X_{[d]}$ is the uniform distribution, that is

$$\tilde{\mathbb{P}}[X_{[d]}] = \frac{1}{\prod_{k\in[d]} m_k} \mathbb{I}\left[X_{[d]}\right]$$

and we construct the conditional probability of the ancilla variable such that

$$\tilde{\mathbb{P}}[A = 1|X_{[d]}] = \lambda \cdot \left(\prod_{k\in[d]} m_k\right) \mathbb{P}\left[X_{[d]}\right] \,.$$

For the latter we can apply the preparation schemes of conditioned distributions, which have been studied above for Bayesian Networks.

We denote for any non-negative tensor $\tau\left[X_{[d]}\right]$ we construct

$$\begin{aligned}\mathcal{A}^{\tau,\lambda}\left[A, X_{[d]}\right] = \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \Big(&\lambda \cdot \big(\prod_{k\in[d]} m_k\big) \cdot \tau\left[X_{[d]} = x_{[d]}\right] \cdot \epsilon_1\left[A\right] \\ &+ \big(1 - \lambda \cdot \big(\prod_{k\in[d]} m_k\big) \cdot \tau\left[X_{[d]} = x_{[d]}\right]\big) \cdot \epsilon_0\left[A\right]\Big) \,.\end{aligned}$$

For this to be a conditional distribution we need to demand that

$$\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \lambda \cdot \big(\prod_{k\in[d]} m_k\big) \cdot \tau\left[X_{[d]} = x_{[d]}\right] \leq 1 \,,$$

which is equivalent to

$$\lambda \leq \left(\prod_{k\in[d]} m_k \right)^{-1} \cdot \left(\max_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \tau \left[X_{[d]} = x_{[d]} \right] \right)^{-1} .$$

Note that if $\tau \left[X_{[d]} \right]$ is the uniform distribution $\mathbb{I} \left[X_{[d]} | \varnothing \right]$, then we can achieve the best acceptance rate $\lambda = 1$.

Given a distribution $\mathbb{P} \left[X_{[d]} \right]$ we add an ancilla variable A and construct the augmented distribution (see Figure 7.4)

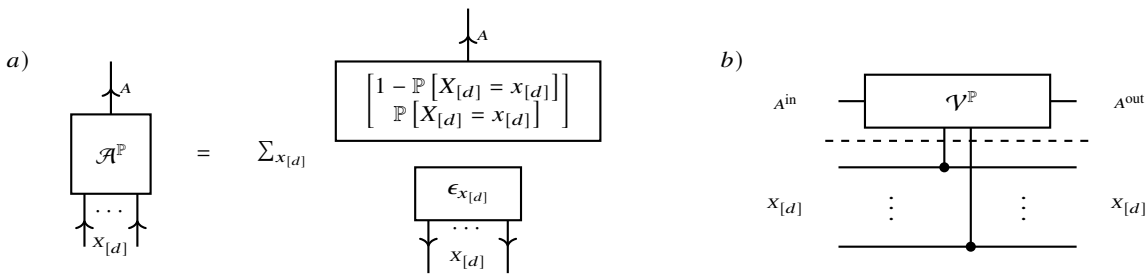


Fig. 7.4 Ancilla augmentation of a distribution $\mathbb{P} \left[X_{[d]} \right]$. a) Augmented distribution $\mathcal{A}^{\mathbb{P}} \left[A, X_{[d]} \right]$ with the property that $\mathbb{P} \left[X_{[d]} \right] = \mathcal{A}^{\mathbb{P}} \left[X_{[d]} | A = 1 \right]$. b) Preparation of the augmented distribution by the activation circuit of $\mathbb{P} \left[X_{[d]} \right]$.

Lemma 7.3 *The activation circuit to a probability distribution $\mathbb{P} \left[X_{[d]} \right]$ applied on the initial state*

$$\sqrt{\frac{1}{\prod_{k\in[d]} m_k}} \mathbb{I} \left[X_{[d]} \right] \otimes \epsilon_0 \left[A \right]$$

prepares a quantum state ψ, which is the Q-sample of the ancilla augmentation

$$\left\langle \mathcal{A}^{\mathbb{P}, \frac{1}{m}} \left[A, X_{[d]} \right] \right\rangle_{\left[A, X_{[d]} | \varnothing \right]}$$

with the acceptance probability $m^{-1} := \left(\prod_{k\in[d]} m_k \right)^{-1}$.

Proof. We first show, that $\left\langle \mathcal{A}^{\mathbb{P}, \frac{1}{m}} \left[A, X_{[d]} \right] \right\rangle_{\left[A, X_{[d]} | \varnothing \right]}$ is indeed an ancilla augmentation. We have

$$\lambda = \left\langle \mathcal{A}^{\mathbb{P}} \left[A, X_{[d]} \right] \right\rangle_{[A=1]} = \frac{1}{m} \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[m_k]} \mathbb{P} \left[X_{[d]} = x_{[d]} \right] = \frac{1}{m} .$$

and for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ it holds that

$$\left\langle \mathcal{A}^{\mathbb{P}, \frac{1}{m}} \left[A, X_{[d]} \right] \right\rangle_{\left[A=1, X_{[d]} = x_{[d]} | \varnothing \right]} = \frac{1}{\lambda} \frac{1}{\prod_{k\in[d]} m_k} \mathbb{P} \left[X_{[d]} = x_{[d]} \right] = \mathbb{P} \left[X_{[d]} = x_{[d]} \right] ,$$

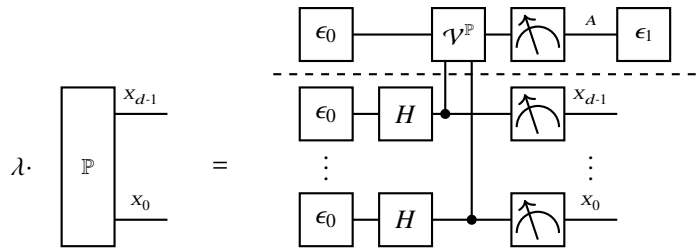
The tensor $\left\langle \mathcal{A}^{\mathbb{P}, \frac{1}{m}} \left[A, X_{[d]} \right] \right\rangle_{\left[A, X_{[d]} | \varnothing \right]}$ is thus an ancilla augmentation for $\mathbb{P}$.

We continue to show that the quantum state ψ prepared by the activation circuit is the Q-sample of $\left\langle \mathcal{A}^{\mathbb{P}, \frac{1}{m}} \left[A, X_{[d]} \right] \right\rangle_{\left[A, X_{[d]} | \varnothing \right]}$. Then we have for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$

$$\psi \left[A, X_{[d]} = x_{[d]} \right] = \sqrt{\frac{1}{\prod_{k\in[d]} m_k}} \left(\sqrt{\mathbb{P} \left[X_{[d]} = x_{[d]} \right]} \cdot \epsilon_1 \left[A \right] + \sqrt{(1 - \mathbb{P} \left[X_{[d]} = x_{[d]} \right])} \cdot \epsilon_0 \left[A \right] \right) ,$$

which is indeed the Q-sample. □

In our tensor network circuit notation, Lem. 7.3 states the equation



for the acceptance probability (see proof of Lem. 7.3)

$$\lambda = \frac{1}{\prod_{k\in[d]} m_k} .$$

Note that the acceptance probability is exponentially small in the number of variables. If $\mathbb{P}\left[X_{[d]}\right]$ is not uniform, we can improve by building the activation circuit to the function

$$\frac{1}{\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right]} \cdot \mathbb{P}\left[X_{[d]}\right] .$$

7.4.3 Best Acceptance Probability by ∞-Renyi Divergence

Let us now show, how the best acceptance probability in sets of ancilla augmented distributions is connected with ∞-Renyi divergences. For $0 < \alpha < \infty$ and $\alpha \neq 1$ the Renyi Divergence is defined as

$$D_\alpha\left[\mathbb{P}||\mathbb{Q}\right] = \frac{1}{1-\alpha} \ln\left[\sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]^\alpha}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]^{\alpha-1}}\right] .$$

The ∞ Renyi Divergence is the limit $\alpha \to \infty$

$$D_\infty\left[\mathbb{P}||\mathbb{Q}\right] = \ln\left[\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\right] .$$

Theorem 7.3 *Let $\mathbb{P}\left[X_{[d]}\right]$ and $\mathbb{Q}[X_{[d]}]$ be distributions, such that for each $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ with $\mathbb{P}\left[X_{[d]} = x_{[d]}\right] > 0$ we also have $\mathbb{Q}[X_{[d]} = x_{[d]}] > 0$. Let us consider all ancilla augmentations $\tilde{\mathbb{P}}[X_{[d]}, A]$ which marginal is $\mathbb{Q}[X_{[d]}]$, that is*

$$\mathbb{Q}[X_{[d]}] = \left\langle \tilde{\mathbb{P}}[X_{[d]}, A]\right\rangle_{[X_{[d]}]} .$$

The best acceptance probability among these ancilla augmentations is then

$$\lambda = \exp\left[-D_\infty\left[\mathbb{P}||\mathbb{Q}\right]\right] ,$$

and achieved when for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$

$$\tilde{\mathbb{P}}[A = 1|X_{[d]} = x_{[d]}] = \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} \cdot \left(\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\right)^{-1} .$$

Proof. Any $\tilde{\mathbb{P}}[X_{[d]}, A]$ with marginal $\mathbb{Q}\left[X_{[d]}\right]$ has a decomposition

$$\tilde{\mathbb{P}}[X_{[d]}, A] = \left\langle \mathbb{Q}\left[X_{[d]}\right], \tilde{\mathbb{P}}[A|X_{[d]}]\right\rangle_{[X_{[d]},A]}$$

and in order to be an ancilla augmentation for $\mathbb{P}\left[X_{[d]}\right]$ we find $\lambda > 0$ with

$$\tilde{\mathbb{P}}[X_{[d]}, A = 1] = \lambda \cdot \mathbb{P}\left[X_{[d]}\right] .$$

It follows that for each $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$

$$1 \geq \tilde{\mathbb{P}}[A = 1|X_{[d]} = x_{[d]}] = \lambda \cdot \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} .$$

and thus

$$\lambda \leq \left(\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\right)^{-1} . \tag{7.2}$$

For the in the claim constructed ancilla augmentation, we have the acceptance probability

$$\begin{aligned}
\tilde{\mathbb{P}}[A = 1] &= \mathbb{E}_{x_{[d]}\sim\mathbb{Q}}\left[\frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]} \cdot \left(\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\right)^{-1}\right] \\
&= \left\langle \mathbb{Q}\left[X_{[d]}\right], \left(\mathbb{Q}\left[X_{[d]}\right]\right)^{-1}, \mathbb{P}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} \left(\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\right)^{-1} \\
&= \left(\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \frac{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]}{\mathbb{Q}\left[X_{[d]} = x_{[d]}\right]}\right)^{-1} \\
&= \exp\left[-D_\infty\left[\mathbb{P}||\mathbb{Q}\right]\right] .
\end{aligned}$$

The constructed ancilla augmented distribution satisfies the bound (7.2) on the acceptance probability holds straight, and it is thus the one with largest acceptance rate. □

Note, that in the section above we have constructed ancilla augmentations with uniform $\mathbb{Q}$ by activation circuits. For these the optimal acceptance rate is

$$\lambda = \left(\prod_{k\in[d]} m_k\right)^{-1} \cdot \left(\max_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \mathbb{P}\left[X_{[d]} = x_{[d]}\right]\right)^{-1} .$$

Thus, $\lambda = 1$ is only achievable if $\mathbb{P}\left[X_{[d]}\right]$ is itself the uniform distribution.

Extension: We could increase the acceptance probability, when we sample from a proposal distribution $\mathbb{Q}$ with smaller ∞ Renyi divergence to $\mathbb{P}$. When sampling with Quantum Circuits, this could be implemented by a state preparation for the distributed variables, before the Computation Activation Circuit is applied. It would be interesting to train variational quantum circuits for this task. However, when we want to apply the same scheme as above, one needs to encode $\mathbb{Q}$ into the ancilla preparing rotations, so $\mathbb{Q}$ would need to be an elementary Computation-Activation Network as well.

7.4.4 Augmentation of Markov Networks

Let us now show how multiple ancilla variables can be designed to sample from arbitrary Markov Networks.

Example 7.5 (Elementary Tensors) Let there be an elementary distribution

$$\mathbb{P}\left[X_{[d]}\right] = \bigotimes_{k\in[d]} \mathbb{P}^k\left[X_k\right] ,$$

that is, all variables are pairwise independent. Then the ancilla augmentation of the distribution is given by the Bayesian Network with the hypergraph shown in Figure 7.5a). Compared with the ancilla augmentation of a generic distribution this augmentation is sparser in the circuit depth (see Figure 7.5b), but introduces multiple ancilla variables. The sparsity originates since each ancilla variable is controlled by a single qubit. The total amount of controlled rotations is thus $\sum_{k\in[d]}[m_k]$ instead of $\prod_{k\in[d]}[m_k]$ for the generic implementation.

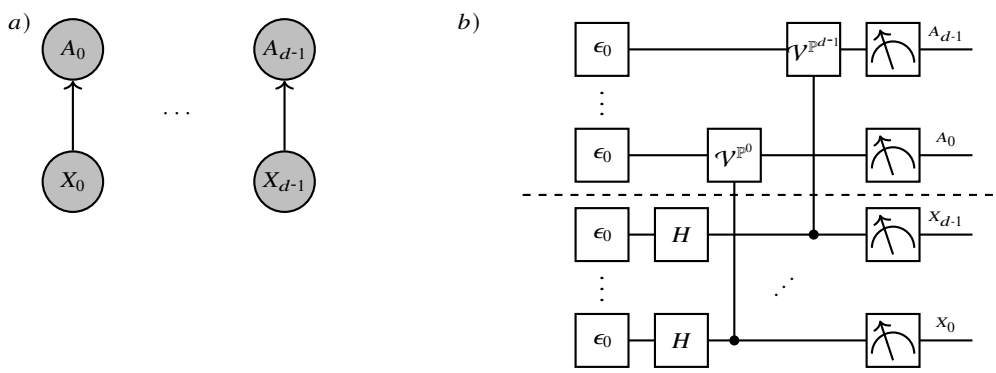


Fig. 7.5 Ancilla augmentation of an elementary distribution (which variables are pairwise independent). a) Hypergraph to the Bayesian Network representing the ancilla augmentation. b) Circuit preparing the ancilla augmented distribution by activation circuits acting on the disentangled initial state.

Let there be a Markov Network, i.e. any non-vanishing tensor network of non-negative tensors, which is understood as a probability disribution after normalization (see [**?**]). For an example of a Markov Network see Figure 7.6a.

We apply a similar ancilla augmentation on any hypercore of a Markov Network, as has been done above for probability distributions. To prepare, we need to scale each core by dividing by its maximum (or any upper bound on its maximum), such that any coordinate is smaller than 1. We introduce for each core an boolean ancilla variable A_e (while X_e is the tuple of distributed variables collected in the hyperedge e) then build directed tensors

$$\mathcal{A}^{\tau^e,\lambda^e}\left[A_e, X_e\right] = \sum_{x_e} \epsilon_{x_e}\left[X_e\right] \otimes \Big(\lambda^e \cdot \big(\prod_{k\in e} m_k\big) \cdot \tau^e\left[X_e = x_e\right] \cdot \epsilon_1\left[A\right] \\ + \big(1 - \lambda^e \cdot \big(\prod_{k\in e} m_k\big) \cdot \tau^e\left[X_e = x_e\right]\big) \cdot \epsilon_0\left[A\right]\Big) .$$

where we choose the maximal values

$$\lambda^e = \left(\prod_{k\in e} m_k\right)^{-1} \cdot \left(\max_{x_e} \tau^e\left[X_e = x_e\right]\right)^{-1} ,$$

such that $\mathcal{A}^e$ stays non-negative and can be interpreted as a conditional probability distribution. Note, that λ^e is not an acceptance rate itself, since τ^e is not always a distribution.

Lemma 7.4 *When augmenting all cores of a Markov Network with distinct ancilla variables in this way (see Figure 7.6b), we get an ancilla augmentation of the Markov Network.*

$$\tilde{\mathbb{P}}[A_{\mathcal{E}}, X_{[d]}] = \left\langle \{\mathcal{A}^{\tau^e,\lambda^e}\left[A_e, X_e\right] \,:\, e \in \mathcal{E}\} \cup \{\mathbb{I}\left[X_{[d]}|\varnothing\right]\}\right\rangle_{\left[A_{\mathcal{E}}, X_{[d]}\right]} .$$

Proof. $\tilde{\mathbb{P}}[A_{\mathcal{E}}, X_{[d]}]$ is a Bayesian Network (see Figure 7.6d) and thus a distribution. We further have

$$\tilde{\mathbb{P}}[A_{\mathcal{E}} = 1_{\mathcal{E}}, X_{[d]}] = \left\langle \{\mathcal{A}^{\tau^e,\lambda^e}\left[A_e = 1, X_e\right] \,:\, e \in \mathcal{E}\} \cup \{\mathbb{I}\left[X_{[d]}|\varnothing\right]\}\right\rangle_{[A_{\mathcal{E}},X_{[d]}]}$$
$$= \frac{\prod_{e\in\mathcal{E}} \lambda^e \left(\prod_{k\in e} m_k\right)}{\prod_{k\in[d]} m_k} \left\langle \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\}\right\rangle_{[X_{[d]}]} .$$

Thus, $\tilde{\mathbb{P}}[A_{\mathcal{E}}, X_{[d]}]$ is an ancilla augmentation of the Markov Network with acceptance probability

$$\frac{\prod_{e\in\mathcal{E}} \lambda^e \left(\prod_{k\in e} m_k\right)}{\left\langle \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\}\right\rangle_{[\varnothing]} \cdot \left(\prod_{k\in[d]} m_k\right)} = \left(\prod_{k\in[d]} m_k\right)^{-1} \cdot \frac{\prod_{e\in\mathcal{E}} \left(\max_{x_e} \tau^e\left[X_e = x_e\right]\right)}{\left\langle \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\}\right\rangle_{[\varnothing]}} .$$

□

The main advantage of this augmentation is, that all cores are directed (see Figure 7.6b, and build therefore a Bayesian Network. Using the activation circuits studied above, we can now determine a rejection sampling procedure for arbitrary Markov Networks.

Lemma 7.5 *Let there be a Markov Network $\tau^{\mathcal{G}} = \{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\}$ on a hypergraph $\mathcal{G}$. To each $e \in \mathcal{E}$ we build a function*

$$q_e = \frac{\tau^e\left[X_e = y_e\right]}{\max_{y_e} \tau^e\left[X_e = y_e\right]} .$$

We then build a concatenation of activation circuits to each function q_e, where each has a distinct ancilla qubit A_e. Then the constructed graph-controlled circuit acting on the state

$$\sqrt{\frac{1}{\prod_{v\in\mathcal{V}} m_v}} \mathbb{I}\left[X_{\mathcal{V}}\right] \otimes \left(\bigotimes_{e\in\mathcal{E}} \epsilon_0\left[A_e\right]\right)$$

prepares an ancilla augmented distribution of the Markov Network.

Proof. We show, that the measurement distribution is the ancilla augmented Markov Network. □

We use this result to prepare samples from Markov Networks by rejection sampling on the ancilla variables, in qcreason (see Sect. **??**).

When the maximum of the hypercores are not known, and only an upper bound is given, it suffice to rescale them such that the maximum is lower than 1.

When the maximal coordinate of the Markov Network is, restricted to each edge, the maximal coordinate of the hypercore of that edge, we say that the Markov Network is not frustrated, otherwise frustrated.

Lemma 7.6 *If the Markov Network is not frustrated, then the acceptance probability λ of the rejection sampling scheme using the ancilla augmented Markov Network (with respect to the uniform distribution) is the ∞-Renyi divergence between the Markov Network and the uniform distribution.*

Proof. If and only if the Markov Network is not frustrated we have

$$\max_{x_{[d]}} \left(\prod_{e\in\mathcal{E}} \tau^e\left[X_e = x_e\right]\right) = \prod_{e\in\mathcal{E}} \left(\max_{x_e} \tau^e\left[X_e = x_e\right]\right) .$$

In this case, the ancilla augmentation has the best acceptance probability, since for the state $x_{[d]}$ maximizing all local cores we have

$$\tilde{\mathbb{P}}[A_{[p]} = 1_{[p]}|X_{[d]} = x_{[d]}] = 1 .$$

Alternatively: Show this with the acceptance rate in proof of Lem. 7.4, comparing with best naive augmentation. □

If the Markov Network is frustrated, we cannot achieve the best acceptance probability by the distributed ancilla preparation approach described above. In this case, one faces a tradeoff between the size of the circuit to prepare ancilla augmentations and the acceptance probability.

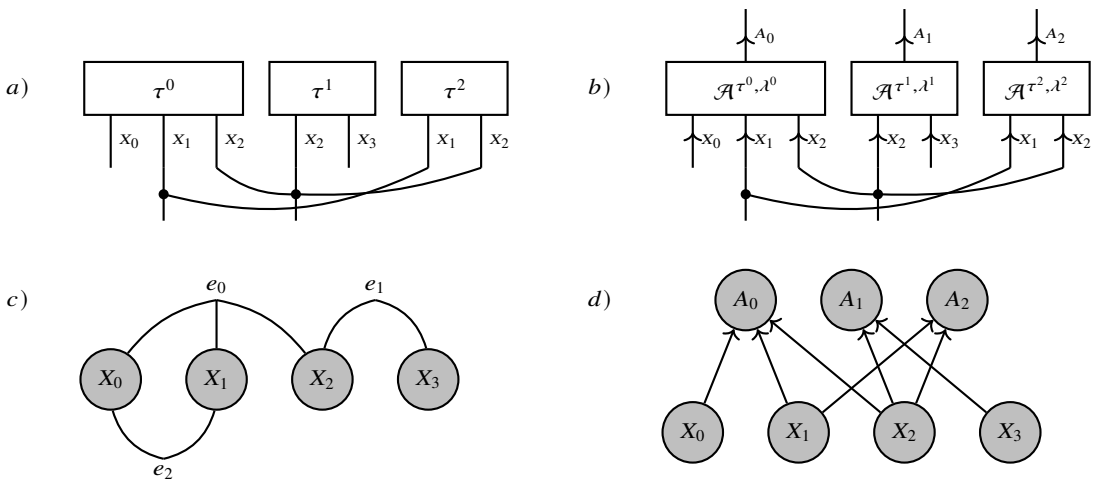


Fig. 7.6 Ancilla augmentation of a Markov Network a) onto a Bayesian Network b). To this end the arbitrary non-negative cores τ^e are replaced by directed ancilla augmented cores $\tilde{\tau}^e$, with ancilly variables outgoing.

7.5 Computation circuits

We here suggest a quantum pendant to basis encodings, which has the decomposition by contraction property.

Definition 7.4 (Computation circuit) Given a boolean function $f : \bigtimes_{k\in[d]}[2] \to [2]$ the computation circuit is the unitary tensor

$$\begin{aligned} C^f\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right] \quad &= \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]\,:\,f\left[x_{[d]}\right]=1} \sigma_1\left[Y_f^{\text{in}}, Y_f^{\text{out}}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \\ &\quad + \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]\,:\,f\left[x_{[d]}\right]=0} \delta\left[Y_f^{\text{in}}, Y_f^{\text{out}}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \ . \end{aligned}$$

Example 7.6 (CNOT gate) The CNOT gate is the controlled Pauli-X, that is

$$\text{CNOT}\left[Y^{\text{in}}, Y^{\text{out}}, X\right] = \sigma_1\left[Y^{\text{in}}, Y^{\text{out}}\right] \otimes \epsilon_1\left[X\right] + \delta\left[Y^{\text{in}}, Y^{\text{out}}\right] \otimes \epsilon_0\left[X\right] \ .$$

For the identity function

$$\text{Id}(x) = x \ .$$

we have

$$C^{\text{Id}}\left[Y^{\text{in}}, Y^{\text{out}}, X\right] = \text{CNOT}\left[Y^{\text{in}}, Y^{\text{out}}, X\right] \ .$$

The CNOT is therefore the computation circuit to the identity function.

Functions with multiple output variables, i.e. $f : \bigtimes_{k\in[d]}[2] \to \bigtimes_{\ell\in[p]}[2]$, can be encoded image coordinate wise as a concatenation of the respective circuits.

7.5.1 Relation with basis encodings

Basis encodings are schemes to encode functions into directed boolean tensors, which enable reasoning about functions by tensor network contractions (see [**?**]). To connect with this formalism, let us now show the relation to the computation circuits.

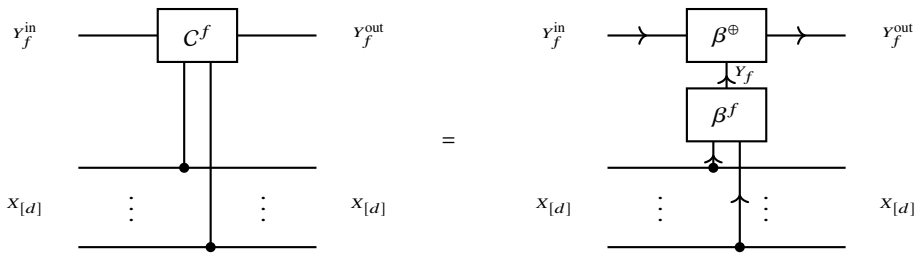


Fig. 7.7 Relation of computation circuit and basis encodings. The computation circuit of a function is the contraction of its basis encoding with the basis encoding of the mod2 sum $\oplus$.

Lemma 7.7 *For any boolean function we have (see Figure 7.7)*

$$C^f\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \beta^{\oplus}\left[Y_f^{\text{out}}, Y_f^{\text{in}}, Y_f\right]\right\rangle_{\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right]} .$$

The computation circuit applied on the initial state

$$\sqrt{\frac{1}{2^d}}\mathbb{I}\left[X_{[d]}^{\text{out}}\right] \otimes \epsilon_0\left[Y^{\text{in}}\right]$$

prepares a q-sample to the normalized basis encoding.

In [**?**] (and more detailed in [**?**]) sparse decompositions of basis encodings have been studied. Based on the equivalence of computation circuits and contracted basis encodings, stated by Lem. 7.7 we transfer these decomposition schemes in the following.

Example 7.7 (Multiple-controlled NOT) Let us consider the d-ary controlled NOT gate

$$\text{MCNOT}^d\left[Y^{\text{in}}, Y^{\text{out}}, X_{[d]}\right] = \sigma_1\left[Y^{\text{in}}, Y^{\text{out}}\right] \otimes \epsilon_{1_{[d]}}\left[X_{[d]}\right] + \delta\left[Y^{\text{in}}, Y^{\text{out}}\right] \otimes \left(\mathbb{I}\left[X_{[d]}\right] - \epsilon_{1_{[d]}}\left[X_{[d]}\right]\right) .$$

This is the computation circuit of the d-ary logical conjunction

$$\wedge^d : [2]^d \to [2] \quad , \quad \wedge^d(x_{[d]}) = \begin{cases} 1 & \text{if} \quad x_{[d]} = 1_{[d]} \\ 0 & \text{else} \end{cases} .$$

Special instances are the CNOT for $d = 1$ (see Example 7.6) and the Toffoli gate for $d = 2$.

7.5.2 Polynomial sparsity

Lemma 7.8 *The composition of two computation circuits to functions q and g (see Figure 7.8), where the computation circuits share the target qubit, is the computation circuit of their sum mod2. In particular, computation circuits commute when they have the same target qubit.*

Proof. This follows from the representation of computation circuits by basis encodings of the corresponding function and the mod2 sum $\oplus$. □

Concatenating two computation circuits, which have the same head qubit, is the computation circuit of their mod2 sum.

A basis+ elementary function can be encoded by a single controlled NOT operation with auxiliary Pauli-X to prepare the control state at the X qubits.

This motivates the mod2-basis+ CP decomposition of tensors, which is exactly the decomposition of boolean polynomials into monomials. Each monomial is called a term (products of x or (1-x) factors), and minterms in case that all variables appear.

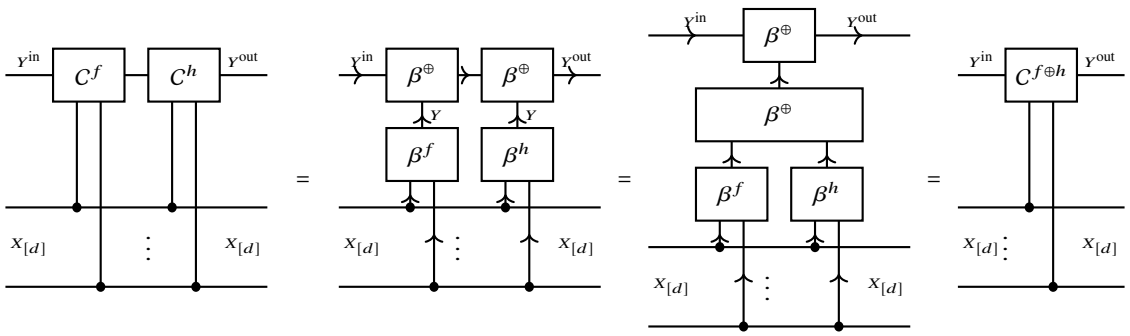


Fig. 7.8 Equivalence of the composition of computation circuits with the computation circuit of their mod2 sum (Lem. 7.8).

Definition 7.5 Given a boolean tensor τ, a mod2-basis+ CP decomposition is a collection $\mathcal{M}$ of tuples (A, x_A) with such that for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$

$$\tau\left[X_{[d]} = x_{[d]}\right] = \bigoplus_{(A,x_A)\in\mathcal{M}} \left\langle \epsilon_{x_A}\left[X_A\right]\right\rangle_{\left[X_{[d]}=x_{[d]}\right]} .$$

This decomposition is closely related to the Algebraic Normal Form of a Boolean polynomial.

Using that basis CP decompositions are a special case of basis+ CP decompositions, we get the following rank bound.

Lemma 7.9 *The mod2-basis+ CP rank is bounded by the basis CP rank.*

Proof. Use $A = [d]$, and x_A to each supported state. Then the mod2-sum is a usual sum and the basis CP decomposition is also a mod2-basis+ CP decomposition. □

This shows in particular, that any propositional formula can be represented by a mod2-basis+ CP decomposition.

Lemma 7.10 *The computation circuit to a boolean tensor τ with a mod2-basis+ CP decomposition $\mathcal{M}$ obeys*

$$\begin{aligned} &C^{\tau}\left[Y^{\text{in}}_{\tau}, Y^{\text{out}}_{\tau}, X_{[d]}\right] \\ &\quad = \left\langle \left\{\delta\left[Y^{\text{in}}_{\tau}, Y_0\right], \delta\left[Y^{\text{out}}_{\tau}, Y_{|\mathcal{M}|-1}\right]\right\} \cup \left\{C^{\epsilon_{x_A}}\left[Y_i, Y_{i+1}, X_A\right] \,:\, (A, x_A) \in \mathcal{M}\right\}\right\rangle_{\left[Y^{\text{in}}_{\tau}, Y^{\text{out}}_{\tau}, X_{[d]}\right]} \end{aligned}$$

where $i \in [|\mathcal{M}|]$ enumerates the tuples in $\mathcal{M}$.

Proof. Can be seen by the basis encoding representation, and the accociativtiy of the sum mod2 ($\oplus$). □

Each computation circuit to each boolean monomial can be prepared by a multiple-controlled σ_1 gate and further pairs of σ_1 gates preparing the control state, see Figure 7.9. When we sum monomials wrt modulus 2 calculus, then the preparation is a sequence of such circuits. In such way, we can prepare the computation circuit to any propositional formula. This encoding strategy exploits a modified (by mod2 calculus) polynomial sparsity.

7.5.3 Decomposition sparsity

The decomposition by contraction property of basis encodings is now a composition of circuits property, as stated in the next lemma. Instead of hidden variables as in the tnreason approach, we here exploit further working qubits.

Lemma 7.11 *We have for functions $q : \bigtimes_{k\in[d]}[2] \to \bigtimes_{\ell\in[p]}[2]$, $g : \bigtimes_{\ell\in[p]}[2] \to \bigtimes_{s\in[r]}[2]$ (see Figure **??**)*

$$\begin{aligned} &C^{g\circ q}\left[Y^{\text{in}}_{g\circ q}, Y^{\text{out}}_{g\circ q}, X_{[d]}\right] \otimes \epsilon_0\left[Y^{\text{out}}_{q}\right] \\ &\quad = \left\langle \epsilon_0\left[Y^{\text{in}}_{q}\right], C^{q}\left[Y^{\text{in}}_{q}, Y^{\text{mid}}_{q}, X_{[d]}\right], C^{g}\left[Y_{\text{in},g\circ q}, Y^{\text{out}}_{g\circ q}, Y^{\text{mid}}_{q}\right], C^{q}\left[Y^{\text{mid}}_{q}, Y^{\text{out}}_{q}, X_{[d]}\right]\right\rangle_{\left[Y^{\text{in}}_{g\circ q}, Y^{\text{out}}_{g\circ q}, X_{[d]}, Y^{\text{out}}_{q}\right]} . \end{aligned}$$

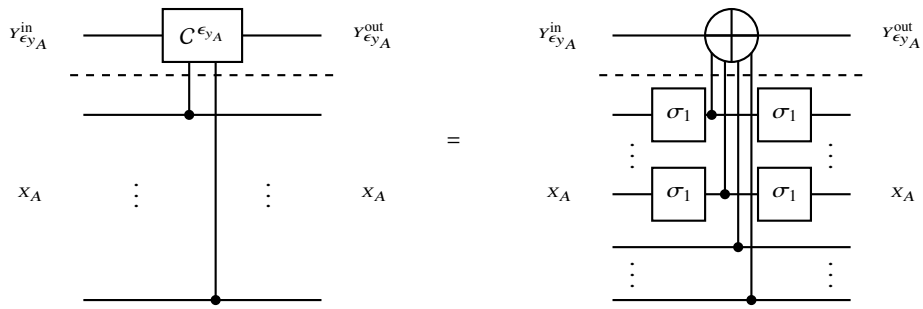


Fig. 7.9 Representation of an slice as a controlled CNOT gate. When the control state to a variable is 0, we introduce auxiliary Pauli-X gates.

Here each copy of $Y_{g\circ q}$ denotes a tuple of r boolean variables and Y_q of p boolean variables.

Proof. We show the claim based on basis calculus and sketch the steps in Figure 7.10. In the first equation we use Lem. 7.7 and represent each computation circuit by a contraction of two basis encodings. In the second equation we use the identity

$$\left\langle \beta^{\oplus}\left[Y^{\text{out}}, Y^{\text{in}}, Y_q\right], \epsilon_0\left[Y^{\text{in}}\right]\right\rangle_{\left[Y^{\text{out}}, Y_q\right]} = \delta\left[Y^{\text{out}}, Y_q\right] .$$

Now in the third equation we use that the contraction of basis encodings is the basis encoding of the composition function, and that $q \oplus q$ is the vanishing function, which basis encoding is a tensor product with $\epsilon_0\left[Y_q^{\text{out}}\right]$. We use we Lem. 7.7 again in the fourth equation to arrive at the claim. □

When having a syntactical decomposition of a propositional formula, we can iteratively apply the computation circuit decomposition theorem and prepare each connective by a circuit. We can decompose any propositional formula into logical connectives and prepare to each a computation circuit (exploiting further sparsity mechanisms). This works, when the target qubit of one connective is used as a value qubit of another.

This entailment of the working qubit with the distributed qubits can be resolved by an un-computation circuit, which is the adjoint of the computation circuit for the working qubits (see Figure 7.10). Note that while this is necessary in some applications, it is not necessary for the quantum rejection sampling presented here.

Example 7.8 Let us reconsider the d-ary controlled NOT gate from Example 7.7. The d-ary logical conjunction is decomposed as a sequence of 2-ary logical conjunctions. The computation circuit of each 2-ary conjunction is a Toffoli gate applied to an input qubit and a worker qubit. Using Decomposition sparsity we therefore find the standard decomposition of d-ary controlled NOT gates into Toffoli gates.

7.6 Function Encoding

We investigate how Boolean functions can be encoded in quantum states using computation circuits.

Using the computation circuit we can prepare two function encoding schemes, depending on the start state.

*Remark 7.1 (Basis and Amplitude Encodings in Quantum Computation [**?**] and* tnreason*)* Basis Encoding in Quantum Computation refers to the representation of classical n bit strings by n qubit basis states, and is called one-hot encoding in tnreason. The Basis Encoding scheme in tnreason goes beyond this scheme and also encodes subsets by sums of one-hot encodings to the members of the set. In this way, relations and functions are represented by boolean tensors and contraction of them is refered as Basis calculus.

Amplitude Encoding in Quantum Computation refers to the storage of complex numbers in the amplitudes of quantum states. The pendant in tnreason is the Coordinate Encoding scheme, where real numbers are stored in the coordinates of real-valued tensors. Compared to Amplitude Encoding, Coordinate Encoding does not have the normalization constraint of quantum states. The Amplitude Encoding of the square root of a probability distribution is sometimes called q-sample.

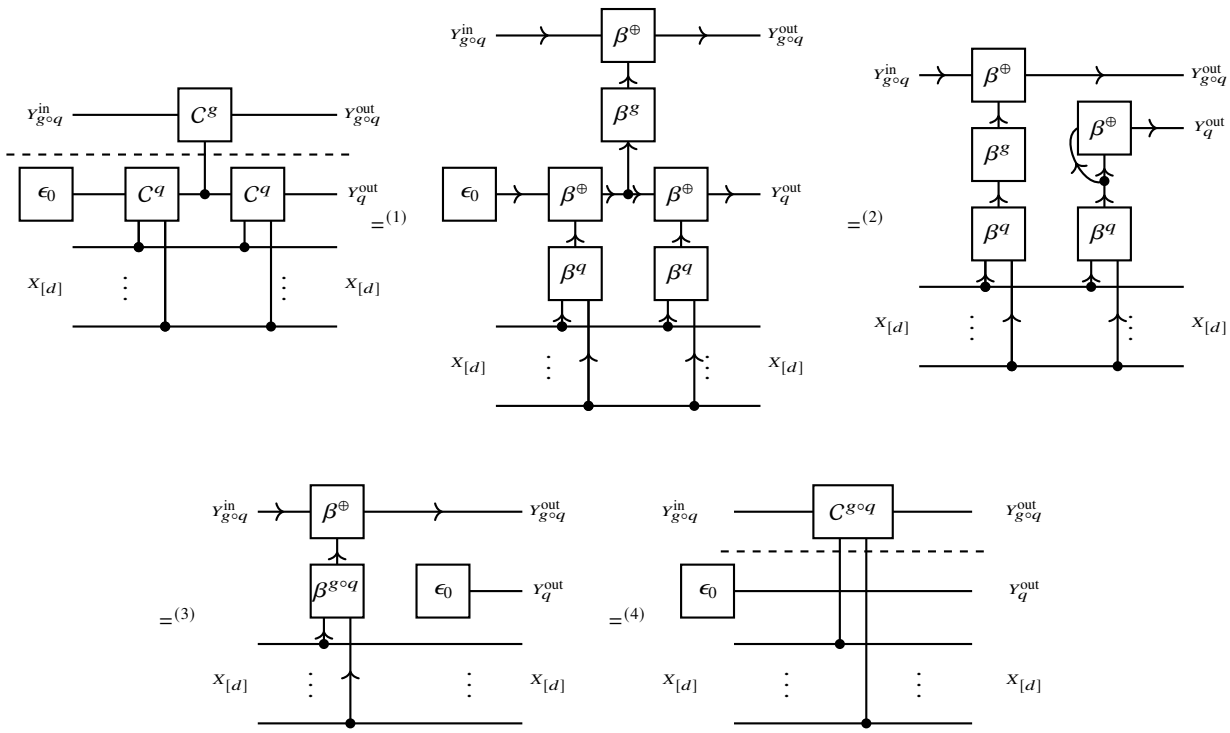


Fig. 7.10 Proof of Lem. 7.11 based on basis calculus.

7.6.1 Basis encoding

Definition 7.6 The basis encoding of a boolean function f is the state

$$\psi_f^{\text{bas}}\left[X_{[d]}, Y\right] = \sqrt{\frac{1}{2^d}} \sum_{x_{[d]} \in \bigtimes_{k\in[d]}[2]} \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \epsilon_{f(x_{[d]})}\left[Y\right] .$$

Lemma 7.12 *When applying the computation circuit on the initial state*

$$\sqrt{\frac{1}{2^d}} \mathbb{I}\left[X_{[d]}\right] \otimes \epsilon_0\left[Y\right]$$

we get the basis encoding state (see Figure 7.11a).

Proof. Since the Pauli-X turns ϵ_0 into ϵ_1, and is applied exactly when $f(x_{[d]}) = 1$. □

Corollary 7.1 *Using Lem. 7.7 we have*

$$\psi_f^{\text{bas}}\left[X_{[d]}, Y\right] = \sqrt{\frac{1}{2^d}} \beta^f\left[X_{[d]}, Y\right]$$

The initial state can be prepared by applying Hadamard gates on all distributed variable qubits and preparing the head variable qubit in the ground state. The overlap of two basis encoding states (which share the same head variable) is proportional to the contraction of the corresponding formulas.

Lemma 7.13 *Let $f, \tilde{f}$ be two Boolean formulas on the states of $X_{[d]}$. Then we have*

$$\left\langle \psi_f^{\text{bas}}\left[X_{[d]}, Y\right], \psi_h^{\text{bas}}\left[X_{[d]}, Y\right] \right\rangle_{[\varnothing]} = \frac{1}{2^d} \left\langle f\left[X_{[d]}\right], \tilde{f}\left[X_{[d]}\right] \right\rangle_{[\varnothing]} .$$

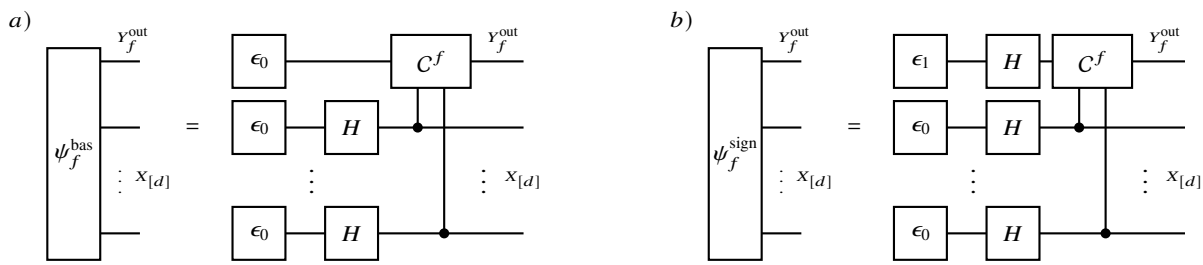


Fig. 7.11 Construction of the basis encoding state (a) and the sign encoding state (b) using the computation circuit of a function.

Proof. We have

$$\begin{aligned}\left\langle \psi_f^{\text{bas}}\left[X_{[d]},Y\right],\psi_h^{\text{bas}}\left[X_{[d]},Y\right]\right\rangle_{[\varnothing]} &= \frac{1}{2^d}\sum_{x_{[d]}\in[2]^d}\left\langle \epsilon_{f\left[X_{[d]}=x_{[d]}\right]}\left[Y\right],\epsilon_{\tilde{f}\left[X_{[d]}=x_{[d]}\right]}\left[Y\right]\right\rangle_{[\varnothing]}\\ &= \frac{1}{2^d}\left\langle f\left[X_{[d]}\right],\tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}.\end{aligned}$$

□

7.6.2 Sign encoding

Definition 7.7 The sign encoding of a boolean function f is the state

$$\psi_f^{\text{sign}}\left[X_{[d]},Y\right]=\left(\sqrt{\frac{1}{2^d}}\sum_{x_{[d]}\in\bigtimes_{k\in[d]}[2]}(-1)^{f(x_{[d]})}\cdot\epsilon_{x_{[d]}}\left[X_{[d]}\right]\right)\otimes\sqrt{\frac{1}{2}}\left(\epsilon_0\left[Y\right]-\epsilon_1\left[Y\right]\right).$$

Sign encodings can be prepared by the phase kickback trick, as we show next.

Lemma 7.14 (Phase kickback trick) *Applying the computation circuit* $C^f\left[Y^{\text{in}},Y^{\text{out}},X_{[d]}\right]$ *on a state*

$$\psi\left[X_{[d]}\right]\otimes\epsilon^-\left[A^{\text{in}}\right]$$

we get

$$\sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]}(-1)^{f(x_{[d]})}\psi\left[X_{[d]}=x_{[d]}\right]\cdot\epsilon_{x_{[d]}}\left[X_{[d]}\right]\otimes\epsilon^-\left[A^{\text{out}}\right].$$

Proof. We get a disentangled ancilla qubit, because $\epsilon^-\left[A^{\text{in}}\right]$ is an eigenvector of the identity and the Pauli-X. For the identity, which is applied when $f(x_{[d]})=0$, the eigenvalue is 1. For the Pauli-X, which is applied when $f(x_{[d]})=0$, the eigenvalue is (-1). □

We get the sign encoding with the phase kickback trick, when we initialize with the q-sample of the uniform distribution.

Corollary 7.2 *When applying the computation circuit on the initial state*

$$\sqrt{\frac{1}{2^d}}\mathbb{I}\left[X_{[d]}\right]\otimes\sqrt{\frac{1}{2}}\left(\epsilon_0\left[Y\right]-\epsilon_1\left[Y\right]\right)$$

we get the sign encoding state (see Figure 7.11b). Note that the initial state is a Hadamard gate acting on the state $\epsilon_1\left[Y\right]$.

The sign encoding is also called the phase encoding. One can ask, whether we can use other ancilla states and get different phase kickbacks while keeping the ancilla disentangled. However, since we construct here computation circuit we are restricted to the eigenvectors of the Pauli-X gate, which is the Hadamard basis

$$\epsilon^{-}[Y] = \sqrt{\frac{1}{2}}\left(\epsilon_0[Y] - \epsilon_1[Y]\right) \quad , \quad \epsilon^{+}[Y] = \sqrt{\frac{1}{2}}\left(\epsilon_0[Y] + \epsilon_1[Y]\right)$$

with eigenvalues (-1) and 0.

The initial state can be prepared by applying Hadamard gates on all distributed variable qubits and preparing the head variable qubit by applying a Hadamard gate on the first basis state, i.e.

$$\frac{1}{\sqrt{2}}\left(\epsilon_1[Y_{\text{out}}] - \epsilon_0[Y_{\text{out}}]\right) = H[Y_{\text{out}}, Y_{\text{in}}]\,\epsilon_0[Y_{\text{in}}] \ ,$$

Lemma 7.15 *Let $f, \tilde{f}$ be two Boolean formulas on the states of $X_{[d]}$. Then we have*

$$\left\langle \psi_f^{\text{sign}}\left[X_{[d]}, Y\right], \psi_h^{\text{sign}}\left[X_{[d]}, Y\right]\right\rangle_{[\varnothing]} = \frac{1}{2^d}\left(\left\langle f\left[X_{[d]}\right], \tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} - \left\langle f\left[X_{[d]}\right], \neg\tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\right) .$$

Proof. We have

$$\begin{aligned}
\left\langle \psi_f^{\text{sign}}\left[X_{[d]}, Y\right], \psi_h^{\text{sign}}\left[X_{[d]}, Y\right]\right\rangle_{[\varnothing]} &= \frac{1}{2^d}\sum_{x_{[d]}\in[2]^d}(-1)^{f\left[X_{[d]}=x_{[d]}\right]+\tilde{f}\left[X_{[d]}=x_{[d]}\right]} \\
&= \frac{1}{2^d}\sum_{x_{[d]}\in[2]^d : f\left[X_{[d]}=x_{[d]}\right]=\tilde{f}\left[X_{[d]}=x_{[d]}\right]} 1 + \sum_{x_{[d]}\in[2]^d : f\left[X_{[d]}=x_{[d]}\right]\neq\tilde{f}\left[X_{[d]}=x_{[d]}\right]} (-1) \\
&= \frac{1}{2^d}\left(\left\langle f\left[X_{[d]}\right], \tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} - \left\langle f\left[X_{[d]}\right], \neg\tilde{f}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\right) .
\end{aligned}$$

□

Limitations:

- The prepared state ψ_f^{sign} is distinguished from the state $\psi_{\neg f}^{\text{sign}}$ only by a non-observable phase factor

$$\psi_f^{\text{sign}} = (-1)\cdot\psi_{\neg f}^{\text{sign}}$$

- With this construction the statistic qubit is disentangled with the distributed qubits. Further manipulation of the statistic qubits alone (as we do with activation circuits) will not retrieve any information about the encoded function.
- When doing the sign encoding procedure for multiple formulas, we get a disentangled state of the head variables with the distributed variables encoded in the sign encoding state of $\bigoplus_{f\in\mathcal{F}} f$ (where by $\bigoplus$ we denote the mod2 sum of boolean functions).

7.6.3 Conclusion

We have seen that preparing some classically hard to compute contraction can be done efficiently by a quantum circuit. We showed in particular that contracted basis encodings can be prepared by circuits, which are classically hard to contract in the presence of undirected loops in the decomposition hypergraph. However, when assessing the prepared contraction state by computational basis measurements, we effectively draw samples from the corresponding distribution, which can also be achieved classically. A quantum advantage could arise from different measurement schemes studied in the quantum state tomography community (see [**?**, **?**, **?**]). To this end, future research could show, how hard to achieve expectation queries can efficiently be prepared by customized measurement schemes.

7.7 Contractions based on Fourier Transforms

Walsh-Hadamard transform and further transforms can be efficiently implemented by quantum circuits. We here investigate how these transforms can be utilized to compute contractions of tensor networks.

7.7.1 Walsh-Hadamard transform

The application of Hadamard gates on the distributed variables afterwards results in a Walsh-Hadamard transform of the prepared state.

The transform is performed by applying Hadamard gates on each variable. This is equivalent with contraction of the tensor

$$H^d\left[X_{[d],\mathrm{in}}, X_{[d],\mathrm{out}}\right] = \bigotimes_{k\in[d]} H\left[X_k^{\mathrm{in}}, X_k^{\mathrm{out}}\right] ,$$

which coordinates are

$$H^d\left[X_{[d],\mathrm{in}} = x_{[d],\mathrm{in}}, X_{[d],\mathrm{out}} = x_{[d],\mathrm{out}}\right] = \sqrt{\frac{1}{2^d}}\,(-1)^{\sum_{k\in[d]} x_{k,\mathrm{in}}\cdot x_{k,\mathrm{out}}} .$$

We have

$$\begin{aligned}\left\langle \epsilon_0\left[X^{\mathrm{in}}\right], H\left[X^{\mathrm{in}}, X^{\mathrm{out}}\right]\right\rangle_{[X^{\mathrm{out}}]} &= \sqrt{\frac{1}{2}}\mathbb{I}\left[X^{\mathrm{out}}\right]\\ \left\langle \epsilon_1\left[X^{\mathrm{in}}\right], H\left[X^{\mathrm{in}}, X^{\mathrm{out}}\right]\right\rangle_{[X^{\mathrm{out}}]} &= \sqrt{\frac{1}{2}}\begin{pmatrix}1\\-1\end{pmatrix}\left[X^{\mathrm{out}}\right]\end{aligned}$$

and conversely

$$\begin{aligned}\left\langle \sqrt{\frac{1}{2}}\mathbb{I}\left[X^{\mathrm{in}}\right], H\left[X^{\mathrm{in}}, X^{\mathrm{out}}\right]\right\rangle_{[X^{\mathrm{out}}]} &= \epsilon_0\left[X^{\mathrm{out}}\right]\\ \left\langle \sqrt{\frac{1}{2}}\begin{pmatrix}1\\-1\end{pmatrix}\left[X^{\mathrm{in}}\right], H\left[X^{\mathrm{in}}, X^{\mathrm{out}}\right]\right\rangle_{[X^{\mathrm{out}}]} &= \epsilon_1\left[X^{\mathrm{out}}\right] .\end{aligned}$$

If and only if the function is constant, then the transformed state is the ground state.

Remark 7.2 (Fourier transform) The Walsh-Hadamard transform is the Fourier transform on the group Z_2^d, the d-ary product of the cyclic group of order 2. The irreducible representations of this group are one-dimensional and their characters are given by the slices of the Hadamard tensor.

7.7.2 Examples

Let us present two known algorithms as the measurement of states prepared by computation circuits.

Example 7.9 (Deutsch-Jozsa Test) We measure the probability of the ground state of the sign encoding after a Walsh-Hadamard transform. The probability of measuring the ground state after a Walsh-Hadamard transform of the distributed qubits is then

$$\frac{1}{2^d}\cdot\left\langle\left|\left\langle \psi_f^{\mathrm{sign}}\left[X_{[d]}, Y\right]\right\rangle_{[Y]}\right|^2\right\rangle_{[\varnothing]} = \frac{1}{2^{2\cdot d}}\left(\langle f\rangle_{[\varnothing]} - \langle \neg f\rangle_{[\varnothing]}\right)^2 .$$

Thus, if and only if the probability of the ground state is 1, we have that the formula is constant, that is $f \in \{\top, \bot\}$. If and only if the probability of the ground state is 0, we have that the formula is balanced, that is $\langle f \rangle_{[\varnothing]} = \langle \neg f \rangle_{[\varnothing]}$. When its known, that the formula is either constant or balanced, we can decide which one it is by measuring the probability of the ground state. This is the Deutsch-Josza algorithm [**?**].

Note that when allowing for an exponentially small error the decision can be efficiently performed by a classical algorithm. When drawing n samples and deciding on $\mu = 0.5$ if and only if in the samples a 0 and a 1 is observed, we have an error probability of 0 when $\mu \in \{0, 1\}$ and of

$$\mathbb{P}^{E|\mu=0.5} = 2 \cdot \frac{1}{2^n} \,.$$

The quantum advantage is thus bound to the demand of an error-free decision, for which a generic classical algorithm would iterate through all states and is therefore exponentially slower than the quantum alternative.

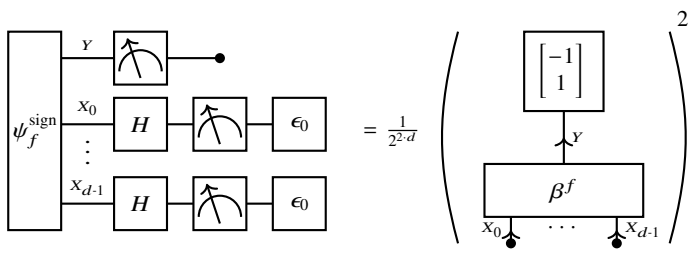


Fig. 7.12 Measurement setup for the Deutsch-Jozsa algorithm on the formula f.

Deutsch-Jozsa can be generalized to estimating the moments of a distribution given its q-sample. Note, that we have to demand a q-sample (i.e. vanishing phase tensor), otherwise the measurement is affected by the phase tensor and the Deutsch-Jozsa test is effectively done wrt another function. If for example the coordinates of the phase tensor are multiples of π, and the phases are thus $+1/-1$, then the test is done wrt the function $f \oplus (\frac{1}{\pi}\phi)$ instead of f.

Example 7.10 (Inversion Test) Concatenating the computation circuits to two formulas f and h prepares the state

$$\psi^{\text{bas}}_{f \oplus h} \left[X_{[d]}, Y \right]$$

The probability of measuring the ground state after a Walsh-Hadamard transform of the distributed qubits (see Figure 7.13) is then

$$\begin{aligned} \frac{1}{2^d} \left| \left\langle \psi^{\text{bas}}_{f \oplus h} \left[X_{[d]}, Y \right] \right\rangle_{[Y]} \right|^2 &= \frac{1}{2^{2\cdot d}} \left[\langle f \oplus h \rangle_{[\varnothing]} \langle \neg (f \oplus h) \rangle_{[\varnothing]} \right] \\ &= \frac{1}{2^{2\cdot d}} \begin{bmatrix} \#\left\{ x_{[d]} \in \bigtimes_{k \in [d]} [2] \,:\, f\left[X_{[d]} = x_{[d]} \right] = h\left[X_{[d]} = x_{[d]} \right] \right\} \\ \#\left\{ x_{[d]} \in \bigtimes_{k \in [d]} [2] \,:\, f\left[X_{[d]} = x_{[d]} \right] \neq h\left[X_{[d]} = x_{[d]} \right] \right\} \end{bmatrix} \,. \end{aligned}$$

The second equation holds since $\neg(f \oplus h)$ is equivalent with the biconditional $f \Leftrightarrow h$.

This is in more generality the inversion test of measuring the overlap of ψ^{bas}_f with ψ^{bas}_h (see [**?**]). Here we used that the computation circuit is self-adjoint and understand the concatenation as the unitaries preparing ψ^{bas}_f and the adjoint (together with the Walsh-Hadamard transform) of ψ^{bas}_h.

Quantum Circuits such as the SWAP test and the Hadamard test ([**?**, Section 3.6.1]) can be used to measure overlaps of quantum states, which are the squared absolutes of contractions of two state tensors.

- When we have preparation schemes for two tensors, we can control them with a common ancilla qubit and measure their contraction by a Hadamard test (alternatively, using the SWAP test and state preparation in two registers).
- Can we extend these schemes to contractions of more general tensor networks?

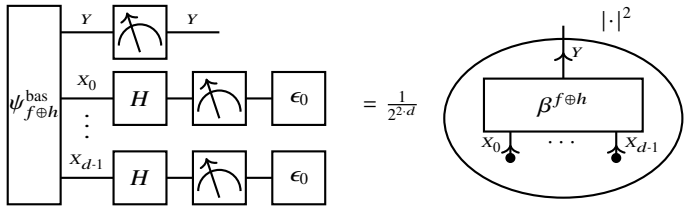


Fig. 7.13 Measurement setup for the inversion test on the formulas f and h. We measure the probability of the statistic qubit Y, when the distributed qubits $X_{[d]}$ are in the ground state after a Walsh-Hadamard transform. This is equal to a scaled and squared contraction of the basis encoding to the formula $f \oplus h$.

While the preparation of contracted basis encodings is done efficiently, accessing the prepared state by measurements is a bottleneck. This is a typical *quantum state tomography* problem [**?**]. Sparsity of the contracted state can be exploited to probabilistically guarantee precise estimates [**?**]. However, when deciding whether f is satisfiable, based on the prepared contraction $\sqrt{\langle \beta^f \left[Y, X_{[d]}\right] \rangle_{[Y|\varnothing]}}$, we have an exponentially small error tolerance. Moreover, the applied sample mean is the UMVUE (Uniformly Minimum Variance Unbiased Estimator, see [**?**]), and can thus not be improved by other unbiased estimators.

7.7.3 Generic Hidden Subgroup Problems

In general [**?**] approached by generic Fourier transforms of the distributed variables of a basis encoding state to a function.

Part II
Computation-Activation Networks

Use the template *part.tex* together with the document class `SNmono` (monograph-type books) or `SNmult` (edited books) to style your part title page and, if desired, a short introductory text (maximum one page) on its verso page.

Chapter 8
Elementary Computation-Activation Networks

Abstract Dicuss here the case of elementary activation, Hard, Markov and Hybrid Logic Networks.

Topics

- Exponential Families (Report I.2.3)

8.1 Exponential Families

8.1.1 Definition

Definition 8.1 (Exponential Family) Given a statistic function

$$t \,:\, \bigtimes_{k\in[d]} [m_k] \to \mathbb{R}^p$$

and a boolean base measure

$$\nu \,:\, \bigtimes_{k\in[d]} [m_k] \to \{0,1\}$$

with $\langle \nu \rangle_{[\varnothing]} \neq 0$, the set $\Gamma^{t,\nu} = \{\mathbb{P}^{(t,\theta,\nu)} \,:\, \theta\,[L] \in \mathbb{R}^p\}$ of probability distributions

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] = \left\langle \exp\left[\left\langle \sigma^t\left[X_{[d]}, L\right], \theta\,[L]\right\rangle_{\left[X_{[d]}\right]}\right], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$$

is called the exponential family to t. We further define for each member with parameters θ the associated energy tensor

$$\phi^{(t,\theta,\nu)}\left[X_{[d]}\right] = \left\langle \sigma^t, \theta\right\rangle_{\left[X_{[d]}\right]}$$

and the cumulant function

$$A^{(t,\nu)}(\theta) = \ln\left[\left\langle \nu, \exp\left[\left\langle \sigma^t, \theta\right\rangle_{\left[X_{[d]}\right]}\right]\right\rangle_{[\varnothing]}\right].$$

Example 8.1 (Exponential family of coin tosses) Recall the family of distributions of boolean $X_{[d]}$ from Example 3.5, which has the order statistic t^+ as a sufficient statistic. We now in addition assume that the variables $X_{[d]}$ are i.i.d. with respect to any member of the family (see Example 3.5). For the variables to be i.i.d., we need $\nu\left[X_{[d]}\right] = \mathbb{I}\left[X_{[d]}\right]$ and can thus choose a representation such that for $z \in \mathrm{val}\,(Z)$

$$\mathbb{P}\left[X_{[d]}\middle|Z = z\right] = \left\langle \beta^{t^+}\left[Y_+, X_{[d]}\right], \xi\left[Y_+, Z = z\right]\right\rangle_{\left[X_{[d]}\right]}$$

where for each $k \in [d+1]$

$$\xi\left[Y_+ = k, Z = z\right] = (1-z)^{d-k} \cdot z^k \,.$$

The marginal distribution $\mathbb{P}\left[Y_+ \middle| Z = z\right]$ is then the binominal distribution $B(d, z)$. When excluding the case of $z \in \{0, 1\}$, the family is a subset of the exponential family with the head count statistic, where each member is reparametrized by

$$\theta := \ln\left[\frac{z}{1-z}\right] \,.$$

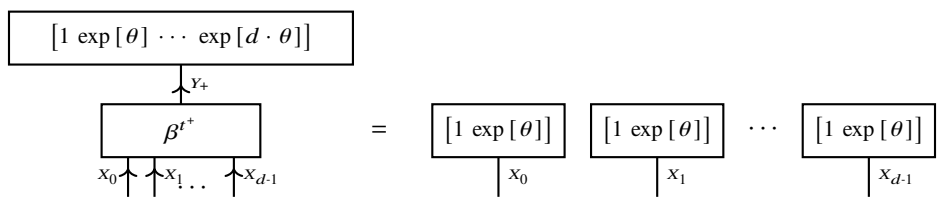
To see that this is true, we observe that the coordinate $y_+ \in [d+1]$ of the activation tensor of $\mathbb{P}^{(t^+,\theta,\mathbb{I})}$ is

$$\alpha^\theta\left[Y_+ = y_+\right] = \exp\left[y_+ \cdot \theta\right] = \frac{z^{y_+}}{(1-z)^{y_+}} \,.$$

Now, with $\mathcal{Z}(\theta) = \frac{1}{(1-z)^d}$ we have for any $x_{[d]}$ with $\sum_{k\in[d]} x_k = y_+$ that

$$\frac{1}{\mathcal{Z}(\theta)} \cdot \left\langle \beta^{t^+}\left[Y_+, X_{[d]} = x_{[d]}\right], \alpha^\theta\left[Y_+\right]\right\rangle_{[\varnothing]} = (1-z)^d \cdot \frac{z^{y_+}}{(1-z)^{y_+}} = z^{y_+} \cdot (1-z)^{d-y_+} \,.$$

Comparing with the activation tensor $\xi\left[Y_+\right]$ above, we note that $\mathcal{Z}(\theta)$ is the partition function of the exponential family and $\mathbb{P}^{(t^+,\theta,\mathbb{I})}\left[X_{[d]}\right]$ coincides with the member $\mathbb{P}\left[X_{[d]} \middle| Z = z\right]$. We further observe that since the statistic t^+ decomposes as a sum of terms depending on single variables only, we have a decomposition of the corresponding CompActNet by



This reproduces the fact that distributions of independent variables are representable by elementary tensors (see Example 3.1).

We used the selection encoding to represent the weighted summation over the statistics, that is the tensor (see Def. **??**)

$$\sigma^t\left[X_{[d]}, L\right] : \bigtimes_{k\in[d]} [m_k] \times [p] \to \mathbb{R}$$

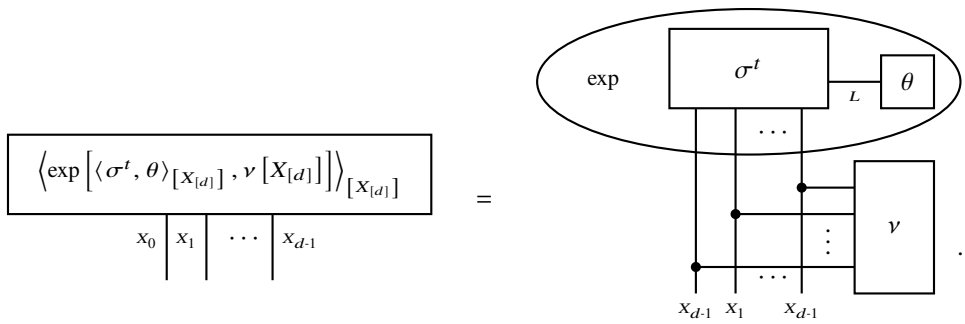
defined for $x_{[d]} \in \bigtimes_{k\in[d]} [m_k]$ and $\ell \in [p]$ as

$$\sigma^t\left[X_{[d]} = x_{[d]}, L = \ell\right] = t_\ell\left[X_{[d]} = x_{[d]}\right] \,.$$

The selection encoding represent the weighted sum of the statistic coordinates by the canonical parameter vector $\theta[L]$ as a contraction

$$\sum_{\ell\in[p]} \theta\left[L = \ell\right] \cdot t_\ell\left[X_{[d]}\right] = \left\langle \sigma^t\left[X_{[d]}, L\right], \theta\left[L\right]\right\rangle_{[X_{[d]}]} \,.$$

For more details on this representation scheme, we refer to Thm. **??** in Chapter **??**. Up to normalization, we sketch the probability distribution of any member by the tensor network diagram



We here denote by an ellipsis the coordinatewise transformation by the exponential function (see Sect. **??**). Since such coordinatewise transformation are nonlinear, they are a caveat for efficient contraction of the diagram.

Since we restrict the discussion to finite state spaces, the distribution $\mathbb{P}^{(t,\theta,\nu)}$ is well-defined for any $\theta\,[L] \in \mathbb{R}^p$. For infinite state space there are sufficient statistics and parameters, such that the partition function $\left\langle \nu, \exp\left[\langle \sigma^t, \theta\rangle_{[X_{[d]}]}\right]\right\rangle_{[\varnothing]}$ diverges and the normalization $\mathbb{P}^{(t,\theta,\nu)}$ is not well-defined. In that cases, the canonical parameters need to be chosen from a subset where the partition function is finite [**?**].

As before, we restrict to boolean base measures, which have to satisfy $\langle \nu\rangle_{[\varnothing]} \neq 0$ for respective distributions to exist. We notice that by positivity of the exponential function, any distribution in an exponential family $\Gamma^{t,\nu}$ is positive with respect to ν (see Def. 3.3). In Chapter **??** we will investigate distributions, where the base measures and the sufficient statistics share a common decomposition framework.

Lemma 8.1 *For any member of an exponential family $\Gamma^{t,\nu}$ we have*

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] = \left\langle \exp\left[\phi^{(t,\theta,\nu)}[X_{[d]}] - A^{(t,\nu)}(\theta)\cdot \mathbb{I}\left[X_{[d]}\right]\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]} .$$

Proof. By definition we have

$$\begin{aligned}
\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] &= \left\langle \exp\left[\left\langle \sigma^t, \theta\right\rangle_{[X_{[d]}]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}|\varnothing]} \\
&= \frac{\left\langle \exp\left[\langle \sigma^t, \theta\rangle_{[X_{[d]}]}, \nu\left[X_{[d]}\right]\right]\right\rangle_{[X_{[d]}]}}{\left\langle \exp\left[\langle \sigma^t, \theta\rangle_{[X_{[d]}]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]}} \\
&= \frac{\left\langle \exp\left[\phi^{(t,\theta,\nu)}[X_{[d]}]\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]}}{\exp\left[A^{(t,\nu)}(\theta)\right]} \\
&= \left\langle \exp\left[\phi^{(t,\theta,\nu)}[X_{[d]}] - A^{(t,\nu)}(\theta)\cdot \mathbb{I}\left[X_{[d]}\right]\right], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]} .\square
\end{aligned}$$

A further useful criterion is that of minimality of an exponential family, as we define next.

Definition 8.2 (Minimal Statistics) We say that a statistic t is minimal with respect to a boolean base measure ν, if there is no pair of a non-vanishing vector $V[L]$ and a scalar $\lambda \in \mathbb{R}$ with

$$\left\langle \sigma^t\left[X_{[d]}, L\right], V[L], \nu\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]} = \lambda\cdot \nu\left[X_{[d]}\right] .$$

If a statistic is not minimal, we can omit coordinates of it without affecting the expressivity $\Gamma^{t,\nu}$. As long as we find a non-vanishing vector $V[L]$ and $\lambda \in \mathbb{R}$ as in Def. 8.2, we can choose a coordinate t_ℓ such that $V[L = \ell] \neq 0$, conclude that the coordinate is linear dependent on the others and drop it as redundant.

8.1.2 Tensor Network Representation

As we have observed, the selection encoding formalism can efficiently represent the energy tensor to a member of an exponential family, but through coordinatewise transform by the exponential does not provide an efficient decomposition scheme of the probability distribution itself. We now overcome this problem with usage of the basis encoding formalism to represent members of exponential families by a single contraction without nonlinear transforms.

Theorem 8.1 (Tensor Network Representation of Exponential Families) *Given any base measure ν and a sufficient statistic t we enumerate for each coordinate $\ell \in [p]$ the image* $\mathrm{im}\,(t_\ell)$ *by a variable Y_ℓ taking values in* $[|\mathrm{im}\,(t_\ell)|]$ *(see for more details on this scheme Chapter* **??***), given an interpretation map*

$$I_\ell \;:\; [|\mathrm{im}\,(t_\ell)|] \to \mathrm{im}\,(t_\ell)\;.$$

For any canonical parameter vector $\theta\,[L] \in \mathbb{R}^p$ we build the activation cores $\alpha^{\ell,\theta}\,[Y_\ell]$ for each coordinate $y_\ell \in$ $[|\mathrm{im}\,(t_\ell)|]$ *by*

$$\alpha^{\ell,\theta}\,[Y_\ell = y_\ell] = \exp\left[\theta\,[L = \ell] \cdot I_\ell(y_\ell)\right]$$

and have

$$\mathbb{P}^{(t,\theta,\nu)}\left[X_{[d]}\right] = \left\langle \{\nu\left[X_{[d]}\right]\} \cup \{\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right] \;:\; \ell \in [p]\} \cup \{\alpha^{\ell,\theta}\,[Y_\ell] \;:\; \ell \in [p]\}\right\rangle_{\left[X_{[d]}|\varnothing\right]}\;.$$

Proof. We embed the image of t in the cartesian product of the coordinate images

$$\mathrm{im}\,(t) \subset \bigtimes_{\ell \in [p]} \mathrm{im}\,(t_\ell)$$

and design enumerate the embedded image of t by the variables $Y_{[p]}$. Thm. **??**, to be shown in Chapter **??**, implies

$$\exp\left[\left\langle \sigma^t, \theta\right\rangle_{\left[X_{[d]}\right]}\right] = \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \exp\left[\langle \cdot, \theta\,[L]\rangle\right]|_{\bigtimes_{\ell \in [p]} \mathrm{im}(t_\ell)}\right\rangle_{\left[X_{[d]}\right]}\;.$$

Here we denote by $\langle \cdot, \theta\,[L]\rangle$ the dual function to $\theta\,[L]$, which assigns to vectors their contraction with $\theta\,[L]$. Its restriction onto the vectors in $\bigtimes_{\ell \in [p]} \mathrm{im}\,(t_\ell)$ is the tensor satisfying

$$\exp\left[\langle \cdot, \theta\rangle\right]|_{\mathrm{im}(t)}\left[Y_{[p]}\right] = \bigotimes_{\ell \in [p]} \exp\left[\cdot\theta\,[L = \ell]\right]|_{\mathrm{im}(t_\ell)}\,[Y_\ell] = \bigotimes_{\ell \in [p]} \alpha^{\ell,\theta}\,[Y_\ell]\;.$$

We further have (see Thm. **??** in Chapter **??**)

$$\beta^t\left[Y_{[p]}, X_{[d]}\right] = \left\langle \{\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right] \;:\; \ell \in [p]\}\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]}\;.$$

Refining the above decomposition of $\exp\left[\left\langle \sigma^t, \theta\right\rangle_{\left[X_{[d]}\right]}\right]$ by these further decompositions we arrive at the claim. □

In the proof of Thm. 8.1 we have observed, that the basis encoding $\beta^t\left[Y_{[p]}, X_{[d]}\right]$ of the statistics decomposed into a tensor network of basis encodings $\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right]$ to the coordinate of the statistic. We can exploit further decomposition mechanisms, which will be discussed in full detail in Chapter **??**, to find even sparser decompositions. This is for example the case, when the coordinates of the statistic are compositions of functions depending on small numbers of variables. When the coordindates of the statistic furthermore share similar parts in their compositions, these parts can be shared in the decomposition. We will investigate such sparsification mechanisms in more detail in Chapter **??**, where the coordinates of the statistic are propositional formulas with a natural decomposition by their syntactical description.

The tensor network representation of an exponential family by Thm. 8.1 is split into computation and activation cores (see Sect. **??**):

- **Computation Cores:** The basis encodings β^{t_ℓ} of the coordinates of a statistic form the computation cores. Our intuition is that they compute the hidden variable Y_ℓ, based on Basis Calculus (see Chapter **??**), which encode the

value of the coordinate with respect to the image interpretation map I_ℓ. We notice that since they are directed with Y_ℓ being the only outgoing variable, they do not influence any contraction with open variables $X_{[d]}$, unless further tensors sharing the variable Y_ℓ are present in the contraction.

- **Activation Cores:** The influence of the contraction is performed by the activation cores $\alpha^{\ell,\theta}[Y_\ell]$, which exploit the computed statistic variables $Y_{[p]}$ and provide in combination with the basis encoding a factor

$$\left\langle \beta^{t_\ell}\left[Y_\ell, X_{[d]}\right], \alpha^{\ell,\theta}[Y_\ell]\right\rangle_{[X_{[d]}]}$$

to the tensor network representing the distribution of the observed variables $X_{[d]}$.

For the implementation of an exponential family as a Computation-Activation Network, see Example **??**.

When the canonical parameter is vanishing at a coordinate, that is $\theta[L = \ell] = 0$, then the corresponding factor is trivial, since $\alpha^{\ell,0}[Y_\ell] = \mathbb{I}[Y_\ell]$ and as a consequence of the directionality of basis encodings we have

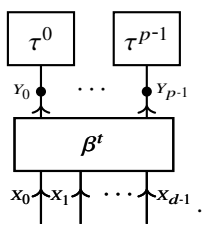
$$\left\langle \beta^{t_\ell}\left[Y_\ell, X_{[d]}\right], \alpha^{\ell,\theta}[Y_\ell]\right\rangle_{[X_{[d]}]} = \left\langle \beta^{t_\ell}\left[Y_\ell, X_{[d]}\right], \mathbb{I}[Y_\ell]\right\rangle_{[X_{[d]}]} = \mathbb{I}\left[X_{[d]}\right] .$$

In that case both the activation core and the corresponding computation core can be dropped from the network without changing its distribution.

By Thm. 8.1 any member of an exponential family is represented by the normalized contraction of a collection of unary activation cores contracted with the computation cores $\beta^{t_\ell}\left[Y_\ell, X_{[d]}\right]$. The unary activation cores form a tensor network on the elementary hypergraph

$$\mathrm{EL} = ([p], \{\{\ell\} : \ell \in [p]\}) ,$$

which hyperedges contain single variables. We call this graph elementary, since it corresponds with elementary tensor network formats consistent of tensor products of vectors. For unary activation cores, that is for the elementary graph EL, any member of $\Lambda^{t,\mathrm{EL}}$ has up to a normalization factor a tensor network decomposition by the diagram

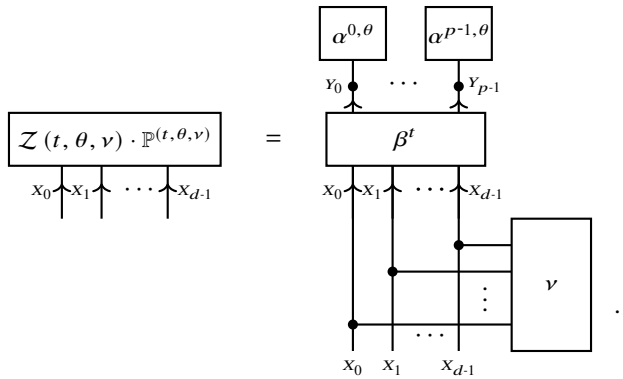


Comparing this representation scheme with Thm. 8.1, we conclude as the next corollary, that any member of an exponential family with trivial base measure can be represented by an elementary activation tensors.

Corollary 8.1 (of Thm. 8.1) *For any statistic* $t : \bigtimes_{k\in[d]}[m_k] \to [p]$ *and trivial base measure* $\nu\left[X_{[d]}\right] = \mathbb{I}\left[X_{[d]}\right]$ *we have*

$$\Gamma^{t,\mathbb{I}} \subset \Lambda^{t,\mathrm{EL}} .$$

For elements of the exponential family with general boolean base measure we have with the activation cores constructed in Thm. 8.1



where the partition function represents the normalizing contraction of the tensor network. Let us note, that when choosing activation cores with nontrivial support, we can also prepare boolean base measures and in principle extend Cor. 8.1 to families of nontrivial base measures. We will investigate such schemes later in Chapter **??**, where we call them Hybrid Logic Networks.

In comparison with the selection encoding representation of energy tensors, we have prepared a contraction without non-linear transforms, which represents the probability distributions being members of an exponential family. However, relation encoding come with the expense of introducing more auxiliary variables compared with selection encodings. To be more precise, while selection encodings bundle the coordinates of the statistic in single selection variables, relation encodings create for each state $\ell \in [p]$ of these selection variable an own auxiliary variable Y_ℓ, which enumerated the image of the coordinate and can therefore be of high dimension. Thus, selection encodings offer in general a more efficient storage format coming at the expense of nonlinear operations in the computation of probabilities. We later will encounter situations, where selection encodings are feasible while relation encodings are not, when applying the formalism of formula selecting networks (see Chapter **??**) in neuro-symbolic reasoning (see Chapter **??**).

Based on Cor. 8.1 a further natural question is, whether $\Gamma^{t,\mathbb{I}}$ is a proper subset of $\Lambda^{t,\mathrm{EL}}$. This is the case for most statistics t, since members of exponential families are positive with respect to their base measure, which is in the corollaries setting trivial, while in $\Lambda^{t,\mathrm{EL}}$ we allow also for activation cores with vanishing coordinates, which in general do not produce positive distributions. The only statistics where $\Gamma^{t,\mathbb{I}}$ is not a proper subset of $\Lambda^{t,\mathcal{G}}$ are along this argumentation constant, since then the activation cores are one-dimensional vectors and vanishing coordinates are prohibited by the need for normalizability. We will follow these intuitions in the discussion of logical reasoning, starting with Chapter **??**, and will use the formats $\Lambda^{t,\mathcal{G}}$ as hybrid formats storing probability distributions and logical knowledge bases.

While we have restricted our discussion on the elementary decomposition of the activation tensor, further decomposition schemes have interesting interpretations as well. Given a CP decomposition of the activation tensor (see for more details Chapter **??**), the corresponding distributions are weighted mixture distributions built from the elementary decompositions. In general, the expressivity increases monotonously with the introduction of additional auxiliary variables and hyperedges in the representation format of activation tensors.

8.1.3 Markov Networks as Exponential Families

As we have claimed before, exponential families can be regarded as a generalization of graphical models. We here show this claim by a construction of an exponential family to any hypergraph, such that any positive Markov Networks on this hypergraph is in the exponential family.

Theorem 8.2 (Exponential Representation of Markov Networks) *Let there be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with a coloring of the nodes by dimensions m_v, we define a sufficient statistics*

$$t^{\mathcal{G}} : \bigtimes_{v\in\mathcal{V}} [m_v] \to \bigtimes_{e\in\mathcal{E}} \left(\bigtimes_{v\in e} [m_v] \right)$$

by a cartesian product $t^{\mathcal{G}} = \bigtimes_{e\in\mathcal{E}} t_e$ of statistics

$$t_e \,:\, \bigtimes_{v\in\mathcal{V}} [m_v] \to \bigtimes_{v\in e} [m_v]$$

defined by the restriction of indices to the respective edge, that is for $x_{\mathcal{V}} \in \bigtimes_{v\in\mathcal{V}} [m_v]$

$$t_e(x_{\mathcal{V}}) = x_e \,.$$

Given any Markov Network with positive tensors $\{\tau^e \,:\, e \in \mathcal{E}\}$ *decorating the hyperedges of* $\mathcal{G}$ *we define*

$$\theta\,[L] = \bigtimes_{e\in\mathcal{E}} \theta_e\,[X_e]$$

where

$$\theta_e\,[X_e] = \ln\left[\tau^e\,[X_e]\right]$$

and L enumerates a concatenation of the states of X_e*. Then, the Markov Network is the member with canonical parameter* θ *of the exponential family with trivial base measure, statistic* $t^{\mathcal{G}}$*, which we denote by* $\Gamma^{t^{\mathcal{G}},\mathbb{I}}$*.*

Proof. We have for any $x_{\mathcal{V}}$

$$\begin{aligned}\prod_{e\in\mathcal{E}} \tau^e\,[X_e = x_e] &= \exp\left[\sum_{e\in\mathcal{E}} \ln\left[\tau^e\,[X_e = x_e]\right]\right] \\ &= \exp\left[\sum_{e\in\mathcal{E}} \theta_e\,[X_e = x_e]\right] \\ &= \exp\left[\sum_{e\in\mathcal{E}} \langle \theta_e\,[X_e]\,, t_e(x_{\mathcal{V}})\rangle_{[\varnothing]}\right] .\end{aligned}$$

By contraction, we further have

$$\langle t\,(X_{\mathcal{V}}, L)\,, \theta\,[L]\rangle_{[X_{\mathcal{V}}]} = \sum_{e\in\mathcal{E}} \langle t_e\,[X_{\mathcal{V}}, X_e]\,, \theta_e\,[X_e]\rangle_{[X_{\mathcal{V}}]}$$

we thus get with the above

$$\langle \{\tau^e \,:\, e \in \mathcal{E}\}\rangle_{[X_{\mathcal{V}}]} = \exp\left[\langle t\,(X_{\mathcal{V}}, L)\,, \theta\,[L]\rangle_{[X_{\mathcal{V}}]}\right] . \tag{8.1}$$

This implies, that the contraction of the tensors in the Markov Network coincides with the exponential of the energy tensor of the constructed member of the exponential family. It follows for the normalization, that

$$\langle \{\tau^e \,:\, e \in \mathcal{E}\}\rangle_{[X_{\mathcal{V}}|\varnothing]} = \left\langle \exp\left[\langle \theta, t\rangle_{[X_{\mathcal{V}}]}\right]\right\rangle_{[X_{\mathcal{V}}|\varnothing]} . \tag{8.2}$$

We thus conclude, that the Markov Network coincides with the constructed member of the exponential family. □

The mean parameter of the Markov Network exponential family is the cartesian product of the marginals $\mu_e\,[X_e]$. They are often referred to as beliefs in the literature, as introduced by [**?**]. For Markov Networks on tree hypergraphs, and their embedding into junction tree formats, the corresponding mean parameter polytope can be characterized by local consistency constraints. More precisely, it can be shown, that for the statistic constructed in Thm. 8.2, in case of tree hypergraphs, the

$$\begin{aligned}\mathcal{M}_{t^{\mathcal{G}}} = \Big\{\mu\,[L] = \left(\mu_e\,[X_e]\right)_{e\in\mathcal{E}} \,:\, &\forall e, \tilde{e} \in \mathcal{E} : \langle \mu_e\,[X_e]\rangle_{[X_{e\cap\tilde{e}}]} = \langle \mu_e\,[X_e]\rangle_{[X_{e\cap\tilde{e}}]}, \\ &\forall e : 0\,[X_e] \prec \mu_e\,[X_e] \wedge \langle \mu_e\,[X_e]\rangle_{[\varnothing]} = 1\Big\} .\end{aligned}$$

That is, the polytope of realizable mean parameters consists of those non-negative and normalized beliefs, which are coinciding on the contraction of shared variables. Capturing these constraints by Lagrange parameters and performing

optimization of certain objectives then results in message-passing schemes, as we will discuss in Chapter **??**. If the hypergraph is not minimally connected, this constructed polytope is only an outer bound of the true mean parameter polytope, but still serves as a motivation of loopy belief propagation schemes (see Chapter 4 in [**?**]).

8.1.4 Representation of Generic Distributions

We now present a universal exponential family, which contains all positive distributions with respect to a base measure. This implies that the formalism of exponential families can capture any probability distribution, when applying statistic functions of large expressivity. Taking for the statistic the identity function $\delta\left[X_{[d]}, L_{[d]}\right]$ defined as

$$\delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right] = \begin{cases} 1 & \text{if} \quad x_{[d]} = \ell_{[d]} \\ 0 & \text{else} \end{cases} ,$$

we can represent any positive probability distribution $\mathbb{P}\left[X_{[d]}\right]$ as a member of the exponential family $\Gamma^{\delta,\mathbb{I}}$. To see this, it is enough to choose

$$\theta\left[L_{[d]}\right] = \left\langle \ln\left[\mathbb{P}\left[X_{[d]}\right]\right], \delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right]\right\rangle_{\left[L_{[d]}\right]} ,$$

where the contraction with δ copies the variables $X_{[d]}$ to $L_{[d]}$. The energy tensor of this member of $\Gamma^{\delta,\mathbb{I}}$ is then

$$\left\langle \theta\left[L_{[d]}\right], \delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = \ln\left[\mathbb{P}\left[X_{[d]}\right]\right]$$

and thus

$$\mathbb{P}^{\delta,\theta,\mathbb{I}}[X_{[d]}] = \left\langle \exp\left[\ln\left[\mathbb{P}\left[X_{[d]}\right]\right]\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]} = \mathbb{P}\left[X_{[d]}\right] .$$

We further note, that the mean parameter of this constructed element of $\Gamma^{\delta,\mathbb{I}}$ is

$$\begin{aligned} \mu\left[L_{[d]}\right] &= \left\langle \mathbb{P}^{\delta,\theta,\mathbb{I}}[X_{[d]}], \delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right]\right\rangle_{\left[L_{[d]}\right]} \\ &= \left\langle \mathbb{P}\left[X_{[d]}\right], \delta\left[X_{[d]} = x_{[d]}, L_{[d]} = \ell_{[d]}\right]\right\rangle_{\left[L_{[d]}\right]} , \end{aligned}$$

and thus coincides with the distribution itself, after a relabelling of the distributed variables. Let us notice, that this family also correspond with the Markov Network on the maximal hypergraph $\mathrm{MAX} = (\mathcal{V}, \{\mathcal{V}\})$. We will further revisit this family in Chapter **??**, where we will refer to it by the minterm family in order to connect with terminology developed for logical reasoning in Chapter **??**.

Chapter 9
Mean Polytopes and Maximum Entropy Distributions

Abstract Beyond elementary: Study generic maximum entropy distributions. Motivation from the chapter before

9.1 Polytopes of Mean Parameters

We in this section investigate properties of probability distributions based on their mean parameters. Given a statistic t, we first define a mean parameter to any distribution by the expectation of the statistic.

Definition 9.1 Let there be a statistic t and a boolean base measure $\nu\left[X_{[d]}\right]$. We call the tensor

$$\mu\left[L\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right]\right\rangle_{[L]}$$

the mean parameter tensor to a distribution $\mathbb{P}\left[X_{[d]}\right]$. The set

$$\mathcal{M}_{t,\nu} = \left\{\left\langle \mathbb{P}, \sigma^t, \nu\right\rangle_{[L]} : \mathbb{P}\left[X_{[d]}\right] \in \Lambda^{\delta,\mathrm{MAX},\nu}\right\}$$

is called the polytope of realizable mean parameters. Here we denote by $\Lambda^{\delta,\mathrm{MAX},\nu}$ the set of all probability distributions representable with respect to ν (see Def. 3.2).

While introduced here as a property of a distribution, the mean parameters will be central to probabilistic inference in Chapter **??**. We in the reminder of this section prepare for this application and derive tensor network representations for distributions having sufficient statistics t, depending on their corresponding mean parameter in the polytope.

9.1.1 Representation by Convex Hulls

First of all, we provide a simple characterization of the sets of mean parameters as the convex hull of the slices to the selection encoding of the statistic (see Figure 9.1). Convex hulls of finite vectors are called $\mathcal{V}$-polytopes (see Lecture 1 in [**?**]).

Theorem 9.1 *For any statistic t the polytope of mean parameters is the convex hull of the slices of σ^t with fixed indices to $X_{[d]}$, that is*

$$\mathcal{M}_{t,\nu} = \mathrm{conv}\left(\sigma^t\left[X_{[d]} = x_{[d]}, L\right] : x_{[d]} \in \bigtimes_{k\in[d]}[m_k],\ \nu\left[X_{[d]} = x_{[d]}\right] = 1\right).$$

Proof. First we realize that the characterization of by ν representable distributions is a standard simplex extended by trivial coordinates, that is

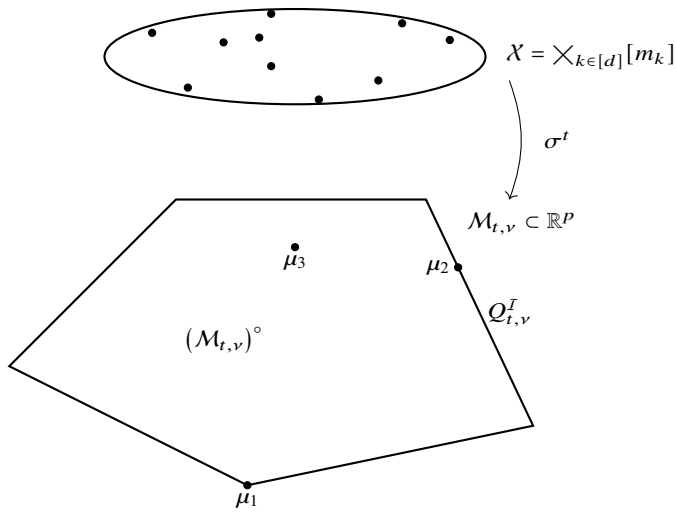


Fig. 9.1 Sketch of the mean polytope $\mathcal{M}_{t,\nu}$ to a statistic t, which is minimal with respect to ν. The mean polytope is a bounded subset of $\mathbb{R}^p$ (here sketched as a 2-dimensional projection). For any vertex μ_1 we find $x_{[d]}$ such that $\mu_1[L] = \sigma^t\left[X_{[d]} = x_{[d]}, L\right]$. Generic mean parameters μ_2 outside the interior are on a face $Q^{\mathcal{I}}_{t,\nu}$. Interior points $\mu_3 \in \mathcal{M}_{t,\mathbb{I}}{}^\circ$ are exactly those reproducible by positive distributions with respect to ν.

$$\Lambda^{\delta,\mathrm{MAX},\nu} = \mathrm{conv}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] \,:\, \nu\left[X_{[d]} = x_{[d]}\right] = 1\right) .$$

This follows from the fact, that the support of any by ν representable distribution is contained in the support of ν. Further, each representable distribution is contained in the convex hull of the one-hot encoded support elements, since any distribution is normalized.

The polytope of mean parameters is a linear transform of the elements in $\Lambda^{\delta,\mathrm{MAX},\nu}$, since the contraction with σ^t is linear. It follows that

$$\begin{aligned}\mathcal{M}_{t,\nu} &= \mathrm{conv}\left(\left\langle\sigma^t\left[X_{[d]}, L\right], \epsilon_{x_{[d]}}\left[X_{[d]}\right]\right\rangle_{[L]} \,:\, \nu\left[X_{[d]} = x_{[d]}\right] = 1\right)\\ &= \mathrm{conv}\left(\sigma^t\left[X_{[d]} = x_{[d]}, L\right] \,:\, \nu\left[X_{[d]} = x_{[d]}\right] = 1\right) .\square\end{aligned}$$

We thus understand the map

$$\sigma^t \,:\, \bigtimes_{k\in[d]}[m_k] \to \mathcal{M}_{t,\nu} \quad , \quad x \to \sigma^t\left[X_{[d]} = x_{[d]}, L\right]$$

as an encoding of states with respect to a statistic t. When the statistic is the universal statistic $t = \delta$, this statistic encoding coincides with the one-hot encoding and the mean polytope is the simplex of dimension $\langle\nu\rangle_{[\varnothing]} - 1$

$$\mathcal{M}_{\delta,\nu} = \mathrm{conv}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] \,:\, \nu\left[X_{[d]} = x_{[d]}\right] = 1\right) .$$

A generic mean polytope $\mathcal{M}_{t,\nu}$ is then the linear transform of the simplex by the contraction with $\sigma^t\left[X_{[d]}, L\right]$, where L is left open.

9.1.2 Representation as Intersecting Half-Spaces

For any vector $a[L] \in \mathbb{R}^p$ and a scalar $b \in \mathbb{R}$, we call the set

$$\left\{\mu[L] \,:\, \langle\mu[L], a[L]\rangle_{[\varnothing]} \le b\right\} \subset \mathbb{R}^p$$

a half-space of $\mathbb{R}^p$. Bounded intersections of finitely many half-spaces are called $\mathcal{H}$-polytopes [**?**]. We state next, that the polytope $\mathcal{M}_{t,\nu}$ of mean parameters is a $\mathcal{H}$-polytopes.

Theorem 9.2 *The set $\mathcal{M}_{t,\nu}$ is for any statistic t and base measure ν a $\mathcal{H}$-polytope, i.e. there exists a finite collection*

$$((a_i\,[L]\,, b_i)\ :\ i \in [n])$$

where $a_i[L]$ a vector and $b_i \in \mathbb{R}$ for all $i \in [n]$ such that

$$\mathcal{M}_{t,\nu} = \left\{\mu\,[L]\ :\ \forall_{i\in[n]}\ \langle \mu\,[L]\,, a_i\,[L]\rangle_{[\varnothing]} \leq b_i\right\}\ .$$

Proof. By Thm. 9.1, the set $\mathcal{M}_{t,\nu}$ is the convex hull of a finite set of vectors and is therefore a $\mathcal{V}$-polytope. We therefore apply the main theorem for polytopes [**?**], which states the equivalence of $\mathcal{V}$-polytopes and $\mathcal{H}$-polytope, for which a proof can be found as Theorem 1.1 in [**?**]. Therefore, $\mathcal{M}_{t,\nu}$ is also a $\mathcal{H}$-polytope and has is thus the intersection of finitely many half-spaces. □

The determination of the half-space parametrizing $((a_i\,[L]\,, b_i)\ :\ i \in [n])$ is, however, in general difficult and the main reason for the intractability of probabilistic inference (see e.g. [**?**]).

9.1.3 Characterization of the Interior

The interior of the mean polytope consists of the mean parameters to positive distributions as we show next.

Theorem 9.3 *For any minimal statistics t with respect to a boolean base measure ν (see Def. 8.2) and a with respect to ν positive distribution $\mathbb{P}\left[X_{[d]}\right]$ we have*

$$\left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right]\right\rangle_{[L]} \in \left(\mathcal{M}_{t,\nu}\right)^\circ\ .$$

Proof. Since by assumption the statistics is minimal, the convex set $\mathcal{M}_{t,\nu}$ is full dimensional (see e.g. Appendix B in [**?**]). We thus use a well-known property for full-dimensional convex sets (see [**?**, **?**]), that $\mu \in \mathcal{M}_{t,\nu}{}^\circ$ if for any non-vanishing vector $V[L]$ there is a there is a $\tilde{\mu}[L]$ with

$$\langle V[L], \mu\,[L]\rangle_{[\varnothing]} < \langle V[L], \tilde{\mu}[L]\rangle_{[\varnothing]}\ .$$

It thus suffices to show for an arbitrary non-vanishing vector $V[L]$ the existence of a distribution $\tilde{\mathbb{P}}$, such that

$$\langle V[L], \mu\,[L]\rangle_{[\varnothing]} < \left\langle V[L], \sigma^t\left[X_{[d]}, L\right], \tilde{\mathbb{P}}[X_{[d]}]\right\rangle_{[\varnothing]}\ .$$

We define for $\epsilon \in \mathbb{R}$

$$\mathbb{P}^\epsilon\left[X_{[d]}\right] = \left\langle \mathbb{P}\left[X_{[d]}\right], \exp\left[\epsilon \cdot \left\langle \sigma^t\left[X_{[d]}, L\right], V[L]\right\rangle_{\left[X_{[d]}\right]}\right]\right\rangle_{\left[X_{[d]}|\varnothing\right]}$$

The derivation of this map at $\epsilon = 0$ is

$$\frac{\partial}{\partial\epsilon}\mathbb{P}^\epsilon\left[X_{[d]}\right]|_{\epsilon=0} = \left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right], V[L]\right\rangle_{\left[X_{[d]}\right]} - \left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right], V[L]\right\rangle_{[\varnothing]} \cdot \mathbb{P}\left[X_{[d]}\right]$$

and thus

$$\begin{aligned}\frac{\partial}{\partial\epsilon}\left\langle \mathbb{P}^\epsilon\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right], V[L]\right\rangle_{[\varnothing]}|_{\epsilon=0} = &\left\langle \mathbb{P}\left[X_{[d]}\right], (\left\langle \sigma^t\left[X_{[d]}, L\right], V[L]\right\rangle^2_{[)]}\right\rangle_{\left[X_{[d]}\right]} \\ &- \left(\left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right], V[L]\right\rangle_{\left[X_{[d]}\right]}\right)^2\ .\end{aligned}$$

We can interpret this quantity as the variance of the random variable

$$\left\langle \sigma^t \left[X_{[d]}, L\right], V[L]\right\rangle_{\left[X_{[d]}=x_{[d]}\right]} ,$$

where $x_{[d]}$ is drawn from $\mathbb{P}\left[X_{[d]}\right]$. The variance is greater than zero, if this random variable is not constant. But from the minimality of t with respect to ν it follows, that this variable is not constant and we therefore have

$$0 < \frac{\partial}{\partial \epsilon} \left\langle \mathbb{P}^{\epsilon}\left[X_{[d]}\right], \sigma^t \left[X_{[d]}, L\right], V[L]\right\rangle_{[\varnothing]} |_{\epsilon=0} .$$

Thus, there is a $\epsilon > 0$ with

$$\langle V[L], \mu[L]\rangle_{[\varnothing]} < \left\langle V[L], \sigma^t \left[X_{[d]}, L\right], \mathbb{P}^{\epsilon}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .\square$$

While Thm. 9.3 only states that the mean parameter of each positive distribution is in the interior, we will construct for each interior point a positive distribution in Chapter **??** by a member of the corresponding exponential family.

9.1.4 Characterization of the Boundary by Faces

Let us now continue with the investigation of the faces of the mean parameter polytope.

Definition 9.2 Given a mean parameter polytope $\mathcal{M}_{t,\nu}$ in the half space representation of Thm. 9.2, and any subset $\mathcal{I} \subset [n]$ we say that the set

$$Q^{\mathcal{I}}_{t,\nu} = \left\{\mu[L] \in \mathcal{M}_{t,\nu} \,:\, \forall_{i\in\mathcal{I}} \ \langle \mu[L], a_i[L]\rangle_{[\varnothing]} = b_i\right\}$$

is the face to the constraints $\mathcal{I}$.

While all inequalities in a half-space representation are satisfied for any element of the polytope, we defined faces by the additional sharp satisfaction of a subset of the half-space inequalities. In this way, the faces build the boundary of $\mathcal{M}_{t,\nu}$. This can be easily verified, since for any vector $\mu[L] \in \mathcal{M}_{t,\nu}$, for which no halfspace inequalities hold sharply, also a neighborhood satisfies the halfspace inequalities. If any halfspace inequality holds sharply, in the other case, the vector is a member of the corresponding face.

If t is not minimal with respect to ν, we find a non-vanishing vector $V[L]$ and a scalar $\lambda \in \mathbb{R}$ such that

$$\left\langle \sigma^t \left[X_{[d]}, L\right], V[L], \nu\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} = \lambda \cdot \nu\left[X_{[d]}\right] .$$

This implies, that any probability distribution $\mathbb{P}\left[X_{[d]}\right]$ representable with ν satisfies

$$\left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t \left[X_{[d]}, L\right], V[L], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \lambda \cdot \left\langle \mathbb{P}\left[X_{[d]}\right], \nu\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \lambda .$$

Any $\mu[L] \in \mathcal{M}_{t,\nu}$ then satisfies

$$\langle \mu[L], V[L]\rangle_{[\varnothing]} = \lambda .$$

Thus, the polytope $\mathcal{M}_{t,\nu}$ is contained in an affine linear subspace and has vanishing interior. We can further understand this equation as two half-space inequalities

$$\langle \mu[L], V[L]\rangle_{[\varnothing]} \leq \lambda \quad \text{and} \quad \langle \mu[L], V[L]\rangle_{[\varnothing]} \geq \lambda ,$$

which can be integrated into any half-space representation. We conclude, that in the case of non-minimal statistics, the whole polytope $\mathcal{M}_{t,\nu}$ is a face itself, since it satisfies these half-space inequalities sharply.

Lemma 9.1 *For each face $Q^{\mathcal{I}}_{t,\nu}$ we have*

$$Q^{\mathcal{I}}_{t,\nu} = \operatorname{conv}\left(\sigma^t \left[X_{[d]} = x_{[d]}, L\right] \,:\, x_{[d]} \in (\sigma^t)^{-1}(Q^{\mathcal{I}}_{t,\nu}),\ \nu\left[X_{[d]} = x_{[d]}\right] = 1\right) .$$

Proof. This holds, since each face is the convex hull of the contained vertices (see Proposition 2.2 and 2.3 in [**?**]). Since the vertices are contained in the image of the statistic encoding σ^t, the vertices contained in $Q^{\mathcal{I}}_{t,\nu}$ are contained in the set

$$\sigma^t\left[X_{[d]} = x_{[d]}, L\right] \,:\, x_{[d]} \in (\sigma^t)^{-1}(Q^{\mathcal{I}}_{t,\nu})\,.\square$$

Lem. 9.1 implies in particular, that faces are mean parameter polytopes with respect to refined base measures. For reference in later chapters, we define these refined base measures next as face measures.

Definition 9.3 The face measure to the face $Q^{\mathcal{I}}_{t,\nu}$ of $\mathcal{M}_{t,\nu}$ is the boolean tensor $\nu^{t,\mathcal{I}}\left[X_{[d]}\right]$ with coordinates to $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ by

$$\nu^{t,\mathcal{I}}\left[X_{[d]} = x_{[d]}\right] = \begin{cases} 1 & \text{if} \quad \gamma^{x_{[d]}} \in Q^{\mathcal{I}}_{t,\nu} \\ 0 & \text{else} \end{cases}\,.$$

We now specify the mean parameter polytope to any face using the face measure as a refinement of the base measure.

Theorem 9.4 *For any face $Q^{\mathcal{I}}_{t,\nu}$ of $\mathcal{M}_{t,\nu}$, we have with the refined base measure*

$$\tilde{\nu}\left[X_{[d]}\right] = \left\langle \nu\left[X_{[d]}\right], \nu^{t,\mathcal{I}}\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]}$$

that

$$Q^{\mathcal{I}}_{t,\nu} = \mathcal{M}_{t,\tilde{\nu}}\,.$$

Proof. We notice that for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$, $x_{[d]} \in (\sigma^t)^{-1}(Q^{\mathcal{I}}_{t,\nu})$ is equal to $\nu^{t,\mathcal{I}}\left[X_{[d]} = x_{[d]}\right] = 1$ and thus

$$\left\{x_{[d]} \,:\, x_{[d]} \in (\sigma^t)^{-1}(Q^{\mathcal{I}}_{t,\nu}),\ \nu\left[X_{[d]} = x_{[d]}\right] = 1\right\} = \left\{x_{[d]} \,:\, \tilde{\nu}\left[X_{[d]} = x_{[d]}\right] = 1\right\}\,.$$

In combination with Lem. 9.1 we then get

$$\begin{aligned} Q^{\mathcal{I}}_{t,\nu} &= \operatorname{conv}\left(\sigma^t\left[X_{[d]} = x_{[d]}, L\right] \,:\, x_{[d]} \in (\sigma^t)^{-1}(Q^{\mathcal{I}}_{t,\nu}),\ \nu\left[X_{[d]} = x_{[d]}\right] = 1\right) \\ &= \operatorname{conv}\left(\sigma^t\left[X_{[d]} = x_{[d]}, L\right] \,:\, x_{[d]} \,:\, \tilde{\nu}\left[X_{[d]} = x_{[d]}\right] = 1\right) \\ &= \mathcal{M}_{t,\tilde{\nu}}\,.\square \end{aligned}$$

Positivity of a distribution with respect to face measures is an equivalent condition for the mean parameter of a distribution to be on a face, as we show next.

Theorem 9.5 *If and only if for a distribution $\mathbb{P}\left[X_{[d]}\right]$ and a face $\mathcal{I}$ we have*

$$\left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right]\right\rangle_{[L]} \in Q^{\mathcal{I}}_{t,\nu}\,,$$

then $\mathbb{P}\left[X_{[d]}\right]$ is representable with respect to the base measure

$$\left\langle \nu\left[X_{[d]}\right], \nu^{t,\mathcal{I}}\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]}\,.$$

Proof. We have

$$\mu\left[L\right] = \sum_{x_{[d]}} \mathbb{P}\left[X_{[d]} = x_{[d]}\right] \cdot \gamma^t\left[X_{[d]} = x_{[d]}, L\right]\,.$$

Now, the $x_{[d]}$ with $\nu^{t,\mathcal{I}}\left[X_{[d]} = x_{[d]}\right] = 1$ are exactly those, for which the conditions $\mathcal{I}$ hold straight. If and only if for a $x_{[d]}$ with $\nu^{t,\mathcal{I}}\left[X_{[d]} = x_{[d]}\right] = 0$ we have $\mathbb{P}\left[X_{[d]} = x_{[d]}\right] > 0$, one of the conditions $\mathcal{I}$ would not hold straight. Thus, if and only if $\mathbb{P}\left[X_{[d]}\right]$ is representable with respect to $\nu^{t,\mathcal{I}}\left[X_{[d]}\right]$, we have $\mu\left[L\right] \in Q^{\mathcal{I}}_{t,\nu}$. $\square$

For members of exponential families, we can make a stronger statement than Thm. 9.5. If for any $\mathbb{P}\left[X_{[d]}\right] \in \Gamma^{t,\nu}$ and a face $\mathcal{I}$ we have $\left\langle \mathbb{P}\left[X_{[d]}\right], \sigma^t\left[X_{[d]}, L\right]\right\rangle_{[L]} \in \left(Q^{\mathcal{I}}_{t,\nu}\right)^{\circ}$ then $\mathbb{P}\left[X_{[d]}\right]$ is positive with respect to the base measure

$$\left\langle \nu\left[X_{[d]}\right], \nu^{t,\mathcal{I}}\left[X_{[d]}\right]\right\rangle_{\left[X_{[d]}\right]} .$$

Let us now investigate tensor network representations of face measures, based on the basis encoding β^t of a statistic. Vertices of $\mathcal{M}_{t,\nu}$ are faces with single elements, that is $\{\mu[L]\}$. By Lem. 9.1 there must be μ must lie in the image of σ^t, since otherwise $\mathcal{M}_{t,\nu}$ would be empty. The vertex measure is then

$$\nu^{t,\mathcal{I}}\left[X_{[d]}\right] = \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \epsilon_\mu\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}\right]}$$

Here we use that each $\mu \in Q^{\mathcal{I}}_{t,\nu} \cap \operatorname{im}(\sigma^t)$ has integer-valued coordinates and denote

$$\epsilon_\mu\left[Y_{[p]}\right] = \bigotimes_{\ell \in [p]} \epsilon_{\mu[L=\ell]}\left[Y_\ell\right] .$$

Theorem 9.6 (Face measure representation) *For any face $Q^{\mathcal{I}}_{t,\nu}$ of $\mathcal{M}$ we have*

$$\nu^{t,\mathcal{I}}\left[X_{[d]}\right] = \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \kappa^{\mathcal{I}}\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}\right]}$$

where

$$\kappa^{\mathcal{I}}\left[Y_{[p]}\right] = \sum_{\mu \in Q^{\mathcal{I}}_{t,\nu} \cap \operatorname{im}(\sigma^t)} \epsilon_\mu\left[Y_{[p]}\right] .$$

Proof. For any $\mu \in Q^{\mathcal{I}}_{t,\nu} \cap \operatorname{im}(\sigma^t)$ the tensor

$$\tau^\mu\left[X_{[d]}\right] = \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \epsilon_\mu\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}\right]}$$

is the indicator of the preimage of μ under σ^t. Since preimages the elements in $Q^{\mathcal{I}}_{t,\nu} \cap \operatorname{im}(\sigma^t)$ are disjoint, the support of $\tau^\mu\left[X_{[d]}\right]$ is disjoint and their sum

$$\sum_{\mu \in Q^{\mathcal{I}}_{t,\nu} \cap \operatorname{im}(\sigma^t)} \tau^\mu\left[X_{[d]}\right]$$

is the indicator of the preimage of $Q^{\mathcal{I}}_{t,\nu}$ under σ^t, which is the face measure $\nu^{t,\mathcal{I}}\left[X_{[d]}\right]$. Exploiting linearity of contraction we have

$$\begin{aligned}
\nu^{t,\mathcal{I}}\left[X_{[d]}\right] &= \sum_{\mu \in Q^{\mathcal{I}}_{t,\nu} \cap \operatorname{im}(\sigma^t)} \tau^\mu\left[X_{[d]}\right] \\
&= \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \sum_{\mu \in Q^{\mathcal{I}}_{t,\nu} \cap \operatorname{im}(\sigma^t)} \epsilon_\mu\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}\right]} \\
&= \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \kappa^{\mathcal{I}}\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}\right]} . \square
\end{aligned}$$

Let us now investigate, which normalized face measures can be computed using t and a hypergraph $\mathcal{G}$.

Example 9.1 (Vertices) Vertices $Q^{\mathcal{I}}_{t,\nu}$ are proper faces of affine dimension 0, that is they consist in single vectors. Since all vertices are in the image $\sigma^t(\mathcal{X})$, there exists an index tuple $x_{[d]} \in \mathcal{X}$ such that $\nu\left[X_{[d]} = x_{[d]}\right] = 1$ and

$$Q^{\mathcal{I}}_{t,\nu} = \left\{\sigma^t\left[X_{[d]} = x_{[d]}, L\right]\right\} .$$

Then $\kappa^{\mathcal{I}}\left[Y_{[p]}\right]$ is the one-hot encoding of the by an interpretation map I assigned index to $\sigma^t\left[X_{[d]} = x_{[d]}, L\right]$, that is

$$\kappa^{\mathcal{I}}\left[Y_{[p]}\right] = \epsilon_{I^{-1}(\sigma^t\left[X_{[d]}=x_{[d]},L\right])}\left[Y_{[p]}\right] .$$

In particular, the activation core is elementary and the face measure to any vertex is in $\Lambda^{t,\mathrm{EL},\nu}$.

Extending Example 9.1, we can provide a coarse estimation of the hypergraph $\mathcal{G}$ required to decompose $\kappa^{\mathcal{I}}$ for generic faces $Q^{\mathcal{I}}_{t,\nu}$. We notice that $\kappa^{\mathcal{I}}\left[Y_{[p]}\right]$ in Thm. 9.6 is a sparse tensor with basis CP rank $\left|Q^{\mathcal{I}}_{t,\nu} \cap \mathrm{im}\,(\sigma^t)\right|$ (see Chapter **??**).

$$\ell_0\left(\kappa^{\mathcal{I}}\left[Y_{[p]}\right]\right) = \left|\sigma^t\left(\nu^{t,\mathcal{I}}\right)\right|$$

where $\left|\sigma^t\left(\nu^{t,\mathcal{I}}\right)\right|$ is the number of different statistic encoding vectors to the support of the face measure $\nu^{t,\mathcal{I}}$. Using the formalism of sparse CP decompositions Chapter **??**, this characterize the basis CP rank of $\kappa^{\mathcal{I}}\left[Y_{[p]}\right]$. However, the basis CP rank is only an upper bound to generic CP rank, which can be loose. By Example 9.2 we provide with the maximal face an example, where the basis CP rank is given by the tensor space dimension, whereas the generic CP rank is one and the normalized face measure is thus still in $\Lambda^{t,\mathrm{EL}}$.

Example 9.2 (Maximal face) The maximal face $Q^{\emptyset}_{t,\nu} = \mathcal{M}_{t,\nu}$ coincides with the mean polytope itself. In this case the corresponding activation tensor to the face measure is trivial, that is

$$\kappa^{\emptyset}\left[Y_{[p]}\right] = \mathbb{I}\left[Y_{[p]}\right] .$$

$\kappa^{\emptyset}$ is elementary and the normalized face measure $\nu^{t,\emptyset}$ to the maximal face is in $\Lambda^{t,\mathrm{EL}}$.

9.1.5 Partition into Effective Interiors of Faces

Let us now introduce effective interiors, which enables us to find disjoint partitions of the mean polytope.

Definition 9.4 (Effective Interior) Let $\mathcal{U} \subset \mathbb{R}^p$ be an arbitrary set and $\mathcal{L}$ the minimal affine subspace of $\mathbb{R}^p$ containing $\mathcal{U}$. Then the effective interior, denoted $(\mathcal{U})^\circ$ is the the interior of $\mathcal{U}$ in the space $\mathcal{L}$.

Lemma 9.2 *Any polytope is a disjoint union of the effective interiors of its faces, that is*

$$\mathcal{M}_{t,\nu} = \dot{\bigcup_{\mathcal{I}\subset[n]}} \left(Q^{\mathcal{I}}_{t,\nu}\right)^\circ .$$

Proof. For any $\mu \in \mathcal{M}_{t,\nu}$ we find a face such that $\mu \in Q^{\mathcal{I}}_{t,\nu}$. If $\mu \notin \left(Q^{\mathcal{I}}_{t,\nu}\right)^\circ$, then there is a face $Q^{\tilde{\mathcal{I}}}_{t,\nu} \subset Q^{\mathcal{I}}_{t,\nu}$ of smaller affine dimension such that $\mu \in Q^{\tilde{\mathcal{I}}}_{t,\nu}$. When continuing this process we reach a face such that $\mu \notin \left(Q^{\mathcal{I}}_{t,\nu}\right)^\circ$, since the faces with affine dimension 0 are vertices and they coincide with their effective interior because they contain a single vector. □

In this way, we find to each $\mu \in \mathcal{M}_{t,\nu}$ a unique exponential family with statistics t and base measure by a face measure, such that μ is reproduced by an element of that exponential family. We will show in Chapter **??**, that these reproducing distributions maximize the entropy among any other reproducing distribution.

9.1.6 Centers

We provide further intuition on the position of the mean parameter, when interpreting ν and t as soft and hard constraint mechanisms on a distribution. To this end, let $\mathcal{X}$ be an arbitrary finite state set, where we so far had the set $\bigtimes_{k\in[d]}[m_k]$ for factored representations.

Definition 9.5 We call the mean parameter of a normalized base measure, that is the vector

$$\mu_{t,0,\nu}\left[L\right] = \left\langle \langle \nu \rangle_{\left[X_{[d]}|\varnothing\right]}, \sigma^t\left[X_{[d]}, L\right]\right\rangle_{[L]}.$$

the center of $\mathcal{M}_{t,\nu}$ (see Figure 9.2).

We have

$$\mu_{t,0,\nu}\left[L\right] = \frac{1}{\langle \nu \rangle_{[\varnothing]}} \sum_{x_{[d]}:\nu\left[X_{[d]}=x_{[d]}\right]=1} \sigma^t\left[X_{[d]} = x_{[d]}, L\right]$$

and therefore understand $\mu_{t,0,\nu}$ as the weighted center of $\mathcal{M}_{t,\nu}$, where weights are allocated on the image of the statistics encoding by the cardinality of their pre-images. These concepts are sketched in Figure 9.2 in blue.

The choice of a base measure ν can be regarded as the hard structure of a distribution. Given ν, we further implement soft structure in form of distributions in exponential families $\Gamma^{t,\nu}$. While the normalized base measure ν reproduces only the weighted center of $\mathcal{M}_{t,\nu}$, canonical parameters can be chosen to alter to any interior point in $\mathcal{M}_{t,\nu}$. This alternations are sketched in Figure 9.2 in red.

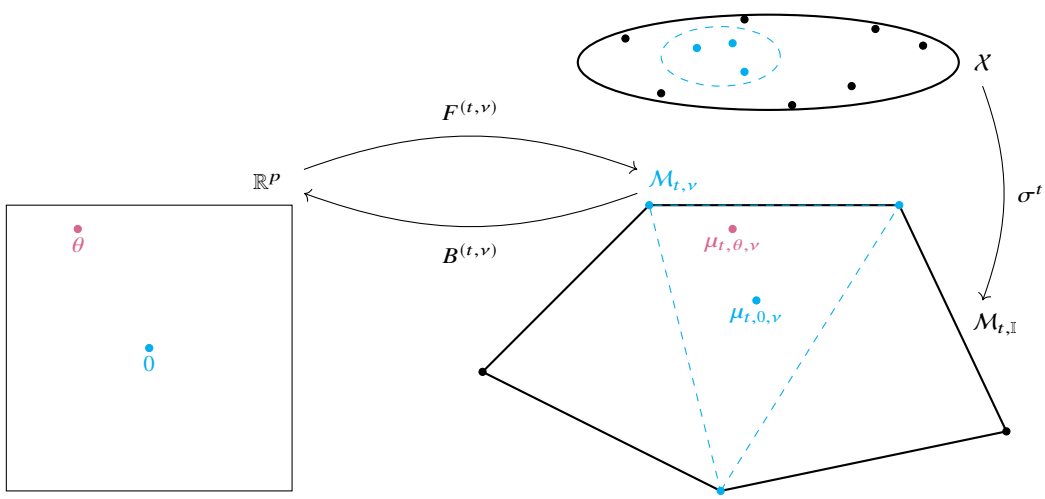


Fig. 9.2 Sketch of hard constraints (blue) and soft constraints (red) determining the position of the mean parameter in $\mu_{t,\mathbb{I}}$. In the set of states $\mathcal{X}$, the support of a base measure ν is marked blue, and the base measure is the corresponding subset encoding. The normalized ν has the mean parameter $\mu_{t,0,\nu}$, and the mean polytope $\mathcal{M}_{t,\nu}$ is the convex hull of the image to the statistics encoding σ^t restricted on the support set of ν. Given a base measure ν, the corresponding forward and backward maps implement soft contraints depending on canonical parameters θ. The forward mapping maps the zero canonical parameter to $\mu_{t,0,\nu}$, correspond with the normalized base measure. Generic parameters θ (red) are mapped onto the interior ${\mathcal{M}_{t,\nu}}^{\circ}$.

Chapter 10
Variational Inference

Abstract We present here variational inference methods to reason on Computation-Activation Networks.
Topics:

- I.3.4 (Exponential family) Variational inference
- I.3.7 Mean field methods

Chapter 11
Inductive Reasoning

Abstract In this chapter, we investigate the problem of selecting statistics.
Topics:

- II.7 Formula Selecting Networks
- II.9.5 Structure Learning in Hybrid Logic Networks

Chapter 12
Probabilistic Guarantees

Abstract Here we show probabilistic guarantees for the success of the reasoning mechanisms on Computation-Activation Networks.

Topics: Chapter II.10

Chapter 13
Quantum Inference

Abstract We here investigate, how inference on Computation-Activation Networks can be executed by quantum algorithms. While we find circuits corresponding to the computation networks, we exploit Grover operators for the activation of the computed qubits.

13.1 Introduction

Tensor network contractions have been identified as central inference mechanism of neuro-symbolic AI. However, their exact execution is NP-hard, and one either has to restrict itself to feasible situations, or apply approximations.

We in this chapter investigate, how specific contractions (especially forward inference on Computation-Activation Networks) can be performed more efficiently on a Quantum Computer.

Tensor Network contractions can be performed by quantum computing as follows:

- **Rejection Sampling based:** Can augment any tensor network by an Bayesian network, which is efficiently inferred by forward sampling. We prepare quantum states, which measurement statistics can be utilized to prepare samples from interesting distributions such as Computation-Activation Networks. The square root quantum advantage is obtained by amplitude amplification.
- **Phase Estimation based:** Measure the eigenvalues of a Grover operator to a formula and a q-sample.
- **Fourier Transform based:** Limited contractions (given promises on their possible values) can be formulated as Hidden Subgroup Problems and solved by Fourier transforms. We are inspired by Deutsch-Jozsa algorithm, which we generalize here to a general moment computation problem.

Note that the rejection sampling scheme and the phase estimation are ignorant under the phase of the prepared state, and only the Fourier transform based scheme depend on the phase.

Generic tensor network contraction is NP-hard, reflecting the hardness of logical and probabilistic inference. Quantum Computers can prepare states, which measurement distribution are specific network contractions, which can be hard to contract classically. We study schemes to retrieve information about these states, for example by sampling schemes, and use the retrieved information in inference.

13.1.1 Circuit Preparation schemes

Circuit preparing schemes based on approximation:

- Q-Alchemy [**?**], Q-Tucker [**?**]
- Tensor-Network Optimization based (alternating schemes) [**?**, **?**]
- Preparing a quantum circuit given an MPS requires logarithmic depth in the tensor order [**?**]

Exact circuit preparing schemes for distributions:

- Grover-Rudolph [**?**], but no quantum advantage [**?**]

- Uniform controlled rotations [**?**]
- Quantum Shannon decomposition [**?**]

Circuit simulation: Since qcreason can prepare quantum circuits to arbitrary tensor networks, it can also be used to simulate quantum circuits (with an overhead!).

- [**?**, **?**]

13.1.2 Related Works

Quantum algorithms for linear algebra:

- Contraction by SWAP and Hadamard test (see Appendix)
- Deutsch-Jozsa Algorithm
- HHL algorithm to solve linear equations [**?**]

13.2 Grover operator

We derive quantum inference schemes based on the Grover operator, which we want to first introduce in full generality.

13.2.1 Definition

Let us assume we have two orthogonal quantum states $\psi^{\parallel}\left[X_{[d]}\right], \psi^{\perp}\left[X_{[d]}\right]$ spanning a subspace $V = \operatorname{span}\left(\psi^{\parallel}\left[X_{[d]}\right], \psi^{\parallel}\left[X_{[d]}\right]\right)$ and let us choose another state

$$\psi^{0}\left[X_{[d]}\right] \in V\,.$$

Then there is $\alpha \in [0, 4\pi)$ with

$$\psi^{0}\left[X_{[d]}\right] = \sin\left(\frac{\alpha}{2}\right)\psi^{\parallel}\left[X_{[d]}\right] + \cos\left(\frac{\alpha}{2}\right)\psi^{\perp}\left[X_{[d]}\right]\,.$$

Consider the reflection about the states $\psi^{\perp}$ and ψ^{0} by

$$\begin{aligned}\mathcal{S}^{\perp}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right] &= 2\cdot\psi^{\perp}\left[X^{\text{in}}_{[d]}\right]\otimes\psi^{\perp}\left[X^{\text{out}}_{[d]}\right] - \delta\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]\\ \mathcal{S}^{0}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right] &= 2\cdot\psi^{0}\left[X^{\text{in}}_{[d]}\right]\otimes\psi^{0}\left[X^{\text{out}}_{[d]}\right] - \delta\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]\,,\end{aligned}$$

and their concatenation

$$G\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right] = \left\langle \mathcal{S}^{\perp}\left[X^{\text{in}}_{[d]}, X^{\text{mid}}_{[d]}\right], \mathcal{S}^{0}\left[X^{\text{mid}}_{[d]}, X^{\text{out}}_{[d]}\right]\right\rangle_{\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]}$$

which is called the *Grover operator*.

13.2.2 Action on projected quantum states

The Grover operator leaves any quantum state ψ invariant, which is orthogonal to $\psi^{\parallel}$ and $\psi^{\perp}$ (such states get a factor (-1) by both reflection operations). The projection of the quantum state onto V gets rotated by the Grover operator (see Figure 13.1). To be more precise, we denote the projection by $(\mathcal{P}\psi)[L]$ which has the coordinates

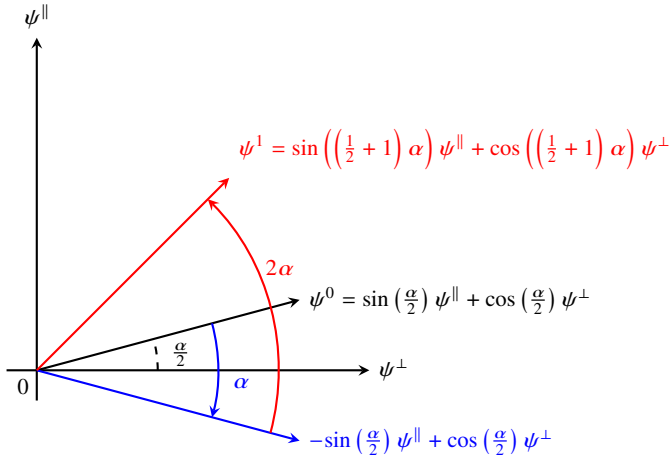


Fig. 13.1 Grover operator as a rotation in span $(\psi^{\parallel}, \psi^{\perp})$.

$$(\mathcal{P}_V\psi)\,[L=0] = \left\langle \left(\psi\left[X_{[d]}\right]\right)^{\dagger}, \psi^{\parallel}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$$
$$(\mathcal{P}_V\psi)\,[L=1] = \left\langle \left(\psi\left[X_{[d]}\right]\right)^{\dagger}, \psi^{\parallel}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} .$$

We define a matrix representation of the Grover operator as

$$G^{\text{red}}\left[L^{\text{out}}, L^{\text{in}}\right] = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{pmatrix}\left[L^{\text{out}}, L^{\text{in}}\right]$$

and it holds for any quantum state $\psi\left[X_{[d]}\right]$ that

$$(\mathcal{P}_V G\psi)\left[L^{\text{out}}\right] = \left\langle G^{\text{red}}\left[L^{\text{out}}, L^{\text{in}}\right], (\mathcal{P}_V\psi)\left[L^{\text{in}}\right]\right\rangle_{[\text{out}]} .$$

The Grover operator therefore has the eigenvalues:

- $\exp[\pm i\alpha]$ with the eigen spaces spanned by $\psi^{\parallel} \pm i\psi^{\perp}$
- 1 with the eigen space by the orthogonal complement of V

Application the Grover operator $j \in \mathbb{N}$ times on ψ^0 results in the state

$$\psi^j\left[X_{[d]}\right] = \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\parallel}\left[X_{[d]}\right] + \cos\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\perp}\left[X_{[d]}\right] .$$

13.2.3 Circuit Realization of the reflection operators

Let us now investigate how the reflection operations can be realized by circuits.

13.2.3.1 Construction based on controlled Pauli-Z

First, the reflection about the state $\epsilon_{1_{[d]}}\left[X_{[d]}\right]$ is realized by a multiple controlled Pauli-Z (see Figure 13.2)

$$-\mathcal{S}^{\epsilon_{1_{[d]}}}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right] = \left\langle \text{MC}^{\text{d}-1}\sigma_3\left[X^{\text{in}}_{d-1}, X^{\text{out}}_{d-1}, X_{[d-1]}\right], \delta\left[X'_{[d-1]}X^{\text{in}}_{[d-1]}, X^{\text{out}}_{[d-1]}\right]\right\rangle_{\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]}$$

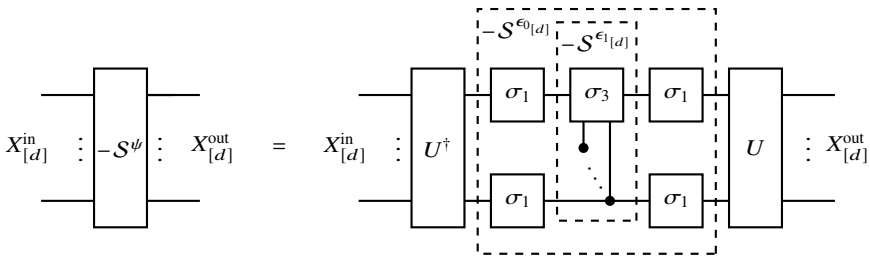


Fig. 13.2 Preparation of the negated reflection operator $-\mathcal{S}^{\psi}$ about a state ψ, which is prepared by a unitary U acting on the ground state $\epsilon_{0_{[d]}}\left[X_{[d]}\right]$. The multiply controlled Pauli-Z is the negated reflection $-\mathcal{S}^{1_{[d]}}$, and extended to the negated reflection $-\mathcal{S}^{1_{[d]}}$ about the ground state by adding layers of single qubit Pauli-X. When concatenating with $U^{\dagger}$ and U, the $-\mathcal{S}^{\psi}$ results.

Note that a global phase of -1 is introduced, which leaves the measurement distribution invariant. Adding layers of Pauli-X gates to each $k \in [d]$ before and after $-\mathcal{S}^{\epsilon_{1_{[d]}}}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]$ (see Figure 13.2) prepares the negated reflection $-\mathcal{S}^{\epsilon_{0_{[d]}}}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]$, since

$$\epsilon_1\left[X^{\text{out}}_k\right] = \left\langle \sigma_3\left[X^{\text{out}}_k, X^{\text{in}}_k\right], \epsilon_0\left[X^{\text{in}}_k\right]\right\rangle_{\left[X^{\text{out}}_k\right]} .$$

Having a unitary U preparing ψ by action on the ground state, we realize the reflection (see Figure 13.2)

$$\mathcal{S}^{\psi}\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right] = \left\langle U^{\dagger}\left[X^{\text{in}}_{[d]}, X^{\text{mid},0}_{[d]}\right], \mathcal{S}^{\epsilon_{0_{[d]}}}\left[X^{\text{mid},0}_{[d]}, X^{\text{mid},1}_{[d]}\right], U\left[X^{\text{mid},1}_{[d]}, X^{\text{out}}_{[d]}\right]\right\rangle_{\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]} .$$

13.2.3.2 Construction based on oracle with ancilla head qubit

Alternatively we can use the phasekick technique when acting on an disentangled ancilla qubit prepared in the Hadamard basis state

$$\epsilon^{-}[Y] = \frac{1}{\sqrt{2}} \cdot \begin{pmatrix} 1 \\ -1 \end{pmatrix}[Y] .$$

We notice, that

$$\left\langle C^f\left[Y^{\text{out}}, Y^{\text{in}}, X_{[d]} = x_{[d]}\right], \epsilon^{-}\left[Y^{\text{in}}\right]\right\rangle_{[Y^{\text{out}}]} = \begin{cases} \epsilon^{-}\left[Y^{\text{in}}\right] & \text{if} \quad f(x_{[d]}) = 0 \\ -\epsilon^{-}\left[Y^{\text{in}}\right] & \text{if} \quad f(x_{[d]}) = 1 \end{cases}$$

Let us consider the span of one-hot encoded models of $\neg f$, namely

$$V^{\neg f} := \text{span}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] \;:\; f(x_{[d]}) = 0\right)$$

and the span of one-hot encoded models

$$V^{f} := \text{span}\left(\epsilon_{x_{[d]}}\left[X_{[d]}\right] \;:\; f(x_{[d]}) = 1\right) .$$

The reflection across $V^{\neg f}$ is then performed by (see Figure 13.3)

$$\mathcal{S}^{V^{\neg f}}\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right] = \left\langle \epsilon^{-}\left[Y^{\text{out}}\right], C^f\left[Y^{\text{out}}, Y^{\text{in}}, X_{[d]}\right], \epsilon^{-}\left[Y^{\text{in}}\right], \delta\left[X_{[d]}, X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right]\right\rangle_{\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right]}$$

We further have

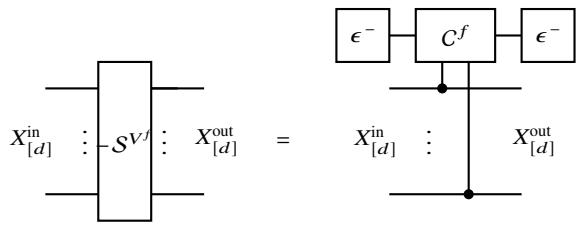


Fig. 13.3 Realization of the reflection across the subspace of one-hot encoded models to a Boolean function f. An ancilla qubit is prepared in the Hadamard basis state ϵ^- and serves as the head qubit of the oracle C^f. By the phase kickback effect, the qubit stays disentangled and the reflection $-\mathcal{S}^{V^f}$ is effectively performed on the qubits $X_{[d]}$.

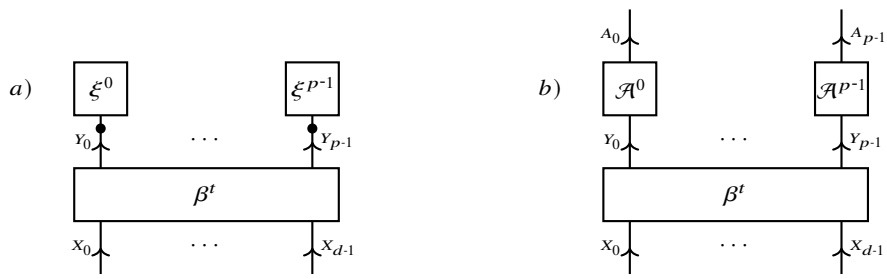


Fig. 13.4 Ancilla augmentation of an elementary Computation-Activation Network a) by replacing each leg vector of the elementary activation tensor by an directed ancilla augmentation.

$$\mathcal{S}^{V^f}\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right] = -\mathcal{S}^{(V^f)^\perp}\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right] = -\mathcal{S}^{V^{\neg f}}\left[X^{\text{out}}_{[d]}, X^{\text{in}}_{[d]}\right].$$

When preparing a disentangled ancilla qubit Y in the basis state $\epsilon^-[Y]$, the oracle C^f is effectively a negated reflection across the span of one-hot encoded models of f.

13.3 Preparation of q-samples to CompActNets

We investigate, how the above circuit encoding schemes can be applied in the preparation of q-samples to Computation-Activation Networks. In particular, we build compositions of computation circuits and activation circuits, for which conditional distributions coincide with Computation-Activation Networks. Note that the resulting circuits are graph-controlled circuits, equivalent to Bayesian networks.

The computation network consists already of directed cores and therefore is already a Bayesian network. The activation network however needs ancilla augmentation.

Elementary Computation-Activation Networks can be augmented by ancilla augmentation of each leg vector in the activation tensor network (see Figure 13.4). We understand ancilla variables as variables in a Bayesian Network having single parents, corresponding with the ancilla augmentation of an elementary tensor. For an example, see Figure 13.7.

To a generic activation hypergraph $\mathcal{G}$, we would do ancilla augmentation for the activation tensor network, where each hidden variable is treated as a distributed qubit. To treat it as a distributed qubit, each hidden activation qubit is prepared in uniform state (i.e. a Hadamard gate acting on a ground state) and will be omitted in the measurement scheme (either not measured, or its measurement ignored). This can be understood from a tensor network perspective, where marginalization means contracting, which is exactly how hidden activation variables are used. For an example in the TT format, see Figure 13.5.

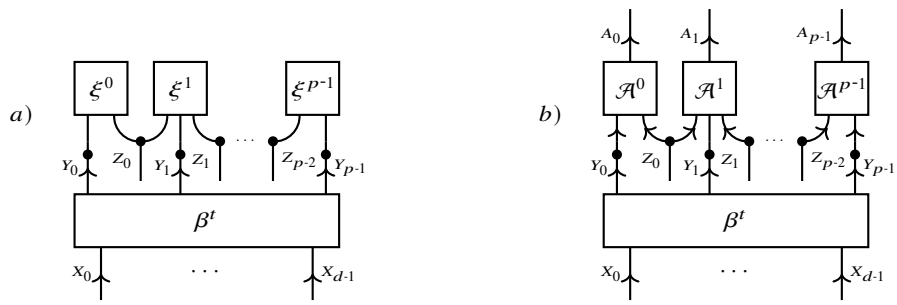


Fig. 13.5 Ancilla augmentation of an TT Computation-Activation Network a) by a Bayesian Network b). The hidden variables in the TT activation tensor are treated as distributed variables. Marginalization over the hidden variables and conditioning on the activation variables in state 1, we reproduce the Computation-Activation Network.

13.3.1 Sampling from Computation-Activation Networks

For Computation-Activation Networks we find a q-sample by concatenating computation circuits and activation circuits (see Figure 13.6).

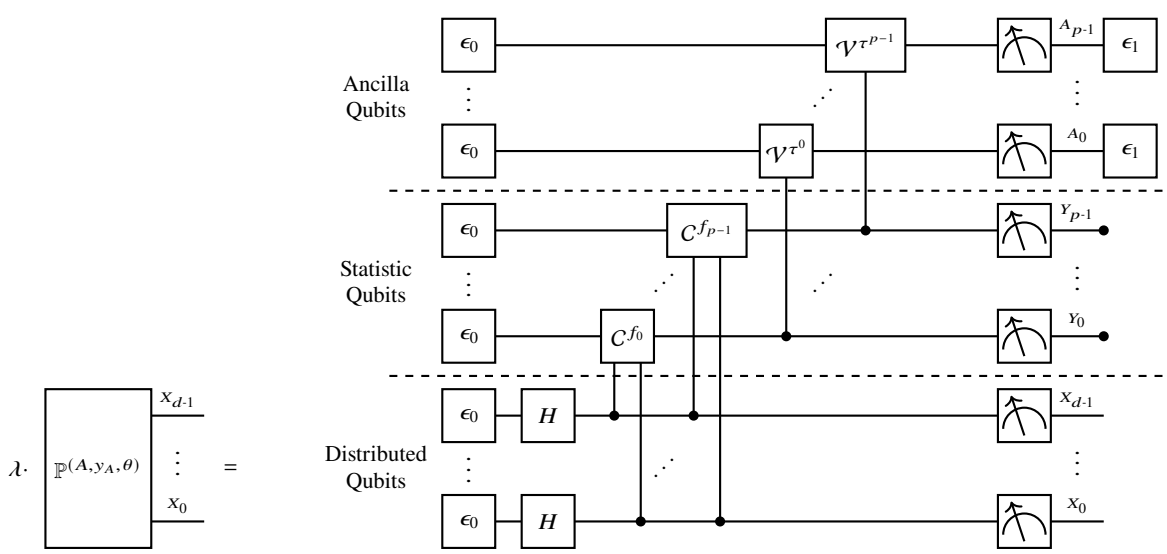


Fig. 13.6 Quantum Circuit to reproduce a q-sample of an augmented Computation-Activation Network. We measure the distributed qubits $X_{[d]}$ and the ancilla qubits $A_{[p]}$ and reject all samples, where an ancilla qubit is measured as 0.

Example 13.1 (Toy Accounting Example) For a more detailed example, let us consider a system of three variables $A1$ Account 1 is booked, $A2$ Account 2 is booked, F a feature on an invoice. Assume the following two rules have to be respected:

- Exactly one account must be booked.
- If feature F is present on the invoice, the account A1 is typically booked.

We formalize this with the statistic

$$t = (X_{A1} \oplus X_{A2}, X_F \Rightarrow X_{A1}) .$$

Any elementary Computation-Activation Network with the statistic t can be realized by the circuit shown in Figure 13.7. For the preparation of the first statistic qubit Y_0 to $t_0 = X_{A1} \oplus X_{A2}$ by a computation circuit we exploit that

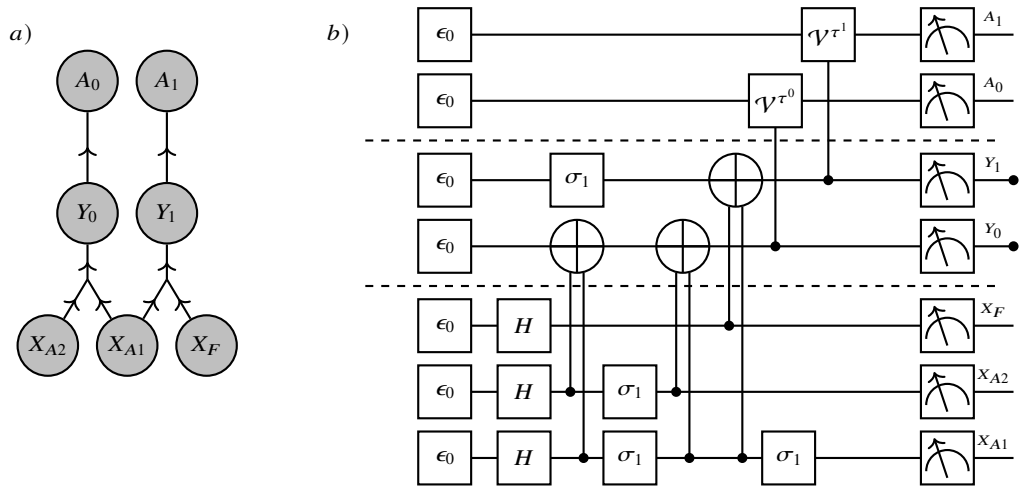


Fig. 13.7 Circuit to sample from the Computation-Activation Network in the toy accounting Example 13.1.

$$\begin{aligned} X_{A1} \oplus X_{A2} &= \epsilon_1 [X_{A1}] \otimes \epsilon_0 [X_{A2}] + \epsilon_0 [X_{A1}] \otimes \epsilon_1 [X_{A2}] \\ &= (\epsilon_1 [X_{A1}] \otimes \epsilon_0 [X_{A2}]) \oplus (\epsilon_0 [X_{A1}] \otimes \epsilon_1 [X_{A2}]) \end{aligned}$$

where by $\oplus$ we denote coordinatewise summation mod 2. The statistic qubit Y_0 is therefore prepared by two controlled NOT gates (see Figure 13.7b).

For the preparation of the second statistic qubit Y_1 to $t_1 = X_F \Rightarrow X_{A1}$ we exploit that the implication is True except for the case where the premise X_F is True and the head X_{A1} is False. In out mod 2 calculus, this amounts to

$$X_F \Rightarrow X_{A1} = \mathbb{I}[X_F, X_{A1}] \oplus \epsilon_1 [X_F] \otimes \epsilon_0 [X_{A1}] \ .$$

It follows that the statistic qubit Y_1 is prepared by a NOT gate (that is a Pauli-X gate σ_1) and a second controlled NOT gate (see Figure 13.7b).

For both cases, the preparation of the statistic qubit can be done by two controlled NOT gates exploiting the polynomial sparsity of the connectives.

Any non-elementary Computation-Activation Network with the statistic t can be prepared by the circuit, when choosing a single ancilla qubit, which is uniformly controlled by the qubits $Y_{[2]}$.

13.4 Quantum Rejection Sampling

We present the Quantum Rejection Sampling method in two steps:

- **Ancilla augmentation:** Find an ancilla augmentation of a distribution, such that there is a unitary U preparing a state with the augmented distribution as measurement distribution
- **Acceptance amplification:** Amplify the amplitude of the ancilla qubits in state 1 by repeated application of the Grover operator

Compared with the rejection sampling approach on the start augmentation of the distribution, we achieve a square-root speed-up in the expected number of gates to be applied per sample.

We in particular apply this scheme to sample from Computation-Activation Networks, where we apply the q-sample preparation scheme of Sect. 13.3.

13.4.1 Acceptance Amplification

Let there be a quantum state, which measurement distribution is an ancilla augmentation $\tilde{\mathbb{P}}[A_{[p]}, X_{[d]}]$ of a target distribution $\mathbb{P}\left[X_{[d]}\right]$. We now show how Grover operators can be used to amplify the acceptance state and improve the efficiency of the rejection sampling method through an increase of the acceptance probability. This is an instance of amplitude amplification.

13.4.1.1 Grover construction

We relate the state $\psi^{\parallel}$ with the event $A_{[p]} = 1_{[p]}$, by constructing it as

$$\psi^{\parallel}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^{t}\left[Y_{[p]}, X_{[d]}\right], \sqrt{\mathbb{P}\left[X_{[d]}\right]}\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]}$$

The complement state $\psi^{\perp}$ is constructed if $\lambda \neq 1$ (in which case the acceptance amplilification scheme would not be necessary)

$$\mathbb{P}^{\perp}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \frac{1}{1-\lambda} \cdot \left(\tilde{\mathbb{P}}[A_{[p]}, Y_{[p]}, X_{[d]}] - \lambda \cdot \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \mathbb{P}\left[Y_{[p]}, X_{[d]}\right]\right)$$

and

$$\psi^{\perp}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sqrt{\mathbb{P}^{\perp}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right]}\,.$$

We thus have

$$\psi^{0}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sin\left(\frac{\alpha}{2}\right)\psi^{0}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] + \cos\left(\frac{\alpha}{2}\right)\psi^{1}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right]$$

where the α is related to the acceptance probability as

$$\sin\left(\frac{\alpha}{2}\right) = \sqrt{\lambda}\,.$$

13.4.1.2 Circuit Realization

Both reflection operators are realized as their negations using the Pauli-Z scheme (see Sect. 13.2). The negations chancel in their concatenation, which is therefore the Grover operator.

13.4.1.3 Ancilla augmentation preserved

The measurement distribution to any quantum state in $V \backslash \mathrm{span}\,(\psi^{\perp})$ is an ancilla augmentation of $\mathbb{P}\left[X_{[d]}\right]$. When amplifying the acceptance probability we stay in the subspace V. When we do not overrotate and since we start with $\lambda > 0$ the amplified state is never in span $(\psi^{\perp})$.

13.4.2 Optimal rotation numbers

Now the application of j grover operators G on ψ^{0} prepares a state

$$\psi^{j}\left[A_{[p]}, X_{[d]}\right] = \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\parallel}\left[A_{[p]}, X_{[d]}\right] + \cos\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\perp}\left[A_{[p]}, X_{[d]}\right]$$

The state is closest to $\psi^{\parallel}$ when

$$\left(\frac{1}{2}+j\right)\alpha \approx \frac{\pi}{2}\,.$$

Estimating for small λ (tight for small λ)

$$\sqrt{\lambda} = \sin\left(\frac{\alpha}{2}\right) \leq \frac{\alpha}{2}$$

we get a bound

$$j^{\text{opt}} \leq \frac{\pi}{4\sqrt{\lambda}}\,.$$

This shows that we have a square root improvement on classical rejection sampling, which would require $\frac{1}{\lambda}$ repetitions to produce a sample. Note however that we compare quantum circuit length with the number of classical repetitions in sampling.

13.4.3 Applications on CompActNets

The above scheme prepares the ancilla augmented Computation-Activation Network

$$\tilde{\mathbb{P}}[A_{[p]}, Y_{[p]}, X_{[d]}] = \lambda \cdot \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \mathbb{P}\left[X_{[d]}\right]\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} + (1-\lambda)\mathbb{P}^{\perp}\left[X_{[d]}, Y_{[p]}, A_{[p]}\right]$$

and the prepared state is the combination of q-samples

$$\psi^0\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sqrt{\lambda} \cdot \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \sqrt{\mathbb{P}\left[X_{[d]}\right]}\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} + \sqrt{(1-\lambda)}\sqrt{\mathbb{P}^{\perp}\left[X_{[d]}, Y_{[p]}, A_{[p]}\right]}\,.$$

Sampling from the augmented Computation-Activation Network can be also done classically by forward sampling: Just draw the variables from uniform, calculate the value qubit by a logical circuit inference and accept with probability by the computed value. To achieve a quantum advantage we manipulate the circuit first by the amplitude amplification scheme and increase the acceptance probability.

13.4.4 Examples

13.5 Moment Estimation

Instead of amplitude amplification we here exploit the Grover operator from a different perspective:

- Good and bad states by the restriction to the support of formulas
- Good state reflection using the oracle on an disentangled ancilla qubit in the Hadamard basis state $\epsilon^{-}[Y]$
- We perform Quantum Phase Estimation to extract the moment through the phase of the eigenvalue of the Grover algorithm

Given a distribution $\mathbb{P}\left[X_{[d]}\right]$ and a propositional formula $f\left[X_{[d]}\right]$, we now investigate how to use Quantum Phase Estimation to estimate

$$\mu = \left\langle \mathbb{P}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\,.$$

We assume that

- a circuit U preparing a state with measurement distribution $\mathbb{P}\left[X_{[d]}\right]$, i.e.

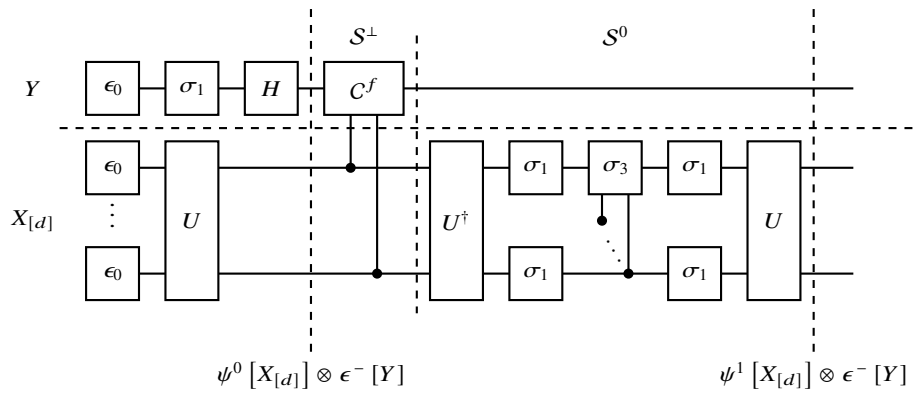


Fig. 13.8 Amplification scheme of the at the models of a Boolean function f supported coordinates of a quantum state. The ancilla variable Y is prepared as an disentagled Hadamard basis vector and stays disentangled during the following reflections. The oracle C^f effectively performs the reflection $\mathcal{S}^{\perp}$ across the bad state supported at the models of f.

$$\psi^0\left[X^{\text{out}}_{[d]}\right] = \left\langle \epsilon_{0_{[d]}}\left[X^{\text{in}}_{[d]}\right], U\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right]\right\rangle_{\left[X^{\text{out}}_{[d]}\right]}$$

and

$$\left|\psi^0\left[X_{[d]}\right]\right|^2 = \mathbb{P}\left[X_{[d]}\right]$$

- a computation circuit for $f\left[X_{[d]}\right]$

13.5.1 Moments as rotation angles

We construct a Grover operator, which rotation angle determines the moment. The moment can therefore be estimated based on the measurement of the operators eigenvalue phases.

We have

$$\sum_{x_{[d]}\in[2]^d\,:\,f(x_{[d]})=1} \left|\psi^0\left[X_{[d]} = x_{[d]}\right]\right|^2 = \left\langle \mathbb{P}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \mu\,.$$

This insight motivates us to define the orthonormal quantum states

$$\psi^{\parallel}\left[X_{[d]}\right] = \frac{1}{\sqrt{\mu}} \sum_{x_{[d]}\in[2]^d\,:\,f(x_{[d]})=1} \psi^0\left[X_{[d]} = x_{[d]}\right]\cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

$$\psi^{\perp}\left[X_{[d]}\right] = \frac{1}{\sqrt{1-\mu}} \sum_{x_{[d]}\in[2]^d\,:\,f(x_{[d]})=0} \psi^0\left[X_{[d]} = x_{[d]}\right]\cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

and split the prepared state into a weighted sum of them as

$$\psi^0\left[X_{[d]}\right] = \sqrt{\mu}\cdot\psi^{\parallel}\left[X_{[d]}\right] + \sqrt{1-\mu}\cdot\psi^{\perp}\left[X_{[d]}\right]\,.$$

Note that here the mean parameter μ is the analogon to the acceptance probability λ in quantum rejection sampling. To be more precise, when aiming to sample from the models of f using a rejection sampling scheme starting with the uniform distribution, the acceptance probability would be μ. Instead of amplifying this acceptance probability we here want to measure it to retrieve the mean parameter.

We prepare the reflection $-\mathcal{S}^0$ in the Pauli-Z scheme and the reflection $\mathcal{S}^{\perp}$ in the oracle scheme (see Sect. 13.2). Their concatenation results in the negated Grover operator, which causes a phase offset by π.

13.5.2 Retrieving the moment from the phase

The eigenvalues of the Grover operator are (see Sect. 13.2) $\exp[\pm i\alpha]$ and 1. Note that the projection of the state ψ^0 onto the eigen space to the eigen value 1 vanishes, and the corresponding trivial phase is not measured by the Quantum Phase Estimation procedure.

The rotation angle is related to μ as

$$\alpha = 2 \cdot \sin^{-1}\left(\sqrt{\mu}\right)$$

Through measuring the phase $\phi \in \{\pm\alpha\}$ of one of these eigenvalues we can therefore retrieve the mean by

$$\mu = \left(\sin\left(\frac{\phi}{2}\right)\right)^2 .$$

13.5.3 Quantum Phase Estimation

The phase of the Grover eigenvalues can be estimated using the Quantum Phase Estimation routine [**?**, **?**]. We prepare auxiliary qubits $A_{[p]}$ and perform for each $\ell \in [p]$ 2^ℓ repetitions of $G^{\mathbb{P},f}$ controlled on A_ℓ. The inverse Fourier transform on the auxiliary qubits is supported on the (2-adic representation of the) phases on the eigenvalues, i.e. $\pm\alpha$, and can be detected by a computational basis measure of the ancilla variables.

When measuring the phase register as $a_{[p]}$ we compute a phase

$$\phi = 2\pi \cdot \sum_{\ell \in [p]} 2^{-(\ell+1)} a_\ell$$

which is either α or $-\alpha$. We retrieve the mean in both cases by

$$\mu = \sin^2\left(\pi \cdot \sum_{\ell \in [p]} 2^{-(\ell+1)} a_\ell\right) .$$

13.6 Moment Amplification

Instead of measuring the phases of the effective Grover operator, we now investigate the effect of uncontrolled Grover operations on the distributions. We find an analogon in discrete increases of the canonical parameters when the measurement distribution of the start state is in an exponential family.

13.6.1 Interpretation of Grover rotations

Rotating j times across the initial state then gives the tensor product of the antisymmetric ancilla state with

$$\psi^j\left[X_{[d]}\right] = \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\mathbb{P}^0,f}\left[X_{[d]}\right] + \cos\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\mathbb{P}^0,\neg f}\left[X_{[d]}\right] .$$

This reflection can be performed by the operator on the distributed qubits

$$U^t\left(\delta\left[X^{\text{in}}_{[d]}, X^{\text{out}}_{[d]}\right] - 2\epsilon_0\left[X^{\text{in}}_{[d]}\right] \otimes \epsilon_0\left[X^{\text{out}}_{[d]}\right]\right) U .$$

13.6.1.1 Mean parameter interpretation

The mean parameter of f with respect to the measurement distribution is

$$\mu^j = \left\langle \mathbb{P}^j \left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)^2 = \sin\left((1+2j)\sin^{-1}\left(\sqrt{\mu^0}\right)\right)^2$$

Here we used that $\alpha = 2\cdot\sin^{-1}\left(\sqrt{\mu^0}\right)$.

13.6.1.2 Canonical parameter interpretation

Lemma 13.1 *Works also for more generic measurement distributions! Let ψ^0 be the q-sample of the elementary Computation-Activation Network with canonical parameters (A, y_A, θ), and let us amplify the formula f_ℓ. Then ψ^j is the q-sample of the elementary Computation-Activation Network with canonical parameters*

$$A^j = \begin{cases} A\cup\{\ell\}, y_A^\ell = (y_A, 1) & \textit{if} \quad \sin\left((1+2\cdot j)\sin^{-1}\left(\sqrt{\mu_0\left[L=\ell\right]}\right)\right) = 1 \\ A, y_A & \textit{else} \end{cases}$$

and

$$\theta_j\left[L=\ell\right] = \begin{cases} 0 & \textit{if} \quad \sin\left((1+2\cdot j)\sin^{-1}\left(\sqrt{\mu_0\left[L=\ell\right]}\right)\right) = 1 \\ \theta\left[L=\ell\right] + \frac{\cos^2\left(\frac{\alpha}{2}\right)\left(1-\cos^2\left(\left(\frac{1}{2}+j\right)\alpha\right)\right)}{\sin^2\left(\frac{\alpha}{2}\right)\cdot\cos^2\left(\left(\frac{1}{2}+j\right)\alpha\right)} & \textit{else} \end{cases}.$$

Proof. Notice, that

$$\left\langle \mathbb{P}^0\left[X_{[d]}\right], \beta^{f_\ell}\left[Y_\ell, X_{[d]}\right]\right\rangle_{[Y_\ell]} = \begin{pmatrix} \left(\cos\left(\frac{\alpha}{2}\right)\right)^2 \\ \left(\sin\left(\frac{\alpha}{2}\right)\right)^2 \end{pmatrix}\left[Y_\ell\right] = \begin{pmatrix} 1-\mu_0\left[L=\ell\right] \\ \mu_0\left[L=\ell\right] \end{pmatrix}\left[Y_\ell\right]$$

and

$$\left\langle \mathbb{P}^j\left[X_{[d]}\right], \beta^{f_\ell}\left[Y_\ell, X_{[d]}\right]\right\rangle_{[Y_\ell]} = \begin{pmatrix} \left(\cos\left(\left(\frac{1}{2}+j\right)\alpha\right)\right)^2 \\ \left(\sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\right)^2 \end{pmatrix}\left[Y_\ell\right]\begin{pmatrix} 1-\mu_j\left[L=\ell\right] \\ \mu_j\left[L=\ell\right] \end{pmatrix}\left[Y_\ell\right]$$

If $\mu_j\left[L=\ell\right] = 1$ (that is $\sin\left(\left(\frac{1}{2}+j\right)\alpha\right) = 1$), the amplitude amplification amounts to adding f_ℓ as a hard constraint. This is equal to $A^j = A\cup\{\ell\}$ and $y_A^j = (y_A, 1)$.

If $\mu_j\left[L=\ell\right] \neq 1$ we choose $\theta_\Delta\left[L=\ell\right] \geq 0$ as in the claim and have after some algebra

$$\frac{\left(\cos\left(\frac{\alpha}{2}\right)\right)^2}{\left(\cos\left(\frac{\alpha}{2}\right)\right)^2 + \theta_\Delta\left[L=\ell\right]\cdot\left(\sin\left(\frac{\alpha}{2}\right)\right)^2} = \left(\cos\left(\left(\frac{1}{2}+j\right)\alpha\right)\right)^2$$

and therefore

$$\left\langle \mathbb{P}^j\left[X_{[d]}\right], \beta^{f_\ell}\left[Y_\ell, X_{[d]}\right]\right\rangle_{[Y_\ell]} = \left\langle \exp\left[\theta_\Delta\left[L=\ell\right]\cdot f_\ell\left[\theta\right]\right]\mathbb{P}^0\left[X_{[d]}\right], \beta^{f_\ell}\left[Y_\ell, X_{[d]}\right]\right\rangle_{[Y_\ell|\varnothing]}.$$

Thus, the rotation of the q-sample corresponds with an increase of the canonical parameter $\theta_0\left[L=\ell\right]$ by $\theta_\Delta\left[L=\ell\right]$. □

13.6.1.3 Maximum Entropy

When $\left(\frac{1}{2}+j\right)\alpha \leq \frac{\pi}{2}$ then all amplitudes remain positive, and $\psi^j\left[X_{[d]}\right]$ is a q-sample. What is more, we can show that the property of maximum entropy with respect to a statistic containing the amplified formula.

Theorem 13.1 *Let ψ^0 be the q-sample of a maximum entropy distribution with respect to the uniform base measure and the statistic $\mathcal{F}$ containing the amplified formula, and let $j \in \mathbb{N}$ be a rotation number such that $\left(\frac{1}{2}+j\right)\alpha \leq \frac{\pi}{2}$. Then also ψ^j is the q-sample of a maximum entropy distribution.*

Proof. By Lem. 13.1 the amplitude amplified state is an elementary Computation-Activation Network and thus a maximum entropy distribution. □

Notice, that we can only increase the canonical parameter by amplitude amplification. Decreasing could be done by amplitude amplification on $\neg f_\ell$ instead.

13.6.2 Moment Matching

We now investigate which Computation-Activation Networks can be prepared directly with amplitude amplification. While we already have studied sparse representations of the computation network parts, we here apply amplitude amplification by Grover rotations as an activation mechanism.

We orient on the iterative proportional fitting algorithm, which is used to estimate the canonical parameters in exponential families [**?**] and more generally of Computation-Activation Networks [**?**].

The algorithm loops over the moment estimation and moment amplification subroutines, which have been introduced in the sections above. The idea is to first compute the current moments of a distribution and second amplify one.

We understand this rotation as a moment matching operation to the formula f, see Algorithm 7.

Algorithm 7 Quantum Circuit Moment Matcher

Require: Formulas $f_p\left[X_{[d]}\right]$ for $\ell \in [p]$, vector $\mu_D[L]$ of mean parameters
Ensure: Circuit U preparing a state (when applied on the ground state) which measurement distribution matched $\mu_D[L]$

$U \leftarrow (\sigma_1 \circ H)[A_{\text{in}}, A_{\text{out}}] \otimes \left(\bigotimes_{k\in[d]} H\left[X_{k,\text{in}}, X_{k,\text{out}}\right]\right)$
while Convergence criterion not met **do**
Estimate $\mu[L]$,

- either by particle bases inference
- or by quantum counting (measuring the eigenvalue of the Grover operator), see Sect. 13.5

Select an $\ell \in [p]$, which moment has to be updated (e.g. by comparison of $\mu[L]$ with $\mu_D[L]$)
Choose whether to amplify f_ℓ or $\neg f_\ell$ based on whether the estimated mean is smaller than the target mean
Compute optimal j to match $\mu_D[L]$
Extend Circuit U by j alternations of

- C^{f_ℓ} (effectively an reflection across the subspace spanned by one-hot encoded models)
- reflections across the current state (by $U(I-2e_0e_0)U$)

end while

We can use Algorithm 7 as a preparation algorithm of a q-sample before doing ancilla augmentation as in the main section. Note that is important to keep track of the canonical parameters in the preparation. When activating the statistic qubits, only the difference of the already prepared canonical parameters and the target canonical parameters has to be used (importance sampling interpretation).

13.7 Extension: Sampling from proposal distributions

We can prepare basis circuit encodings to selection augmented formulas, in this way introducing formula selecting networks.

Idea for an inductive reasoning scheme: Prepare a q-sample from the empirical distribution and the current distribution. Then prepare the basis circuit encodings, where the selection variables are shared and the distributed variables assigned to the prepared samples. Now, the ancilla qubits can be designed to ϵ_1 and ϵ_0 accordingly. The rejection sampling scheme on both ancillas being 1 and the measurement of L prepares then the distribution

$$\left\langle \left\langle \mathbb{P}^D \left[X_{[d]}\right], \sigma^t \left[X_{[d]}, L\right]\right\rangle_{[L]}, \left\langle \tilde{\mathbb{P}} \left[X_{[d]}\right]\right\rangle_{[L]} \right\rangle_{[L|\varnothing]}$$

That is, the probability of selecting ℓ is proportional to

$$\mu_D \left[L = \ell\right] \cdot \left(1 - \tilde{\mu} \left[L = \ell\right]\right)$$

and thus prefers formulas, which have a large empirical mean, but a small current mean.

Open Question: Since the distribution is "similar" to $\exp\left[\mu_D \left[L = \ell\right] - \tilde{\mu} \left[L = \ell\right])\right]$ (terms appear in Taylor of first order), can we tune the distribution with an inverse temperature parameter β?

Chapter 14
Probabilistic Relational Models

Abstract We investigate in this chapter, how Compuation-Activation Networks can be trained given data in the form of a first-order logic world.

14.1 Semantic Structure

Understand semantics as model sets, over which distributions can be defined.

We now investigate the semantic structure of a first-order logic theory, when restricting to database semantics. This involves the representation of collections of worlds, where each has a substitution structure as discussed above.

14.1.1 World Enumerating Variables

Given database semantics, we have a finite set of worlds, which we enumerate by tuples of variables X.

Definition 14.1 (World enumerating variables) Given a first-order logic theory with constant symbols $\mathcal{U}$, function symbols $\{v_l : l \in [r]\}$ and predicate symbols $\{g_k : k \in [d]\}$, we enumerate the world under database semantics by a tuple

$$X_W = \left(\bigtimes_{\tilde{k} \in [d]} \bigtimes_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1}} X_{\tilde{k}, o_{\tilde{k}}} \right) \times \left(\bigtimes_{k \in [d]} \bigtimes_{o_k \in \in [r]^{n_k}} X_{k, o_k} \right)$$

of boolean variables. In the world indexed by x_W, the grounding tensor of the function v_l is

$$\beta^{v_l|_{x_W}} \left[O_l, O_{[r_l]} \right] = \sum_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1} : o_{\tilde{k}} = 1} \epsilon_{o_{\tilde{k}}} \left[O_{\tilde{k}} \right]$$

and of the predicate g_k

$$g_k|_{x_W} \left[O_{[r_k]} \right] = \sum_{o_{\tilde{k}} \in [r]^{n_k} : o_k = 1} \epsilon_{o_k} \left[O_k \right] .$$

We further have the restriction that each basis encoded function is a directed tensor. To restrict to those worlds, where this is true, we further introduce the validation base measure $\nu \left[X_W \right]$ with coordinates

$$\nu \left[X_W = x_W \right] = \begin{cases} 1 & \text{if} \quad \forall l \in [r] : \sum_{o_l \in [r]^{n_{\tilde{k}}+1}} = 1 \\ 0 & \text{else} \end{cases} .$$

14.1.2 Representation of Terms and Formulas

Combining its substitution structure and semantic structure we represent a first-order logic term v with arity r_v as a tensor

$$\beta^v \left[O_v, O_{[r_v]}, X_W\right] = \sum_{x_W : \nu[X_W = x_W]=1} \beta^{v|_{x_W}} \left[O_v, O_{[r_v]}\right] \otimes \epsilon_{x_W} \left[X_W\right] \ .$$

and a formula as the tensor

$$q \left[O_q, X_W\right] = \sum_{x_W : \nu[X_W = x_W]=1} q|_{x_W} \left[O_q\right] \otimes \epsilon_{x_W} \left[X_W\right] \ .$$

From Def. 14.1 the representation of predicate and function symbols are directly derived from the X_W, that is

$$g_k \left[O_{[r_k]}, X_W = x_W\right] = \sum_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1} : o_{\tilde{k}}=1} \epsilon_{o_{\tilde{k}}} \left[O_{\tilde{k}}\right]$$

and

$$\beta^{v_l} \left[O_l, O_{[r_l]}, X_W = x_W\right] = \sum_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1} : o_{\tilde{k}}=1} \epsilon_{o_{\tilde{k}}} \left[O_{\tilde{k}}\right]$$

14.1.3 Case of Propositional Logic

Propositional logic as discussed in (see Chapter 2) is the special case of first-order logic with empty sets of constant and function symbols, and $n_k = 0$ for the predicate symbols. With the convention $[]^0 = [2]$ the worlds are enumerated by the tuple

$$X_W = \bigtimes_{k \in [d]} X_k$$

which we have in previous chapters abbreviated by $x_{[d]}$.

To represent logical formulas as sets of possible worlds, and distributions of worlds, we applied in Part **??** one-hot encodings of possible worlds. For the case of propositional logic, this is

$$\epsilon_{x_W} \left[X_{[d]}\right] = \bigotimes_{k \in [d]} \epsilon_{x_k} \left[X_k\right] \ .$$

14.1.4 One-hot Encoding of Worlds

Let us now generalize the one-hot encodings of propositional logic worlds to worlds of first-order logic. To encode the boolean tensors x_W describing first-order logics as basis elements of a tensor space, we take the one-hot encoding

$$\epsilon : \left(\bigtimes_{\tilde{k} \in [d]} \bigtimes_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1}} [2]\right) \times \left(\bigtimes_{k \in [d]} \bigtimes_{o_k \in [r]^{n_k}} [2]\right) \to \left(\bigotimes_{\tilde{k} \in [d]} \bigotimes_{o_{\tilde{k}} \in [r]^{n_{\tilde{k}}+1}} \mathbb{R}^2\right) \otimes \left(\bigotimes_{k \in [d]} \bigotimes_{o_k \in [r]^{n_k}} \mathbb{R}^2\right)$$

defined by

$$\epsilon_{x_W}\left[X_W\right] = \left(\bigotimes_{\tilde{k}\in[d]} \bigotimes_{o_{\tilde{k}}\in[r]^{n_{\tilde{k}}+1}} \epsilon_{x_{\tilde{k},o_{\tilde{k}}}}\left[X_{\tilde{k},o_{\tilde{k}}}\right]\right) \otimes \left(\bigotimes_{k\in[d]} \bigotimes_{o_k\in[r]^{n_k}} \epsilon_{x_{k,o_k}}\left[X_{k,o_k}\right]\right)$$

This is a tensor of order $d \cdot r^n$, in a tensor space of dimension $2^{(d\cdot r^n)}$. Storage of such tensors in naive formats would not be possible. However, the basis CP format discussed in Chapter **??** still provides storage with demand linear in the order $d \cdot r^n$.

14.1.5 Probability Distributions

Having established the formalism of one-hot encodings also in the case of first-order logic worlds, we can now proceed with the definition of distributions and formulas, analogously to the development in Part **??**. Probability distributions over worlds coinciding on their domain are then non-negative and normalized tensors $\mathbb{P}\left[X_W\right]$ where each coordinate of a world x_W is captured by a boolean random variable $X_{k,o_{[n]}}$, indicating whether the k-th predicate holds on the object tuple indexed by $o_{[n]}$.

We notice that by definition these probability distributions are distributions of

$$\left(\sum_{k\in[d]} r^{n_k}\right) + \left(\sum_{\tilde{k}\in[d]} r^{n_{\tilde{k}}}\right)$$

Booleans. Unfortunately, it is not possible to design encoding spaces of smaller dimension, when our aim is to get any distribution over possible worlds by an element in the encoding space. This is due to the fact, that one-hot encodings provide a basis in the tensor space, as will be shown in Chapter **??**. The reason for the large encoding space dimension is therefore rooted in the equal number of possible worlds and not in an overhead in the dimension of the one-hot encoding space. We will later in this chapter investigate methods to handle such high-dimensional distributions in the formalism of exponential families.

14.2 Probabilistic Relational Models

So far we have studied Markov Logic Networks in propositional logic as probability distributions over worlds. In FOL they define probability distributions over relations in worlds with a fixed set of objects. More generally, such models are probabilistic relational models (see for an overview [**?**].

We in this section treat random worlds in first-order logic with fixed domains $\mathcal{U}$.

We in this section show, when and how we can interpret likelihoods of Markov Logic Networks in first-order logic in terms of samples of a Markov Logic Network in propositional logic.

14.2.1 Hybrid First-Order Logic Networks

Following [**?**] Markov Logic Networks in first-order logic are templates for distributions, which instantiate random worlds when choosing a set of objects $\mathcal{U}$. Given a fixed set of constants, they then define a distribution over the worlds, which objects correspond with the constants. This applies database semantics, where only those worlds are considered, where the unique name and domain closure assumptions given a set of constants are satisfied.

We count the number of true groundings to a formula by

$$\#q_\ell|_{x_W} = \left\langle q_\ell|_{x_W}\left[O_{q_\ell}\right]\right\rangle_{[\varnothing]} .$$

Using these counts as statistics, we now define Hybrid First-Order Logic Networks as a Computation Activation Network.

Definition 14.2 (Hybrid First-Order Logic Networks (HFLN)) Let there be a set $\mathcal{U}$ of objects and a set Q of first-order logic formulas. Further let (A, y_A, θ) as in Def. **??** be a tuple of a subset $A \subset [p]$, a tuple $y_A \in \bigtimes_{\ell \in A}[2]$ and $\theta\,[L] \in \mathbb{R}^p$. We then define the Hybrid First-Order Logic Network as the distribution

$$\mathbb{P}^{\#Q,(A,y_A,\theta)}\,[X_W] = \Big\langle \{\beta^{\#q_\ell}\,[Y_\ell, X_W] \,:\, \ell \in [p]\} \\ \cup \{\alpha^{\ell,\theta}\,[Y_\ell] \,:\, \ell \in [p]\} \cup \{\kappa^{\ell,(A,y_A)}\,[Y_\ell] \,:\, \ell \in [p]\}\Big\rangle_{\left[X_W|\varnothing\right]}$$

where

$$\kappa^{\ell,(A,y_A)}\,[Y_\ell] = \begin{cases} \epsilon_0\,[Y_\ell] & \text{if} \quad \ell \in A,\ y_\ell = 0 \\ \epsilon_{n_\ell - 1}\,[Y_\ell] & \text{if} \quad \ell \in A,\ y_\ell = 1 \\ \mathbb{I}\,[Y_\ell] & \text{else}\,. \end{cases}$$

The mean parameter polytope is the convex hull of the vectors

$$\sigma^Q\,[X_W = x_W, L]$$

to the worlds x_W with $\nu\,[X_W = x_W] = 1$. These vectors store are the counts of satisfied groundings to each formula, that is

$$\sigma^Q\,[X_W = x_W, L] = \left|\{o_{q_\ell} \,:\, q_\ell|_{x_W}\left[O_{q_\ell} = o_{q_\ell}\right] = 1\}\right|\,.$$

14.2.2 Propositionalization

Let us notice, that different to the case of propositional Hybrid Logic Networks treated in Chapter **??**, the statistic does not consist of boolean features, when formulas contain variables and we have multiple objects. One could, however, replace each q_ℓ by the set of the possible groundings, i.e. substitutions of the formulas variables by any tuple of objects in $\mathcal{U}$. The resulting distribution would be an Hybrid Logic Network with boolean statistic, which coincides with the Hybrid First-Order Logic Network when posing certain weight sharing conditions on θ. The downside of this construction is the increase in the number of features from p to $\sum_{\ell \in [p]} |\mathcal{U}|^{|O_{q_\ell}|}$. This polynomial in the cardinality of the domain set increase poses significant computational challenges, see [**?**].

We now show that the Hybrid First-Order Logic Network can be propositionalized to a Hybrid Logic Network in propositional logic.

Lemma 14.1 *We have*

$$\Big\langle \beta^{\#q_\ell}\,[Y_\ell, X_W]\,, \alpha^{\ell,\theta}\,[Y_\ell]\Big\rangle_{[X_W]} \\ = \left\langle \{\beta^{q_\ell|_{x_W}\left[x_W, o_{q_\ell}\right]}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell} \in [r_{q_\ell}]\} \cup \{\alpha^{\ell,\theta}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell} \in [r_{q_\ell}]\}\right\rangle_{[X_W]}$$

and

$$\Big\langle \beta^{\#q_\ell}\,[Y_\ell, X_W]\,, \kappa^{\ell,(A,y_A)}\,[Y_\ell]\Big\rangle_{[X_W]} \\ = \left\langle \{\beta^{q_\ell|_{x_W}\left[x_W, o_{q_\ell}\right]}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell} \in [r_{q_\ell}]\} \cup \{\kappa^{\ell,(A,y_A)}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell} \in [r_{q_\ell}]\}\right\rangle_{[X_W]}\,.$$

Proof. The first claim holds, since for any world x_W we have

$$\begin{aligned}
&\left\langle \beta^{\#q_\ell}\left[Y_\ell, X_W\right], \alpha^{\ell,\theta}\left[Y_\ell\right]\right\rangle_{[X_W=x_W]} \\
&\quad = \prod_{o_{q_\ell}\,:\,q_\ell|_{x_W}\left[x_W,o_{q_\ell}\right]=1} \exp\left[\theta\left[L=\ell\right]\right] \\
&\quad = \left\langle \{\beta^{q_\ell|_{x_W}\left[x_W,o_{q_\ell}\right]}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell}\in[r_{q_\ell}]\} \cup \{\alpha^{\ell,\theta}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell}\in[r_{q_\ell}]\}\right\rangle_{[X_W=x_W]} .
\end{aligned}$$

For $\ell \notin A$ the second claim is trivial, since any contraction of a basis encoding with a trivial head is trivial. If $\ell \in A$ and $y_\ell = 0$ then

$$\begin{aligned}
&\left\langle \beta^{\#q_\ell}\left[Y_\ell, X_W\right], \kappa^{\ell,(A,y_A)}\left[Y_\ell\right]\right\rangle_{[X_W=x_W]} \\
&\quad = \begin{cases} 1 & \text{if} \quad \forall o_{q_\ell} \,:\, q_\ell|_{x_W}\left[x_W, o_{q_\ell}\right] = 0 \\ 0 & \text{else} \end{cases} \\
&\quad = \left\langle \{\beta^{q_\ell|_{x_W}\left[x_W,o_{q_\ell}\right]}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell}\in[r_{q_\ell}]\} \cup \{\kappa^{\ell,(A,y_A)}\left[Y_{o_{q_\ell}}\right] \,:\, o_{q_\ell}\in[r_{q_\ell}]\}\right\rangle_{[X_W=x_W]} .
\end{aligned}$$

For $y_\ell = 1$ this holds analogously. □

Each substitution of the variables in q_ℓ by objects in $\mathcal{U}$, which satisfies the formula in the world x_W, therefore provides a factor of $\exp\left[\theta\left[L=\ell\right]\right]$ to the probability of x_W.

14.2.3 Conditioning on an Importance Formula

To reduce the number of object tuples influencing the probability distribution, we now condition Hybrid First-Order Logic Networks on situations where a formula, called the importance formula, has a fixed grounding tensor.

To this end, we mark pairs of term indices relevant to the distributions by an auxiliary index $j \in [m]$. Given a set $\{o^j_{[n]} \,:\, j \in [m]\}$ of indices to the important tuples we build a set encoding (see Def. **??**)

$$\underline{p} = \sum_{j\in[m]} \left(\bigotimes_{l\in[n]} \epsilon_{o^j_l}\left[O_l\right] \right) .$$

We interpret the tensor $\underline{p}$ as the grounding of a formula, which we call the importance formula.

To have a constant importance formula we define a syntactic representation and restrict the support of the Hybrid First-Order Logic Network to those world coinciding with groundings of the importance formula coinciding with $\underline{p}$ by designing a base measure

$$\nu^{\underline{p}}\left[X_W\right] = \begin{cases} 1 & \text{if} \quad p|_{x_W}\left[O_p\right] = \underline{p} \\ 0 & \text{else} \end{cases} .$$

The base measure restricts the Hybrid First-Order Logic Network to be those worlds, where $p|_{x_W}$ is coincides with the fixed tensor $\underline{p}$. Intuitively, $p|_{x_W}$ represents certain evidence about a first-order logic world, whereas other formulas are uncertain.

14.2.4 Decomposition of the Log Likelihood

To reduce the likelihood of a world to we make the assumption that all formulas in a Hybrid First-Order Logic Network are of the form

$$q_\ell\left[O_{q_\ell}\right] = \left(p\left[O_{[n]}\right] \Rightarrow h_\ell\left[O_{q_\ell}\right]\right) \tag{14.1}$$

that is a rule with the importance formula being the premise. In particular, we assume, that they depend on all term variables $O_{[n]}$. If this is not the case, we extend the formula trivially on the missing term variables. When this assumption holds, we can think of the importance formula as a conditions on individuals to satisfy a statistical relation given by h.

Towards connecting with propositional logic, we further make the assumption, that we can decompose the formula h_ℓ in what we will call extraction formulas.

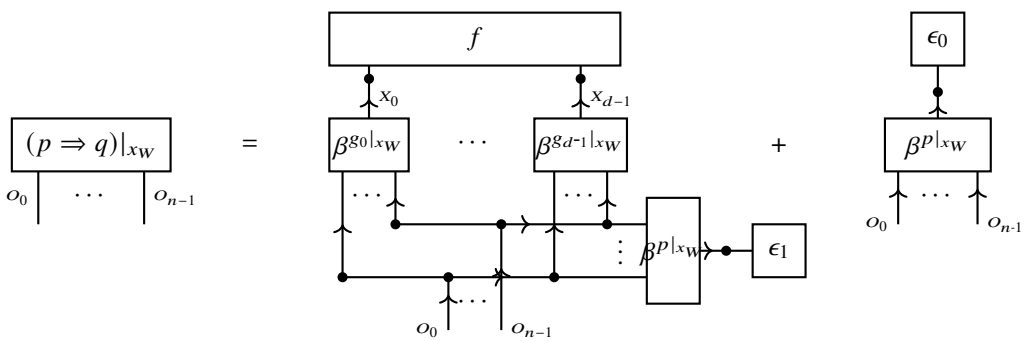
assumption 1. *We assume that there exist formulas* $\{g_k\left[X_W, O_{[n]}\right] : k \in [d]\}$*, which we refer to as atom extraction formulas, and an importance formula* $p\left[O_{[n]}\right]$ *such that the following holds. To each first-order logic formula* q_ℓ *there is another first-order logic formula* $h_\ell\left[O_{q_\ell}\right]$ *and a propositional formula* $f_\ell\left[X_{[d]}\right]$ *such that*

$$q_\ell\left[X_W, O_{[n]}\right] = \left(p\left[X_W, O_{[n]}\right] \Rightarrow h_\ell\left[X_W, O_{[n]}\right]\right)$$

and

$$h_\ell\left[O_{[n]}\right] = \left\langle \{f_\ell\left[X_{[d]}\right]\} \cup \{\beta^{g_k}\left[X_k, X_W, O_{[n]}\right] : k \in [d]\}\right\rangle_{\left[O_{[n]}\right]} .$$

We depict the assumption, that any formula is of the form (14.1) in the diagram



where the second summand depends only on the query p and therefore does not appear in the likelihood.

14.2.5 Reduction to Propositional Logic

We now make additional assumptions to decompose the partition function of an Hybrid First-Order Logic Network as a product of Hybrid Logic Network partition functions.

assumption 2. *Given an importance formula* p *and a tensor* $\underline{p}$ *we consider the base measure*

$$\nu^{\underline{p}}\left[X_W\right] = \begin{cases} 1 & \text{if} \quad p|_{x_W}\left[O_p\right] = \underline{p}\left[O_p\right] \\ 0 & \text{else} \end{cases}$$

of worlds with grounding of p *by* $\underline{p}$*. Let further enumerate by* $j \in [m]$ *the support of* $\underline{p}$*, that is* $o^j_{[n]}$ *are the indices of* $O_{[n]}$ *with* $\underline{p}\left[O_{[n]} = o^j_{[n]}\right] = 1$*. We assume that with respect to* $\underline{p}$ *the variables*

$$\left(g_k|_{x_W}\left[O_{[n]} = o^j_{[n]}\right]\right) \tag{14.2}$$

are for $k \in [d]$ *and* $j \in [m]$ *independent and uniformly distributed.*

When the above assumption holds, we now show that the probability of a first-order logic world with respect to a Hybrid First-Order Logic Network coincides with the likelihood of a dataset in a propositional Hybrid Logic Network.

Theorem 14.1 *Let there be a set of formulas* Q *such that Assumption 1 and Assumption 2 hold with an importance formula* p *and a tensor* $\underline{p}$*. Then for any* (A, y_A, θ) *and any world* x_W *with* $p|_{x_W} = \underline{p}$ *we have*

$$\frac{1}{m}\ln\left[\mathbb{P}^{\#Q,(A,y_A,\theta)}\left[X_W = x_W \middle| p|_{x_W} = \underline{p}\right]\right] = \mathbb{H}\left[\mathbb{P}^{D}, \mathbb{P}^{t,(A,y_A,\theta)}\right] - \frac{\ln\left[\left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}\right]}{m}$$

where $\mathcal{F}$ is the set of propositional equivalents to Q (see Assumption 1) and D the data map with evaluation at $j \in [m]$ by the enumerated non-vanishing coordinates of $\underline{p}\left[O_p\right]$

$$D(j) = \left(g_0|_{x_W}\left[O_{[n]} = o^j_{[n]}\right], \ldots, g_{d-1}|_{x_W}\left[O_{[n]} = o^j_{[n]}\right]\right).$$

To show the theorem, we show first in the following lemma the factorization of the partition function of the Hybrid First-Order Logic Network.

Lemma 14.2 *Under the assumptions of Thm. 14.1, we have*

$$\begin{aligned}
&\left\langle \beta^{Q|_{\mathcal{U}}}\left[Y_{[p]}, X_W\right], \tau^{(A,y_A,\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]} \\
&= \left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]} \cdot \left(\prod_{o_{[n]}:\underline{p}\left[O_{[n]}=o_{[n]}\right]=0} \tau^{\ell,(A,y_A,\theta)}\left[Y=1\right]\right) \cdot \\
&\left(\frac{1}{2^d}\cdot\left\langle \beta^{t}\left[Y_{[p]}, X_{[d]}\right], \tau^{(A,y_A,\theta)}\left[Y_{[p]}\right]\right\rangle_{[\varnothing]}\right)^m .
\end{aligned}$$

Proof. Using the assumption on the structure of the formulas we have

$$\#q_\ell|_{x_W} = \#\neg p|_{x_W} + \#(p \wedge h_\ell)|_{x_W}$$

and for any x_W

$$\begin{aligned}
&\left\langle \beta^{\#q_\ell}\left[Y_\ell\right], \tau^{\ell,(A,y_A,\theta)}\left[Y\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[X_W=x_W]} \\
&= \left(\prod_{o_{[n]}:\underline{p}\left[O_{[n]}=o_{[n]}\right]=0} \tau^{\ell,(A,y_A,\theta)}\left[Y=1\right]\right) \cdot \left\langle \beta^{\#(p\wedge h_\ell)}\left[Y_\ell\right], \tau^{\ell,(A,y_A,\theta)}\left[Y_\ell\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[X_W=x_W]}
\end{aligned}$$

Here by $\tau^{\ell,(A,y_A,\theta)}\left[Y\right]$ we denote the two-dimensional realization of the activation core. We use that the grounding tensor of p is constant among the by $\nu^{\underline{p}}\left[X_W\right]$ supported worlds and thus

$$\begin{aligned}
&\left\langle \beta^{\#(p\wedge h_\ell)}\left[Y_\ell\right], \tau^{\ell,(A,y_A,\theta)}\left[Y_\ell\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[X_W]} \\
&= \left\langle \bigcup_{j\in[m]}\left\{\beta^{h_\ell|_{o^j_{[n]}}}\left[Y_{\ell,j}\right], \tau^{\ell,(A,y_A,\theta)}\left[Y_{\ell,j}\right]\right\} \cup \left\{\nu^{\underline{p}}\left[X_W\right]\right\}\right\rangle_{[X_W]} .
\end{aligned}$$

For each $j \in [m]$ we have by Assumption 1

$$\begin{aligned}
&\left\langle \beta^{h_\ell|_{o^j_{[n]}}}\left[Y_{\ell,j}\right], \tau^{\ell,(A,y_A,\theta)}\left[Y_{\ell,j}\right]\right\rangle_{[X_W]} \\
&= \left\langle \left\{\beta^{g_k|_{o^j_{[n]}}}\left[Y_{k,j}\right] : k \in [d]\right\} \cup \left\{\beta^{f_\ell}\left[Y_f, Y_{k,j}\right], \tau^{\ell,(A,y_A,\theta)}\left[Y_f\right]\right\}\right\rangle_{[X_W]} .
\end{aligned}$$

From Assumption 2 we know

$$\left\langle \left\{\beta^{g_k|_{o^j_{[n]}}}\left[Y_{k,j}\right] : k \in [d],\ j \in [m]\right\} \cup \left\{\nu^{\underline{p}}\left[X_W\right]\right\}\right\rangle_{\left[Y_{[d]\times[m]}\right]} = \frac{\left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}}{2^{d\cdot m}} \cdot \mathbb{I}\left[Y_{[d]\times[m]}\right] .$$

Combining the above we get

$$\begin{aligned}
&\left\langle \beta^{Q|\mathcal{U}}\left[Y_{[p]}, X_W\right], \tau^{(A,y_A,\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]} \\
&= \frac{\left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}}{2^{d\cdot m}} \cdot \\
&\left(\prod_{o_{[n]}:\underline{p}\left[O_{[n]}=o_{[n]}\right]=0} \tau^{\ell,(A,y_A,\theta)}\left[Y=1\right]\right) \prod_{j\in[m]} \left\langle \beta^t\left[Y_{[p]}, Y_{[d]\times\{j\}}\right], \tau^{(A,y_A,\theta)}\left[Y_{[p]}\right]\right\rangle_{[\varnothing]} \\
&= \left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]} \cdot \left(\prod_{o_{[n]}:\underline{p}\left[O_{[n]}=o_{[n]}\right]=0} \tau^{\ell,(A,y_A,\theta)}\left[Y=1\right]\right) \\
&\left(\frac{1}{2^d} \cdot \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \tau^{(A,y_A,\theta)}\left[Y_{[p]}\right]\right\rangle_{[\varnothing]}\right)^m .\square
\end{aligned}$$

We notice that for the event $p|_{x_W} = \underline{p}$ to be of non-vanishing probability, we need to have $y_A = \mathbb{I}_A$ or $\underline{p}\left[O_p\right] = \mathbb{I}\left[O_p\right]$. With this lemma, we are now show Thm. 14.1.

Proof of Thm. 14.1. We have

$$\begin{aligned}
\ln\left[\mathbb{P}^{\#Q,(A,y_A,\theta)}\left[X_W = x_W \middle| p|_{x_W} = \underline{p}\right]\right] &= \ln\left[\left\langle \beta^{Q|\mathcal{U}}\left[Y_{[p]}, X_W\right], \tau^{(A,y_A,\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[X_W=x_W]}\right] \\
&\quad - \ln\left[\left\langle \beta^{Q|\mathcal{U}}\left[Y_{[p]}, X_W\right], \tau^{(A,y_A,\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}\right]
\end{aligned}$$

While the second term is decomposed by Lem. 14.2 we now derive a decomposition of the first term. By Assumption 1 and $p|_{x_W} = \underline{p}$ we have

$$\begin{aligned}
&\left\langle \beta^{Q|\mathcal{U}}\left[Y_{[p]}, X_W\right], \tau^{(A,y_A,\theta)}\left[Y_{[p]}\right], \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[X_W=x_W]} \\
&= \left(\prod_{o_{[n]}:\underline{p}\left[O_{[n]}=o_{[n]}\right]=0} \tau^{\ell,(A,y_A,\theta)}\left[Y=1\right]\right) \cdot \\
&\prod_{j\in[m]} \left\langle \{\beta^{g_k|_{o^j_{[n]}}}\left[Y_k, X_W = x_W\right] : k\in[d]\} \cup \{\beta^t\left[Y_{[p]}, X_{[d]}\right], \tau^{(A,y_A,\theta)} Y_{[p]}\}\right\rangle_{[\varnothing]} .
\end{aligned}$$

With Lem. 14.2 we then have

$$\begin{aligned}
&\frac{1}{m}\ln\left[\mathbb{P}^{\#Q,(A,y_A,\theta)}\left[X_W = x_W \middle| p|_{x_W} = \underline{p}\right]\right] \\
&= \frac{1}{m}\sum_{j\in[m]} \ln\left[\left\langle \{\beta^{g_k|_{o^j_{[n]}}}\left[Y_k, X_W = x_W\right] : k\in[d]\} \cup \{\beta^t\left[Y_{[p]}, X_{[d]}\right], \tau^{(A,y_A,\theta)} Y_{[p]}\}\right\rangle_{[\varnothing]}\right] \\
&\quad - \ln\left[\left\langle \{\beta^{g_k|_{o^j_{[n]}}}\left[Y_k, X_W\right] : k\in[d]\} \cup \{\beta^t\left[Y_{[p]}, X_{[d]}\right], \tau^{(A,y_A,\theta)} Y_{[p]}\}\right\rangle_{[\varnothing]}\right] - \frac{\ln\left[\left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}\right]}{m} \\
&= \mathbb{H}\left[\mathbb{P}^D, \mathbb{P}^{t,(A,y_A,\theta)}\right] - \frac{\ln\left[\left\langle \nu^{\underline{p}}\left[X_W\right]\right\rangle_{[\varnothing]}\right]}{m} .\square
\end{aligned}$$

Let us now investigate, in which cases the Assumption 2 of independent data can be matched.

Example 14.1 If the p and $g_0, \ldots, g_{d-1}$ are predicates applied on variables, and the index tuples $o^j_{[n]}$ are pairwise different, then Assumption 2 is met. This is the case, since the values $g_k\left[O_{[n]} = o^j_{[n]}\right]|_{x_W}$ are determined by different variables in X_W.

There are situations, where Assumption 2 is violated.

- extraction formula being a) conjunctions of predicates: Probability that they are satisfied decreases b) disjunctions of predicates: Probability that they are satisfied increases
- extraction formula coinciding with importance formula: Always satisfied, in this case still boolean
- extraction formulas contradicting each other, more general not independent from each other

Remark 14.1 (Approximation by Independent Samples) As argued above, we do not have independent samples in general. As a consequence, we cannot apply Lem. 14.2 to decompose the partition function term of the log-probability into factors to each solution map of p. In this case, it might be still benefitial to use the reduction to the likelihood of a HLN, but needs to understand it as a approximation to the true world probability.

If the expectations of each sample with respect to the marginalized distributions coincide, the average of empirical distribution also coincides with these (by linearity). When the creation of samples has sufficient mixing properties, the empirical distribution converges to this expectation in the asymptotic case of large numbers of samples.

14.2.6 Sample Extraction from First-Order Logic Worlds

We have observed that in certain situations the log-likelihood of a first-order logic world with respect to a Hybrid First-Order Logic Network coincides with the likelihood of a data set in a propositional Hybrid Logic Network. Let us now investigate the extraction process of these data set. We model the extraction process as a relation between a tuple of individuals and the extracted world in the factored system of atoms X_k.

Definition 14.3 Given a first-order logic world x_W, an importance formula p and extraction formulas g_k for $k \in [d]$, we define the extraction relation

$$\mathcal{R} \subset \left(\bigtimes_{l \in [n]} [r] \right) \otimes \left(\bigtimes_{k \in [d]} [2] \right)$$

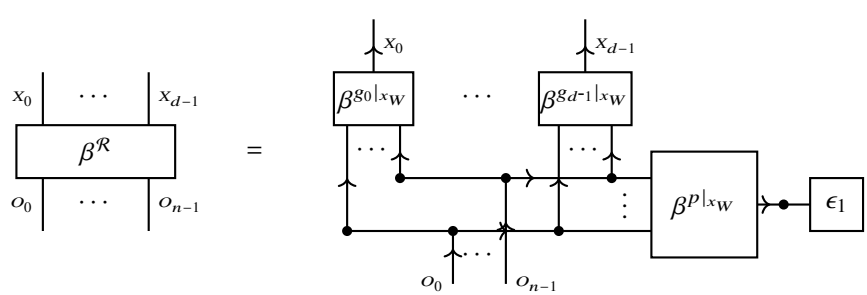
by

$$\mathcal{R} = \{ (o_{[n]}, x_{[d]}) \,:\, p|_{x_W} \left[O_{[n]} = o_{[n]} \right] = 1, \forall k \in [d] : x_k = g_k \left[O_{[n]} = o_{[n]} \right] \} .$$

The encoding of an extraction relation is the tensor

$$\beta^{\mathcal{R}} \left[O_{[n]}, X_{[d]} \right] \subset \left(\bigotimes_{l \in [n]} \mathbb{R}^r \right) \otimes \left(\bigotimes_{k \in [d]} \mathbb{R}^2 \right)$$

and drawn in a contraction diagram by



Here the contraction of β^p with the truth vector ϵ_1 represents the matching condition posed by p when extracting pairs of individuals.

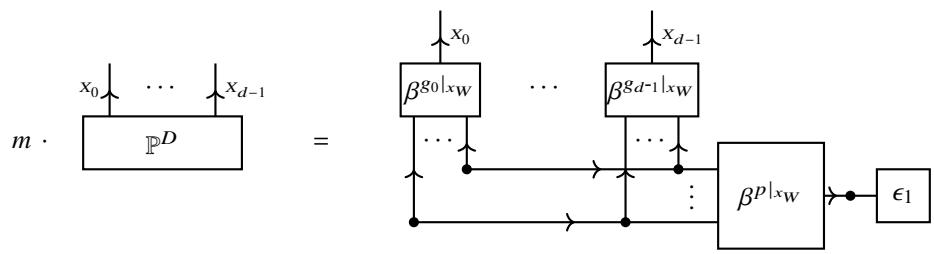
The empirical distribution of the extracted data is then the normalized contraction leaving only the legs to the extracted atomic formulas open, that is

$$\mathbb{P}^D = \frac{\left\langle \beta^{\mathcal{R}} \right\rangle_{[X_{[d]}]}}{\left\langle \beta^{\mathcal{R}} \right\rangle_{[\varnothing]}} .$$

Here the number of extracted data is the denominator

$$m = \left\langle \beta^{\mathcal{R}} \right\rangle_{[\varnothing]} = \left\langle \beta^{p} \left[Y_p, O_{[n]} \right], \epsilon_1 \left[Y_p \right] \right\rangle_{[\varnothing]} .$$

We depict this by



To connect with the empirical distribution introduced in Sect. 3.8.1 we now show how the empirical distribution extracted from the interpretations of the formulas $p, g_0, \ldots, g_{d-1}$ on a first-order logic world x_W can be represented by tensor networks.

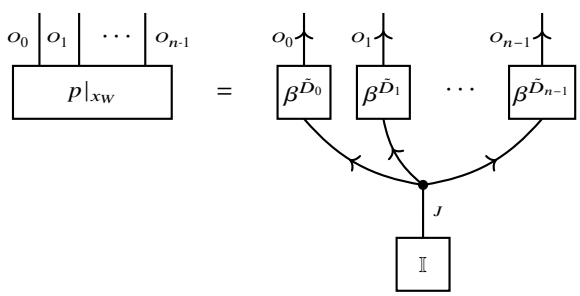
First of all, we decompose the importance formula into a basis CP format (see Chapter **??**), that is a decomposition

$$p|_{x_W} \left[O_{[n]} \right] = \left\langle \{ \rho^l \left[O_l, J \right] \, : \, l \in [n] \} \right\rangle_{[O_{[n]}]}$$

such that all $\rho^l \left[O_l, J \right]$ are directed and boolean tensors. Here an auxiliary variables J taking values in $[m]$ is introduced, which we call the data variable, which enumerates the non-vanishing coordinates of $p|_{x_W}$. With this decomposition, we can understand the decomposition of $p|_{x_W} \left[O_{[n]} \right]$ as a basis encoding of an term selection map $\tilde{D}$ with coordinate maps defined such that

$$\beta^{\tilde{D}_l} \left[O_l, J \right] = \rho^l \left[O_l, J \right] .$$

We depict this decomposition by:



Based on these construction, we now provide a tensor network decomposition of the extracted empirical distribution.

Theorem 14.2 *Given a first-order logic world x_W, an importance formula p and extraction formulas g_k for $k \in [d]$, we have*

$$\beta^{\mathcal{R}} \left[O_{[n]}, X_{[d]} \right] = \left\langle \{ \beta^{g_k|_{x_W}} \left[X_k, O_{[n]} \right] \, : \, k \in [d] \} \cup \{ \beta^{\tilde{D}_l} \left[O_l, J \right] \, : \, l \in [n] \} \right\rangle_{[O_{[n]}, X_{[d]}]}$$

and thus

$$\mathbb{P}^D \left[X_{[d]} \right] = \frac{1}{m} \left\langle \{ \beta^{g_k|_{x_W}} \left[X_k, O_{[n]} \right] \, : \, k \in [d] \} \cup \{ \beta^{\tilde{D}_l} \left[O_l, J \right] \, : \, l \in [n] \} \right\rangle_{[X_{[d]}]} .$$

Proof. To show the first claim, let us choose arbitrary state tuples $o_{[n]}$ and $x_{[d]}$. We then have

$$\left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] \,:\, k \in [d]\} \cup \{\beta^{\tilde{D}_l}\left[O_l, J\right] \,:\, l \in [n]\}\right\rangle_{\left[O_{[n]}=o_{[n]}, X_{[d]}=x_{[d]}\right]}$$
$$= \left\langle \{\beta^{g_k|_{x_W}}\left[X_k = x_k, O_{[n]} = o_{[n]}\right] \,:\, k \in [d]\} \cup \{\beta^{\tilde{D}_l}\left[O_l = o_l, J\right] \,:\, l \in [n]\}\right\rangle_{[\emptyset]} \,.$$

This contraction evaluates to 1, if and only if for all $k \in [d]$ we have $\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] = 1$ and

$$\left\langle \{\beta^{\tilde{D}_l}\left[O_l = o_l, J\right] \,:\, l \in [n]\}\right\rangle_{[\emptyset]} = 1 \,.$$

The first condition is equal to $x_k = g_k\left[O_{[n]} = o_{[n]}\right]$ for all $k \in [d]$ and the second to

$$p|_{x_W}\left[O_{[n]} = o_{[n]}\right] = 1 \,.$$

Comparing with the definition of the extraction relation (see Def. 14.3), we notice that these conditions are equal to $(o_{[n]}, x_{[d]}) \in \mathcal{R}$ and therefore to

$$\beta^{\mathcal{R}}\left[O_{[n]} = o_{[n]}, X_{[d]} = x_{[d]}\right] \,.$$

The first claim follows, since $\beta^{\mathcal{R}}$ is boolean, as is the contraction of the cores $\beta^{g_k|_{x_W}}$ with the cores $\beta^{\tilde{D}_l}$, which leaves the outgoing variables $X_{[d]}$ open. The second claim follows from the first using that $\mathbb{P}^D\left[X_{[d]}\right] = \frac{1}{m}\left\langle \beta^{\mathcal{R}}\right\rangle_{\left[X_{[d]}\right]}$. □

To connect with the representation of empirical distributions based on data cores (see Sect. 3.8.1), we now form data cores by contractions with the grounding of extraction formulas with the cores $\beta^{\tilde{D}_l}$ (see Figure 14.1),

$$\beta^{D_k}[X_k, J] = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right]\} \cup \{\rho^l\left[O_l, J\right] \,:\, l \in [n]\}\right\rangle_{[X_k, J]} \,.$$

The empirical distribution is then a tensor network of these tensors, as we show next.

Theorem 14.3 *We have*

$$\left\langle \beta^{\mathcal{R}}\right\rangle_{\left[X_{[d]}\right]} = \left\langle \{\beta^{D_k}[J, X_k] \,:\, k \in [d]\}\right\rangle_{\left[X_{[d]}\right]}$$

and thus

$$\mathbb{P}^D\left[X_{[d]}\right] = \frac{1}{m}\left\langle \{\beta^{D_k}[J, X_k] \,:\, k \in [d]\}\right\rangle_{\left[X_{[d]}\right]} \,.$$

Proof. By Thm. 14.2 we have

$$\beta^{\mathcal{R}}\left[O_{[n]}, X_{[d]}\right] = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] \,:\, k \in [d]\} \cup \{\beta^{\tilde{D}_l}\left[O_l, J\right] \,:\, l \in [n]\}\right\rangle_{\left[O_{[n]}, X_{[d]}\right]} \,.$$

Since $\beta^{\tilde{D}_l}\left[O_l, J\right]$ are directed and boolean, they can be copied and separately contracted with each $g_k|_{x_W}$, without changing the contraction. We arrive at

$$\beta^{\mathcal{R}}\left[O_{[n]}, X_{[d]}\right]$$
$$= \left\langle \left\{ \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right]\} \cup \{\beta^{\tilde{D}_l}\left[O_l, J\right] \,:\, l \in [n]\}\right\rangle_{[X_k, J]} \,:\, k \in [d]\right\}\right\rangle_{\left[O_{[n]}, X_{[d]}\right]}$$
$$= \left\langle \{\beta^{D_k}[J, X_k] \,:\, k \in [d]\}\right\rangle_{\left[X_{[d]}\right]} \,,$$

which established the claim. □

When many atom extraction formulas differ only by a constant, we can replace the constant by an auxiliary term variable. The atoms are then the atomizations of this variable (see Sect. 2.4.2.2), treated as a categorical variable, with respect to the constant in the extraction query. The advantages are that we can avoid the β-formalism and directly model the categorical distributions.

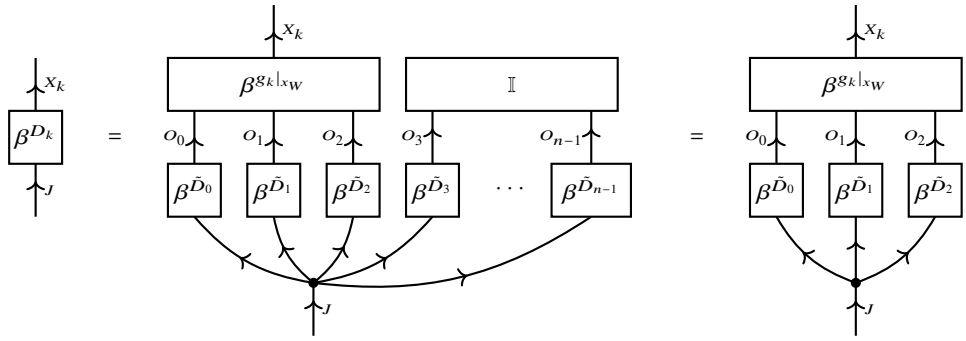


Fig. 14.1 Generation of a data core for the variable X_k given an extraction formula g_k and an importance formula, which grounding is decomposed into a basis CP format with leg vectors $\beta^{\tilde{D}_l}[O_l, J]$. Term variables, which are appearing in the importance formula, but not in the extraction formula g_k can be treated trivally by contraction with the trivial tensor (here $O_4, \ldots, O_{n-1}$).

This also enables a batchwise computation of multiple SPARQL queries, which differ only in one constant.

14.3 Generation of First-Order Logic Worlds

So far we have discussed, how Probabilistic Relational Models for first-order logic Knowledge Bases such as Knowledge Graphs can be built by extracting data. Conversely, any binary tensor can be interpreted as a Knowledge Graph. To be more precise, we follow the intuition that the ones coordinates mark possible worlds compatible with the knowledge about a factored system. Each possible world can then be encoded in a subgraph of the Knowledge Graph representing the world. This amounts to an "inversion" of the data generation process described in the subsection above.

In the previous section we have described a way to extract an effective empirical distribution for the likelihood of a first-order logic world given a Hybrid First-Order Logic Network. We now want to investigate methods to reproduce an empirical distribution based on a constructed first-order logic world.

Definition 14.4 (Reproduction of Empirical Distributions) Given an empirical distribution $\mathbb{P}^D \in \bigotimes_{k\in[d]} \mathbb{R}^2$, we say that a triple $(x_W, p, g_{[d]})$ of a first-order logic world x_W an importance formula p and extraction formulas $g_{[d]} = \{g_k : k \in [d]\}$ reproduces $\mathbb{P}^D$, when

$$\mathbb{P}^D = \left\langle \{p|_{x_W}\left[O_{[n]}\right]\} \cup \{\beta^{g_k|_{x_W}}\left[X_k, O_{[n]}\right] : k \in [d]\}\right\rangle_{[X_{[d]}|\varnothing]} .$$

Note that for distribution $\mathbb{P}$ to be reproducible, it needs to have rational coordinates. If any only if all coordinates are rational, we find a $m \in \mathbb{N}$ such that $\operatorname{im}(m \cdot \mathbb{P}) \subset \mathbb{N}$. We can then interpret m as the number of samples, and construct a sample selector map by understanding each coordinate of $m \cdot \mathbb{P}$ as the number of appearances of the respective world in the samples.

We show different schemes and give examples on Knowledge Graphs, where we provide examples for importance and extraction formulas by SPARQL queries.

14.3.1 Samples by Single Objects

In the first reproduction scheme we construct datapoints by dedicated objects, which represent a sample, that is we choose a domain $\mathcal{U} = [m]$.

Theorem 14.4 *Let there be an empirical distribution $\mathbb{P}^D$ to a sample selector map D (see Def. 3.16), we construct a world $x_W[L, O]$ with d unary predicates by*

$$x_W[L, O] = \sum_{k \in [d]} \sum_{j \in [m] : D_k(j)=1} \epsilon_k [L] \otimes \epsilon_j [O] \ .$$

We further choose a trivial importance query, that is

$$p|_{x_W} [O] = \mathbb{I} [O] \ ,$$

and extraction queries coinciding with the unary predicates, that is for $k \in [d]$

$$g_k = g_k \ .$$

Then, the triple $(x_W, p, g_{[d]})$ *reproduces* $\mathbb{P}^D$.

Proof. By Thm. 14.3 it is enough to show, that the data cores constructed from the data extraction process coincide with those of $\mathbb{P}^D$. We enumerate to this end the non-vanishing coordinates of $p|_{x_W}$ by the data variable J taking values $j \in [m]$, as

$$p|_{x_W} [O = j] = 1$$

and choose

$$\tilde{D} = \delta \ .$$

For arbitrary $k \in [d]$ and $j \in [m]$ we now have

$$\begin{aligned} \beta^{D_k} [X_k, J = j] &= \left\langle \beta^{g_k|_{x_W}} [X_k, O] \, , \rho^0 [O, J = j] \right\rangle_{[X_k, J]} \\ &= \left\langle \beta^{g_k|_{x_W}} [X_k, O] \, , \epsilon_{\tilde{D}(j)} [O] \right\rangle_{[X_k, J=j]} \\ &= \epsilon_{D_k(j)} [X_k] \ . \end{aligned}$$

This coincides with the slice of the data core of the CP representation of empirical distributions used in Thm. 3.16. Since the slice and the core was arbitrary, the tensor network representations in Thm. 3.16 and Thm. 14.3 are equal and thus the triple $(x_W, p, g_{[d]})$ reproduces $\mathbb{P}^D$. □

We now give by the next theorem an example of a Knowledge Graph with SPARQL queries reproducing and arbitrary empirical distribution.

Theorem 14.5 *Let* $\mathbb{P}^D$ *be an empirical distribution to the sample selector* D. *We construct a Knowledge Graph of the resources* $\mathcal{U} = \{s_j \ : \ j \in [m]\} \cup \{C\} \cup \{C_k \ : \ k \in [d]\}$, *where* s_j *represent samples and* C_k *unary predicates, by*

$$\mathrm{RDF}|_{x_W} = \sum_{j \in [m]} \epsilon_{I_{s_j}} O_s \otimes \epsilon_{I_{\mathrm{rdftype}}} O_p \otimes \epsilon_{I_C} O_o + \sum_{j \in [m]} \sum_{k \in [d] : D_k(j)=1} \epsilon_{I_{s_j}} O_s \otimes \epsilon_{I_{\mathrm{rdftype}}} O_p \otimes \epsilon_{I_{C_k}} O_o \ .$$

We further define an importance formula by the SPARQL *query*

p= *SELECT* { *?x* } *WHERE* { *?x* rdf : type C .}

and for each $k \in [d]$ *an extraction formula by the query*

g_k = *SELECT* { *?x* } *WHERE* { *?x* rdf : type C_k .} .

Then the triple $(\mathrm{KG}|_{x_W}, p, g_{[d]})$ *reproduces* $\mathbb{P}^D$.

Proof. We show the theorem analogously to Thm. 14.4, with the slide difference in the importance formula. We have for the grounding of p on $\mathrm{KG}|_{x_W}$ that

$$p|_{x_W} [O] = \sum_{j \in [m]} \epsilon_{I_{s_j}} O$$

and enumerate the non-vanishing coordinates by J.

For each extraction formula we have

$$g_k|_{x_W}\,[O] = \sum_{j\in[m]\,:\,D_k(j)=1} \epsilon_{I_{s_j}} O\,.$$

It follows that the data cores used in Thm. 14.3 are

$$\beta^{D_k}\,[X_k, j] = \epsilon_0\,[X_k] \otimes \left(\sum_{j\in[m]\,:\,D_k(j)=0} \epsilon_j\,[J]\right) + \epsilon_1\,[X_k] \otimes \left(\sum_{j\in[m]\,:\,D_k(j)=1} \epsilon_j\,[J]\right)$$

and they thus coincide with those in the decomposition in Thm. 3.16. The claim follows therefore with the same argumentation as in the proof of Thm. 14.4. □

Let us provide some more insights on the construction of the reproducing Knowledge Graph in Thm. 14.5. By the insertions to the one-hot encodings $\epsilon_{I_{s_j}} O_s \otimes \epsilon_{I_{\text{rdftype}}} O_p \otimes \epsilon_{I_C} O_o$ we mark each sample representing resource by a class and ensure its appearance as a owl : NamedIndividual in the graph. The insertions $\epsilon_{I_{s_j}} O_s \otimes \epsilon_{I_{\text{rdftype}}} O_p \otimes \epsilon_{I_{C_k}} O_o$ on the other side encode the sample selecting map, by inserting exactly the assertions corresponding with the respective sample. In this simple Knowledge Graph, Description Logic is expressive enough to represent any formula q composed of the formulas $g_0, \ldots, g_{d-1}$.

14.3.2 Samples by Tuples of Objects

We now instantiate multiple objects for each datapoint, one for each variable of the importance formula, i.e. $\mathcal{U} = [m] \times [n]$ Label individuals $s_{j,l}$ by data index and variable index.

Lemma 14.3 *Let there a data map D, queries $p, g_{[d]}$ and a first-order logic world containing objects $s_{j,l}$ for $j \in [m]$ and $l \in [n]$ If*

$$p|_{x_W} = \sum_{j\in[m]} \bigotimes_{l\in[n]} \epsilon_{I_{s_{j,l}}}\,[O_l]$$

and for any $k \in [d]$

$$g_k|_{x_W} = \sum_{j:D_k(j)=1} \bigotimes_{O_l\in O_{g_k}} \epsilon_{I_{s_{j,l}}}\,[O_l]\,.$$

Then the tuple $(\mathrm{KG}|_{x_W}, p, \{g_0, \ldots, g_{d-1}\})$ *reproduces* $\mathbb{P}^D$.

Proof. We notice that the grounding of the importance formula is in a basis CP format, since by assumption

$$p|_{x_W} = \sum_{j\in[m]} \bigotimes_{l\in[n]} \epsilon_{I_{s_{j,l}}}\,[O_l]\,.$$

We choose J to enumerate the non-vanishing entries and get a term selecting map

$$\tilde{D}_l(j) = I_{s_{j,l}}\,.$$

From this we have

$$\left\langle \{\beta^{g_k|_{x_W}}\,[X_k, O_{g_k}]\} \cup \{\beta^{\tilde{D}_l}\,[O_l, J]\,:\,l \in [n]\}\right\rangle_{[X_k,J]} = \beta^{D_k}\,[X_k, J]$$

and the claim follows with the same argumentation as in the proof of Thm. 14.4. □

14.4 Discussion

Statistical Models are called Probabilistic Relational Models. Extensions are models that also handle structural uncertainty, i.e. distributions of worlds with varying $\mathcal{U}$.

In the emerging area of network science [**?**, **?**], statistical models for random graphs are investigated. Statistical Models of first-order logic go beyond the typical single edge type perspective of network science.

Remark 14.2 (Alternative Representation of empirical distributions) So far, we have motivated the representation of empirical distributions based on basis CP decompositions based on data maps. In this section, based on the extraction queries, we have observed that empirical distributions might have more efficient representation formats. In many applications such as the computation of log-likelihoods we can use any representation of the empirical distribution by tensor networks. It is thus not necessary to compute the data cores as above, unless one requires a list of the extracted samples.

Part III
Contraction Calculus

Glossary

Use the template *glossary.tex* together with the Springer Nature document class SVMono (monograph-type books) or SVMult (edited books) to style your glossary in the Springer Nature layout.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.